Differential Privacy with OpenDP

Manuel V. Loureiro

Version 2026-10-26

Table of contents

Preface **1**
How this book is organized 1

I. Getting started **3**

1. A first release **5**
1.1. Install the library 5
1.2. One noisy number 6
1.3. The five steps of an analysis 7
1.4. A count and a mean 8
1.5. What the context refuses 9
1.6. Summary 10
1.7. References 11

2. The data and the context **12**
2.1. The Labour Force Survey sample 12
2.2. Loading the data lazily 13
2.3. A context on the frame 14
2.4. A query is not a frame 16
2.5. The first release: how many responses 17
2.6. A margin, in preview 18
2.7. Reading the released count 19
2.8. Summary 19
2.9. References 20

II. Tabular data **21**

3. Essential statistics **23**
3.1. The context for this chapter 23
3.2. Counting responses 24
3.3. Where bounds come from 26
3.4. The sum of hours 26
3.5. The mean 28
3.6. The median and the quartiles 29
3.7. A check against the truth 30
3.8. Summary 31
3.9. References 32

4. Grouping **33**
4.1. Why grouping keys are dangerous . . . 33
4.2. Stable keys . . . 34
4.3. Explicit keys . . . 35
4.4. Invariant keys . . . 36
4.5. Margins as public descriptors . . . 38
4.6. The noise per group and the ungrouped count . . . 39
4.7. Checking the release against the truth . . . 39
4.8. Summary . . . 40
4.9. References . . . 41

5. Identifier truncation and bounds **42**
5.1. A privacy unit with an identifier . . . 42
5.2. Truncating each person's rows . . . 44
5.3. Truncating the groups a person touches . . . 45
5.4. One row per person . . . 48
5.5. Bounds in the privacy unit . . . 48
5.6. The check against the truth . . . 50
5.7. Summary . . . 51
5.8. References . . . 52

6. Preparing microdata **53**
6.1. Prepare inside the query . . . 53
6.2. Columns, filters and groups . . . 54
6.3. The types OpenDP tracks . . . 56
6.4. Casting, filling and clipping . . . 57
6.5. Predicates and arithmetic . . . 59
6.6. Strings and dates . . . 59
6.7. A worked preparation: hours lost to absence . . . 60
6.8. What a refused expression looks like . . . 62
6.9. Summary . . . 62
6.10. References . . . 64

7. Assessing utility and choosing the noise **65**
7.1. What an accuracy statement means . . . 65
7.2. A target error bar on the survey . . . 66
7.3. Laplace under pure DP, Gaussian under zCDP . . . 67
7.4. One person in many groups: L1 against L2 . . . 68
7.5. Stating the guarantee in epsilon . . . 70
7.6. Signed and unsigned counts . . . 71
7.7. Which noise to choose . . . 72
7.8. The releases of one run . . . 72
7.9. Summary . . . 73
7.10. References . . . 74

III. The framework **75**

8. A framework to understand differential privacy **77**
8.1. Two kinds of object . . . 77
8.2. Functions . . . 78
8.3. Domains . . . 79
8.4. Metrics . . . 80
8.5. Measures . . . 81
8.6. Stability maps and privacy maps . . . 81
8.7. Chaining a transformation and a measurement . . . 82
8.8. The typical workflow in both APIs . . . 84
8.9. Summary . . . 86
8.10. References . . . 88

9. Transformations **89**
9.1. Constructors and chains . . . 89
9.2. Preprocessing . . . 90
9.3. The sum . . . 93
9.4. The mean and the variance . . . 95
9.5. Quantile scores . . . 97
9.6. Reading a chain . . . 98
9.7. Summary . . . 99
9.8. References . . . 100

10. Measurements **101**
10.1. Additive noise . . . 101
10.2. Canonical noise . . . 104
10.3. Thresholded noise . . . 104
10.4. The approximate Laplace projection . . . 107
10.5. Noisy max and selection . . . 107
10.6. Randomized response . . . 109
10.7. Summary . . . 111
10.8. References . . . 112

11. Combinators **114**
11.1. Chaining . . . 114
11.2. Non-adaptive composition . . . 116
11.3. Adaptive composition . . . 117
11.4. Odometers . . . 119
11.5. Privacy filters . . . 119
11.6. Chaining and nesting compositors . . . 120
11.7. Measure casting . . . 121
11.8. Amplification by subsampling . . . 123
11.9. Summary . . . 124
11.10. References . . . 125

12. Utilities and the Context API inside **126**
12.1. Types . . . 126

12.2. Parameter search . . . 128
12.3. Serialization . . . 130
12.4. The compositor behind a context . . . 131
12.5. The query behind a release . . . 132
12.6. Nested compositors and other measures . . . 133
12.7. Summary . . . 134
12.8. References . . . 135

IV. Extending the library **136**

13. Plugins **138**
13.1. Trust and the honest-but-curious flag . . . 138
13.2. A measurement plugin . . . 139
13.3. A transformation plugin . . . 141
13.4. Your own domains, metrics and measures . . . 142
13.5. Registering a constructor for the Context API . . . 143
13.6. A Theil-Sen regression . . . 144
13.7. Choosing grouping columns privately . . . 147
13.8. Summary . . . 150
13.9. References . . . 151

14. Statistical modelling **152**
14.1. A model or a table? . . . 152
14.2. Linear regression . . . 153
14.3. Principal component analysis . . . 155
14.4. The NumPy domain underneath . . . 157
14.5. What a reviewer should ask . . . 158
14.6. The releases of one run . . . 158
14.7. Summary . . . 159
14.8. References . . . 160

15. Limitations and troubleshooting **161**
15.1. What OpenDP does not promise . . . 161
15.2. The feature flags . . . 162
15.3. What lies outside the guarantee . . . 164
15.4. The error messages you will meet . . . 165
15.5. Reading an OpenDPException . . . 167
15.6. What a reviewer verifies . . . 168
15.7. Summary . . . 169
15.8. References . . . 170

V. Synthetic data **171**

16. Contingency tables **173**
16.1. From grouped counts to marginals . . . 173
16.2. The context under zCDP, with a delta . . . 174
16.3. Keys and cuts . . . 175

16.4. The Fixed algorithm and its workload . . . 176
16.5. The released table . . . 177
16.6. Adaptive estimation . . . 178
16.7. Checking the release against the truth . . . 178
16.8. Summary . . . 180
16.9. References . . . 181

17. Synthetic data with AIM and MST **182**
17.1. When you do not know the questions . . . 182
17.2. The AIM release . . . 183
17.3. A spanning tree of pairs with MST . . . 185
17.4. A first look at the one-way marginals . . . 186
17.5. Nulls and the key set . . . 186
17.6. Guaranteeing marginals with Sequential . . . 187
17.7. Generating the rows . . . 188
17.8. The privacy statement . . . 190
17.9. The measured marginals against the real ones . . . 191
17.10. Summary . . . 192
17.11. References . . . 192

18. Evaluating and tuning synthetic data **194**
18.1. What good means . . . 194
18.2. The workload error . . . 196
18.3. A query nobody measured . . . 197
18.4. Train on synthetic, test on real . . . 198
18.5. What the budget and the algorithm buy . . . 199
18.6. How many rows, and how to draw them . . . 200
18.7. What to publish . . . 201
18.8. Summary . . . 202
18.9. References . . . 203

Appendices **205**

A. Glossary **205**

B. Index **206**

Preface

This book teaches the OpenDP library, from installing it and releasing one noisy number to generating a synthetic copy of a dataset. OpenDP is a library for differential privacy (DP), the guarantee that a release reveals almost nothing about any one person in the data. The library's documentation is a reference, organized by what the library contains. This book is a course, organized by what you learn next, and it follows the documentation closely enough that every chapter names the pages it covers, so you can read the two side by side.

Every piece of code here has been run. A chapter is built around one script in the repository's `code/` directory, which a test runs and checks against every number the chapter quotes. A short block on a page is either a snippet that runs on its own or an excerpt of that script, so the code you read is the code that produced the output next to it. OpenDP draws its noise from a cryptographic generator that cannot be seeded, so a released value is different on every run. A chapter therefore quotes the numbers that are the same on every run, such as noise scales, error bars and thresholds, and shows released values in a block marked as one run.

One dataset runs through the book: the sample of the French Labour Force Survey (LFS) that the OpenDP documentation uses in its own tutorials, about 3.8 million quarterly survey responses from 1.7 million people. Each chapter releases something new about it, so by the end you will have counted, summed, grouped, bounded and modelled it, and finally replaced it with a synthetic table.

You need Python and enough NumPy or Polars to read a short query. You do not need to know differential privacy already. Each chapter says in a sentence what the results it uses mean, and links to the full edition of *Privacy-Preserving Machine Learning* for their derivations, which this book never repeats.

How this book is organized

1. **Getting started** installs the library, releases a first number, and sets up the context that mediates every later query to the data.
2. **Tabular data** releases counts, sums, means, quantiles and grouped tables with Polars, bounds what one person can contribute, prepares the data inside the privacy guarantee, and reads error bars.
3. **The framework** opens the box: domains, metrics and measures, transformations and measurements, how they chain and compose, and the utilities that search for parameters.
4. **Extending the library** writes new transformations and measurements as plugins, fits regression and principal component models, and knows what the library does not promise.
5. **Synthetic data** releases contingency tables and generates synthetic microdata with the AIM and MST algorithms, then measures how good the copy is.

The code runs on OpenDP 0.16.0 with Polars 1.36.1, pinned in the repository so that every number stays reproducible. To rerun a chapter, run `uv run python code/<part>/opendp_<topic>.py` from the repository's root.

Part I.

Getting started

This part takes you from an empty Python environment to a context that holds a real dataset and a privacy budget. It assumes nothing about differential privacy. The one idea it teaches is simple to state. A differentially private release is a randomized computation whose noise is calibrated to how much one person can change the answer. OpenDP does that calibration for you, and it keeps count of what has been spent.

The first chapter installs the library and releases a single noisy number. It does so first with the low-level framework, where you build the mechanism by hand, and then with the Context API, where you state the privacy unit and the budget and let the library find the noise. On the way it introduces the five steps of every analysis in OpenDP. You identify the privacy unit, set the budget, collect public information, mediate access to the data, and submit queries.

The second chapter brings in the dataset that runs through the book. It is the sample of the French Labour Force Survey (LFS) that the OpenDP documentation uses in its tutorials. The chapter shows how to load the data lazily with Polars and why a person who answers the survey in many quarters needs a privacy unit of many rows. It then explains how a context splits one budget over several queries, and what a margin is.

Table 1.: The chapters of this part and the questions they answer.

Chapter	Question it answers
A first release	How do I install OpenDP, and what happens when I release one noisy number?
The data and the context	How do I put a real dataset behind a context, and what do its unit, budget and margins mean?

By the end of this part you will be able to install OpenDP and explain the privacy unit and the budget of a release in a sentence each. You will also be able to build a context for a dataset in which one person owns many rows.

1. A first release

Suppose you hold a list of a hundred people's scores and want to publish how many there are and their average, without letting anyone work out any one person's score. This chapter does exactly that with OpenDP, and on the way it introduces the one idea the whole library is built on. A **differentially private** release is a randomized computation whose noise is calibrated to how much one person can change the answer. OpenDP's job is to do the calibration, to refuse a computation it cannot calibrate, and to keep count of what has been spent.

The chapter follows the documentation's quickstart and its typical workflow. You will install the library, build a Laplace mechanism by hand and read its guarantee, then let the Context API do the work: state the privacy unit and the budget, hand over the data once, and release a count and a mean. By the end you will be able to explain what each line of a first OpenDP program promises.

The script opendp_first_release.py runs every step on OpenDP 0.16.0, with a hundred integers drawn by a seeded generator standing in for the scores. The noise scales, the error bars and the privacy maps it prints are the same on every run. The released values are not, because OpenDP draws its noise from a cryptographic generator that cannot be seeded, so this page shows them in one block marked as one run.

```
uv run python code/part01-getting-started/opendp_first_release.py
```

1.1. Install the library

OpenDP is a Python package with a Rust core, and it reads tables through Polars, so install it with the Polars extra.

```
pip install 'opendp[polars]'
```

The repository pins `opendp[polars]==0.16.0` with Polars 1.36.1, which is what every number in this book comes from. Then import the prelude, which gathers the library's modules under one name, and switch on the `contrib` feature.

```
import opendp.prelude as dp

dp.enable_features("contrib")
print(dp.__version__)  # 0.16.0
```

The feature flag is a statement about trust. Every component of OpenDP is meant to come with a written proof that it does what it claims, and a review of that proof. Components whose proof has not finished that process are marked `contrib`, and the library will not run them unless you opt in. Almost everything in this book, from the Context API to the Polars queries, is still `contrib` in version 0.16.0, so the line goes at the top of every script. A reviewer who needs only vetted components can leave it out and see what stops working.

1.2. One noisy number

The simplest private computation adds noise to one number. OpenDP builds it from a **space**, which is a pair of a domain and a metric, and a constructor that turns the space into a **measurement**, which is a randomized function with a privacy guarantee.

```
import opendp.prelude as dp

dp.enable_features("contrib")
space = dp.atom_domain(T=float, nan=False), dp.absolute_distance(T=float)
laplace = space >> dp.m.then_laplace(scale=1.0)
print(laplace.map(1.0))  # 1.0
print(laplace.map(2.0))  # 2.0
print(laplace.check(1.0, 0.5))  # False
```

The domain says what the input may be, here any finite float. The metric says how two inputs are compared, here by their absolute difference. The constructor `then_laplace` adds a draw from the Laplace distribution with scale 1 to its input, and the `>>` operator attaches it to the space. Calling `laplace(123.0)` gives a noisy value near 123, different each time.

The guarantee lives in the **privacy map**. Its argument is a bound on how far apart two inputs can be, which differential privacy calls the **sensitivity** of the quantity you are releasing. Its value is the largest privacy loss ε the mechanism can incur on two such inputs. For Laplace noise the map is the sensitivity divided by the scale, so a sensitivity of 1 at scale 1 costs $\varepsilon = 1$ and a sensitivity of 2 costs $\varepsilon = 2$. The `check` method asks whether the map at a given sensitivity stays within a given ε, and it answers that sensitivity 1 at scale 1 does not fit within $\varepsilon = 0.5$. Doubling the scale halves the cost, which the script confirms.

```
laplace scale 1: map(1.0)=1.0 map(2.0)=2.0
laplace scale 1: check(1.0, 1.0)=True check(1.0, 0.5)=False
laplace scale 2: map(1.0)=0.5
```

What ε promises is that changing one person's data changes the probability of any output by a factor of at most e^{ε}. The full edition derives the Laplace mechanism's guarantee, and why the scale must be the sensitivity over ε, in its chapter on the Laplace and Gaussian mechanisms in the full edition. For now the point is where the guarantee comes from: not from the noise alone, but from the noise together with a bound on what one person can change. You supplied that bound as the argument of `map`. The rest of the chapter shows how the library works it out for you.

1.3. The five steps of an analysis

The documentation's typical workflow has five steps, and every analysis in this book follows them. The first three are decisions you make before touching the data. The fourth hands the data to the library, and the fifth asks it questions.

The first step is to identify the **privacy unit**, which is what the guarantee protects. The documentation calls it the unit of privacy. Here each person contributes one row, so the unit is one row, and two datasets are **neighbours** when one is the other with a row added or removed. OpenDP writes that as a metric, the symmetric distance, and a distance of 1 under it.

```
import opendp.prelude as dp

dp.enable_features("contrib")
print(dp.unit_of(contributions=1))  # (SymmetricDistance(), 1)
print(dp.loss_of(epsilon=1.0))  # (MaxDivergence, 1.0)
print(dp.loss_of(epsilon=1.0, delta=1e-7))  # (Approximate(MaxDivergence), (1.0, 1e-07))
print(dp.loss_of(rho=0.1))  # (ZeroConcentratedDivergence, 0.1)
```

The second step is to set the **privacy budget**, which the documentation calls the privacy loss. It is a measure, which says how the distributions of two releases are compared, and a bound on that comparison. Pure differential privacy compares them by the max divergence and bounds it by ε, so `loss_of(epsilon=1.0)` is the pair you see printed. The other two lines show the budgets later chapters use. Adding a slack δ gives approximate differential privacy, and a budget ρ gives zero-concentrated differential privacy, which Assessing utility and choosing the noise introduces. A budget of $\varepsilon = 1$ is a common starting point, and the documentation points to the OpenDP deployments registry for the values real releases have used.

The third step is to collect **public information**: facts about the data that you choose not to protect because they are known anyway. Here they are the bounds on a score, 0 to 100, and a value to stand in for a missing score, the midpoint 50. Bounds must come from the domain or from another public source, never from looking at the private data, because a bound read off the data leaks it.

The fourth step is to **mediate access**. The script builds a **context**, which holds the data, the privacy unit, the budget and the rule for splitting it, and from then on every computation on the data goes through it.

```
def make_context(data: list[float]) -> dp.Context:
    """The context: the data behind a privacy unit of one row, a budget of epsilon = 1, spli
    return dp.Context.compositor(
        data=data,
        privacy_unit=dp.unit_of(contributions=1),  # neighbours add or remove one person's r
        privacy_loss=dp.loss_of(epsilon=EPSILON),  # pure differential privacy
        split_evenly_over=QUERIES,  # each query gets epsilon / 3
    )
```

The context is a **compositor**, because the budget of several releases adds up. Three queries at a third of ε each cost ε in all, which is the basic composition rule that the full edition's chapter on properties proves. The script prints the split.

```
privacy unit: (SymmetricDistance(), 1)
privacy loss: (MaxDivergence, 1.0)
bounds: (0.0, 100.0)  imputed value: 50
budget per query: [0.3333, 0.3333, 0.3333]
```

The fifth step is to submit queries, which the next section does. One habit from the documentation is worth adopting now: once the context exists, stop touching the data directly, and delete your own reference to it. Anything you compute outside the context is outside the guarantee.

1.4. A count and a mean

A query is built by chaining methods on `context.query()`, each a constructor of the framework with its space left out, because the context knows the space. The count query counts the rows and adds Laplace noise.

```
count_query = context.query().count().laplace()
count_scale = count_query.param()  # the scale the budget allows: sensitivity 1 over epsilon
count_accuracy = dp.discrete_laplacian_scale_to_accuracy(scale=count_scale, alpha=0.05)
dp_count = count_query.release()
```

Nothing in `laplace()` says what scale to use. The context fills it in, by searching for the smallest scale whose privacy map at the unit's distance still fits within the query's share of the budget, and `param` reports what it found. A larger scale would also fit, but it would add more noise than the budget requires. Adding or removing one row changes a count by 1, so the sensitivity is 1, and the share is $\varepsilon/3$, so the scale is 3. The accuracy function turns that scale into an error bar: with probability 0.95, the released count is within about 9.4 of the true count. The count is an integer, so the noise is the discrete Laplace distribution and the accuracy function is the discrete one.

```
count: scale=3.0000 95% accuracy=9.4457
```

Read those two numbers before you release anything. They depend on the budget and the sensitivity, not on the data, so you can decide whether the error bar is acceptable without spending anything.

The mean needs more preparation, because the mean of a list is not a well-behaved function of the list until three things are fixed.

```
mean_query = (
    context.query()
    .impute_constant(IMPUTED)  # a missing value becomes the public midpoint
    .clamp(BOUNDS)  # every value is clipped into the public bounds
```

```
    .resize(size=dp_count, constant=IMPUTED)  # the mean needs a known size: the released co
    .mean()
    .laplace()
)
mean_scale = mean_query.param()
dp_mean = mean_query.release()
```

Imputation removes missing values, since the mean of a list with a hole in it is undefined. Clamping bounds every value, since without bounds one person could move the mean by any amount and no finite noise would hide them. Resizing fixes the number of rows, since the mean divides by it, and the sensitivity of a mean is only bounded when that divisor is known. The size passed in is the count just released, which is allowed because a released value is public and using it again costs nothing. That is the post-processing property, proved in the full edition. If the noisy count is too small, the resize pads the list with the imputed value, and if it is too large, it drops rows, so the released mean is the mean of a slightly altered list. The error bar below does not cover that alteration.

Because the scale of the mean depends on the released count, it changes from run to run. At the true size of 100 the script computes the scale and its error bar at development time, which is fine here because the data are a public toy.

```
mean at size 100: scale=3.0000 95% accuracy=8.9872
```

A scale of 3 on a mean of values between 0 and 100 gives an error bar of about 9 points, which is wide. The reason is the budget. A third of $\varepsilon = 1$ on a hundred rows is little, and the Laplace mechanism's error does not shrink with the number of rows when the sensitivity is a fixed fraction of the range. Essential statistics shows how a larger dataset and a sum released alongside a count do better.

1.5. What the context refuses

The context keeps count. After the count and the mean, two of the three shares are spent, and the script reads them back.

```
budget spent after two queries: [0.3333, 0.3333]
```

A third query takes the last share. A fourth has nothing left to take, and the context raises an error instead of releasing anything.

```
try:
    context.query().count().laplace().release()
    refusal = "released"
except ValueError as err:
    refusal = str(err)
```

```
fourth query: Privacy allowance has been exhausted
```

This refusal is the reason to use a context rather than loose mechanisms. With the framework alone, nothing stops you from calling a measurement twice and spending twice, and the guarantee you can claim is whatever you tracked by hand. With a context, the budget is a hard limit, and the claim is what you declared at the start.

Finally the releases of one run, read against the error bars.

```
released (one run):
  count 105  (true 100, within 9.4 with probability 0.95)
  mean scale 2.8571  mean 53.32  (true 50.93)
  third query, a second count 108
```

The count landed within its error bar, as it should about 19 times in 20. The mean's scale came out below 3 because the released count was above 100, and the mean itself landed within its bar too. Comparing a release with the truth is possible here only because the data are a toy. On real data you cannot run that check, so you make these decisions on public or historical data while you develop, and release the real data once.

1.6. Summary

Table 1.1.: The calls of a first release, what each does, and the rule of thumb that goes with it.

Call or concept	What it does	Rule of thumb
`dp.enable_features("contrib")`	Switches on the components whose proofs are not yet vetted	Put it at the top of every script; leave it out to see what is vetted
`space >> dp.m.then_laplace(scale)`	A Laplace measurement on a domain and a metric	The scale is the sensitivity over ε
`measurement.map(d_in)`	The largest ε for inputs at most `d_in` apart	Read it before releasing; it costs nothing
`measurement.check(d_in, d_out)`	Whether the map at `d_in` stays within `d_out`	What the context runs for you when it searches for a scale
`dp.unit_of(contributions=1)`	The privacy unit: neighbours add or remove one row	Choose it before anything else; one row per person is the default case
`dp.loss_of(epsilon=1.0)`	The privacy budget under pure differential privacy	$\varepsilon = 1$ is a common start; the registry lists real values
`dp.Context.compositor(...)`	Holds the data, the unit, the budget and its split	One context per release; delete your own reference to the data
`context.query().count().laplace()`	A query whose scale the context fills in from the budget	Read `param()` and the accuracy before `release()`

Call or concept	What it does	Rule of thumb
`impute_constant`, `clamp`, `resize`	Make the mean's sensitivity finite	Bounds and imputed values come from public information
A fourth query on a budget split in three	`ValueError: Privacy allowance has been exhausted`	The refusal is the point of the context

1.7. References

- OpenDP documentation: "Quickstart", "Typical Workflow", and "Context" in the API user guide.
- Dwork, McSherry, Nissim and Smith, "Calibrating Noise to Sensitivity in Private Data Analysis" (2006): the Laplace mechanism and the definition it satisfies.
- Gaboardi, Hay and Vadhan, "A Programming Framework for OpenDP" (2020): the framework of domains, metrics, measures and maps behind every call.
- The full edition's chapters on the definition in the full edition, post-processing and composition in the full edition and the Laplace and Gaussian mechanisms in the full edition derive the results this chapter states.

2. The data and the context

The first chapter released a count and a mean from a list of a hundred numbers that fit in one line of Python. Real data arrive as a table with millions of rows, read from a file rather than typed in, and one person appears in the table many times. This chapter puts such a table behind a context. OpenDP is a community library for differential privacy, built on a Rust core with an independent review of its proofs, and it reads tables through Polars. The one idea this chapter adds is that the privacy unit must cover everything one person can contribute. A person who answers a survey every quarter therefore needs a unit of many rows, and every noise scale grows with it.

The chapter follows the documentation's introduction to tabular data and its page on how OpenDP's Polars differs from plain Polars. You will meet the dataset that runs through the whole book, load it lazily, and build a context with a unit of 36 rows and a budget split five ways. Then you will see how a query on a context differs from one on a frame, release the count of responses, and preview the margin that sums need. You will then be able to put a real dataset behind a context and defend every argument of the call.

The script opendp_data_context.py runs every step on OpenDP 0.16.0 with Polars 1.36.1. The sizes of the sample, the budget splits, the scale and error bar of the count, the margin and the true count are the same on every run. The released count is not, because OpenDP draws its noise from a cryptographic generator that cannot be seeded, so this page shows it in one block marked as one run.

```
uv run python code/part01-getting-started/opendp_data_context.py
```

2.1. The Labour Force Survey sample

Every quarter, Eurostat and the national statistical offices survey the working hours of people across the European Union in the Labour Force Survey (LFS). Eurostat publishes a public sample of the microdata. The OpenDP project prepared a slice of it for its tutorials, with a few columns of the French sample over the nine years from 2005 to 2013. The public microdata are already protected by statistical disclosure control, the classical toolkit of recoding categories, suppressing rare values and adding a little noise. So nothing you release from this sample can harm anyone, and you can compare every release with the truth, which is why the book uses it.

Table 2.1.: The columns of the LFS sample, with the coding of the survey's user guide.

Column	Meaning	Coding
`PIDENT`	Person identifier	An integer, generated by the OpenDP project
`YEAR`	Reference year	2005 to 2013

Column	Meaning	Coding
QUARTER	Reference quarter	1 to 4
SEX	Sex	1 male, 2 female
AGE	Age during the reference week	Single years, as a float with a few missing values
HWUSUAL	Hours per week usually worked in the main job	1 to 98 hours; 0 when the hours vary too much to give; 99 when not applicable; missing when there was no answer
HWACTUAL	Hours actually worked in the reference week	Coded like HWUSUAL
ILOSTAT	Labour status during the reference week	1 worked for pay; 2 had a job but was absent; 3 laid off; 4 conscript; 5 neither worked nor had a job; 9 under 15 years old

The survey publishes no identifier for a person, so the OpenDP project generated PIDENT, a synthetic column that links rows as if they were one person's answers over time. It exists to show how the library handles a person who owns many rows, and it does not point to a real person. The script reads the sizes of the sample before any context exists, which is fine only because the sample is public.

```
rows: 3,811,932  people: 1,682,306
years: 2005 to 2013  quarters: 36
responses per person: max 15  distinct quarters per person: max 4
```

There are about 3.8 million responses from about 1.7 million identifiers, so a typical identifier appears two or three times. The busiest appears 15 times, well below the 36 quarters of the survey. Its rows fall into at most four distinct quarters, so the synthetic identifier sometimes links several rows of one quarter. Neither number is used below. The privacy unit comes from the survey's design, which is public, and the two numbers only confirm that the design's bound holds with room to spare.

2.2. Loading the data lazily

Polars offers two ways to read a file. An eager read loads the whole table into memory at once. A lazy scan records only where the file is and what its columns are, and it reads the rows when a query finally runs. OpenDP targets the lazy API, so the script opens the sample with scan_parquet and gets a LazyFrame. Printing its schema reads nothing but the file's header.

```
frame = pl.scan_parquet(practitioner_datasets.france_lfs())  # a LazyFrame: the file is read
print("schema: " + ", ".join(f"{name} {dtype}" for name, dtype in frame.collect_schema().ite
```

```
schema: PIDENT Int64, YEAR Int64, QUARTER Int64, SEX Int64, AGE Float64, HWUSUAL Float64, HW
```

The documentation reads the same sample as a CSV file, through `dp.examples.get_france_lfs_path()` and `scan_csv`. The repository caches it as Parquet instead, for a reason the documentation spells out in a warning. A CSV file carries no types, so Polars infers them from the values it sees, and the type of a column then depends on the data. A column of whole numbers becomes an integer column, and the same column with one decimal value in it becomes a float column.

```
import io

import polars as pl

integers = pl.scan_csv(io.BytesIO(b"hours\n35\n40\n"))
decimals = pl.scan_csv(io.BytesIO(b"hours\n35\n40.5\n"))
strings = pl.scan_csv(io.BytesIO(b"hours\n35\n40\n"), infer_schema=False)
print(integers.collect_schema())  # Schema({'hours': Int64})
print(decimals.collect_schema())  # Schema({'hours': Float64})
print(strings.collect_schema())  # Schema({'hours': String})
```

Why does that matter for privacy? The guarantee covers the queries that go through the context, and nothing else. Loading the data is outside it, and the documentation leaves that step to a trusted curator. Suppose one person's unusual value changes an inferred type or makes parsing fail, and that difference becomes visible outside the trusted environment. Then the data have leaked through a channel the guarantee never counted. Inside a trusted environment a parsing error is useful, because it points to malformed data before any wrong statistic is released. There are two safe habits. One is to read a format that carries its own schema, such as Parquet. The other is to load every CSV column as a string with `infer_schema=False`, as the last line of the snippet does, and to cast it yourself.

The context should hold the `LazyFrame` itself, not a collected table. A lazy frame lets OpenDP read the plan of every query before a single row is touched. It also keeps the 3.8 million rows out of memory until a release needs them.

2.3. A context on the frame

The context of the first chapter took a list, and this one takes the frame. Everything else about it is the same, but every argument has a different value, and each deserves a sentence.

```
def make_context(frame: pl.LazyFrame, margins: Sequence[dp.polars.Margin] | None = None) ->
    """A context on the LazyFrame: a privacy unit of 36 rows, a budget of epsilon = 1 split
    return dp.Context.compositor(
        data=frame,
        privacy_unit=dp.unit_of(contributions=CONTRIBUTIONS),  # neighbours add or remove on
        privacy_loss=dp.loss_of(epsilon=EPSILON),  # pure differential privacy
        split_evenly_over=QUERIES,  # each query gets epsilon / 5
```

```
        margins=margins,  # public facts about the frame; the count needs none
    )
```

```
privacy unit: (SymmetricDistance(), 36)
privacy loss: (MaxDivergence, 1.0)
budget split evenly over 5: [0.2, 0.2, 0.2, 0.2, 0.2]
budget split by weights [2, 1, 1, 1]: [0.4, 0.2, 0.2, 0.2]
```

The privacy unit is `dp.unit_of(contributions=36)`, and the 36 comes from the survey's design. The unit is what the guarantee protects, so it must cover everything one person can contribute to the table. The survey runs every quarter over nine years, and a person answers at most once a quarter, so a person owns at most 36 rows. Two datasets are therefore **neighbours** when one is the other with up to 36 rows of one person added or removed, which OpenDP writes as a symmetric distance of 36. A later chapter, on identifier truncation, replaces the fixed 36 by `dp.unit_of(contributions=1, identifier="PIDENT")`, which lets the library enforce the bound instead of trusting it.

The unit has a price. Adding or removing one person changes a count by up to 36, so every count here is 36 times as sensitive as with a unit of one row. Every noise scale grows by the same factor. That price is not negotiable. A unit of one row would protect each quarter's answer on its own. For a person with 15 rows, the guarantee would then weaken to 15 times the budget, by the group-privacy property that the full edition's chapter on properties proves.

The budget is `dp.loss_of(epsilon=1.0)`, pure differential privacy with a privacy parameter ε of 1. It promises that changing one person's data, all 36 rows of it, changes the probability of any output by a factor of at most e^{ε}. The full edition's chapter on the definition sets that out. The documentation calls this argument the privacy loss, and this book calls it the budget, because the context spends it.

The budget must be split, because the losses of several releases add up. With `split_evenly_over=5` each query gets $\varepsilon/5$, and the five shares of 0.2 sum to the whole budget, which is the basic composition rule from the same chapter on properties. A limit on the number of queries is essential, since without it the total loss has no bound. The documentation's advice is to split evenly when the queries are alike, and by weights when some matter more. The script builds a second context only to print the weighted split.

```
weighted = dp.Context.compositor(
    data=frame,
    privacy_unit=dp.unit_of(contributions=CONTRIBUTIONS),
    privacy_loss=dp.loss_of(epsilon=EPSILON),
    split_by_weights=WEIGHTS,  # the first query gets twice the share of each of the other t
)
```

The first query gets 0.4 and the other three get 0.2 each, so the weights are shares of the budget and not values of ε. The documentation recommends one context for all your queries, and every release below comes from one.

```
context = make_context(frame)
del frame  # from here on, every computation on the data goes through a context
```

The last line is a habit the documentation insists on. Once the context exists, delete your own reference to the frame. Anything you compute from the frame itself is outside the guarantee, and the sizes printed above are exactly such a computation, allowed here only because the sample is public.

2.4. A query is not a frame

With a `LazyFrame` behind it, `context.query()` returns a `LazyFrameQuery`, and the documentation's page on Polars versus OpenDP lists four ways in which it differs from the frame. A tiny frame shows the first two.

```
import opendp.prelude as dp
import polars as pl

dp.enable_features("contrib")
context = dp.Context.compositor(
    data=pl.LazyFrame({"hours": [35, 40, 0, 99]}),
    privacy_unit=dp.unit_of(contributions=1),
    privacy_loss=dp.loss_of(epsilon=1.0),
    split_evenly_over=5,
)
query = context.query().select(dp.len(signed=True))
print(type(query).__name__)  # LazyFrameQuery
scale = query.summarize()["scale"].item()
print(f"scale {scale:.1f}")  # scale 5.0
```

The first difference is how you specify the data. You never touch the frame again, and you write every query against `context.query()` instead. The documentation calls it a mock for the real data, and the script shows how good a mock it is.

```
query type: LazyFrameQuery
query schema: Schema({'PIDENT': Int64, 'YEAR': Int64, 'QUARTER': Int64, 'SEX': Int64, 'AGE':
collect before release: You must call `.release()` before executing a query.
plain Polars mean: MakeMeasurement("Expr is not recognized at this time: col("AGE").mean().
```

The mock knows the schema of the frame, so a query written against it is checked against the real columns and types. The documentation describes it as an empty `LazyFrame` with extra methods. The installed version makes it a class of its own rather than a subclass of `LazyFrame`. That changes nothing in how you use it.

The second difference is how you construct a query. OpenDP extends the Polars API in two places. The query gains the methods `summarize`, which reports the noise each aggregate will get,

and `release`, which spends the budget. Expressions gain the namespace `.dp`, whose methods are the aggregates that add noise, such as `dp.len`, `.dp.sum` and `.dp.median`. In the snippet, a unit of one row and a share of $\varepsilon/5$ give the count a scale of 5, the sensitivity over the share.

The third difference is how you run a query. In plain Polars you call `collect` on a lazy frame, and in OpenDP you must call `release` first, as the refusal in the text block shows. The release accounts for the privacy loss, takes the query's share of the budget, and returns a `OnceFrame`, which you then collect. The last section shows what happens when you collect it twice.

The fourth difference is what a query may contain. OpenDP makes guarantees only about the query plans and expressions it has a proof for, so it behaves like an allow-list. A plain Polars `mean` of the age column is refused with the message in the text block, because that expression adds no noise. The private mean is `.dp.mean`, which the next chapter uses. OpenDP also changes the arguments of some expressions it does accept, most often so that a failure cannot raise an error that depends on the data.

2.5. The first release: how many responses

The count of rows is the one statistic that needs no public information beyond the unit. The query selects `dp.len(signed=True)`, and the script reads its summary before releasing it.

```
count_query = context.query().select(dp.len(signed=True))  # signed: the noise may take the
count_summary = count_query.summarize(alpha=0.05)  # the scale and the 95% error bar, before
released = count_query.release()  # spends the share; a OnceFrame that collects once
dp_count = released.collect().item()
```

```
shape: (1, 5)
┌────────┬──────────────┬──────────────────┬────────┬────────────┐
│ column ┆ aggregate    ┆ distribution     ┆ scale  ┆ accuracy   │
│ ---    ┆ ---          ┆ ---              ┆ ---    ┆ ---        │
│ str    ┆ str          ┆ str              ┆ f64    ┆ f64        │
╞════════╪══════════════╪══════════════════╪════════╪════════════╡
│ len    ┆ Frame Length ┆ Integer Laplace  ┆ 180.0  ┆ 539.731115 │
└────────┴──────────────┴──────────────────┴────────┴────────────┘
count at alpha=0.01: accuracy=829.4299
```

The summary is a small table with one row per aggregate. The count is a frame length, its noise is integer Laplace, which is the discrete Laplace distribution because a count is an integer, and its scale is 180. That scale is the sensitivity over the share of ε, so 36 over 0.2. Here the price of the unit shows, because the snippet of the last section got a scale of 5 from the same share with a unit of one row. The accuracy column appears because `alpha` was given. It says that, with probability 0.95, the released count is within about 540 of the true count. Asking for more confidence widens the bar, and at `alpha=0.01` it is about 829. Neither number depends on the data, so you can decide whether the bar is acceptable before spending anything.

The argument `signed=True` decides what happens when the noise is large and negative. An unsigned count is clamped at zero, so a noisy count can never be negative, but the clamping

biases small counts upwards. A signed count is released as it comes, below zero if need be, and stays unbiased. On a count of 3.8 million with a scale of 180 the difference never arises, but the habit matters for small groups later. The installed version still defaults to the unsigned count and says so.

```
dp.len() without signed: FutureWarning: The default value is currently signed=False, and cou
```

So write `signed=True` out, as the documentation does, and the code will behave the same when the default changes.

2.6. A margin, in preview

A count needs nothing but the unit, and the next statistic the documentation releases, a sum of working hours, needs one more thing. The script asks for that sum on the context above and prints the refusal, then builds a context that carries the missing fact.

```
margin = dp.polars.Margin(
    # at most 150,000 responses per quarter (50,000 households of under three people, both p
    # over the 36 quarters of the survey: a bound on the length of the frame that costs no p
    max_length=QUARTER_RESPONSES * CONTRIBUTIONS,
)
bounded = make_context(frame, margins=[margin])
```

```
margin: Margin(by=[], max_length=5400000, max_groups=None, invariant=None)
sum without a margin: MakeTransformation("must specify 'max_length' in a margin with by=[]")
sum with the margin: scale=14,405.2  (36 x 80 over epsilon / 5 = 14,400)
```

A **margin** is a set of facts about the frame, grouped by some columns, that you declare public. With `by=[]` the group is the whole frame, and `max_length` is a bound on how many rows it has. A sum needs that bound because OpenDP's guarantee covers overflow and floating-point rounding too. Adding millions of floats loses a little precision, and the proof that the loss cannot leak anything needs to know how many numbers were added. Without the bound the library refuses to build the sum, which is the first line of the block. With it, a sum of clipped usual hours gets a scale a little above 14,400, which is the plain sensitivity over the share. The small extra pays for the rounding.

The bound must be public, and the documentation shows how to find one. Eurostat publishes that the French survey reaches about 50,000 households a quarter, and public statistics put the average household under three people. So 150,000 responses a quarter is a generous bound. Over the 36 quarters that gives 5.4 million rows, which is above what the sample holds. The bound was not read off the row count printed earlier, and it must not be, because a bound read from the data leaks the data. You give up a fact that an adversary could look up anyway. In exchange, the library can prove that arithmetic cannot undermine the guarantee. A margin can also carry a bound on the number of groups and declare the keys or the lengths of groups invariant, which Grouping takes up. Essential statistics builds the sum itself.

2.7. Reading the released count

The releases of one run, read against the error bar and the true count of this public sample.

```
true count: 3,811,932
released (one run):
  count 3,811,974  (true 3,811,932, within 539.7 with probability 0.95)
  budget spent: [0.2]
  second collect: FailedFunction("OnceFrame has been exhausted")
```

The released count landed within its error bar, as it should about 19 times in 20. On a count of 3.8 million, an error of a few hundred is small enough for almost any use. The noise that swamped a mean of a hundred numbers in the first chapter is negligible on a table this size. The context recorded one share spent. The last line shows what a `OnceFrame` is for. It hands out its release once, and a second `collect` fails, so keep the collected frame rather than the released query.

The error bar covers the noise and only the noise. It does not cover the sampling error between this sample and the population of France, nor the recoding and suppression that Eurostat applied before publication. Nor does it cover anything a filter removes before the count runs. It covers the count of responses, not of people, and the gap between 3.8 million rows and 1.7 million identifiers is the reason a later chapter counts by identifier. The comparison with the truth is possible only because the sample is public. On confidential data you decide on the summary, which costs nothing, and release once.

2.8. Summary

Table 2.2.: The calls and concepts of this chapter, what each does, and the rule of thumb that goes with it.

Call or concept	What it does	Rule of thumb
The LFS sample	Public microdata of the French Labour Force Survey, 3,811,932 responses from 1,682,306 synthetic identifiers	Already protected by statistical disclosure control, so releases can be checked against the truth
`pl.scan_parquet(path)`	A `LazyFrame` that reads rows only when a query runs	Give the context the lazy frame, not a collected table
`pl.scan_csv` and schema inference	Types inferred from the values, so they depend on the data	Read a format with a schema, or load strings with `infer_schema=False` and cast
`dp.unit_of(contributions=36)`	The privacy unit: neighbours add or remove up to 36 rows of one person	Take the bound from the design of the data, never from the data

Call or concept	What it does	Rule of thumb
`dp.loss_of(epsilon=1.0)`	The budget, under pure differential privacy	One person's data changes any output's probability by at most e^{ε}
`split_evenly_over`, `split_by_weights`	How the budget is divided; `context.d_mids` lists the shares	Weights are shares, not values of ε; the shares sum to the budget
`del frame`	Removes your own reference to the data	Anything computed outside the context is outside the guarantee
`context.query()`	A `LazyFrameQuery`: a mock that knows the schema and accepts only proven expressions	Build and summarize against the mock; `collect` works only after `release`
`dp.len(signed=True)`	The count of rows, with noise that may go below zero	Write `signed=True` out; the default is unsigned in 0.16.0 and will change
`summarize(alpha=0.05)`	Scale, distribution and error bar of each aggregate, before any release	Decide on the summary; it depends on the budget and the unit, not on the data
`release().collect()`	Spends the share and returns the released frame once	Keep the collected frame; a second `collect` fails
`dp.polars.Margin(max_length=...)`	A public bound on the length of the frame, which sums need	Build it from published figures, as 150,000 responses a quarter here

2.9. References

- OpenDP documentation: "Working with Tabular Data" (the dataset description, loading data, and mediating access with `Context`), "Polars vs. OpenDP" and "Context" in the API user guide, the count of "Essential Statistics", and the "Why OpenDP?" paragraph of the front page.
- Eurostat, *EU Labour Force Survey*, public microdata and the EU-LFS database user guide: the survey, its columns and their coding.
- Dwork, McSherry, Nissim and Smith, "Calibrating Noise to Sensitivity in Private Data Analysis" (2006): the Laplace mechanism behind the count.
- The full edition's chapters on the definition in the full edition and on post-processing, group privacy and composition in the full edition derive the results this chapter states.

Part II.

Tabular data

This part does the everyday work of a private analysis, which is statistics on a table. OpenDP does it through Polars, a dataframe library whose lazy queries OpenDP can read before they run. So every filter, cast and aggregation is accounted for in the privacy guarantee. The thread of the part is that every statistic needs two things from you. One is a bound on what one person can contribute, and the other is a share of the budget. The library turns those into a noise scale that you can read before you release anything.

The chapters follow the documentation's tutorial on tabular data. Essential statistics releases counts, sums, means, medians and quantiles. It also shows how `summarize` reports the noise scale and the error bar of a query. Grouping releases a table with one row per group, either with a public list of keys or with keys the library selects privately. Identifier truncation and bounds handles a person who owns many rows, by truncating their rows per group and bounding the groups they touch. Preparing microdata covers the expressions OpenDP allows before an aggregation, from casting and filling nulls to string and date operations, and the data types it understands. Assessing utility and choosing the noise turns scales into error bars and back. It then compares Laplace with Gaussian noise under zero-concentrated differential privacy, and shows what the error bars leave out.

Table 2.3.: The chapters of this part and the questions they answer.

Chapter	Question it answers
Essential statistics	How do I release a count, a sum, a mean or a quantile, and how large is its error?
Grouping	How do I release one statistic per group, when the groups are known and when they are not?
Identifier truncation and bounds	What do I do when one person owns many rows?
Preparing microdata	Which transformations may I apply before aggregating, and how do types and nulls behave?
Assessing utility and choosing the noise	How do I read an error bar, and when is Gaussian noise the better choice?

By the end of this part you will be able to release a table of statistics about a dataset in which each person owns many rows. Every cell will carry its error bar, and the whole table a guarantee you can state.

3. Essential statistics

Once a context mediates access to a table, the everyday questions follow. They ask how many responses there are, how many hours people work in total and on average, and what a typical week looks like. This chapter answers them on the Labour Force Survey (LFS) sample with OpenDP's Polars queries, and one idea runs through every answer. A statistic is released by adding noise, and the library works out the scale of that noise from two things you supply. One is a bound on what one person can change, and the other is a share of the budget. You read that scale and its error bar before you release anything, and only then decide whether the release is worth its share.

The chapter follows the documentation's tutorial on essential statistics and its page on aggregation. You will count rows, read a `summarize` table and decide where bounds come from. Then you will sum a column, release a mean in two ways, and choose a median and quartiles among candidates. By the end you will be able to say, for each of these statistics, what the library asked you for and what its error bar covers. The practitioner edition's workflow on releasing statistics does the same job on a retail dataset in which a customer owns a varying number of rows. Here the privacy unit is the simplest one the survey allows.

The script opendp_essential_statistics.py runs every query on OpenDP 0.16.0. It reads the sample lazily through `practitioner_datasets.france_lfs()`, as the documentation reads the same file through `dp.examples.get_france_lfs_path()`. The noise scales, the error bars, the refusals and the true values it prints are the same on every run. The released values are not, because OpenDP's noise cannot be seeded, so this page shows them in one block marked as one run.

```
uv run python code/part02-tabular-data/opendp_essential_statistics.py
```

3.1. The context for this chapter

The previous chapter chose the privacy unit for this survey. A person answers at most once per quarter over nine years, so one person owns at most 36 rows, and `dp.unit_of(contributions=36)` says so. Two neighbouring datasets are therefore two tables that differ by one person's rows, up to 36 of them, added or removed. The budget is a privacy loss ε of 1 under pure differential privacy (DP), split evenly over five queries, so each query gets $\varepsilon = 0.2$.

The context adds one thing the first chapter's did not: a **margin**, which is a set of facts about the table that you declare public. Here the only fact is `max_length`, an upper bound on the number of rows. The survey reaches about 50,000 households a quarter, with fewer than three people in each, so 150,000 responses a quarter is a safe public bound. Over 36 quarters that gives 5,400,000 rows. The sum, the mean and the quantiles below refuse to run without this bound, for a reason the section on the sum explains.

```
def make_context(frame: pl.LazyFrame, invariant: Literal["keys", "lengths"] | None = None) -
    """The chapter's context: 36 rows per person, epsilon = 1 split evenly over 5 queries, a
    return dp.Context.compositor(
        data=frame,
        privacy_unit=dp.unit_of(contributions=CONTRIBUTIONS),  # neighbours add or remove on
        privacy_loss=dp.loss_of(epsilon=EPSILON),  # pure differential privacy
        split_evenly_over=QUERIES,  # each query gets epsilon / 5
        margins=[dp.polars.Margin(max_length=MAX_LENGTH, invariant=invariant)],  # a public
    )
```

```
privacy unit: (SymmetricDistance(), 36)
privacy loss: (MaxDivergence, 1.0)
budget per query: [0.2, 0.2, 0.2, 0.2, 0.2]
margin: max_length=5,400,000
```

The chapter's five releases come from this one context: a count, a sum, a sum with a count, a median and three quartiles. A query takes the next share of the budget when it is built. So the comparisons on this page, which read scales without releasing anything, run on fresh contexts with the same settings. The documentation's tutorial does the same, though a real analysis builds one context and plans its shares before the first query.

3.2. Counting responses

The simplest statistic is the number of rows. The query selects `dp.len`, OpenDP's private version of Polars' `len`, and the context fills in the noise.

```
count_query = context.query().select(dp.len(signed=True))
count_summary = count_query.summarize(alpha=0.05)
dp_count = count_query.release().collect().item()
```

The method to call before `release` is `summarize`. It describes the measurement the context built without running it, one row per statistic.

```
shape: (1, 5)
┌────────┬──────────────┬──────────────────┬───────┬──────────────┐
│ column ┆ aggregate    ┆ distribution     ┆ scale ┆ accuracy     │
│ ---    ┆ ---          ┆ ---              ┆ ---   ┆ ---          │
│ str    ┆ str          ┆ str              ┆ f64   ┆ f64          │
╞════════╪══════════════╪══════════════════╪═══════╪══════════════╡
│ len    ┆ Frame Length ┆ Integer Laplace  ┆ 180.0 ┆ 539.731115   │
└────────┴──────────────┴──────────────────┴───────┴──────────────┘
```

The column is `len`, the aggregate is the length of the frame, and the distribution is the integer Laplace distribution, because a count is an integer. The scale is 180, and the accuracy is the error

bar at the level you passed as `alpha`. With probability 0.95, the released count is within 539.73 of the true one. The scale comes from the privacy unit. Removing a person removes up to 36 rows, so the **sensitivity** of the count, the most one person can change it, is 36. The Laplace scale is the sensitivity over the query's ε, and the full edition's chapter on the Laplace and Gaussian mechanisms derives why that scale gives ε-DP.

The error bar depends on `alpha`, not on the data. Asking for more confidence widens it, so at `alpha=0.01` the same scale gives an error bar of 829.43. Both numbers cost nothing to read.

```
count: scale=180.0 95% accuracy=539.73 99% accuracy=829.43
sensitivity 36 over epsilon 0.2: scale 180.0
```

The argument `signed=True` keeps the released count a signed integer, so a small count can come out negative when the noise outweighs it. Without the argument, the library clamps a negative count to zero and warns that this default will change.

```
without signed=True: FutureWarning: The default value is currently signed=False, and counts
```

A column has its own counts, and the documentation's aggregation page lists them. On a column, `.dp.len` counts the rows with nulls included, which equals the frame length unless an earlier expression changed the number of rows. The expression `.dp.count` counts the rows that are not null and `.dp.null_count` the rows that are, so the two add up to the length. Finally `.dp.n_unique` counts distinct values, with null as one of them. Each is as sensitive as the frame length, because one person's rows can change any of them by at most 36. So each gets the same scale of 180 when it is the only statistic in a query.

Which count you want depends on the question. The frame length counts responses, the non-null count counts those that answered the hours question, and the null count measures non-response. The distinct count tells you how many different answers appear, which is how you would notice that a column is coded rather than free. The excerpt asks for all five in one query.

```
five_counts = variants.query().select(
    dp.len(signed=True),  # rows of the frame
    hwusual.dp.len(signed=True).alias("expr_len"),  # rows of the column, nulls included
    hwusual.dp.count(signed=True).alias("count"),  # rows of the column that are not null
    hwusual.dp.null_count(signed=True).alias("null_count"),  # rows of the column that are n
    hwusual.dp.n_unique(signed=True).alias("n_unique"),  # distinct values, null among them
)
five_summary = five_counts.summarize(alpha=0.05)
```

```
HWUSUAL len alone: scale=180.0
HWUSUAL count alone: scale=180.0
HWUSUAL null_count alone: scale=180.0
HWUSUAL n_unique alone: scale=180.0
five counts in one query: scale=900.0 each, 95% accuracy=2696.7
```

Five statistics in one query share that query's $\varepsilon = 0.2$ equally, so each gets a fifth of it and five times the scale. This is the rule for every query on this page. A query's share is divided evenly among its statistics unless you set their scales yourself. Putting several counts in one query is therefore cheap in budget but not in noise. Grouping shows a better way to get the null and the non-null count together, by grouping on whether the value is null, which costs one count's noise for both.

3.3. Where bounds come from

Every statistic after the count needs to know the range of its values. A sum and a mean need a lower and an upper bound, and a median or a quantile needs a list of candidates. The bounds set the sensitivity, because one person's rows can move a sum by at most their number times the width of the bounds. That is the clipping idea, and the full edition's chapter on sensitivity derives it. Bounds that are too wide buy more noise than the data need. Bounds that are too narrow clip real values and bias the result, and the error bar says nothing about that bias.

Three sources of bounds are legitimate. The first is domain knowledge, such as a working week of at most about 80 hours. The second is a comparable public dataset, from which you read the range. The third is the budget itself. You spend a share on a quantile query with widely spaced candidates to estimate the tails, and the rest on the statistic you wanted.

The one source that is never legitimate is the private data. A bound read off the data, such as its maximum, depends on one person's value and leaks it, and nothing in the guarantee accounts for that. This page takes its bounds from domain knowledge, with hours between 0 and 80. The candidates for the median are then the whole hours from 20 to 59.

3.4. The sum of hours

The column `HWUSUAL` holds the hours a person usually works in a week. The column codes "not applicable" as 99, so the query first filters those rows out, and OpenDP accounts for the filter as part of the query. Then `.dp.sum` clips every value into the bounds, sums, and adds noise.

```
hours: Any = pl.col("HWUSUAL").cast(int)  # Any: the .dp namespace is added at run time
sum_query = (
    context.query()
    .filter(pl.col("HWUSUAL") != 99.0)  # 99 means "not applicable"
    .select(hours.fill_null(35).dp.sum(bounds=(0, 80)))  # fill, clip, sum, add noise
)
sum_summary = sum_query.summarize(alpha=0.05)
dp_sum = sum_query.release().collect().item()
```

```
sum HWUSUAL: Sum, Integer Laplace, scale=14,400.00 95% accuracy=43,139.0
sensitivity 36 x 80 = 2,880 over epsilon 0.2: scale 14,400
```

One person owns at most 36 rows of at most 80 hours each. Adding or removing those rows changes the sum by at most 2,880, which is the sensitivity, and the scale is that sensitivity over the query's ε. The error bar of 43,139 hours looks large on its own, but it is small against a total near 56 million. The noise does not grow with the number of rows, so the relative error shrinks as the table grows.

The sum needs the margin, and it says so when the margin is missing.

```
without a margin: must specify 'max_length' in a margin with by=[]
```

Why does a sum need a bound on the number of rows when the count did not? Because a sum is computed in finite arithmetic. Integers can overflow, and every addition of floats rounds, so the result of summing a column depends slightly on how many values were summed. The library accounts for that in the sensitivity, and it can only do so if it knows the largest number of rows a table can have. The `by=[]` in the message names the margin of the whole table, since no grouping is in play; Grouping adds margins per group. The refusal does not depend on the data, as a snippet on three rows shows.

```
import opendp.prelude as dp
import polars as pl

dp.enable_features("contrib")
context = dp.Context.compositor(
    data=pl.LazyFrame({"hours": [35.0, 39.0, 40.0]}),
    privacy_unit=dp.unit_of(contributions=1),
    privacy_loss=dp.loss_of(epsilon=1.0),
    split_evenly_over=1,
)
try:
    context.query().select(pl.col("hours").dp.sum(bounds=(0.0, 80.0))).summarize()
except dp.OpenDPException as err:
    print(err.message)  # must specify 'max_length' in a margin with by=[]
```

The integer cast is the second adjustment. The column is stored as floats, and a float sum pays for the rounding of every addition, up to `max_length` of them. At this chapter's bound the price is small, a scale of 14,405.18 instead of 14,400. A careless bound of a billion rows raises the float scale to 192,035.68, while the integer scale does not move. Casting to an integer removes the rounding and with it the dependence on `max_length`, so cast whenever the values are whole numbers.

```
float sum, max_length 5,400,000: HWUSUAL: Sum, Float Laplace, scale=14,405.18 95% accuracy=4
float sum, max_length 1,000,000,000: HWUSUAL: Sum, Float Laplace, scale=192,035.68 95% accur
int sum, max_length 1,000,000,000: HWUSUAL: Sum, Integer Laplace, scale=14,400.00 95% accura
```

The third adjustment is `fill_null`, and it exists because a sum has no defined value when a row is null. The library therefore insists that the column it sums has no nulls, and if you write the clip and the sum by hand, it refuses a nullable column. The shortcut `.dp.sum` instead fills a null

with the midpoint of the bounds, 40 here, before clipping. Writing `fill_null(35)` makes the choice yours, and 35 is the legal working week in France, a public fact. On this data the choice turns out not to matter, because the filter has already dropped every null. A null is not equal to 99, and Polars drops a row whose test is null, so the check at the end of the chapter finds none left. To confirm the midpoint rule on a release, the script also sums the whole column, where the nulls do remain.

```
sum without fill_null: scale=14,400.0 (accepted; a null becomes the midpoint 40)
clip, sum and laplace by hand, without fill_null: input data (HWUSUAL) might contain nulls.
```

Finally, consider what the error bar covers. The released sum is within 43,139 of the *clipped* sum with probability 0.95. A value above 80 would be pulled down to 80 before summing, and no error bar reports that. Here no applicable value exceeds the bound, as the check at the end shows, so the clipped sum is the true sum.

3.5. The mean

A mean is a sum divided by a count, and under the add/remove relation both are private quantities. The documentation's recommendation is to release the two in one query and divide afterwards. Dividing two released numbers is post-processing, which costs nothing, as the full edition's chapter on properties proves.

```
joint_query = (
    context.query()
    .filter(pl.col("HWUSUAL") != 99.0)
    .select(hours.dp.sum(bounds=(0, 80)), dp.len(signed=True))  # a null becomes the midpoin
)
joint_summary = joint_query.summarize(alpha=0.05)
joint = joint_query.release().collect().with_columns(mean=pl.col("HWUSUAL") / pl.col("len"))
```

```
joint HWUSUAL: Sum, Integer Laplace, scale=28,800.00 95% accuracy=86,277.6
joint len: Frame Length, Integer Laplace, scale=360.00 95% accuracy=1,079.0
```

The query's share is split evenly between its two statistics, so each has twice the scale it had alone: 28,800 for the sum and 360 for the count. The pair costs the same share as either did on its own, which is why the joint query is the recommended one. The sum here has no `fill_null`, so a null would become the midpoint, as the comment says. One released count can also serve several means, by dividing the sums of other columns by it. Calling `.dp.mean` on this context changes nothing: it releases the same noisy sum and noisy length, as the script's comparison shows.

```
dp.mean on the add/remove context: HWUSUAL: Sum, Integer Laplace, scale=28,800.00 95% accura
dp.mean on the add/remove context: HWUSUAL: Length, Integer Laplace, scale=360.00 95% accura
```

The alternative is to declare the number of rows public. The documentation calls this bounded DP. In the full edition's terms the neighbouring relation becomes replace-one, where two

neighbours have the same number of rows and differ in one person's values. In a margin, `invariant="lengths"` declares the number of rows an invariant, and `.dp.mean` then adds noise to the sum alone and divides by the exact length.

```
applicable_frame = frame.filter(pl.col("HWUSUAL") != 99.0)  # filtered outside the context:
bounded = make_context(applicable_frame, invariant="lengths")  # the number of rows is publi
mean_query = bounded.query().select(hours.dp.mean(bounds=(0, 80)))
mean_summary = mean_query.summarize(alpha=0.05)
dp_mean = mean_query.release().collect().item()
```

```
bounded mean HWUSUAL: Sum, Integer Laplace, scale=7,200.00 95% accuracy=21,569.8
bounded mean HWUSUAL: Length, Integer Laplace, scale=0.00 95% accuracy=none
```

The sum's scale is 7,200, half of what the sum had with a whole share under add/remove. When the length is fixed, changing one row counts as one removal and one addition. A distance of 36 under the privacy unit's metric therefore means at most 18 replaced rows, so the sensitivity halves. The length row shows a scale of zero and no accuracy, because the length is released exactly. That is the weaker guarantee you accept. The release still hides what a person answered, but not how many rows the table has, and so not whether a person is in it.

There is a second cost, visible in the excerpt. The filter on the code 99 runs outside the context, on the LazyFrame itself. Inside the context, a filter makes the number of remaining rows unknown again, so the invariant is lost. Then `.dp.mean` falls back to a noisy sum and a noisy length, which the script confirms. Filtering outside the context is only sound if the number of applicable responses is truly public information, as the documentation says. If it is not, use the joint query.

```
dp.mean after a filter inside the bounded context: HWUSUAL: Sum, Integer Laplace, scale=28,8
dp.mean after a filter inside the bounded context: HWUSUAL: Length, Integer Laplace, scale=3
```

3.6. The median and the quartiles

A median cannot be released by adding noise to it. When few values lie near the middle, one person's rows can move the median by an arbitrary amount, so no finite scale covers every dataset. OpenDP instead chooses the median among **candidates** you supply. It scores each candidate by how unbalanced the counts below and above it are, adds exponentially distributed noise to every score, and releases the candidate with the smallest noisy score. That is the exponential mechanism, which the full edition's chapter on selection derives. The candidates here are the whole hours from 20 to 59.

```
candidates = list(range(20, 60))  # whole hours from 20 to 59
median_query = context.query().filter(pl.col("HWUSUAL") != 99.0).select(hours.dp.median(cand
median_summary = median_query.summarize(alpha=0.05)
dp_median = median_query.release().collect().item()
```

```
median HWUSUAL: 0.5-Quantile, ExponentialMin, scale=360.00 95% accuracy=none
score sensitivity 36: scale 2 x 36 / 0.2 = 360
median without a margin: Must know max_length
```

The aggregate reads 0.5-quantile because `.dp.median` calls `.dp.quantile` with `alpha=0.5`. The distribution is named for the mechanism, exponential noise and a minimum. Its scale is twice the sensitivity of the score over the query's ε, with the factor of two that the exponential mechanism's analysis carries. The accuracy is empty, and it has to be. The noise went into the scores, not into the answer, and the answer is whichever candidate won, so there is no error bar in hours. A candidate whose score is far above the best one is very unlikely to win, and the check at the end of the chapter puts numbers on that. The quantile also needs the margin, as the refusal shows, because the scores are computed in integers whose size depends on the number of rows.

A quantile other than the median takes `alpha`, the share of values that lie below it. The excerpt asks for the three quartiles in one query.

```
quartile_query = (
    context.query()
    .filter(pl.col("HWUSUAL") != 99.0)
    .select(hours.dp.quantile(a, candidates).alias(f"{a}-Quantile") for a in [0.25, 0.5, 0.7
)
quartile_summary = quartile_query.summarize()
dp_quartiles = quartile_query.release().collect().row(0)
```

```
quartiles 0.25-Quantile: 0.25-Quantile, ExponentialMin, scale=3,240.00 95% accuracy=none
quartiles 0.5-Quantile: 0.5-Quantile, ExponentialMin, scale=1,080.00 95% accuracy=none
quartiles 0.75-Quantile: 0.75-Quantile, ExponentialMin, scale=3,240.00 95% accuracy=none
0.25-quantile alone: scale=1,080.0
```

Three quantiles in one query split its share three ways, so the median's scale triples from 360 to 1,080. The quartiles' scales are three times larger again, which looks as if the median got more of the budget, and the documentation says it did not. The reason is internal to the score. A quantile's score is counted in integers scaled by the denominator of `alpha`, which is four for a quartile and two for the median. So a quartile's score moves three times as far when one person's rows change, and its noise scales with it. The last line shows the even split directly. The first quartile on its own gets a scale of 1,080, a third of what it has inside the trio, so each of the three had the same ε.

3.7. A check against the truth

The sample is public, so the script computes the true values at development time and reads one run's releases against them. On private data you cannot do this, which is why you do it on public or historical data while you develop, and release the real data once.

```
true responses: 3,811,932
true responses with HWUSUAL not 99: 1,496,192 (nulls among them: 0; in the whole column: 20,
true sum of hours after the filter, clipped to [0, 80]: 56,315,594 (values above 80: 0)
true sum of the whole column, clipped: 240,742,034 with nulls as 40, 239,909,274 with nulls
true mean: 37.64  (bounded mean 95% accuracy: 0.0144)
true quartiles: 35, 37, 40
score gap from the true quartile to the nearest other candidate: 0.25: 219,506  0.5: 74,383
```

Two of these lines settle questions left open above. The filter leaves no nulls among the applicable responses, so the imputed value never entered the sum, and no applicable value exceeds 80, so clipping changed nothing. The bounded mean's error bar is the sum's error bar divided by the exact length, which gives 0.0144 hours. The last line is the quantiles' substitute for an error bar. The nearest competing candidate scores at least 74,383 above the true quartile, against a noise scale of at most 3,240. A wrong candidate is therefore far too unlikely to see in any number of runs. The test therefore requires the exact quartiles.

```
released (one run):
  responses 3,811,961  (true 3,811,932, within 539.7 with probability 0.95)
  sum of hours 56,308,788  (true 56,315,594, within 43,139.0)
  joint: sum 56,335,491  count 1,495,681  mean 37.67  (true 37.64)
  bounded mean 37.6411  (true 37.6393, within 0.0144)
  median 37  quartiles 35, 37, 40  (true 37; 35, 37, 40)
  sum of the whole column 240,726,542  (true with nulls as 40: 240,742,034)
```

The count and the sum landed inside their error bars, as they should about 19 times in 20. The joint mean is off in the second decimal, because it divides two noisy numbers, each with twice the scale of a lone release. The bounded mean is far closer, inside a bar of about a hundredth of an hour, which is what declaring the length public buys. The median and the quartiles came out exactly as the score gaps predicted. The sum of the whole column landed inside its bar of the total that counts each null as 40 hours, and more than 800,000 hours from the total that drops them, which confirms what `.dp.sum` does with a null.

3.8. Summary

Table 3.1.: The calls of this chapter, what each does, and the rule of thumb that goes with it.

Call or concept	What it does	Rule of thumb
`dp.unit_of(contributions=36)`	The privacy unit: neighbours differ by one person's rows, up to 36	Count the most rows one person can own, not the typical number
`dp.polars.Margin(max_length=A, ...)`	A public bound on the number of rows	Sums, means and quantiles refuse to run without it; take it from public facts

Call or concept	What it does	Rule of thumb
`dp.len(signed=True)`	The number of rows, with Laplace noise of scale sensitivity over ε	Without `signed=True` a negative count is clamped to zero
`.dp.len`, `.dp.count`, `.dp.null_count`, `.dp.n_unique`	A column's rows, non-nulls, nulls and distinct values	Each is as sensitive as the frame length; several in one query share its ε
`query.summarize(alpha=0.05)`	The column, aggregate, distribution, scale and error bar of each statistic	Read it before `release`; a smaller `alpha` widens the bar
`.dp.sum(bounds=(0, 80))`	Fills nulls with the midpoint, clips, sums, adds noise	Cast to an integer; the error bar excludes the bias of clipping
Sum and `dp.len` in one query	A mean under add/remove, by dividing afterwards	Each statistic gets twice the scale; the division is free post-processing
`invariant="lengths"` and `.dp.mean`	A bounded-DP mean: noise on the sum, the exact length	Weaker guarantee; filter outside the context only if the row count is truly public
`.dp.median(candidates)`, `.dp.quantile(alpha, candidates)`	The candidate nearest the quantile, by the exponential mechanism	No error bar; candidates from domain knowledge; several quantiles split the share evenly

3.9. References

- OpenDP documentation: "Essential Statistics" in the tutorial on tabular data, and "Aggregation" in the Polars user guide.
- Dwork, McSherry, Nissim and Smith, "Calibrating Noise to Sensitivity in Private Data Analysis" (2006): the Laplace mechanism behind the counts and sums.
- McSherry and Talwar, "Mechanism Design via Differential Privacy" (2007): the exponential mechanism behind the median and the quantiles.
- The full edition's chapters on the Laplace and Gaussian mechanisms in the full edition, post-processing and composition in the full edition, selection in the full edition and sensitivity in the full edition derive the results this chapter states.

4. Grouping

Most private releases are tables with one row per group, such as counts by age and employment status or the mean hours worked by sex. Grouping adds a danger that a single count does not have. The keys of the table are data too, because a group that one person fills alone announces that person just by appearing in the table. This chapter is about the three ways OpenDP lets you release a grouped table, and what each one costs.

You will release counts by age and labour status with keys the library selects, so that only the groups whose noisy count clears a threshold appear. You will read that threshold and the δ it costs. Then you will release the same kind of table with a public key list, and with margins that declare the keys or the group sizes public. By the end you will be able to choose between the three, say what each gives up, and read a grouped release against its error bars.

The script opendp_grouping.py runs every query of this chapter on OpenDP 0.16.0 against the full survey sample. The scales, error bars, thresholds and error messages it prints are the same on every run. So are the true group counts, which this public dataset lets the script compute. The released counts and means are not, because OpenDP's noise cannot be seeded, so the page shows them in one block marked as one run.

```
uv run python code/part02-tabular-data/opendp_grouping.py
```

4.1. Why grouping keys are dangerous

The dataset is the France Labour Force Survey (LFS) sample of the OpenDP tutorials, which Essential statistics introduced. It holds 3,811,932 quarterly responses from 1,682,306 people. A person answers at most once per quarter over nine years, so at most 36 times, and the privacy unit of the whole chapter is `dp.unit_of(contributions=36)`. Two datasets are neighbours when one is the other with all of one person's rows added or removed.

Grouping the responses by `AGE` and by `ILOSTAT`, the labour status code, gives 16 groups in truth. The largest, the children too young to work, holds 688,977 rows. The smallest, the responses with no recorded age, holds 255. A noisy count of that small group would come with an error bar of a few hundred, which hides how many rows it has but not that it exists. Now imagine a finer grouping, say by age, sex and town, in which one person fills a group alone. The noise hides their count, but the row's presence in the table says that someone with those keys answered the survey. So the keys of a grouped table need protection as much as the counts do.

OpenDP enforces this. Ask a context with a pure budget for a count by `AGE` and `ILOSTAT`, and `summarize` refuses before anything is computed.

```
without delta: The key-set of {col("AGE"), col("ILOSTAT")} is private and cannot be released
```

The message names the two ways out, and the next two sections take them in turn. A filter keeps only the groups whose noisy count clears a threshold, which needs a δ in the budget. A join against a public key list releases only keys you already knew. The snippet below shows both on a toy frame, so you can see the shape of each summary without the dataset.

```
import opendp.prelude as dp
import polars as pl

dp.enable_features("contrib")
data = pl.LazyFrame({"region": ["A", "A", "B"], "hours": [35, 40, 20]})
unit = dp.unit_of(contributions=1)
pure = dp.Context.compositor(data=data, privacy_unit=unit, privacy_loss=dp.loss_of(epsilon=1
counts = pure.query().group_by("region").agg(dp.len(signed=True))
try:
    counts.summarize()
except dp.OpenDPException as err:
    print("cannot be released without a filter or join" in str(err))  # True
loss = dp.loss_of(epsilon=1.0, delta=1e-6)
approx = dp.Context.compositor(data=data, privacy_unit=unit, privacy_loss=loss, split_evenly
summary = approx.query().group_by("region").agg(dp.len(signed=True)).summarize(alpha=0.05)
print(summary.columns)  # ['column', 'aggregate', 'distribution', 'scale', 'accuracy', 'thre
print(int(summary["threshold"].item()))  # 14
keyed = counts.with_keys(pl.LazyFrame({"region": ["A", "B", "C"]}))
print(keyed.summarize().columns)  # ['column', 'aggregate', 'distribution', 'scale']
```

4.2. Stable keys

The first way is to let the library decide which groups to show. The documentation calls the keys it keeps **stable keys**, because a group is stable when it would still be there after any one person was removed from the data. The release keeps a group only when its noisy count exceeds a threshold. The threshold is set so that a group one person fills alone is released with probability at most δ. That slack is why the budget needs a δ, and the chapter uses the documentation's values, `dp.loss_of(epsilon=0.25, delta=1e-7)`. This is **partition selection**, and the full edition's chapter on sensitivity derives why a thresholded count satisfies approximate differential privacy and where its δ comes from.

```
context = dp.Context.compositor(
    data=frame,
    privacy_unit=dp.unit_of(contributions=CONTRIBUTIONS),  # a person answers in at most 36
    privacy_loss=dp.loss_of(epsilon=EPSILON, delta=DELTA),  # the delta pays for the thresho
    split_evenly_over=1,
)
stable = context.query().group_by("AGE", "ILOSTAT").agg(dp.len(signed=True))
```

```
stable_summary = stable.summarize(alpha=0.05)
stable_hat = stable.release().collect()
```

```
stable keys: privacy loss (Approximate(MaxDivergence), (0.25, 1e-07))
stable keys len: Frame Length, Integer Laplace, scale=144.00 95% accuracy=431.9 threshold=27
```

Read the summary before releasing. The scale is 144, for the same reason as in the ungrouped count of the previous chapter. One person's 36 rows can all sit in one group, so the count of any group has sensitivity 36. That sensitivity over $\varepsilon = 0.25$ is the scale. The accuracy says that with probability 0.95 a released count is within 431.9 of the truth. The new column is the threshold, 2773, and a group whose noisy count does not exceed it is dropped from the release.

The threshold is calibrated to the worst case for one person. Their 36 rows could all land in one otherwise empty group, whose true count would then be 36. That group is released only if the noise lifts it above the threshold, which for discrete Laplace noise of scale 144 has probability 2.77×10^{-9}. But the same person's rows could instead spread over 36 groups with one row each, and any of them might slip through. So the library adds up the chances over 36 groups. That gives 9.98×10^{-8}, just within $\delta = 10^{-7}$. One step lower, the total would exceed δ. The script computes both tails, so the threshold you see is the smallest one that keeps the promise.

```
one person's 36 rows clear 2773 with probability 2.77e-09; over 36 groups 9.98e-08 <= delta
at threshold 2772: over 36 groups 1.01e-07 > delta
```

The threshold moves with the scale and with δ. A looser slack of $\delta = 10^{-5}$ lowers it to 2110 at the same scale. A budget of $\varepsilon = 1$ shrinks the scale to 36. The threshold falls to 720 with it. A larger privacy unit raises both, because a person who owns more rows can build a bigger group alone.

```
threshold with delta=1e-05: scale=144.00 threshold=2110
threshold with epsilon=1: scale=36.00 threshold=720
```

On this public data the script can also count what the threshold will do. Of the 16 groups, 14 have a true count above 2773, and the two below it are the groups with 255 and with 847 rows. So the release should come back with 14 rows, and it did in the run shown at the end of the chapter. A dropped group does not announce itself. The release has no row saying that something was removed, and the number of rows you get back is itself noisy information. If you see fewer groups than you expected, do not look at the data to find out which are missing, because that look is outside the guarantee.

4.3. Explicit keys

If you know the keys in advance, you can give them to the library and keep a pure budget. **Explicit keys** are a key list that you supply, and every key in it is released, whether or not the data holds rows for it. The list must be public, which means fixed without looking at the data, because a key list read off the data is a release in itself. Here the list is every survey year crossed with every status code of the codebook, 54 keys in all. It includes the conscripts and the "other" category, which never occur in this sample.

```
explicit = (
    context.query()
    .group_by("YEAR", "ILOSTAT")
    .agg(dp.len(signed=True))
    .with_keys(public_keys())  # the years crossed with the codebook's codes, fixed without
)
explicit_summary = explicit.summarize(alpha=0.05)
explicit_hat = explicit.release().collect()
```

```
explicit keys: 54 keys, 36 with rows in the data, 18 without
explicit keys len: Frame Length, Integer Laplace, scale=144.00 95% accuracy=431.9
```

The scale and the accuracy are unchanged, and the threshold column is gone, since no group is dropped. Of the 54 keys, 36 have rows in the data. The other 18 come back as noise around zero, which can be negative because the counts are signed. A reader of the table cannot tell an empty group from a small one, and that is the point. The documentation reuses the keys of a stable release as the explicit keys of a later query, which is allowed because a released key set is public.

`with_keys` stands for a join. The library rewrites the query as a right join of the aggregation onto the key list, and you can write that join yourself with the same summary.

```
right join len: Frame Length, Integer Laplace, scale=144.00 95% accuracy=431.9
```

Only a left or a right join against a key list releases a key set. An inner join would keep just the keys that have rows, which reveals which keys the data holds, so the library refuses it.

```
inner join: only left or right joins can be used to release key-sets
```

4.4. Invariant keys

The third way is to declare the keys of a grouping public through a **margin**, which is OpenDP's description of what is known about a grouped table, as in the margins of a printed table. The margin `dp.polars.Margin(by=["YEAR", "QUARTER"], invariant="keys")` says that the set of quarters in the data is public. The library then releases every group's key in the clear, with no threshold and no δ.

```
context = dp.Context.compositor(
    data=frame,
    privacy_unit=dp.unit_of(contributions=CONTRIBUTIONS),
    privacy_loss=dp.loss_of(epsilon=EPSILON),
    split_evenly_over=1,
    margins=[dp.polars.Margin(by=["YEAR", "QUARTER"], invariant="keys")],  # the quarters ar
)
quarterly = context.query().group_by("YEAR", "QUARTER").agg(dp.len(signed=True))
quarterly_summary = quarterly.summarize(alpha=0.05)
quarterly_hat = quarterly.release().collect()
```

```
invariant keys len: Frame Length, Integer Laplace, scale=144.00 95% accuracy=431.9
true quarterly counts: 36 quarters, smallest 82,504 (2006 Q3), largest 131,656 (2011 Q4)
error bar as a share of the smallest quarter: 0.52%, of the largest: 0.33%
```

An **invariant** is a fact about the data that the guarantee no longer covers, and the documentation calls the resulting guarantee weaker for that reason. With invariant keys, the presence and the absence of each key are released exactly, so a quarter with a single respondent would appear in the table and point at them. Here nothing is lost, because the quarters are the survey's design and were known before anyone answered. The documentation adds that invariants have been used in high-profile releases, by the United States Census Bureau among others, and that you should still be reluctant to use them. The error bar is the same as before, but the quarters are smaller than the age groups, so it is a larger share of each count, about half a percent of the smallest quarter.

4.4.1. Invariant lengths

A margin can go one step further and declare the group sizes public, with `invariant="lengths"`. That implies the keys are public too, so it gives up more. The documentation's example is the mean of `HWUSUAL`, the hours usually worked, by sex. The code 99 means not applicable and would bias the mean, so those rows must go first. But a filter inside the query changes the group sizes and voids the margin, and the library then treats the keys as private again.

```
filter inside the query: The key-set of {col("SEX")} is private and cannot be released witho
```

The way around it is to filter the LazyFrame before the context is built, so that the margin describes the filtered data. That leaves 1,496,192 rows.

```
context = dp.Context.compositor(
    data=working,
    privacy_unit=dp.unit_of(contributions=CONTRIBUTIONS),
    privacy_loss=dp.loss_of(epsilon=MEAN_EPSILON),
    split_evenly_over=1,
    margins=[
        dp.polars.Margin(
            by=["SEX"],
            invariant="lengths",  # the number of responses per sex is public, keys included
            max_length=MAX_LENGTH,  # needed by any sum: a public bound on the rows in a gro
            max_groups=3,  # male, female or null
        )
    ],
)
mean_hours = context.query().group_by("SEX").agg(hours().dp.mean(HOURS_BOUNDS))
mean_summary = mean_hours.summarize(alpha=0.05)
mean_hat = mean_hours.release().collect()
```

```
invariant lengths HWUSUAL: Sum, Integer Laplace, scale=1,764.00 95% accuracy=5,285.0
invariant lengths HWUSUAL: Length, Integer Laplace, scale=0.00 95% accuracy=none
true lengths: SEX=1 779,714  SEX=2 716,478
true mean hours: SEX=1 40.778  SEX=2 34.224
mean 95% accuracy: SEX=1 0.0068  SEX=2 0.0074
```

The mean is released as a sum divided by a length, and the summary has a row for each. The length has scale 0, because it is public and used exactly. The sum has scale 1764 at $\varepsilon = 1$, which is its sensitivity. Since the group sizes cannot change, one person's rows can only be swapped for other rows. That is at most 18 swaps, and each swap moves the sum by at most 98 hours. The documentation's text quotes 7056 for this scale, but its own table and the installed version give 1764.

An error bar of 5285 hours on a sum looks large, but the sum is divided by a public length of several hundred thousand. On the mean the error bar is below a hundredth of an hour for either sex, as the script computes from the true lengths. The released means are floats in version 0.16.0, where the documentation shows them rounded to whole hours.

4.5. Margins as public descriptors

A margin describes one grouping, named by its `by` columns, and a context takes a list of margins. So one context can protect most groupings fully while relaxing a few, which is how a real analysis uses them. Besides `invariant`, a margin carries two bounds. The first, `max_groups`, is a public upper bound on the number of groups the grouping can have. It lowers the threshold of a stable-keys release, because a person's rows can then create at most that many groups and the δ is spread over fewer of them. With a bound of 16 the threshold falls from 2773 to 2656.

```
context = dp.Context.compositor(
    data=frame,
    privacy_unit=dp.unit_of(contributions=CONTRIBUTIONS),
    privacy_loss=dp.loss_of(epsilon=EPSILON, delta=DELTA),
    split_evenly_over=1,
    margins=[dp.polars.Margin(by=["AGE", "ILOSTAT"], max_groups=GROUPS_IN_TRUTH)],  # at mos
)
bounded_summary = context.query().group_by("AGE", "ILOSTAT").agg(dp.len(signed=True)).summar
```

```
max_groups=16: threshold=2656
```

The second, `max_length`, is a public upper bound on the number of rows in any one group of the grouping. A sum needs it, because the library's guarantee covers overflow and a sum of unbounded length could overflow. A count needs neither bound. Ask for a sum of hours per sex with a margin that lacks it, and the query is refused.

```
sum per SEX without max_length: must specify 'max_length' in a margin with by=[col("SEX")]
```

Both bounds are promises about the data that the guarantee does not check. The bound of 16 above was read off the public data at development time, which is fine for a tutorial and wrong for a release. Set them from the design of the survey, never from the private data, and set them conservatively, because a bound that the data violates voids the guarantee without any error.

4.6. The noise per group and the ungrouped count

Every count in this chapter had the same scale, 144, whether it was one group among 16, one quarter among 36 or the ungrouped count of all rows.

```
ungrouped count len: Frame Length, Integer Laplace, scale=144.00 95% accuracy=431.9
error bar as a share of the true count: all rows 0.011%, largest group 0.063%, smallest grou
```

The reason is the privacy unit. All 36 rows of a person can sit in one group, so a group's count is as sensitive as the total, and grouping does not spread the noise thinner. What changes is the relative error. The same error bar of 431.9 is a hundredth of a percent of all rows. It is 3.3% of the smallest group that clears the threshold. Finer groupings therefore cost utility twice, since more groups fall below the threshold and the survivors carry a larger relative error.

The documentation's advice follows from this. Group as finely as the question needs and no finer, and ask for several statistics in one query rather than releasing the same keys again and again. Assessing utility and choosing the noise returns to this grouping with Gaussian noise, whose scale depends on how a person's rows spread over the groups. Identifier truncation and bounds shows how to bound that spread.

4.7. Checking the release against the truth

Because the LFS sample is public, the script compares each release with the true table, which a real release never can. This is the check to run on public or historical data while you develop. For the stable keys it answers two questions, how many groups should survive and how close to the threshold the dropped ones were. The smallest group above the threshold has 13,157 rows, and the largest below it has 847. So no group sits within reach of the threshold, and the release should return 14 groups on every run. A group within an error bar or two of the threshold would appear on some runs and not on others, and the script would print it here.

```
smallest true count above the threshold: 13,157  largest below it: 847
test bound for a table: 4 x accuracy = 1,727.5
released (one run):
  stable groups released: 14  smallest released count: 13,321  smallest true count among the
  stable AGE=47 ILOSTAT=1: 652,892  (true 652,801)
  stable AGE=65 ILOSTAT=2: 13,321  (true 13,157)
  stable largest |released - true|: 349
  explicit YEAR=2005 ILOSTAT=4: 312  (true 0)
  explicit YEAR=2013 ILOSTAT=1: 188,396  (true 188,349)
  explicit keys released: 54  largest |released - true|: 530
```

```
  quarter 2005 Q1: 87,307  (true 87,372)
  quarters released: 36  within their 95% accuracy: 36 of 36  largest |released - true|: 307
  mean hours: SEX=1 40.786 (true 40.778)  SEX=2 34.229 (true 34.224)
```

Read the run against the error bar of 431.9. The stable release has its 14 groups, and every one of them landed within the bar. The explicit release has all 54 keys, including the conscripts of 2005, whose true count is zero and whose released count is a noise draw. Its largest error is a little over the bar, which is what a table of 54 counts should show, since each count misses a 95% bar about once in twenty. All 36 quarters landed within theirs. The means are within a hundredth of an hour of the truth, as their error bars said.

The test checks the deterministic lines exactly. For the stable release it checks three things. It must have 14 groups, each must exceed the threshold, and each must have a true count above it. Every named count must be within three error bars of its truth. For a whole table the test allows the largest error four bars, the 1727.5 the script prints, because a table has up to 54 counts and the chance that any of them misses four bars is small enough. Each mean must be within three of its own bar. The practitioner edition's workflow Release statistics with a library in the practitioner edition runs the same kind of check on a shop's customers. There the keys are products and the privacy unit is a customer with many lines.

4.8. Summary

Table 4.1.: The calls and concepts of a grouped release, what each does, and the rule of thumb that goes with it.

Call or concept	What it does	Rule of thumb
`group_by(...).agg(dp.len(public_keys=True))` without keys or margins	Refuses, the key set is private	Choose stable keys, explicit keys or a margin before you query
Stable keys, `dp.loss_of(epsilon, delta)`	Releases only groups whose noisy count exceeds a threshold	The threshold costs δ; read it in `summarize`
The `threshold` column	The smallest value such that a one-person group clears it with probability at most δ, over every group that person could create	Grows with the scale, with the privacy unit and with a smaller δ
`with_keys(keys)`	Joins a public key list; every key is released, zeros as noise	The list is fixed without the data; a released key set counts as public
`join(keys, how="right")`	The join that `with_keys` stands for	Only left or right joins release a key set
`Margin(by, invariant="keys")`	The keys of that grouping are released in the clear	Presence and absence of a key are unprotected; use for a design the public knows

Call or concept	What it does	Rule of thumb
`Margin(by, invariant="lengths")`	The group sizes are public, keys included	Gives up more than keys; filter before the context, not inside the query
`max_groups`, `max_length`	Public bounds on the number of groups and the rows per group	`max_groups` lowers the threshold; a sum needs `max_length`; set both from the design
One scale for every group	A person's rows can all sit in one group	Finer groups keep the noise and lose relative accuracy

4.9. References

- OpenDP documentation: "Grouping" in the tabular-data tutorial, and "Aggregation" and "Polars" in the API user guide.
- Korolova, Kenthapadi, Mishra and Ntoulas, "Releasing Search Queries and Clicks Privately" (2009): thresholded noisy counts as a way to release keys privately.
- Desfontaines, Voss, Gipson and Mandayam, "Differentially Private Partition Selection" (2022): the threshold, its δ, and why one person's groups add up.
- The full edition's chapters on sensitivity in the full edition, whose stability-based release is the thresholded count, and on post-processing and composition in the full edition, which is why a released key set can be reused for free.

5. Identifier truncation and bounds

A person in the survey answers in several quarters, so one person owns several rows. Grouping handled that with a privacy unit of 36 contributions, the most quarters anyone could appear in over nine years, and every noise scale grew 36-fold with it. Hardly anyone appears that often. This chapter replaces the fixed number with the column that says which rows belong together, `PIDENT`, and lets each query decide how many of a person's rows it keeps. The guarantee then protects a person's whole record, and the noise is calibrated to what the query actually lets one person contribute.

The chapter follows the documentation's page on identifier truncation and its user guide on truncation and bounds. You will declare a privacy unit with an identifier, then truncate rows per person and groups per person inside a query. You will then aggregate to one row per person before a statistic, and declare finer bounds in the unit when you must. By the end you will know which of these the library enforces and which it takes on trust, because each call is run in version 0.16.0 end to end.

The script opendp_identifier_truncation.py runs every query on the Labour Force Survey (LFS) sample with OpenDP 0.16.0. The scales, error bars, thresholds, error messages and facts about the data that it prints are the same on every run. The released values are not, because OpenDP's noise cannot be seeded, so the page shows them in one block marked as one run.

```
uv run python code/part02-tabular-data/opendp_identifier_truncation.py
```

5.1. A privacy unit with an identifier

The privacy unit says what the guarantee protects. In the chapters before this one it was a number of rows, and two datasets were neighbours when one was the other with that many rows added or removed. With an identifier, the unit is a value of a column instead. Two datasets are neighbours, written $D \sim D'$, when one is the other with every row of one `PIDENT` value added or removed. That is **user-level privacy**, the guarantee that hides a person's whole record rather than one row of it, which the full edition's chapter on user-level privacy defines and analyses. OpenDP writes the unit as a metric on identifiers and a distance of one.

```
import opendp.prelude as dp

dp.enable_features("contrib")
print(dp.unit_of(contributions=1, identifier="PIDENT"))  # (SymmetricIdDistance { identifier
```

The argument `contributions=1` now counts identifiers, not rows. It says that a person appears under at most one value of `PIDENT`, which is a fact about how the survey assigns its identifiers, so

it is public. The identifier may also be a Polars expression, as long as it is computed row by row. Nothing in the unit says how many rows a person owns, and that is the point. The old number 36 was a bound that had to hold for the whole dataset before any query ran. Under the new unit, each query states its own bound as a truncation that the library applies to the data, so the bound holds because the query made it hold.

The script first looks at what an identifier owns in this file. The data are public, so the look is harmless here. On real data these are exactly the facts you must not read off the data, because a bound read off the data leaks it. There you reason from the survey's design instead.

```
identifiers: 1,682,306  rows: 3,811,932
rows per identifier: p50=2 p90=4 p99=7 max=15
identifiers with more than 10 rows: 195
quarters per identifier: max 4  rows per identifier and quarter: max 8
```

Half of the identifiers own two rows or fewer, and none owns more than 15. An identifier spans at most four quarters, and within one quarter it can own several rows, up to eight. So the documentation's worst case of 36 rows is far from this file. Even the truncation at ten rows that the next section applies touches only 195 identifiers.

The context is the one the documentation builds, with the public bound on the number of rows from Essential statistics. It takes a privacy budget ε of 1 and a slack δ of 10^{-8}, split evenly over four queries. The slack is the small probability with which the guarantee may fail. The table per quarter later in the chapter needs it, because the library decides from the data which quarters to publish.

```
def make_context(frame: pl.LazyFrame, privacy_unit: tuple[dp.Metric, Any]) -> dp.Context:
    """The documentation's context: approximate differential privacy, split evenly over four
    return dp.Context.compositor(
        data=frame,
        privacy_unit=privacy_unit,  # dp.unit_of(contributions=1, identifier="PIDENT") below
        privacy_loss=dp.loss_of(epsilon=EPSILON, delta=DELTA),
        split_evenly_over=QUERIES,
        margins=[dp.polars.Margin(max_length=MAX_LENGTH)],
    )
```

```
budget per query: epsilon=0.25 delta=2.5e-09
```

A query on this context that does not truncate is refused. The library cannot bound how far one identifier's rows move a statistic, so it cannot calibrate the noise. The error it raises in version 0.16.0 is not the one you would hope for.

```
try:
    context.query().filter(pl.col("HWUSUAL") != 99).select(hours().dp.mean(HOURS)).summarize
    untruncated = "accepted"
except dp.OpenDPException as err:
    untruncated = reason(err)
```

```
mean without truncation: A step in your query is not recognized at this time: "DF ["PIDENT",
```

The message blames the scan of the data rather than the missing truncation. When you see it on a context with an identifier, the first thing to check is whether every statistic sits behind a truncation.

5.2. Truncating each person's rows

The documentation's first query is the mean of hours usually worked, `HWUSUAL`, over the rows where the value is not 99, which is the survey's code for "not applicable". The mean needs public bounds on the hours, 0 to 80, and a bound on the rows per person, which `truncate_per_group(10)` supplies. It keeps at most ten rows per identifier, chosen at random, and drops the rest before the aggregation sees them.

```
mean_query = (
    context.query()
    .filter(pl.col("HWUSUAL") != 99)  # 99 means "not applicable"
    .truncate_per_group(ROWS_PER_IDENTIFIER)  # at most 10 rows per identifier, chosen at ra
    .select(hours().dp.mean(HOURS))
)
mean_summary = mean_query.summarize(alpha=0.05)
dp_mean = mean_query.release().collect().item()
```

```
mean, 10 rows per identifier: HWUSUAL Sum, Integer Laplace, scale=6,400.0 95% accuracy=19,17
mean, 10 rows per identifier: HWUSUAL Length, Integer Laplace, scale=80.0 95% accuracy=240.2
```

A mean is released as a sum and a count, as Essential statistics showed, and the query's share of the budget is split between the two, so each gets $\varepsilon/8$. One person now moves the count by at most ten rows, so the count's sensitivity is 10. Its Laplace scale is that sensitivity over $\varepsilon/8$, which is 80. Each row adds at most 80 hours to the sum, so the sum's sensitivity is ten times that. Its scale is 6,400. The sensitivity is the most one privacy unit can change a statistic, and truncation and clipping are the two ways of forcing it to be finite. The full edition's chapter on sensitivity derives what each of them does to the answer.

Compare the unit of 36 contributions, which the script also builds. Under it the same mean gets scales of 23,040 and 288. That is the 36-fold increase the opening mentioned, and the truncation replaces it with a tenfold one. The price is bias, because the rows beyond the tenth are dropped and the mean is computed on what remains. On this file the price is nothing. Only 18 identifiers have more than ten rows left after the filter. The truncation drops 20 of their rows, out of about 1.5 million.

```
rows with HWUSUAL not 99: 1,496,192 by 809,710 identifiers
identifiers with more than 10 such rows: 18  rows dropped: 20
unit of 36 contributions, no identifier: scales 23,040.0, 288.0
```

The `keep` argument says which rows survive. The default, `"sample"`, keeps a random ten, and the plan the library builds shows how. It numbers each identifier's rows in a shuffled order and keeps the numbers below ten. The documentation notes that `truncate_per_group(10)` is the same as writing that filter yourself. With `"first"` or `"last"` the numbering follows the file's order, which is cheaper but biased whenever that order means something, such as early or late quarters. A `SortBy` keeps the rows that sort first by an expression of your choice. The documentation sorts by `ILOSTAT`, the employment status, to prefer the rows where the person worked for pay. None of the choices changes the scales, because the bound of ten rows holds whichever rows are kept.

```
truncate_per_group(10) plan: FILTER [(0.int_range([len().cast(Int64)]).shuffle().over([col("
keep=first: scales 6,400.0, 80.0
keep=last: scales 6,400.0, 80.0
keep=SortBy(ILOSTAT): scales 6,400.0, 80.0
ILOSTAT values among them: [1]
```

The last line is a small lesson about this dataset. After the filter on `HWUSUAL`, every remaining row has `ILOSTAT` equal to 1, so the sort by employment status changes nothing here. It would matter on a query that keeps the other statuses.

5.3. Truncating the groups a person touches

The second query is a table with one row per quarter, grouped by `QUARTER` and `YEAR`, with the number of responses and the sum of their hours. A person's rows now spread over groups, and the noise on a grouped table depends on two bounds. One is how many rows a person contributes to any one group, and the other is how many groups a person touches. The vector of counts can change by at most the first bound in each of at most the second number of groups. So its total sensitivity is the product of the two. The full edition's chapter on user-level privacy derives this in its section on contribution bounding in the full edition.

So the query truncates twice. `truncate_per_group(1, by=quarterly)` keeps one row per identifier and quarter, and `truncate_num_groups(10, by=quarterly)` keeps at most ten quarters per identifier. Without the second truncation the query is refused, and this time the message says what is missing. It goes on to suggest the call that supplies it, `truncate_num_groups` with the grouping columns. The script leaves that part out of the printed line, because the order of the columns in the suggestion changes from run to run.

```
per quarter without truncate_num_groups: num_groups (None), total contributions (None), and
```

The query below is the documentation's, with one change. It passes `keep="first"` to the group truncation, for a reason the rest of this section explains.

```
quarter_query = (
    context.query()
    .filter(pl.col("HWUSUAL") != 99)
    .truncate_per_group(1, by=QUARTERLY)  # one row per identifier and quarter
    .truncate_num_groups(GROUPS_PER_IDENTIFIER, by=QUARTERLY, keep="first")  # at most 10 qu
```

```
    .group_by(QUARTERLY)
    .agg(dp.len(signed=True), hours().dp.sum(HOURS))
)
quarter_summary = quarter_query.summarize(alpha=0.05)
quarters_hat = quarter_query.release().collect()
```

```
per quarter, 10 quarters per identifier: len Frame Length, Integer Laplace, scale=80.0 95% a
per quarter, 10 quarters per identifier: HWUSUAL Sum, Integer Laplace, scale=6,400.0 95% acc
```

The scales are the ones of the mean, because one row in each of ten quarters is again ten rows per person. The table also has a threshold. The list of quarters is not given as public keys, so the library releases a quarter only if its noisy count exceeds 1,714. That is the private selection of keys that Grouping explained. Every quarter here has at least 27,228 responses, so all 36 appear.

Now the two things to know about `truncate_num_groups` in version 0.16.0. The first concerns the default, `keep="sample"`. The query passes `summarize`, and `release` spends the query's share of the budget, but `collect` fails inside Polars.

```
sample_query = grouped_query(context, keep="sample")  # the same table, with the default kee
sample_summary = sample_query.summarize(alpha=0.05)
released = sample_query.release()  # spends the query's share of the budget
try:
    released.collect()
    sample_outcome = "collected"
except dp.OpenDPException as err:
    sample_outcome = reason(err)
```

```
keep=sample: summarize scales 80.0, 6,400.0; collect: multiple fields with name 'QUARTER' fo
shares spent after the failed collect: 4 of 4
```

The share is gone, because the budget is spent when the measurement is built, not when its output is read. You can catch the failure before spending anything, because the fault is in the Polars plan, and Polars checks a plan without touching data. Call `explain()` on the query, as the snippet below does on a toy frame with one identifier and three quarters, and the same error appears at no cost.

```
import opendp.prelude as dp
import polars as pl

dp.enable_features("contrib")
toy = pl.LazyFrame({"PIDENT": [7, 7, 7], "YEAR": [2005] * 3, "QUARTER": [1, 2, 3], "HWUSUAL"
context = dp.Context.compositor(
    data=toy,
    privacy_unit=dp.unit_of(contributions=1, identifier="PIDENT"),
    privacy_loss=dp.loss_of(epsilon=1.0, delta=1e-8),
    split_evenly_over=1,
```

```
    margins=[dp.polars.Margin(max_length=100)],
)
quarterly = [pl.col("QUARTER"), pl.col("YEAR")]
query = context.query().truncate_per_group(1, by=quarterly).truncate_num_groups(10, by=quart
query = query.group_by(quarterly).agg(dp.len(signed=True))
print(f"{query.summarize()['scale'].item():.1f}")  # 10.0
try:
    query.explain()
except pl.exceptions.DuplicateError as err:
    print(str(err).splitlines()[0])  # multiple fields with name 'QUARTER' found
```

The second thing concerns `keep="first"` and `keep="last"`, which do run. The plan they build ranks each identifier's quarters and keeps the ranks below the number you asked for. Polars counts ranks from one, so nine ranks survive a request for ten, and the truncation keeps one group fewer than asked. The snippet shows the two numberings side by side on one identifier with three quarters.

```
import polars as pl

toy = pl.DataFrame({"PIDENT": [7, 7, 7], "YEAR": [2005] * 3, "QUARTER": [1, 2, 3]})
quarterly = [pl.col("QUARTER"), pl.col("YEAR")]
numbered = toy.with_columns(
    index=pl.int_range(pl.len()).over("PIDENT"),  # what truncate_per_group compares with k
    rank=pl.struct(quarterly).rank("dense").over("PIDENT"),  # what truncate_num_groups comp
)
print(numbered["index"].to_list(), numbered["rank"].to_list())  # [0, 1, 2] [1, 2, 3]
```

On this file the request for ten quarters drops nothing, because no identifier spans more than four. The script therefore measures the effect at a smaller `k`, using the filter that the documentation gives as the equivalent of the call. A request for four keeps at most three quarters, and the 13,948 identifiers that have four lose one. A request for five keeps all four, at the scale and threshold of five.

```
truncate_num_groups(10, keep=first) plan: FILTER [(col("QUARTER").as_struct([col("YEAR")]).r
identifier-quarters after the filter: 1,241,896  identifiers with 4 quarters: 13,948
truncate_num_groups(4, keep=first): keeps at most 3 quarters, 1,227,948 identifier-quarters;
truncate_num_groups(5, keep=first): keeps at most 4 quarters, 1,241,896 identifier-quarters;
```

So in version 0.16.0 you ask for one group more than you mean to keep, and you pay the noise of that larger number. The guarantee is not at risk either way, since the library calibrates to the number you asked for and keeps fewer. What is at risk is the estimate, because the dropped group is always the last in sort order, which here is the latest quarter of each person's year. The practitioner edition's release workflow, which bounds a customer's invoice lines, met both problems and avoided the call altogether.

One more rule about truncations comes from the library itself. The truncations of a query must sit next to each other, and the group truncation must come last before the aggregation. The

script moves the filter on `HWUSUAL` between the two truncations, and the query is refused with a message that names the truncation's own expression as unrecognized.

```
a filter between the two truncations: Expr is not recognized at this time: dyn int: 0.int_ra
```

5.4. One row per person

For a mean there is a better move than keeping ten rows per person. Group by the identifier first, aggregate each person's rows to one row with any Polars expression, and take the private mean of those rows. The documentation calls the group-by the point where truncation begins, because after it no identifier owns more than one row.

```
person_query = (
    context.query()
    .filter(pl.col("HWUSUAL") != 99)
    .group_by("PIDENT")  # one row per identifier: the truncation
    .agg(pl.col("HWUSUAL").mean())  # any expression may summarize the person's rows
    .select(hours().dp.mean(HOURS))
)
person_summary = person_query.summarize(alpha=0.05)
dp_person_mean = person_query.release().collect().item()
```

```
mean of one row per identifier: HWUSUAL Sum, Integer Laplace, scale=640.0 95% accuracy=1,917
mean of one row per identifier: HWUSUAL Length, Integer Laplace, scale=8.0 95% accuracy=24.5
```

The count's sensitivity is now 1 and the sum's is 80, so both scales are a tenth of what the truncation at ten gave. No row of anyone's is dropped. They are the scales a dataset with one row per person would get, however many rows a person really owns.

What changes is the question the statistic answers. The first query averaged over responses, so a person with four responses counted four times. This one averages each person's own average, so every person counts once. On this file the two differ, with 37.64 hours over responses against 37.24 over people. The cast to an integer in the documentation's query changes it a third time. It truncates each person's mean towards zero before the private mean sees it, which lowers the true value to 37.10. Noise is not the issue in any of these, since the error bar of the last query is about 0.0035 hours. The choice of what to average is.

```
true mean of one row per identifier: 37.0970  (before the cast to int: 37.2359; all rows: 37
per-identifier mean error bar at the true size: 0.0035
```

5.5. Bounds in the privacy unit

So far the unit held one number, and the query held every bound. The unit can hold finer knowledge, as a list of `dp.polars.Bound` objects. Each names the grouping it applies to, a bound

on identifiers per group and a bound on the groups one person touches. The documentation's example imagines data merged from two sources that assigned identifiers separately. A person may then appear under two identifiers, which would double the noise, but never under two in the same quarter.

```
two_sources = dp.unit_of(
    contributions=[
        dp.polars.Bound(per_group=2),  # a person may appear under two identifiers
        dp.polars.Bound(by=QUARTERLY, per_group=1),  # but under one identifier in any quart
    ],
    identifier="PIDENT",
)
bounded = make_context(frame, two_sources)
bounded_query = grouped_query(bounded, keep="first", k=5)  # each identifier may affect up t
bounded_summary = bounded_query.summarize(alpha=0.05)
bounded_hat = bounded_query.release().collect()
```

```
two identifiers, one per quarter, 5 quarters: len Frame Length, Integer Laplace, scale=80.0
two identifiers, one per quarter, 5 quarters: HWUSUAL Sum, Integer Laplace, scale=6,400.0 95
one identifier as a bound, 5 quarters: scales 40.0, 3,200.0
```

The query keeps one row per identifier and quarter and five quarters per identifier. Two identifiers times five quarters is ten rows per person, so the scale is 80 again. The second bound tells the library that the two identifiers never meet in one quarter, which caps what one person adds to any single quarter at one row. It does not lower the count's scale, because the ten rows can still fall in ten different quarters. With a single identifier declared as a bound, the same query gets half the scale.

A bound in the unit can also carry `num_groups`, the number of groups a person touches. Then the query needs no `truncate_num_groups`, and the script shows that a promise of ten quarters gives the same scale and threshold as the truncation at ten did. But the two are not the same kind of thing. The truncation is enforced, since the library drops the rows that would break it. The bound is a promise about the data that the library takes on trust. The script makes a promise that this file breaks, that no identifier spans more than three quarters, and the release goes through without a complaint.

```
promised = dp.unit_of(
    contributions=[
        dp.polars.Bound(per_group=1),
        dp.polars.Bound(by=QUARTERLY, per_group=1, num_groups=3),  # a promise: at most 3 qu
    ],
    identifier="PIDENT",
)
promise_query = (
    make_context(frame, promised)
    .query()
    .filter(pl.col("HWUSUAL") != 99)
```

```
    .truncate_per_group(1, by=QUARTERLY)  # no truncate_num_groups: the unit already bounds
    .group_by(QUARTERLY)
    .agg(dp.len(signed=True), hours().dp.sum(HOURS))
)
promise_summary = promise_query.summarize(alpha=0.05)
promise_hat = promise_query.release().collect()
```

```
num_groups=10 promised in the unit: len Frame Length, Integer Laplace, scale=80.0 95% accura
num_groups=3 promised in the unit: len Frame Length, Integer Laplace, scale=24.0 95% accurac
num_groups=3 promised in the unit: HWUSUAL Sum, Integer Laplace, scale=1,920.0 95% accuracy=
identifiers that break the promise of 3 quarters: 13,948
```

The noise was calibrated for three quarters, and 13,948 identifiers have four. For those people the release can move further than the noise was built to hide. Their privacy loss then exceeds the ε the context declared, and nothing in the output says so. This is the documentation's closing advice, and the chapter's. Truncate in the query, where the library enforces the bound. Declare a bound in the unit only when no truncation can express it, as with the two identifiers above, which no filter can recognize as one person. The same advice applies to clipping: clip the values in the query rather than declaring their range in the input domain. Assessing utility and choosing the noise returns to bounds per group when it switches to Gaussian noise, whose scale depends on them differently.

5.6. The check against the truth

The data are public, so the script computes what each query would return without noise and compares. For the mean over ten rows per identifier, the script draws its own random ten with a seed. The truncated mean comes out at 37.6393 hours, and the truncation of 20 rows leaves it unchanged to four decimals. The quarters' true values use the first row of each identifier in each quarter. The released sums keep a random row instead, so for the hours the check is approximate, while for the counts it is exact.

```
true mean, 10 rows per identifier: 37.6393  (1,496,172 rows)
true per quarter (first row per identifier and quarter): 36 quarters, rows from 27,228 to 41
true 2005Q1: rows 29,561  hours 1,110,610
true 2005Q2: rows 29,406  hours 1,106,170
true 2005Q3: rows 28,364  hours 1,070,826
true 2005Q4: rows 28,681  hours 1,080,009
```

The releases of one run follow, read against their error bars. The script derives an error bar for each mean from the bars of its sum and its count, since the mean is one divided by the other, and that bar holds whenever both of them hold.

```
released (one run):
  mean, 10 rows per identifier: 37.6366  (true 37.6393, within 0.0189 with probability at le
  per quarter: 36 quarters released; rows within their 95% accuracy: 36 of 36; hours: 35 of
  2005Q1: rows 29,644  hours 1,105,518
  2005Q2: rows 29,460  hours 1,098,965
  2005Q3: rows 28,244  hours 1,063,846
  2005Q4: rows 28,645  hours 1,087,028
  mean of one row per identifier: 37.0970  (true 37.0970, within 0.0035 with probability at
  two bounds, 5 quarters: 36 quarters released; rows within four times their 95% accuracy: 3
  num_groups=3 promised: 36 quarters released; rows within four times their 95% accuracy: 36
```

Every release landed within its bar, which on a run like this is what you should expect about 19 times in 20 for each value. The test behind the page checks the two means within three times their bars and the 36 quarters within four times theirs, so that a correct run almost never fails it. On real data you cannot run this comparison, so you run it on public or historical data while you develop, as here, and release the real data once.

5.7. Summary

Table 5.1.: The calls of identifier truncation and bounds, what each does, and the rule of thumb that goes with it.

Call or concept	What it does	Rule of thumb
`dp.unit_of(contributions=1, identifier="PIDENT")`	The privacy unit is every row of one identifier	`contributions` now counts identifiers; the rows per person are bounded by the query
A query with no truncation	Refused, with a message that blames the data scan	On a context with an identifier, every statistic sits behind a truncation
`truncate_per_group(k, keep=...)`	Keeps at most `k` rows per identifier, and per group with `by`	`"sample"` is unbiased; `"first"`, `"last"` and `SortBy` are cheaper or deliberate, and biased when order matters
`truncate_num_groups(k, by=...)`	Keeps at most `k` groups per identifier	In 0.16.0, `"sample"` fails at `collect` after the budget is spent, and `"first"` keeps `k - 1` groups; call `explain()` before `release()`
Truncations in a query	Must sit next to each other, the group truncation last	A filter between them makes the query unrecognized
`group_by("PIDENT").agg(...)` before a statistic	One row per person, so the sensitivity is that of one row	The best choice for a mean, if the mean over people is the one you want

Call or concept	What it does	Rule of thumb
`dp.polars.Bound(by, per_group, num_groups)` in `unit_of`	Finer bounds on identifiers per group and groups per person	A promise the library trusts; a person who breaks it loses more than ε
Truncate or declare	The query enforces a truncation; the unit declares a bound	Truncate, and declare only what no truncation can express

5.8. References

- OpenDP documentation: "Identifier Truncation" in the tabular-data tutorial, and "Identifier Truncation and Bounds" in the Polars user guide.
- Dwork, McSherry, Nissim and Smith, "Calibrating Noise to Sensitivity in Private Data Analysis" (2006): sensitivity and the Laplace mechanism.
- Amin, Kulesza, Muñoz Medina and Vassilvitskii, "Bounding User Contributions: A Bias-Variance Trade-off in Differential Privacy" (2019): why truncating a person's rows trades bias for variance.
- Wilson, Zhang, Lam, Desfontaines, Simmons-Marengo and Gipson, "Differentially Private SQL with Bounded User Contribution" (2020): bounding rows per group and groups per user in grouped queries.
- The full edition's chapters on sensitivity in the full edition and user-level privacy in the full edition derive the results this chapter states.

6. Preparing microdata

Survey data rarely arrive in the form a statistic needs. Hours of work come as floats, with a 99 for "not applicable" and a null where the answer is missing. The date of a response sits in two integer columns, the year and the quarter. Before you release anything you cast, fill, clip, filter and recode. This chapter is about where that work happens. The one idea is that OpenDP reads the whole lazy query before it runs, preparation included, and accounts for every step. So preparation belongs inside the query, where the guarantee covers it, and not in a script that runs before the context exists.

The chapter follows the documentation's tutorial on preparing microdata, its page on data types and its pages on the expressions it accepts. You will see which Polars contexts and expressions OpenDP allows, and what it changes about them and why. You will also see how it tracks nulls, undefined floats and bounds in the domain of a column. The chapter ends with a worked preparation of the Labour Force Survey (LFS) that releases one statistic. It is the mean difference between the hours people actually worked and the hours they usually work. The last section shows what an expression OpenDP refuses looks like.

The script opendp_preparing_microdata.py runs every step on OpenDP 0.16.0 with Polars 1.36.1. The noise scales, the error bars, the thresholds, the error messages and the true values it prints are the same on every run. The released counts and the released mean are not, because OpenDP draws its noise from a generator that cannot be seeded. This page therefore shows them in one block marked as one run.

```
uv run python code/part02-tabular-data/opendp_preparing_microdata.py
```

6.1. Prepare inside the query

A context built around a Polars LazyFrame reads only the frame's schema, which the library treats as public. The data are not touched until a query is released. Each query you submit is a lazy plan, and OpenDP parses it into a chain of transformations before anything runs, the chain that Transformations opens. For each step the library works out how much one person's rows can change the output. So a filter or a fill in the middle of the plan is part of the accounting.

Preparation before the context is different. What the library sees is the prepared frame, and its guarantee is a guarantee about that frame. Suppose the step was a fixed function of each row, such as renaming a column or parsing a date with a known format. Then every person still owns the same rows afterwards, and the guarantee means what it says. But the library did not check it, so the claim is yours. Suppose instead that the step read the data. An imputation by the column's own mean, a bound set at its maximum, or a filter at a quantile each carries information about everyone into the release. The accounting never saw that path, so the stated guarantee

no longer covers the original data. The facts you may use freely are the public ones, such as the schema, the code book and the published bound on the sample's size.

The context is the one the documentation uses on these pages. The privacy unit is 36 rows, because a person answers at most once a quarter over nine years. So two neighbouring datasets differ by one person's rows, at most 36 of them. The budget is approximate differential privacy with the privacy budget ε at 1 and the slack δ at 10^{-7}, as in Essential statistics, split evenly over four queries. The margin is the public bound on the number of rows from Identifier truncation and bounds.

```
def make_context(frame: pl.LazyFrame) -> dp.Context:
    """The documentation's context: 36 rows per person, (epsilon = 1, delta = 1e-7) split ev
    return dp.Context.compositor(
        data=frame,
        privacy_unit=dp.unit_of(contributions=CONTRIBUTIONS),
        privacy_loss=dp.loss_of(epsilon=EPSILON, delta=DELTA),
        split_evenly_over=QUERIES,
        margins=[dp.polars.Margin(max_length=MAX_LENGTH)],  # public: a bound on the number
    )
```

```
privacy unit: (SymmetricDistance(), 36)
privacy loss: (Approximate(MaxDivergence), (1.0, 1e-07))
budget per query: epsilon=0.25 delta=2.5e-08
schema: PIDENT=Int64  YEAR=Int64  QUARTER=Int64  SEX=Int64  AGE=Float64  HWUSUAL=Float64  HW
```

Each query gets a quarter of the budget. The schema line is all the context knows about the data at this point, and it assumes that any column may hold nulls.

6.2. Columns, filters and groups

Polars applies an expression in one of five contexts, and OpenDP treats them in two groups. Expressions in `.select` and `.agg` may change the number and the order of rows, because an aggregation does exactly that. Expressions in `.with_columns`, `.filter` and `.group_by` must be **row-by-row**, which means they could be written as a function applied to each row on its own. The reason is the privacy unit. A row-by-row expression keeps each person's rows aligned with their contributions, so 36 rows in means 36 rows out. An expression that sorted or shifted the column would mix people's rows, and the library could no longer say what one person changes.

The documentation's first example adds a column with `.with_columns` and groups on it. The expression `cut` bins the usual hours into bands at fixed breaks, which is row-by-row, and it shadows `HWUSUAL` with its bin. The labels of the bins depend only on the breaks, not on the data, so they are public and can be passed as keys.

```
def binned_counts(context: dp.Context) -> LazyFrameQuery:
    """How many responses fall in each band of usual hours: a row-by-row `cut`, then a count
    labels = pl.Series("HWUSUAL", cut_labels(BREAKS, left_closed=True), dtype=pl.Categorical
```

```
    return (
        lazy_query(context)
        .with_columns(pl.col("HWUSUAL").cut(breaks=BREAKS, left_closed=True))  # shadows HWU
        .group_by("HWUSUAL")
        .agg(dp.len(signed=True))
        .with_keys(pl.LazyFrame([labels]))  # the bin labels are public, so no threshold is
    )
```

```
binned counts: len Frame Length, Integer Laplace, scale=144.00 95% accuracy=431.9
without keys: len Frame Length, Integer Laplace, scale=144.00 threshold=2,973
true bins: null=20,819  [-inf, 0)=0  [0, 20)=89,495  [20, 40)=908,061  [40, 60)=406,947  [60
```

The scale is 144, which is the 36 rows one person can add to a bin, divided by the query's share of the budget. The error bar on each count is about 432. With the keys supplied, every bin is released, including the empty one below zero, whose noisy count can be negative because the count is signed. Without the keys the library would have to learn from the data which bins exist. The same query then carries a threshold of 2,973 below which a bin is dropped, as Grouping explained. The rows with a null `HWUSUAL` fall in a null bin that is not among the keys, so they are released nowhere. One more thing changes when a column is shadowed: any margin you declared about `HWUSUAL` applied to the original column, and the new column has none.

A `.filter` masks rows with a boolean row-by-row expression. The documentation's example casts the hours to integers, drops the rows coded 99 and sums the rest. Filtering only removes rows, so it keeps most of what the margins say. But it discards any invariant about the group keys and the group sizes, because those were facts about the dataset before the filter. Filtering also tells the library nothing about what remains. A column filtered to non-null values is still nullable in its domain, which the worked example will run into.

```
filtered sum: HWUSUAL Sum, Integer Laplace, scale=11,520.00 95% accuracy=34,511.3
```

The last context of the tutorial is a `.group_by` followed by `.agg` *before* the private aggregation. The documentation groups the rows a thousand ways by `PIDENT` modulo 1,000 and takes the minimum usual hours in each group. It then releases the mean of those thousand minima, which is its version of sample and aggregate. Any expression may go in that inner `.agg`, which is its appeal, but the price is noise. The script prints the scales of the mean with the context above, and then with a privacy unit that says each person's rows all land in one group. That unit is the `Bound` of Identifier truncation and bounds.

```
sample and aggregate: HWUSUAL Sum, Integer Laplace, scale=17,280.00
sample and aggregate: HWUSUAL Length, Integer Laplace, scale=576.00
one group per person: HWUSUAL Sum, Integer Laplace, scale=480.00
one group per person: HWUSUAL Length, Integer Laplace, scale=16.00
```

The first context does not know that a person's rows share a `PIDENT`, so it allows for 36 changed groups, and the bound brings the scales down by that factor. Even then the inner aggregation doubles the noise, because changing one row changes one aggregated row, which counts as removing a value and adding another.

6.3. The types OpenDP tracks

Every Polars column can hold nulls, and OpenDP assumes that every column does until an expression proves otherwise. Beyond that, the library keeps a few **domain descriptors** for each column, which are facts about its values that later aggregations need. For an integer column it tracks lower and upper bounds, which a `.clip` establishes and which some later steps drop. The supported integer types are the signed ones and the unsigned ones of 32 and 64 bits, so a `UInt8` column is refused. A float column has one more descriptor, whether the values may be NaN, the float that stands for an undefined number. A float sum therefore needs both `.fill_null` and `.fill_nan` before it. Booleans and strings carry no descriptors, and strings are the slowest type to work with.

Categorical columns look like strings but store an integer code per value, with a table of labels. Two things about that table depend on the data, and the library refuses both. First, an expression that could add or remove a category, such as `.fill_null` or a binary operator, makes Polars rewrite the codes and emit a warning. That is a side effect that depends on the data. Second, the codes are assigned in the order the values first appear, so grouping by them would reveal the order of the rows. An `Enum` column fixes its categories in the schema, so neither problem arises. The `cut` expression is fine too, because its encoding depends only on the breaks.

```
import polars as pl
import opendp.prelude as dp

dp.enable_features("contrib")
status = ["working", "absent", "other"] * 100
unit, loss = dp.unit_of(contributions=1), dp.loss_of(epsilon=1.0, delta=1e-7)


def context_for(column: pl.Series) -> dp.Context:
    return dp.Context.compositor(data=pl.LazyFrame([column]), privacy_unit=unit, privacy_los


categorical = context_for(pl.Series("status", status, dtype=pl.Categorical))
try:
    categorical.query().group_by("status").agg(dp.len(signed=True)).summarize()
except dp.OpenDPException as err:
    print(err.message)  # Categories are data-dependent, which may reveal sensitive record o
enum = context_for(pl.Series("status", status, dtype=pl.Enum(["working", "absent", "other"]))
print(f"{enum.query().group_by('status').agg(dp.len(signed=True)).summarize()['scale'].item(
```

The error names the fix, which is to cast to string before grouping, and the enum query goes through. A released enum column comes back typed categorical, because the frame crosses from Rust to Python through Arrow, which does not tell the two apart. Dates, datetimes and times are supported too. A datetime's time zone is part of the domain, so a column must have one zone throughout. The components of a date are read with the `.dt` expressions of Section 6.6.

6.4. Casting, filling and clipping

The schema is read off the frame, so in ordinary use the columns are already typed and a cast is a choice, not a repair. The documentation's reason to cast is the float sum. OpenDP accounts for the rounding error of floating-point addition, and on hundreds of thousands of rows with wide bounds that term shows in the noise scale. The script sums the usual hours both ways.

```
def hours_sum(context: dp.Context, as_integers: bool) -> pl.DataFrame:
    """The noise scale of a sum of usual hours, on the float column or after casting it to i
    usual: Any = pl.col("HWUSUAL")  # typed Any: OpenDP adds the `.dp` namespace to expressi
    column = usual.cast(int) if as_integers else usual
    return lazy_query(context).select(column.dp.sum(HOUR_BOUNDS)).summarize(alpha=0.05)
```

```
float sum: HWUSUAL Sum, Float Laplace, scale=14,405.18 95% accuracy=43,154.1
integer sum: HWUSUAL Sum, Integer Laplace, scale=14,400.00 95% accuracy=43,139.0
```

On integers the scale is exactly 36 times the upper bound of 100 over the share of the budget, and on floats it is a little more. The difference is small here and grows with the bounds and the number of rows. A cast has one more effect. In Polars a value that cannot be converted raises an error, and whether it raises depends on the data. So OpenDP makes a failed cast return a null instead. The output domain of every cast is therefore nullable, and a statistic on a cast column needs a `.fill_null` first.

```
import polars as pl
import opendp.prelude as dp

dp.enable_features("contrib")
context = dp.Context.compositor(
    data=pl.LazyFrame({"hours": ["40", "35", "n/a"] * 100}),
    privacy_unit=dp.unit_of(contributions=1),
    privacy_loss=dp.loss_of(epsilon=1.0),
    split_evenly_over=1,
    margins=[dp.polars.Margin(max_length=1000)],
)
hours = pl.col("hours").cast(int)  # OpenDP turns "n/a" into a null instead of raising an er
try:
    context.query().select(hours.clip(0, 80).sum().dp.noise()).summarize()
except dp.OpenDPException as err:
    print(err.message)  # input data (hours) might contain nulls. Preprocess your data with
query = context.query().select(hours.fill_null(0).clip(0, 80).sum().dp.noise())
print(query.summarize()["scale"].item())  # 80.0
```

The `.dp.sum` of the earlier chapters inserts the imputation and the clip for you. The snippet instead prepares the column by hand and adds noise to a plain `.sum`, which shows what the library checks. A `.clip` establishes the bounds descriptor, and the sum looks for it, since without bounds one person could move the sum by any amount. The order of the steps matters, because

`.fill_null` and `.fill_nan` drop the bounds. Fill first, then clip. The script runs the same preparation on the survey in `.with_columns` and tries the wrong orders.

```
def prepared_sum(context: dp.Context, steps: pl.Expr) -> pl.DataFrame:
    """A sum with noise on a column prepared in with_columns: OpenDP checks the steps left i
    total: Any = pl.col("HWUSUAL").sum()
    return lazy_query(context).with_columns(steps).select(total.dp.noise()).summarize()
```

```
prepared sum: HWUSUAL Sum, Integer Laplace, scale=14,400.00
cast without fill_null: input data (HWUSUAL) might contain nulls. Preprocess your data with
clip before fill_null: input domain must consist of bounded data. Either specify bounds in t
fill_null before fill_nan: input data (HWUSUAL) might contain nans. Preprocess your data wit
float prepared sum: HWUSUAL Sum, Float Laplace, scale=14,405.18
```

The fill value need not be a constant. The documentation fills a person's missing usual hours with their actual hours, which is a fill from another column. Then both the column and the fill must be row-by-row, so that they have the same length and line up. The fill must also be free of nulls itself, or the output could still hold one. The script fills `HWACTUAL` with 0 first and then uses it as the fill, which is accepted, and tries the unfilled column, which is not.

```
filled from a column: HWUSUAL Sum, Integer Laplace, scale=14,400.00
filled from a nullable column: fill expression must not be nullable
```

Two expressions remove rows rather than fill them. `.drop_nulls` and `.drop_nans` guarantee a clean column, but they change its length. So the library forbids them in a row-by-row context, and they cost the invariants on the dataset's size.

```
drop_nulls in with_columns: drop_nulls is not allowed in a row-by-row context
```

Recoding has two forms. The expression `.replace` swaps listed old values for new ones and keeps the type, while `.replace_strict` maps every value and may change the type. So a value that the second mapping leaves out needs somewhere to go, and it takes the `default`. The documentation says that OpenDP sets a missing `default` to null, so that the query cannot fail on an unexpected value. Version 0.16.0 does not do that. The script maps a toy column of three codes with a mapping for two of them. Then `summarize` accepts the query, but `release` raises the Polars error about an incomplete mapping. Give `replace_strict` a `default` yourself.

```
replace_strict without default: summarize scale=2.0; release: InvalidOperation(ErrString("in
replace_strict with default: released keys one other two
```

The last expression of the page is `.to_physical`, which returns the integer codes behind a categorical or temporal column, under the same condition on the encoding.

6.5. Predicates and arithmetic

OpenDP accepts all of Polars's comparison, conjunction and arithmetic operators, and treats each as row-by-row. Both sides of a binary operator must be row-by-row even inside `.select` or `.agg`, so that they have the same length and the rows line up. A mismatch would raise an error that depends on the data. The boolean tests are row-by-row too. They are `is_null` and `is_not_null` for any type and `is_nan`, `is_not_nan`, `is_finite` and `is_not_finite` for floats, and `.not_` negates a predicate. These expressions build filters and grouping columns. The documentation counts the respondents over retirement age by grouping on a comparison.

```
def over_retirement_age(context: dp.Context) -> LazyFrameQuery:
    """How many responses come from people over 64: a boolean expression as the grouping col
    return (
        lazy_query(context)
        .group_by(pl.col("AGE") > RETIREMENT_AGE)
        .agg(dp.len(signed=True))
        .with_keys(pl.LazyFrame({"AGE": [True, False]}))  # a null age is in neither group a
    )
```

```
over 64: len Frame Length, Integer Laplace, scale=144.00 95% accuracy=431.9
true over 64: true=1,190,845  false=2,620,832  null=255
```

A comparison with a null is null, so the 255 rows without an age form a third group that the keys leave out. The same question can be asked as a filter inside the aggregation, `pl.col("HWUSUAL").filter(pl.col("AGE") > 64)`, which is not row-by-row but requires row-by-row inputs. The grouped form answers both halves of the question for the same price.

Two operators you might reach for are not available in 0.16.0. Membership in a list, `is_in`, is refused with a message that says it is not yet supported, so write the test as a disjunction of equalities. The script counts the employed, who are working or absent from work, that way.

```
is_in: IsIn { nulls_equal: false } is not currently supported
employed: len Frame Length, Integer Laplace, scale=144.00 95% accuracy=431.9
```

The absolute value is the other, and Section 6.8 shows its message. One more restriction comes from the columns page: an expression selects one column at a time, so a query over several columns lists one expression per column.

6.6. Strings and dates

Of the string expressions, OpenDP supports only parsing into a temporal type, with `.str.strptime` and its variants `to_date`, `to_datetime` and `to_time`. The `format` argument is mandatory. Polars can infer a format from the data, but the inferred format could change when one person's rows are added or removed, which makes the step unstable. Parsing into nanosecond datetimes is refused as well, because the parser can panic on some inputs, and a panic is an error that depends on the data.

```
to_date without format: format must be specified; otherwise Polars will attempt to infer the
nanosecond datetime: Nanoseconds are not currently supported due to potential panics when pa
```

The survey has no date column, only `YEAR` and `QUARTER`. The documentation builds one with `concat_str` before the context, because concatenation is not a supported expression. The script confirms that it is refused inside a query.

```
concat_str: Expr is not recognized at this time: ConcatHorizontal { delimiter: "-", ignore_n
```

Arithmetic is supported, so the same date can be built inside the query. Multiplying the year by 100 and adding three times the quarter gives an integer such as 200503. Cast to a string, it parses with the format `%Y%m` into the first day of the quarter's last month. The components of the date are then read with the `.dt` expressions. For a date they cover the year, the quarter, the month, the week, the weekday and the day. For a time they cover the hour down to the nanosecond.

```
def quarter_end() -> pl.Expr:
    """The last month of the quarter as a date, built from YEAR and QUARTER and parsed with
    year_month = pl.col("YEAR") * 100 + pl.col("QUARTER") * 3  # 2005 and 1 become 200503
    return year_month.cast(str).str.to_date(format="%Y%m")


def responses_per_month(context: dp.Context) -> LazyFrameQuery:
    """How many responses carry each quarter-end month, read off the parsed date."""
    return (
        lazy_query(context)
        .with_columns(DATE=quarter_end())
        .group_by(pl.col("DATE").dt.month().alias("MONTH"))
        .agg(dp.len(signed=True))
        .with_keys(pl.LazyFrame({"MONTH": pl.Series(QUARTER_END_MONTHS, dtype=pl.Int8)}))
    )
```

```
first dates: 2005 Q1 -> 2005-03-01  2005 Q2 -> 2005-06-01
per month: len Frame Length, Integer Laplace, scale=144.00 95% accuracy=431.9
true per month: 3=953,695  6=958,396  9=933,257  12=966,584
```

The four months are public, since every quarter ends in one of them, so the keys are given and the counts cost no threshold. The key series is typed `Int8` because that is what `.dt.month` returns, and keys must match the grouping column's type exactly.

6.7. A worked preparation: hours lost to absence

The statistic is the mean difference between the hours a person actually worked in the survey week and the hours they usually work, among people with a job. A negative mean means that holidays, illness and short weeks outweigh overtime. Each step of the preparation is a decision

about the data, and each is inside the query. The rows are first restricted to `ILOSTAT` equal to 1, which the code book defines as working for pay or profit. Both hour columns are cast to integers, so that the sum later uses the integer noise of Section 6.4. The usual hours are filled with 0 before the next filter rather than after it. The filter then keeps the rows whose usual hours are known. That excludes 0, the code for hours that vary, and 99, the code for not applicable.

The reason for filling before filtering is the rule from Section 6.2 that a filter proves nothing to the library. After the filter no null usual hours remain, but the column is still nullable in its domain. The next step wants to use it as a fill value, which is refused unless it is non-nullable. A row whose actual hours are missing gets the usual hours instead, which makes its difference zero, the least informative choice. Finally the difference is clipped to the public bounds of plus or minus 40 hours, because a sum needs bounds. The range of possible differences, plus or minus 80, would double the noise.

```
def prepare[Frame: (LazyFrameQuery, pl.LazyFrame)](frame: Frame) -> Frame:
    """The rows of people who work, with their usual hours known, and the hours they lost th
    low, high = LOST_BOUNDS
    return (
        frame.filter(pl.col("ILOSTAT") == 1)  # working for pay or profit
        .with_columns(pl.col("HWUSUAL").cast(int).fill_null(0), pl.col("HWACTUAL").cast(int)
        .filter((pl.col("HWUSUAL") > 0) & (pl.col("HWUSUAL") < 99))  # 0 is "varies", 99 "no
        .with_columns(pl.col("HWACTUAL").fill_null(pl.col("HWUSUAL")))  # unknown actual hou
        .with_columns(LOST=(pl.col("HWACTUAL") - pl.col("HWUSUAL")).clip(low, high))
    )
```

The function takes either a query or a plain LazyFrame. So the script runs one preparation through the context for the release, and on the frame for the dev-time check. The query ends in `pl.col("LOST").dp.mean(LOST_BOUNDS)`, and `summarize` shows what the mean costs.

```
hours lost: LOST Sum, Integer Laplace, scale=11,520.00 95% accuracy=34,511.3
hours lost: LOST Length, Integer Laplace, scale=288.00 95% accuracy=863.3
hours lost, bounds (-80, 80): LOST Sum, Integer Laplace, scale=23,040.00 95% accuracy=69,022
```

A mean is released as a noisy sum over a noisy count, and the query's share of the budget is split between the two. So the sum's scale is the 36 rows times the bound of 40, over an eighth of the budget. The count's scale divides the same 36 rows by the same eighth. With bounds of plus or minus 80 the sum's scale doubles, which is the third line. The clip is the trade of this chapter, since a narrower bound means less noise and more bias. How the sensitivity of a sum follows from its bounds is derived in the full edition's chapter on sensitivity.

Because the survey is public, the script measures the bias at development time, which on real data you can do only with a public or historical copy.

```
true prepared rows: 1,490,113  true sum of hours lost: -9,289,648  true mean: -6.2342
rows the clip changes: 42,830 (2.874%)  mean without the clip: -6.5319
mean error bar at 95%: 0.0268
```

The preparation keeps 1,490,113 rows and the clip changes 42,830 of them, under three percent. Those rows move the true mean from -6.53 hours to -6.23. So the clip costs a bias of about a third of an hour. The noise is far smaller. The script turns the two accuracies into an error bar for the mean, which is the sum's bar plus the mean times the count's bar, all over the count. It comes to 0.027 hours. On this many rows the clip, not the noise, is what limits the result, and a wider bound would trade some of that bias for noise you can read off the table.

```
released (one run):
  bins: [-inf, 0)=-129  [0, 20)=89,491  [20, 40)=907,895  [40, 60)=406,594  [60, 80)=78,436
  over 64: true=1,190,894  false=2,620,771
  per month: 3=953,684  6=958,313  9=933,077  12=966,807
  hours lost: mean -6.2147  (true -6.2342, within 0.0268 with probability 0.95)
budget spent: 4 of 4 queries
```

Every count landed within its bar of about 432, and the empty bin came out negative, as a signed count may. The mean landed within its bar of the clipped truth. The four releases used the four shares, and a fifth query would be refused. Assessing utility and choosing the noise takes the error bars further, and shows what they leave out.

6.8. What a refused expression looks like

An expression that OpenDP cannot parse raises an `OpenDPException` while the query is built, before any data are read. The exception carries a variant and a message. The absolute value of the difference in hours is a case in point.

```
abs: Expr is not recognized at this time: [(col("HWACTUAL")) - (col("HWUSUAL"))].abs(). If y
```

The messages on this page come in two kinds. "Not recognized at this time" means the expression has not been implemented, and the message asks for an issue, which is how `concat_str` and `abs` fail. The others state a rule, such as that a format must be specified or that a fill must not be nullable. Each of those names its fix, which is to prepare differently inside the query. For an unimplemented expression you have two choices. Rewrite it with supported operators where you can, as `is_in` became a disjunction above. Otherwise apply it before the context, as the documentation does with `concat_str`, but only if it is a fixed function of each row. Then state that step alongside the release, because the library did not check it.

6.9. Summary

Table 6.1.: The contexts and expressions of this chapter, what each does, and the rule of thumb that goes with it.

Call or concept	What it does	Rule of thumb
Preparation inside the query	OpenDP parses every step and accounts for it	Prepare before the context only with fixed row-by-row steps and public facts, and say so
`.with_columns`, `.filter`, `.group_by`	Row-by-row contexts: each person's rows stay their own	A new column has no margins; a filter drops key and length invariants and proves nothing about what remains
`.select`, `.agg`	May change the number and order of rows	Binary operators and fills inside them still need row-by-row inputs
`.cut(breaks)`	Bins a number into categories with a data-independent encoding	Pass the labels as keys to avoid a threshold
Domain descriptors	Nullable, may be NaN, bounds	Every column starts nullable; floats need `.fill_null` and `.fill_nan`; `.clip` sets bounds
`.cast`	A failed cast returns null instead of raising	Follow it with `.fill_null`; cast to integers for a cheaper sum
`.fill_null`, `.fill_nan`	Impute; a column fill must be row-by-row and non-nullable	Fill first, then clip, because filling drops the bounds
`pl.Categorical` vs `pl.Enum`	Categories from the data vs fixed in the schema	Group by enums; cast categoricals to string
`.replace_strict(mapping)`	Recodes every value	Pass a `default`: in 0.16.0 a missing one fails at release
Comparisons, `&`, `|`, arithmetic	Row-by-row predicates and new columns	`is_in` and `abs` are not supported in 0.16.0
`.str.to_date(format=...)`	Parses a string into a date	The format is mandatory; nanosecond datetimes are refused
`.dt.month()` and the other components	Reads a part of a date or time	Keys for a component must match its type (`Int8` for a month)
`OpenDPException`	Raised when the plan is parsed, before any data are read	"Not recognized" is unimplemented; the other messages state a rule and its fix

6.10. References

- OpenDP documentation: “Preparing Microdata” in the tabular data tutorial; “Data Types” and the “Expressions” pages “Boolean”, “Columns”, “Manipulation”, “Operators”, “String” and “Temporal” in the Polars user guide.
- Nissim, Raskhodnikova and Smith, “Smooth Sensitivity and Sampling in Private Data Analysis” (2007): the sample and aggregate framework behind the stable `group_by/agg`.
- Gaboardi, Hay and Vadhan, “A Programming Framework for OpenDP” (2020): the stable transformations that each expression is parsed into.
- The full edition’s chapter on sensitivity in the full edition derives how the bounds of a clipped column fix the noise of its sum and mean.

7. Assessing utility and choosing the noise

Every query in the earlier chapters printed a scale and an accuracy before it was released. This chapter is about reading those two numbers and changing them. An **accuracy** is the half-width of an error bar: with a stated probability, the released value lies within that distance of the value the noise was added to. OpenDP converts a noise scale to an accuracy and back, for Laplace and for Gaussian noise. So you can work out what a target error bar costs before you spend anything.

The second half of the chapter is about which noise to add. Under pure differential privacy (DP) the context adds Laplace noise, and under zero-concentrated differential privacy (zCDP) it adds Gaussian noise. Switching between them is one argument of `dp.loss_of`. The choice matters most when one person's records fall in many groups. Gaussian noise is then calibrated to a sensitivity that grows with the square root of the number of groups, rather than with the number itself. On the survey, telling the library that a person answers at most once per quarter cuts the noise on a quarterly count six-fold, but only under zCDP. By the end you will be able to say what an error bar covers and what it leaves out. You will also convert a budget in ρ back to an ε, and choose the noise for a table.

The script opendp_utility_and_noise.py runs every step on OpenDP 0.16.0 against the Labour Force Survey (LFS) sample of the earlier chapters. The conversions, scales, accuracies, thresholds and budgets it prints are the same on every run. The released counts and the means of a small clamping experiment are not, because OpenDP's noise cannot be seeded, so this page shows them in blocks marked as one run.

```
uv run python code/part02-tabular-data/opendp_utility_and_noise.py
```

7.1. What an accuracy statement means

The documentation gives a template for reading the accuracy functions, and it is worth learning by heart. When the noise has a given scale, the released value differs from the true value by no more than the accuracy at a significance level α, or with confidence $(1 - \alpha) \times 100\%$. Every accuracy on a page of this book has $\alpha = 0.05$, so the error bar holds with probability 0.95, and about one release in twenty lands outside it. A smaller α buys a wider bar, which Essential statistics showed by summarizing the same count at $\alpha = 0.01$.

The module `opendp.accuracy` has eight conversion functions, and the prelude exposes them under `dp`. Four convert a scale to an accuracy and back for continuous noise, `laplacian_scale_to_accuracy`, `accuracy_to_laplacian_scale`, `gaussian_scale_to_accuracy` and `accuracy_to_gaussian_scale`. The other four do the same for the discrete distributions, and their names carry `discrete_`. The documentation's example asks for the Gaussian scale at which a release differs from its input by at most 2 with 95% confidence.

```
import opendp.prelude as dp

print(dp.accuracy_to_gaussian_scale(accuracy=2.0, alpha=0.05))  # 1.020426913849308
print(dp.gaussian_scale_to_accuracy(scale=180.0, alpha=0.05))  # 352.7935172172097
print(dp.discrete_gaussian_scale_to_accuracy(scale=180.0, alpha=0.05))  # 354.0
print(dp.laplacian_scale_to_accuracy(scale=180.0, alpha=0.05))  # 539.2318092397184
print(dp.discrete_laplacian_scale_to_accuracy(scale=180.0, alpha=0.05))  # 539.7311147961669
```

The continuous functions are closed forms. For Laplace noise the accuracy is the scale times $\ln(1/\alpha)$. For Gaussian noise it is the scale times the two-sided normal quantile, which is about 1.96 at the usual level. The discrete functions sum the probabilities of the integer noise instead, which is why a discrete Gaussian accuracy is a whole number. Use the discrete ones for counts, because OpenDP adds integer noise to an integer, and the continuous ones for sums and means of floats. The difference is small at a scale of 180, as the snippet shows. At a scale near 1 the discrete bars are noticeably wider, but `summarize` always picks the right one for you.

The error bar is a statement about the noise and nothing else. The documentation says so in a note, and the point deserves a paragraph here, because it is the mistake a reader of an error bar makes most often. The accuracy functions describe how far the release can fall from the input to the noise mechanism. That input is the count, sum or mean of the data after every preparation step has run. Clipping values into public bounds, imputing a constant for missing values, truncating a person's rows and resizing the frame all change that input. The error bar says nothing about the gap between the input and the quantity you had in mind.

Clipping a sum pulls it toward the bounds, and truncation drops rows. Both are biases that no number of repeated releases averages away. The practitioner edition's release workflow in the practitioner edition measures such a gap on another dataset, where truncation and clipping keep about half of the revenue. Here the quantity is a count of rows with no preparation, so the bar is the whole story.

7.2. A target error bar on the survey

Suppose you want the count of survey responses within 100 of the truth with 95% confidence. The count under the tutorial's pure-DP budget of $\varepsilon = 1$, split evenly over five queries, gives you a place to start. The excerpt summarizes it and then asks the discrete Laplace conversion for the scale the target needs.

```
laplace = laplace_context(frame, EPSILON)
count_laplace = laplace.query().select(dp.len(signed=True))
laplace_summary = count_laplace.summarize(alpha=ALPHA)
scale_needed = float(dp.accuracy_to_discrete_laplacian_scale(accuracy=TARGET, alpha=ALPHA))
epsilon_needed = CONTRIBUTIONS / scale_needed  # the Laplace scale is the sensitivity over e
```

```
laplace count: Integer Laplace scale=180 95% accuracy=539.731
budget per query: epsilon=0.2000
error bar relative to the true count: 0.014%
```

```
discrete laplace scale for accuracy 100: 33.2152
epsilon that scale needs at sensitivity 36: 1.0838  (5.42 times the share)
```

The scale of 180 comes from the privacy unit and the budget. A person contributes at most 36 rows, so adding or removing a person changes the count by at most 36. That is the sensitivity, and the Laplace scale is the sensitivity over the query's share of ε, which is 0.2. The error bar of about 540 is 0.014% of the true count, which is small for a headline figure. The target of 100 needs a scale of 33.2. At sensitivity 36, that scale costs $\varepsilon = 1.08$ on this one query alone, which is more than the whole budget. The honest conclusion is that a bar of 100 is not available at this budget, not that the budget should grow to meet it.

The conversion also tells you the exchange rate. Because the Laplace scale is the sensitivity over ε, and the accuracy is proportional to the scale, halving the error bar means doubling the budget. The script checks this by rebuilding the context at $\varepsilon = 2$, where the same query gets a scale of 90 and about half the error bar.

```
laplace count at epsilon 2: Integer Laplace scale=90 95% accuracy=270.115
```

The bar is not exactly half of 539.7, because the discrete conversion adds a constant for the rounding of integer noise, but the rule of thumb holds. The full edition's chapter on the Laplace mechanism derives both the scale and the tail bound behind the conversion.

7.3. Laplace under pure DP, Gaussian under zCDP

OpenDP chooses the noise distribution from the kind of budget you give it. If the budget is `dp.loss_of(epsilon=...)`, the guarantee is pure DP and the context adds Laplace noise. If it is `dp.loss_of(rho=...)`, the guarantee is zCDP and the context adds Gaussian noise. Nothing else in a query changes, as long as the query does not name a distribution itself. So the two context constructors of the script differ only in the budget line.

```
def laplace_context(frame: pl.LazyFrame, epsilon: float) -> dp.Context:
    """Pure differential privacy: the context adds Laplace noise."""
    return dp.Context.compositor(
        data=frame,
        privacy_unit=dp.unit_of(contributions=CONTRIBUTIONS),
        privacy_loss=dp.loss_of(epsilon=epsilon),
        split_evenly_over=QUERIES,
    )


def gaussian_context(frame: pl.LazyFrame, rho: float, delta: float | None = None) -> dp.Cont
    """Zero-concentrated differential privacy: the context adds Gaussian noise."""
    return dp.Context.compositor(
        data=frame,
        privacy_unit=dp.unit_of(contributions=CONTRIBUTIONS),
```

```
        privacy_loss=dp.loss_of(rho=rho, delta=delta),
        split_evenly_over=QUERIES,
    )
```

What does ρ mean? Pure DP bounds the privacy loss, the log of how much more likely an output is with a person in the data than without, by ε in the worst case. Zero-concentrated DP bounds it on average instead. A mechanism is ρ-zCDP when its privacy loss has mean at most ρ and tails no heavier than those of a normal variable with variance 2ρ. The Gaussian mechanism fits this definition exactly, which is why the two go together, and budgets in ρ add up across queries just as budgets in ε do. The full edition's chapter on concentrated DP defines it through Rényi divergences and derives all of this. The documentation's Gaussian page repeats the count under $\rho = 0.1$, split over the same five queries.

```
gaussian = gaussian_context(frame, RHO)
count_gaussian = gaussian.query().select(dp.len(signed=True))
gaussian_summary = count_gaussian.summarize(alpha=ALPHA)
```

```
privacy loss: (ZeroConcentratedDivergence, 0.1)
gaussian count: Integer Gaussian scale=180 95% accuracy=354
budget per query: rho=0.0200  sqrt(2 rho)=0.2000
continuous gaussian accuracy at scale 180: 352.7935
```

The scale is again 180, and that is no accident. Write σ for the standard deviation of the Gaussian noise, which is what OpenDP calls its scale, and Δ for the sensitivity of the quantity it is added to. Then the mechanism is $\Delta^2/(2\sigma^2)$-zCDP, so the context solves for the scale and finds the sensitivity over $\sqrt{2\rho}$. With a share of $\rho = 0.02$, that root is 0.2, which is exactly the Laplace share, and the documentation chose $\rho = 0.1$ so that its two pages compare at equal scale. At equal scale the Gaussian bar is narrower, 354 against 540, because Gaussian tails fall faster than Laplace tails. Equal scale is not equal privacy, though. Section 7.5 converts this share of ρ to an ε of about 1 at a small δ. That is five times the Laplace share, so the two pages compare different guarantees.

The exchange rate also differs. Because the Gaussian scale is the sensitivity over $\sqrt{2\rho}$, halving the error bar means quadrupling ρ, not doubling it. The script confirms this at $\rho = 0.4$.

```
gaussian count at rho 0.4: Integer Gaussian scale=90 95% accuracy=177
```

So on a single count, Gaussian noise under zCDP is no bargain. Its advantage appears when a query answers many things at once, which is the subject of the next section.

7.4. One person in many groups: L1 against L2

A grouped count returns a vector, one number per group, and the noise is calibrated to how far one person can move the whole vector. Laplace noise measures that distance in the L_1 norm, which adds up the changes across groups. Gaussian noise measures it in the L_2 norm, which adds

up their squares and takes the root. The documentation puts it in one picture. Walking between two corners of a unit square takes 2 along the streets and $\sqrt{2}$ as the crow flies. In d dimensions the gap grows to d against $\sqrt{d}$.

The survey shows the gap at its most useful. A person answers at most once per quarter, so their 36 records fall in 36 different year-and-quarter groups, each changed by one. The L_1 distance between the two count vectors is 36, but the L_2 distance is $\sqrt{36} = 6$. Without being told that, the library must assume the worst case, in which all 36 records land in one group, and then both distances are 36. Identifier truncation and bounds introduced `dp.polars.Bound` for exactly this statement, and the excerpt uses it.

```
def bounded_context(frame: pl.LazyFrame, privacy_loss: tuple[Measure, Any]) -> dp.Context:
    """A budget of either kind, with the privacy unit also bounded to one record per year an
    return dp.Context.compositor(
        data=frame,
        privacy_unit=dp.unit_of(
            contributions=[
                dp.polars.Bound(by=[], per_group=CONTRIBUTIONS),  # at most 36 records overa
                dp.polars.Bound(by=["YEAR", "QUARTER"], per_group=1),  # and one in each yea
            ]
        ),
        privacy_loss=privacy_loss,
        split_evenly_over=QUERIES,
    )
```

The budget now carries a δ, because the query groups by keys the library has not been given. As Grouping explained, the context then releases only the groups whose noisy count clears a threshold. The small probability δ pays for the chance that a group of one person slips through. The script summarizes the quarterly count three ways. The first is under zCDP with the plain bound of 36 contributions, the second adds the per-group bound, and the third uses the per-group bound under pure DP.

```
bounded = bounded_context(frame, dp.loss_of(rho=RHO, delta=DELTA))
count_by_quarter = bounded.query().group_by("YEAR", "QUARTER").agg(dp.len(signed=True))
grouped_summary = count_by_quarter.summarize(alpha=ALPHA)
```

```
privacy loss with a threshold: (Approximate(ZeroConcentratedDivergence), (0.1, 1e-07))
grouped count, 36 contributions: Integer Gaussian scale=180 95% accuracy=354 threshold=1133
grouped count, 1 per year and quarter: Integer Gaussian scale=30 95% accuracy=60 threshold=1
grouped count, 1 per year and quarter, pure DP: Integer Laplace scale=180 95% accuracy=539.7
true groups: 36  smallest 82,504  largest 131,656
```

The per-group bound cuts the Gaussian scale from 180 to 30, the factor of six that the arithmetic predicted. The threshold falls with it, from 1,133 to 184. Under pure DP the same bound changes nothing, because the L_1 distance is 36 either way, and the Laplace scale stays at 180. The thresholds are far below the smallest quarterly count of 82,504, so every group is released and the

error bar of 60 is what the table buys. The full edition's chapter on the two mechanisms proves why Gaussian noise scales with the L_2 sensitivity and compares the two in many dimensions.

The lesson generalizes beyond this survey. Whenever a person's records are spread thinly over many groups, say one purchase per day, the L_2 sensitivity is far below the L_1 one. Then Gaussian noise under zCDP is the right tool. The gain depends on telling the library about the spread with a bound per group. A bound you cannot justify from public knowledge must be enforced by truncation first, as the identifier truncation chapter showed.

7.5. Stating the guarantee in epsilon

A budget in ρ is convenient for accounting but unfamiliar to most readers of a release. OpenDP converts it to the usual pair (ε, δ) with two combinators, which Combinators covers in general. The first, `make_zCDP_to_approxDP`, turns a measurement's zCDP guarantee into a curve of ε against δ. The second, `make_fix_delta`, picks a δ on that curve. The script applies them to a Gaussian measurement built by hand, whose scale is set so that a sensitivity of 1 costs exactly ρ.

```
def epsilon_of_rho(rho: float, delta: float) -> float:
    """The epsilon that OpenDP's conversion gives for rho-zCDP at delta, read off a Gaussian
    space = dp.atom_domain(T=float, nan=False), dp.absolute_distance(T=float)
    gaussian = dp.m.make_gaussian(*space, scale=1 / math.sqrt(2 * rho))  # sensitivity 1 at
    approx = dp.c.make_fix_delta(dp.c.make_zCDP_to_approxDP(gaussian), delta)
    epsilon, _ = approx.map(1.0)
    return float(epsilon)
```

```
rho=0.1 at delta=1e-07: epsilon=2.3484
closed form rho + 2 sqrt(rho ln(1/delta)): 2.6391
rho=0.02 (one query) at delta=1e-07: epsilon=0.9951
the whole context converted: epsilon=2.3484 delta=1e-07
```

Take $\delta = 10^{-7}$. The whole budget of $\rho = 0.1$ is then worth $\varepsilon = 2.35$. One query's share of $\rho = 0.02$ is worth about $\varepsilon = 1$ at the same δ. The textbook conversion is

$$\varepsilon = \rho + 2\sqrt{\rho \ln(1/\delta)}, \tag{7.1}$$

which gives 2.64 here, and OpenDP's smaller number is not a mistake. The library uses a tighter conversion that optimizes over the orders of the Rényi divergence, which the full edition's chapter on conversions derives. You can also convert a whole context rather than one measurement. Its `accountant` attribute is the measurement that composes every query. The same two combinators applied to it give the same 2.35, which is the number to write in the release's statement.

The δ here is your choice. It should be far below one over the number of people, because a δ of that size would allow a mechanism that publishes one person's record outright. With 1.7 million people, 10^{-7} is a defensible value. This δ is a conversion cost, separate from the δ that paid for the threshold in the grouped query. A release that needs both should budget them both.

A last detail is which accuracy the zCDP context reports. The Gaussian count's summary said 354, and the continuous conversion at the same scale says 352.79. The context adds a discrete Gaussian to an integer count, so it reports the discrete accuracy, which Measurements explains when it builds the integer mechanisms. The difference is a rounding. But if you recompute an accuracy by hand from a scale, use the discrete function for counts to match what `summarize` prints.

7.6. Signed and unsigned counts

Every count on this page passed `signed=True`, and the reason is bias. A noisy count can come out negative when the true count is small, and OpenDP's older behaviour was to raise such a value to zero. Version 0.16.0 adds the `signed` argument. Pass `signed=True`, and the count is cast to a signed integer before the noise, so a negative result is kept. Pass `signed=False`, and the result is clamped at zero. Leaving the argument out still clamps and prints a warning, because the default is about to change.

Clamping looks harmless for one count, and for a single small cell it even reduces the error. The trouble starts when counts are added. A clamped count can be too high but never too low, so each small cell is biased upward, and a total over hundreds of cells piles those biases up. The OpenDP cookbook demonstrates this on a table of 400 cells, most of them nearly empty. There the clamped total overshoots on every repetition, while the signed total is right on average.

A signed release can be clamped afterwards for display, because clamping a released value is post-processing and costs nothing. The rule is to release signed counts whenever anyone might sum, average or model them, and to tidy up the presentation last. The script makes the point in miniature. It counts an empty frame a thousand times at a fixed Gaussian scale of 30 through the framework API, which bypasses a context and so suits toy data only.

```
def mean_noisy_count_of_nobody(signed: bool, scale: float, repeats: int) -> float:
    """The mean of `repeats` noisy counts of an empty frame, at a fixed Gaussian scale, sign
    empty = pl.LazyFrame(schema={"A": pl.Int32})
    domain = dp.with_margin(
        dp.lazyframe_domain([dp.series_domain("A", dp.atom_domain(T="i32"))]),
        dp.polars.Margin(by=[], max_length=1000),
    )
    count = dp.m.make_private_lazyframe(
        domain, dp.symmetric_distance(), dp.zero_concentrated_divergence(), empty.select(dp.
    )
    return sum(int(count(empty).collect().item()) for _ in range(repeats)) / repeats
```

```
dp.len() without signed: warns=True clamped=True
expected clamped mean at scale 30: scale/sqrt(2 pi)=11.9683
standard error of the signed mean: scale/sqrt(1000)=0.9487
clamping experiment (one run): 1000 noisy counts of nobody at gaussian scale 30
  unsigned mean 11.48  signed mean 1.02  (true count 0)
```

The true count is zero. The signed mean sits near zero, within the standard error of 0.95 that a thousand draws at scale 30 leave. The unsigned mean sits near 12, which is the mean of the positive half of a Gaussian of scale 30, and no number of repetitions brings it down. A table of empty and near-empty cells released unsigned carries that 12 in every cell.

7.7. Which noise to choose

The choice comes down to what you release and to whom you must explain it. Table 7.1 gathers the cases of this chapter.

Table 7.1.: Which noise to choose, by what is released.

Situation	Noise and budget	Why
One count, sum or mean, and a guarantee in pure ε	Laplace under `dp.loss_of(epsilon=...)`	The guarantee has no δ and is the easiest to state
A grouped table where a person touches many groups	Gaussian under `dp.loss_of(rho=..., delta=...)` with a `Bound` per group	The L_2 sensitivity grows with the root of the number of groups
Many queries composed into one release	Gaussian under zCDP	Budgets in ρ add up tightly and convert once at the end
A release that others will analyse assuming normal errors	Gaussian	The noise is normal, so standard intervals apply
A guarantee that must be reported as (ε, δ)	Either; convert ρ with `make_zCDP_to_approxDP` and `make_fix_delta`	OpenDP's conversion is tighter than the closed form
Counts that anyone might add up	Either, with `signed=True`	Clamping at zero biases totals upward

Two cautions go with the table. Gaussian noise cannot satisfy pure DP, so a policy that demands $\delta = 0$ rules it out whatever the sensitivity. And the gain from the L_2 sensitivity exists only when the library knows how a person's records spread across groups. So the per-group bound is part of the choice, not an optional extra.

7.8. The releases of one run

The script ends by releasing the count under both budgets and the quarterly table under the bounded zCDP context, and reads each against its error bar.

```
released (one run):
  laplace count 3,811,867  (true 3,811,932, within 539.7 with probability 0.95)
  gaussian count 3,811,994  (true 3,811,932, within 354.0 with probability 0.95)
```

```
2005 Q1 count 87,442  (true 87,372, within 60)
2005 Q2 count 86,457  (true 86,469, within 60)
2005 Q3 count 83,536  (true 83,518, within 60)
2005 Q4 count 84,746  (true 84,739, within 60)
groups released 36 of 36; within 60 of the truth: 32
```

Both counts landed well inside their bars, as they should about nineteen times in twenty. Of the 36 quarterly counts, 32 landed within their bar. The first quarter of 2005 is one of the misses. That is what an error bar at $\alpha = 0.05$ means across a table: about one cell in twenty falls outside it, and you cannot tell which. Comparing with the truth is possible here only because the survey is public. On your own data you make these choices on the scales and accuracies, which cost nothing to read, and release once.

7.9. Summary

Table 7.2.: The calls and concepts of this chapter, what each does, and the rule of thumb that goes with it.

Call or concept	What it does	Rule of thumb
`summarize(alpha=0.05)`	The scale and the 95% accuracy of each statistic, before release	Read it before `release()`; it costs nothing
`laplacian_scale_to_accuracy`, `gaussian_scale_to_accuracy`	A continuous scale to a 95% error bar	For sums and means of floats
`discrete_laplacian_scale_to_accuracy`, `discrete_gaussian_scale_to_accuracy`	The same for integer noise	For counts, to match `summarize`
`accuracy_to_laplacian_scale` and the three others	A target error bar to the scale it needs	Divide the sensitivity by the scale to see the ε it costs
The error bar	Covers the noise only	Clipping, imputation, truncation and resizing bias the input, and the bar does not see it
`dp.loss_of(epsilon=...)`	Pure DP; the context adds Laplace noise	Halving the bar doubles ε
`dp.loss_of(rho=...)`	zCDP; the context adds Gaussian noise	Halving the bar quadruples ρ
`dp.polars.Bound(by=[...], per_group=1)`	One record per person per group	Cuts the L_2 sensitivity to the root of the number of groups; does nothing for Laplace

Call or concept	What it does	Rule of thumb
`make_zCDP_to_approxDP` then `make_fix_delta`	ρ to (ε, δ), tighter than the closed form	Apply to `context.accountant` for the whole release
`signed=True`	Keeps negative noisy counts	Always, if the counts may be summed; clamp for display afterwards

7.10. References

- OpenDP documentation: "Assessing Utility" in Getting Started, "Gaussian Noise" in the Polars user guide, and "Aggregation" in the Polars expressions guide for `signed`. The OpenDP Cookbook's "Signed Counts: Why Clipping at Zero Biases Totals" is the experiment the signed-counts section summarizes.
- Bun and Steinke, "Concentrated Differential Privacy: Simplifications, Extensions, and Lower Bounds" (2016): zCDP, the Gaussian mechanism under it, and the conversion to (ε, δ).
- Canonne, Kamath and Steinke, "The Discrete Gaussian for Differential Privacy" (2020): the integer Gaussian that the context adds to counts and its tail bounds.
- Kamath, Mouzakis, Regehr, Singhal and Steinke, "A Bias-Variance-Privacy Trilemma for Statistical Estimation" (2023): why the bias of clipping cannot be removed in general, cited by the documentation's note on accuracy.
- The full edition's chapters on the Laplace and Gaussian mechanisms in the full edition, concentrated DP in the full edition and conversions to (ε, δ) in the full edition derive the results this chapter states.

Part III.

The framework

Everything the Context API did in the previous parts was built from a small number of pieces, and this part opens the box. OpenDP's framework describes a private computation as a chain of two kinds of object. A transformation maps data to data, and it promises how much its output can change when its input changes, which is its stability. A measurement maps data to a random release, and it promises how much the distribution of that release can change, which is its privacy. Chaining the two multiplies the promises through, and composing measurements adds them up. Once you can read these objects, you can see exactly what guarantee any query carries. You can also build queries the Context API does not offer, and check a claim someone else makes.

A framework to understand differential privacy introduces the vocabulary, namely domains, metrics and measures, and the two maps, the stability map and the privacy map. Transformations builds the preprocessing and aggregation steps, from clamping and resizing to sums, means and quantile scores. Measurements covers the mechanisms that add noise or select, from Laplace and Gaussian to thresholded noise, noisy max and randomized response. Combinators chains and composes them, converts between privacy measures, and shows the adaptive compositors and odometers behind a context. Utilities and the Context API inside covers the type descriptors, the parameter search that finds a scale from a budget, and serialization. It ends by showing how the Context API maps onto the framework.

Table 7.3.: The chapters of this part and the questions they answer.

Chapter	Question it answers
A framework to understand differential privacy	What are domains, metrics, measures, transformations and measurements, and what does each promise?
Transformations	How do I build and chain the data-to-data steps, and how does stability flow through them?
Measurements	Which mechanisms does OpenDP offer, and how do I calibrate each one?
Combinators	How do I compose measurements, convert between privacy measures, and account adaptively?
Utilities and the Context API inside	How do types, parameter search and serialization work, and what does a context do under the hood?

By the end of this part you will be able to build a private query from its pieces and read off the guarantee of any chain with `map` and `check`. You will also be able to explain what the Context API does when it resolves a query.

8. A framework to understand differential privacy

Every query you released in the first two parts went through the Context API. It chose a noise scale, spent a share of the budget and refused the query that went over. This chapter opens that box. Underneath, OpenDP describes a private computation with two kinds of object. A transformation maps data to data, and it promises how far apart its outputs can be when its inputs are close. A measurement maps data to a random release, and it promises how far apart the distributions of its outputs can be. Each promise is a map from an input distance to an output distance. Differential privacy (DP) in OpenDP is the bookkeeping of such maps.

The chapter follows the documentation's Framework API pages. You will meet the two objects and their fields, then the supporting elements one at a time, with code for each. Domains say what the data may be. Metrics say how two datasets or two numbers are compared, and measures say how two distributions of releases are compared. The maps tie them together, and chaining a transformation with a measurement carries the guarantee from the data to the release. The chapter ends by redoing the first chapter's workflow with the framework API next to the Context API, which gives the same scales. By the end you will be able to read the guarantee of any OpenDP object off its fields and its map.

The script opendp_framework.py runs every step on OpenDP 0.16.0. The vector of the sum example is the `HWUSUAL` column of the Labour Force Survey (LFS) sample, collected to a list of integers. The workflow runs on the first chapter's hundred toy scores. The domains, metrics, measures, maps, scales and error bars the script prints are the same on every run. The released values are not, because OpenDP's noise cannot be seeded, so the page shows them in one block marked as one run.

```
uv run python code/part03-framework/opendp_framework.py
```

8.1. Two kinds of object

The first chapter built a Laplace measurement from a space and a constructor without saying what came back. The documentation's running example is the same mechanism with a scale of 5. It is built with `make_laplace`, which takes the domain and the metric as its first two arguments instead of through `>>`.

```
def laplace_measurement(scale: float) -> dp.Measurement:
    """Laplace noise of the given scale on one finite float, compared by absolute distance,
    return dp.m.make_laplace(
        dp.atom_domain(T=float, nan=False),  # input domain: any float that is not NaN
        dp.absolute_distance(T=float),  # input metric: |a - b|, the units of the sensitivit
```

```
        scale=scale,
    )
```

What comes back is a `Measurement`, and the script reads its fields.

```
laplace scale 5: type=Measurement
laplace scale 5: input domain=AtomDomain(T=f64)
laplace scale 5: input metric=AbsoluteDistance(f64)
laplace scale 5: output measure=MaxDivergence
laplace scale 5: function=Function
```

A transformation and a measurement share four fields. The `input_domain` is the set of values the function accepts, and the `output_domain` is the set it produces. The `function` is the computation itself, and the `input_metric` is the rule for the distance between two inputs. The two objects differ in their last two fields, because a **transformation** is deterministic while a **measurement** is randomized. A transformation's outputs are data again, so it has an `output_metric` and a **stability map**, which bounds the distance between two outputs given a bound on the distance between the inputs. A measurement's output is a draw from a distribution, so it has an `output_measure` and a **privacy map**, which bounds the distance between two output distributions in the same way.

Read the Laplace measurement's fields in those terms. It accepts one finite float and compares two inputs by their absolute difference. It measures the gap between two output distributions by the max divergence, which is the quantity that ε bounds. The documentation says that a transformation is (`d_in`, `d_out`)-stable when inputs at most `d_in` apart give outputs at most `d_out` apart. A measurement is (`d_in`, `d_out`)-differentially private when inputs at most `d_in` apart give output distributions at most `d_out` apart. Both sentences have the same shape, and that is the point of the framework. Every element of either object is one ingredient of that sentence, and the rest of the chapter takes them one at a time.

8.2. Functions

The function is the part you would have written anyway. A transformation or a measurement is called like a function, and `invoke` does the same thing under a name. The clamp below is built on the space of vectors of finite floats under the symmetric distance. Its bounds are 0 to 5, and it clips every element into them.

```
import opendp.prelude as dp

dp.enable_features("contrib")
space = dp.vector_domain(dp.atom_domain(T=float, nan=False)), dp.symmetric_distance()
clamp = space >> dp.t.then_clamp(bounds=(0.0, 5.0))
print(type(clamp).__name__)  # Transformation
print(clamp([1.3, 7.8, -2.5, 7.0]))  # [1.3, 5.0, 0.0, 5.0]
print(clamp.invoke([10.0]))  # [5.0]
print(type(clamp.function).__name__)  # Function
```

The `function` field holds the computation as an object of its own, which the plugin chapter of the next part uses to wrap a Python function. Calling a transformation gives you exactly what the function computes, with no noise, so the result is not private. That is by design. A transformation prepares or aggregates the data inside the guarantee, and only a measurement may release anything. So a transformation has to be chained with a measurement before its output leaves the box. If you ever find yourself printing the output of a transformation, you have stepped outside the guarantee.

8.3. Domains

A domain is a set of values, and the library uses it in two ways. The proof behind a map holds only for inputs in the domain, so the domain states the assumptions the map relies on. And when two objects are chained, the library checks that the first one's output domain equals the second one's input domain. That way every intermediate value is one the next function was proved on. The most common domain is `atom_domain`, which describes one value of a type `T`, and `vector_domain` lifts it to vectors of such values.

```
import opendp.prelude as dp

floats = dp.atom_domain(T=float, nan=False)
print(floats)  # AtomDomain(T=f64)
print(floats.member(1.0), floats.member(float("nan")))  # True False
print(dp.atom_domain(T=float))  # AtomDomain(nan=true, T=f64)
small = dp.atom_domain(bounds=(-2, 2))
print(small)  # AtomDomain(bounds=[-2, 2], T=i32)
print(small.member(-2), small.member(3))  # True False
pairs = dp.vector_domain(dp.atom_domain(T=bool), size=2)
print(pairs)  # VectorDomain(AtomDomain(T=bool), size=2)
print(pairs.member([True, True]), pairs.member([True, True, True]))  # True False
```

The `member` method tests whether a value belongs to the domain, which is how you check an assumption by hand. Three arguments shape an atom domain. The type `T` is written in Python terms, where `float` means a 64-bit float and `int` a 32-bit integer. That is why the printed forms say `f64` and `i32`. The `nan` flag says whether a float may be NaN. The installed version prints the flag only when it is true, so `atom_domain(T=float)` allows NaN and `atom_domain(T=float, nan=False)` does not. A measurement that adds noise needs the second, because the absolute distance between two values is not defined when one of them is NaN.

The `bounds` restrict the values to an interval. A domain with bounds is what the sum transformation demands, because the sum of unbounded values has no finite sensitivity. A vector domain adds a `size`, which fixes the length of the vector. The mean transformation needs it, for the same reason that the first chapter's mean needed a resize.

8.4. Metrics

A metric is a rule for the distance between two members of a domain, and the framework never evaluates it. It only uses the metric to give `d_in` its meaning. Under the **symmetric distance**, two datasets are compared by the number of additions and removals that turn one into the other, counted with multiplicity. The function below computes it for two small vectors from the documentation's notebook, where the second lacks the first person's record.

```
from collections import Counter

import opendp.prelude as dp


def symmetric_distance(u: list[int], v: list[int]) -> int:
    """The number of additions and removals that turn the multiset u into v."""
    cu, cv = Counter(u), Counter(v)
    return sum(((cu - cv) + (cv - cu)).values())


print(symmetric_distance([12, 10, 8, 7], [10, 8, 7]))  # 1
print(symmetric_distance([12, 10, 8, 7], [10, 10, 8, 7]))  # 2
print(dp.symmetric_distance(), dp.absolute_distance(T=float))  # SymmetricDistance() Absolut
print(dp.unit_of(contributions=1, ordered=True))  # (InsertDeleteDistance(), 1)
print(dp.unit_of(changes=1))  # (ChangeOneDistance(), 1)
```

Removing one record gives a distance of 1. Changing a 12 into a 10 gives 2, because a change is one removal and one addition. So under the symmetric distance, `d_in` is the largest number of records one person can add or remove. That is the privacy unit of the first chapter in the framework's words. Two datasets are **neighbouring**, written $D \sim D'$, when they are at most `d_in` apart under the input metric, and here that relation is add/remove. The symmetric distance is the metric behind `dp.unit_of(contributions=k)`. A person who answers the LFS at most 36 times is at most 36 away from a dataset without them.

The **absolute distance** compares two numbers by the absolute value of their difference. It is the metric of an aggregate, so under it `d_in` is a **sensitivity**, the most an aggregate can change when one person's records change. You rarely supply a sensitivity yourself. It appears midway through a chain, as the output distance of an aggregation and the input distance of the noise. The full edition's chapter on sensitivity in the full edition works out the sensitivities of the common aggregates.

The other dataset metrics are variations on the symmetric distance. The insert-delete distance counts additions and removals at specific positions, so it also protects the order of the records. The change-one distance counts replaced records in datasets of a fixed size, which is the replace-one relation that other books call bounded DP. The Hamming distance counts the positions at which two vectors of the same length differ, so it is the ordered version of change-one.

8.5. Measures

A measure is a rule for the distance between two probability distributions. It is what makes a measurement's `d_out` a privacy guarantee. The **max divergence** is the largest value, over every set of outputs, of the log of the ratio of the two probabilities of that set. Bounding it by ε is pure differential privacy, which the full edition defines in its chapter on the definition in the full edition. The library offers four measures, and `d_out` has a different form under each.

```
import opendp.prelude as dp

dp.enable_features("contrib")
print(dp.max_divergence(), dp.zero_concentrated_divergence())  # MaxDivergence ZeroConcentra
print(dp.smoothed_max_divergence(), dp.fixed_smoothed_max_divergence())  # SmoothedMaxDiverg
space = dp.atom_domain(T=float, nan=False), dp.absolute_distance(T=float)
gaussian = space >> dp.m.then_gaussian(scale=1.0)
print(gaussian.output_measure, gaussian.map(1.0))  # ZeroConcentratedDivergence 0.5
profile = dp.c.make_zCDP_to_approxDP(gaussian).map(1.0)
print(f"{profile.epsilon(delta=1e-7):.4f}")  # 5.6709
print(dp.c.make_fix_delta(dp.c.make_zCDP_to_approxDP(gaussian), delta=1e-7).map(1.0))  # (5.
```

Under the max divergence, `d_out` is a single number, ε. Under the **zero-concentrated divergence**, `d_out` is the parameter ρ of zero-concentrated differential privacy (zCDP). That measure bounds a family of Rényi divergences at once, and it is the native measure of the Gaussian mechanism. A Gaussian of scale 1 at sensitivity 1 costs $\rho = 0.5$, which is the sensitivity squared over twice the scale squared.

The two smoothed measures carry approximate differential privacy, whose guarantee holds up to a slack δ. Under the smoothed max divergence, `d_out` is a whole curve, a **privacy profile** that gives the smallest ε for any δ you ask for, and the snippet asks it for $\delta = 10^{-7}$. Under the fixed smoothed max divergence, `d_out` is instead one pair of ε and δ. The installed version prints that measure as `Approximate(MaxDivergence)`, and the first chapter's `dp.loss_of(epsilon=1.0, delta=1e-7)` is a budget under it. The combinator `make_fix_delta` turns the curve into the pair by choosing the δ. The conversions between the measures are the subject of Combinators. Here the point is only that a measurement's `output_measure` tells you how to read its `d_out`.

8.6. Stability maps and privacy maps

A map is the function that turns an input distance into an output distance, and it is where the guarantee lives. For a measurement, `map(d_in)` returns the smallest `d_out` the library can prove. The measurement is then (`d_in`, `d_out`)-differentially private. For a transformation, the same call returns the smallest `d_out` for which it is (`d_in`, `d_out`)-stable. The `check` method compares the map's value with a `d_out` you propose and answers whether the guarantee fits inside it. The Laplace measurement of the first section is the running example, and the script reads its map at two sensitivities.

```
epsilon_at_1 = laplace.map(d_in=1.0)  # the sensitivity over the scale: 1 / 5
epsilon_at_10 = laplace.map(d_in=10.0)
fits = laplace.check(d_in=1.0, d_out=0.2)  # is map(1.0) <= 0.2?
too_tight = laplace.check(d_in=1.0, d_out=0.19)
```

```
laplace scale 5: map(1.0)=0.2 map(10.0)=2.0
laplace scale 5: check(1.0, 0.2)=True check(1.0, 0.19)=False
```

The Laplace mechanism's privacy map is the sensitivity divided by the scale, because the scale that gives ε is the sensitivity over ε. So a sensitivity of 1 at scale 5 costs $\varepsilon = 0.2$. A sensitivity ten times larger costs ten times more. The check at $\varepsilon = 0.2$ passes and the check at 0.19 fails, which is all `check` ever does. It is also all the Context API does when it searches for a scale. It tries scales until `check` passes at the query's share of the budget. The full edition derives the map in its chapter on the Laplace and Gaussian mechanisms in the full edition.

The value a map returns is an upper bound that the library can prove, not always the smallest true one, and the sum shows why that matters.

```
import opendp.prelude as dp

dp.enable_features("contrib")
float_sum = dp.t.make_sum(dp.vector_domain(dp.atom_domain(bounds=(0.0, 5.0))), dp.symmetric_
int_sum = dp.t.make_sum(dp.vector_domain(dp.atom_domain(bounds=(0, 5))), dp.symmetric_distan
print(float_sum([1.3, 3.8, 0.0, 5.0]))  # 10.1
print(float_sum.map(d_in=2))  # 10.00000004656613
print(int_sum.map(d_in=2))  # 10
print(int_sum.check(d_in=2, d_out=10), float_sum.check(d_in=2, d_out=10.0))  # True False
```

A person who contributes two records of at most 5 can move the sum by 10, and the integer sum's stability map says exactly that. The float sum's map says a little more than 10. Adding floats rounds, and a proof that covers the rounding must give the rounding its share. That extra is tiny, but `check` is a comparison. So the float sum fails at a `d_out` of exactly 10 while the integer sum passes. The lesson is to let the maps produce the distances rather than typing in the number you know from theory. The workflow section shows the same lesson once more, for a scale.

8.7. Chaining a transformation and a measurement

A chain is where the framework pays off. Suppose a transformation's output domain and output metric equal a measurement's input domain and input metric. Then the `>>` operator joins them into a new measurement. Its function applies the transformation and then the measurement. Its input domain and metric are the transformation's, and its output measure is the measurement's. Its privacy map is the measurement's map applied to the transformation's map. The script builds the documentation's chain of a clamp, a sum and Laplace noise on the LFS. The vector is the `HWUSUAL` column, the hours a respondent usually works per week. The nulls and the code 99 for "not applicable" are dropped, which leaves 1,496,192 whole numbers of hours from 0 to 80.

```
clamp = dp.t.make_clamp(dp.vector_domain(dp.atom_domain(T=int)), dp.symmetric_distance(), bo
bounded_sum = dp.t.make_sum(clamp.output_domain, clamp.output_metric)  # the clamp's output
laplace_int = dp.m.make_laplace(bounded_sum.output_domain, bounded_sum.output_metric, scale=
noisy_sum = clamp >> bounded_sum >> laplace_int  # the chain: its input space is the clamp's
sum_sensitivity_1 = bounded_sum.map(d_in=1)  # one row of at most 80 hours
sum_sensitivity_36 = bounded_sum.map(d_in=CONTRIBUTIONS)  # 36 rows
epsilon_at_scale_5 = noisy_sum.map(d_in=CONTRIBUTIONS)
```

Each constructor takes the previous object's output space as its input space, so the domains match by construction. The clamp turns a vector of any integers into a vector of integers in the public bounds 0 to 80, and the sum accepts exactly that domain. The script prints the fields along the chain and the maps.

```
clamp: input domain=VectorDomain(AtomDomain(T=i32))
clamp: output domain=VectorDomain(AtomDomain(bounds=[0, 80], T=i32))  map(36)=36
sum: output domain=AtomDomain(T=i32)  output metric=AbsoluteDistance(i32)
sum: map(1)=80 map(36)=2880
chain scale 5: input domain=VectorDomain(AtomDomain(T=i32))
chain scale 5: input metric=SymmetricDistance()  output measure=MaxDivergence
chain scale 5: map(36)=576.0
```

Read the distances from left to right. The clamp is 1-stable, since clipping values changes no record's count, so 36 records in give 36 records out. The sum's stability map turns a dataset distance into a sensitivity. One record of at most 80 hours moves the sum by at most 80. A person with 36 records moves it by 36 times as much, which is the 2880 printed. The chain's privacy map is the Laplace map applied to that sensitivity, and at a scale of 5 it says $\varepsilon = 576$. That is no privacy at all. The scale was chosen for one float in the documentation's example, not for a sum over 36 records of 80 hours. The map tells you so before anything is released.

```
sum_scale = dp.binary_search_param(
    lambda scale: clamp >> bounded_sum >> dp.m.then_laplace(scale), d_in=CONTRIBUTIONS, d_ou
)
noisy_sum = clamp >> bounded_sum >> dp.m.then_laplace(sum_scale)
sum_accuracy = dp.discrete_laplacian_scale_to_accuracy(scale=sum_scale, alpha=0.05)
released_sum = noisy_sum(hours)
```

```
chain for epsilon 1 at d_in 36: scale=2880.0 map(36)=1.0
chain for epsilon 1 at d_in 36: 95% accuracy=8,628.2 (0.01532% of the sum)
```

The right scale comes from the map, not from arithmetic. The utility `binary_search_param` takes a function from a scale to a chain. It searches for the largest scale whose `check` passes at the given distances, and it returns 2880, the sensitivity over $\varepsilon = 1$. Here `then_laplace` replaces `make_laplace`, since a `then_` constructor takes its input space from the object before it in the chain. The sum is an integer, so the noise is discrete Laplace and the error bar comes from the discrete accuracy function. An error of about 8,600 hours on a sum of 56 million hours is tiny. Sums over large datasets are the easy case in differential privacy. The release itself is below, in

the block of one run. Transformations works out the stability of each transformation the library offers, and Measurements does the same for the mechanisms.

8.8. The typical workflow in both APIs

The Context API is a layer over the framework, so the first chapter's five steps can be written in either. The script does both on the same hundred toy scores as A first release, drawn by the same seeded generator. So the scales can be compared with the ones you saw there. The first two steps are the privacy unit and the budget. The framework states the first as a distance and a metric, and the second as a distance and a measure.

```
d_in = 1  # neighbouring datasets differ by at most one row...
input_metric = dp.symmetric_distance()  # ...counted as additions and removals
input_domain = dp.vector_domain(dp.atom_domain(T=float))  # a vector of floats, NaN allowed
d_out = EPSILON  # the distributions of two releases differ by at most epsilon...
privacy_measure = dp.max_divergence()  # ...measured by the max divergence, which is pure DP
```

```
unit: d_in=1 input metric=SymmetricDistance()  (Context API: (SymmetricDistance(), 1))
loss: d_out=1.0 output measure=MaxDivergence  (Context API: (MaxDivergence, 1.0))
input domain: VectorDomain(AtomDomain(nan=true, T=f64))
```

The pairs that `dp.unit_of` and `dp.loss_of` returned in the first chapter are these same objects. The framework also asks for the input domain up front, which the context inferred from the list you handed it. The domain allows NaN because a list of floats may contain one. The third step, the public bounds and the imputed value, is identical in both. The fourth step is to mediate access, and here the framework shows what a context is made of.

```
compositor = dp.c.make_adaptive_composition(
    input_domain=input_domain,
    input_metric=input_metric,
    output_measure=privacy_measure,
    d_in=d_in,
    d_mids=[d_out / QUERIES] * QUERIES,  # the shares of the budget, one per query
)
queryable = compositor(data)  # a measurement; invoking it on the data gives the queryable t
```

```
compositor: type=Measurement map(1)=1.0
queryable: type=Queryable
context accountant: type=Measurement d_mids=[0.3333, 0.3333, 0.3333]
```

An **adaptive composition** is itself a measurement. Its privacy map at `d_in` is the sum of the shares, because the budgets of sequential releases add up. That is the composition property that the full edition's chapter on properties proves. Invoking the composition on the data does not release anything. It returns a **queryable**, an object that holds the data and answers measurements submitted to it, one share at a time. The composition is adaptive because each measurement

may depend on the answers before it. The context's `accountant` field is this same object, and its `d_mids` are the same three shares. The fifth step submits the queries, and each is built by hand.

```
count = dp.t.make_count(input_domain, input_metric)
count_sensitivity = count.map(d_in)  # adding or removing one row changes a count by 1
count_scale = dp.binary_search_param(
    lambda scale: count >> dp.m.then_laplace(scale), d_in=d_in, d_out=d_out / QUERIES
)
count_measurement = count >> dp.m.then_laplace(count_scale)
dp_count = queryable(count_measurement)
```

```
count: sensitivity=1  output metric=AbsoluteDistance(i32)
count at scale 3.0: map(1)=0.33333333333333337 check(1, 1/3)=False
count by search: scale=3.0000000000000004 check(1, 1/3)=True
count scale: framework=3.0000 context=3.0000 equal=True
count: 95% accuracy=9.4457
```

The count is a transformation whose stability map gives the sensitivity 1, and the Laplace measurement after it needs a scale. You know from theory that the scale is the sensitivity over a third of ε, which is 3. Typing in 3.0 fails the check, because one over three in floating point lands one rounding step above the share. The binary search returns a scale one rounding step above 3, which passes. It is bit for bit the scale that `context.query().count().laplace().param()` returned in the first chapter, because the context runs the same search. Submitting the measurement to the queryable spends the first share and returns the noisy count. The mean follows the first chapter's recipe, with the released count fixing the size.

```
mean = (
    dp.t.make_impute_constant(input_domain, input_metric, IMPUTED)  # NaN becomes the public
    >> dp.t.then_clamp(BOUNDS)
    >> dp.t.then_resize(size=dp_count, constant=IMPUTED)  # the released count fixes the siz
    >> dp.t.then_mean()
)
mean_scale = dp.binary_search_param(
    lambda scale: mean >> dp.m.then_laplace(scale), d_in=d_in, d_out=d_out / QUERIES
)
dp_mean = queryable(mean >> dp.m.then_laplace(mean_scale))
```

```
mean: output domain=AtomDomain(T=f64)  output metric=AbsoluteDistance(f64)
mean at size 100: sensitivity=1.0000
mean scale at size 100: framework=3.0000 context=3.0000 equal=True
mean at size 100: 95% accuracy=8.9872
mean scale at the released count: equal=True
```

Four transformations chain into one, from a vector of floats with NaN allowed to a single float. The stability of the whole is the maps of the parts composed. At the true size of 100, one row moves the mean by at most 1, so the scale is again 3. The context gives the same scale, both at

the true size and at the released count. The scale of the mean changes with the released count, so the page quotes the one at the true size, which is a development-time number on a public toy. A third query takes the last share, and a fourth is refused.

```
try:
    queryable(count_measurement)
    refusal = "released"
except dp.OpenDPException as err:
    refusal = f"{err.variant}: {err.message}"
```

```
fourth query: FailedFunction: out of queries
```

The queryable raises the library's own exception, with the variant `FailedFunction` and the message "out of queries". The context wrapped the same refusal in a `ValueError` about an exhausted allowance. The queryable also refuses a single query whose map exceeds its share, before it touches the data. What the framework version makes plain is that a context is three objects you can build yourself, namely a space, an adaptive composition over that space, and a queryable. So Utilities and the Context API inside follows a query through the context to show where each is used. Finally the releases of one run, read against their error bars.

```
released (one run):
  laplace scale 5 on 0.0: 12.84  (within 15.0 with probability 0.95)
  hours sum 56,316,146  (true 56,315,594, within 8,628 with probability 0.95)
  count 99  (true 100, within 9.4 with probability 0.95)
  mean scale 3.0303  mean 50.43  (true 50.93)
  third query, a second count 104
```

The Laplace draw at scale 5 landed within its bar of about 15, and the sum of hours within its bar of about 8,600. The count and the mean landed within theirs. The comparison with the truth is possible only because the LFS sample and the toy scores are public. On real data you would read the error bars, which cost nothing, and release once.

8.9. Summary

Table 8.1.: The objects and elements of the framework, what each does, and the rule of thumb that goes with it.

Call or concept	What it does	Rule of thumb
`Transformation`	A deterministic map from data to data, with an `output_metric` and a stability map	Its output is not private; chain it with a measurement before anything leaves

Call or concept	What it does	Rule of thumb
`Measurement`	A randomized map from data to a release, with an `output_measure` and a privacy map	Its `map(d_in)` is the guarantee; read it before releasing
`trans(x)`, `trans.invoke(x)`	Evaluate the `function` field	Printing a transformation's output steps outside the guarantee
`atom_domain(T, bounds, nan)`	One value of type `T`, optionally bounded, with or without NaN	Noise needs `nan=False`; a sum needs `bounds`; the printed form shows `nan` only when true
`vector_domain(domain, size)`	Vectors of a domain, optionally of fixed length	A mean needs the `size`; `member` tests a value by hand
`symmetric_distance()`	Additions and removals, with multiplicity	`d_in` is the records one person can add or remove
`absolute_distance(T)`	The difference of two numbers	`d_in` is a sensitivity; it appears midway through a chain
`insert_delete`, `change_one`, `hamming`	Ordered add/remove, replace-one, ordered replace-one	Pick the one that matches how your data can change
`max_divergence()`	Pure differential privacy; `d_out` is ε	The measure behind `loss_of(epsilon=...)`
`zero_concentrated_divergence()`	zCDP; `d_out` is ρ	The Gaussian mechanism's native measure
`smoothed_max_divergence()`, fixed	Approximate DP; `d_out` is a privacy profile, or one (ε, δ) pair	`make_fix_delta` turns the curve into the pair
`obj.map(d_in)`, `obj.check(d_in, d_out)`	The smallest provable `d_out`; whether a proposed one fits	Let the map produce the distance; a hand-typed 3.0 fails by one rounding step
`trans >> meas`	A measurement whose map composes the two maps	Domains and metrics must match; `then_` constructors match them for you
`binary_search_param`	The largest scale whose chain passes `check`	What the Context API runs for `param()`
`make_adaptive_composition`	A measurement whose invocation returns a queryable	A context is this, over a space, plus the queryable

8.10. References

- OpenDP documentation: “A Framework to Understand DP”, “Differential Privacy with OpenDP”, “Core Structures” and “Supporting Elements” in the Framework API user guide, and “API Comparison”.
- Gaboardi, Hay and Vadhan, “A Programming Framework for OpenDP” (2020): the framework of domains, metrics, measures, stability maps and privacy maps that the library implements.
- Dwork, McSherry, Nissim and Smith, “Calibrating Noise to Sensitivity in Private Data Analysis” (2006): sensitivity and the Laplace mechanism.
- The full edition’s chapters on the definition in the full edition, post-processing and composition in the full edition, the Laplace and Gaussian mechanisms in the full edition and sensitivity in the full edition derive the results this chapter states.

9. Transformations

Every query of the previous parts began with the data and ended with noise, and in between stood a chain of steps that cast, imputed, clamped, resized and aggregated. In OpenDP each of those steps is a **transformation**, a function from data to data that comes with a promise about how much its output can change when its input changes. A framework to understand differential privacy introduced that promise as the stability map. This chapter builds the transformations themselves, from the preprocessing steps to the sum, the mean and the scores behind a quantile, and shows how the promise travels down a chain.

The idea that holds the chapter together is that a chain is only as good as the match between its links. Each transformation has an input space and an output space, and the output space of one must be exactly the input space of the next. That match is what makes the stability of a chain the product of the stabilities of its parts. It is also why so many constructors exist, because each one takes the data one provable step closer to the shape an aggregator needs. By the end you will be able to build a sum, a mean or a median's scores from the framework alone and read off the sensitivity for your privacy unit. You will also recognize the error OpenDP raises when two spaces do not match.

The script opendp_transformations.py runs every step on OpenDP 0.16.0 with the Labour Force Survey (LFS) sample. Its vector is the `HWUSUAL` column, the hours a respondent usually works, without its nulls and without the code 99 for "not applicable", cast to integers. The sensitivities, scales, accuracies, scores and true values it prints are the same on every run. The released values are not, because OpenDP's noise cannot be seeded, and neither is a float sum that truncates its input by a random sample. Those appear in one block marked as one run. The short snippets use toy vectors instead and print only what is deterministic.

```
uv run python code/part03-framework/opendp_transformations.py
```

9.1. Constructors and chains

Every constructor comes in two forms. The `make_` form takes the input space first, as a domain and a metric, and then the constructor's own arguments, so `make_clamp(input_domain, input_metric, bounds)` is the full call. The `then_` form leaves the space out. It returns a partial constructor, and the `>>` operator fills in the space from whatever stands to its left, either a space written as a tuple or a transformation whose output space is used. The two forms build the same object, and the snippet shows both.

```
import opendp.prelude as dp

dp.enable_features("contrib")
```

```
space = dp.vector_domain(dp.atom_domain(T=int)), dp.symmetric_distance()
clamp = dp.t.make_clamp(*space, bounds=(0, 10))
print(clamp([-5, 3, 50]))  # [0, 3, 10]
print(clamp.output_space)  # (VectorDomain(AtomDomain(bounds=[0, 10], T=i32)), SymmetricDist
total = dp.t.make_sum(*clamp.output_space)
chain = space >> dp.t.then_clamp(bounds=(0, 10)) >> dp.t.then_sum()
print(chain([-5, 3, 50]), (clamp >> total)([-5, 3, 50]))  # 13 13
print(chain.output_space)  # (AtomDomain(T=i32), AbsoluteDistance(i32))
print(clamp.map(1), total.map(1), chain.map(1))  # 1 10 10
```

Clamping changes the domain and not the metric. The input domain was the vectors of integers, and the output domain is the vectors of integers between 0 and 10. The metric is still the symmetric distance, which counts the rows you must add or remove to turn one vector into the other. The sum then takes that output space as its input space, which `make_sum(*clamp.output_space)` says literally, and its own output is a single integer under the absolute distance. Reading `output_space` after each step is the habit this chapter teaches, because the next constructor must accept exactly that space. When it does not, the chain refuses to build, as Section 9.6 shows.

The number that `map` returns is the transformation's **stability**, the largest change in the output for inputs at most `d_in` apart, each change measured in its own metric. For the clamp it is 1, because one added row of the input is one added row of the output. For the sum it is 10, because one added row can add at most the larger bound to the total. For the chain it is the product of the two. When the last step of a chain is an aggregator, its stability is what differential privacy calls the sensitivity of the query, which the full edition's chapter on sensitivity derives for sums, means and histograms. That number is what a measurement needs to calibrate its noise.

9.2. Preprocessing

Preprocessing turns the data into the domain an aggregator accepts, and every aggregator states that domain. The sum wants bounded values, for example, while the mean wants bounded values and a known size. The documentation lists the preprocessing constructors in the order you typically chain them, which is casting, imputing, clamping and resizing, and the subsections below keep that order. All but one of them work row by row, so one added row of the input is one added row of the output and the stability is 1. The exception is the resize.

9.2.1. Casting and imputing

Data read from a file arrive as strings, and `make_cast` turns a vector of strings into a vector of another type. A string that does not parse becomes a missing value, so the output domain is a vector of options, a type that holds either a value or nothing. No aggregator accepts options, for a reason the documentation states plainly. An aggregator that skipped the missing values would understate the sensitivity of the people with values, and one that propagated them would reveal whether anything was missing. So the missing values have to go first. `make_impute_constant` replaces each with a value of your choice, and `make_drop_null` removes the rows. `make_cast_default`

folds the cast and the imputation into one step, with the type's default standing in, which is 0 for numbers.

```
import opendp.prelude as dp

dp.enable_features("contrib")
strings = dp.vector_domain(dp.atom_domain(T=str)), dp.symmetric_distance()
cast = strings >> dp.t.then_cast(TOA=int)
print(cast.output_domain)  # VectorDomain(OptionDomain(AtomDomain(T=i32)))
imputed = cast >> dp.t.then_impute_constant(0)
print(imputed(["1", "2", "x"]))  # [1, 2, 0]
print(imputed.output_domain)  # VectorDomain(AtomDomain(T=i32))
by_default = strings >> dp.t.then_cast_default(TOA=int)
print(by_default(["1", "2", "x"]))  # [1, 2, 0]
floats = dp.vector_domain(dp.atom_domain(T=float)), dp.symmetric_distance()
print(floats[0])  # VectorDomain(AtomDomain(nan=true, T=f64))
uniform = floats >> dp.t.then_impute_uniform_float(bounds=(0.0, 10.0))
print(uniform.output_domain)  # VectorDomain(AtomDomain(T=f64))
print(cast.map(1), imputed.map(1), uniform.map(1))  # 1 1 1
```

Floats carry their own missing value, the not-a-number value, and the domain says whether it is allowed. The plain `atom_domain(T=float)` allows it, which the `nan=true` in the printed domain records. `make_impute_constant` accepts that domain too, and `make_impute_uniform_float` replaces each missing float by a uniform draw from the bounds you pass, so that imputed values do not pile up at one point. Both output the domain without `nan`, which is the one the aggregators want. The stability of every step here is 1, as the last line shows. Preparing microdata does the same preparation on a whole table through Polars, inside the guarantee.

9.2.2. Clamping and resizing

Clamping pins every value into public bounds. A value below the lower bound becomes the lower bound, and a value above the upper becomes the upper. It changes nothing in between, so it costs no accuracy on data that already lie within the bounds, and it is what makes the sum's sensitivity finite. Without a bound, one person could move a sum by any amount, and no finite noise could hide them. The bounds must come from public knowledge, such as a codebook, never from looking at the data, because a bound read off the data leaks it.

Resizing is the step with a stability of 2. It takes a target size and makes every vector that size: a shorter vector is padded with a constant, and a longer one is cut down by a random sample. It exists because some aggregators, the mean and the variance among them, must know how many rows they divide by, and the number of rows is itself private. The resize writes a size into the domain, which from then on OpenDP treats as public, and that size can be a guess, a released count or public knowledge. The stability follows from one case. Add one row to an input that was shorter than the target. The output gains that row and loses one padded row, so two rows changed for one.

```
import opendp.prelude as dp

dp.enable_features("contrib")
floats = dp.vector_domain(dp.atom_domain(T=float, nan=False)), dp.symmetric_distance()
clamp = floats >> dp.t.then_clamp(bounds=(0.0, 10.0))
print(clamp([-5.0, 3.0, 50.0]), clamp.map(1))  # [0.0, 3.0, 10.0] 1
resize = clamp >> dp.t.then_resize(size=5, constant=5.0)
print(resize([1.0, 2.0, 3.0]))  # [1.0, 2.0, 3.0, 5.0, 5.0]
print(resize.output_domain)  # VectorDomain(AtomDomain(bounds=[0.0, 10.0], T=f64), size=5)
print(resize.map(1))  # 2
```

The documentation's resize page shows by simulation what a wrong target does. A target below the true size throws rows away, which widens the spread of the mean that follows. A target above it pads with the constant, which pulls that mean toward it. Neither error appears in an error bar, because the size is treated as true. Nor can the resize warn you, since an error that depended on the true size would leak it. So spend a little budget on a count first, as the previous chapters did, because a count has a sensitivity of 1 and lands close to the truth.

9.2.3. Categories and bins

Categorical data go through three constructors that map between values and indices. `make_find` replaces each value by its position in a category list, and a value outside the list becomes a missing index, so an imputation follows and gives it the next index. `make_find_bin` turns a number into the index of the bin it falls in, given the bin edges, and `make_index` goes the other way, from an index back to a label.

```
import opendp.prelude as dp

dp.enable_features("contrib")
strings = dp.vector_domain(dp.atom_domain(T=str)), dp.symmetric_distance()
find = strings >> dp.t.then_find(categories=["A", "B", "C"]) >> dp.t.then_impute_constant(3)
print(find(["A", "B", "C", "A", "D"]))  # [0, 1, 2, 0, 3]
floats = dp.vector_domain(dp.atom_domain(T=float, nan=False)), dp.symmetric_distance()
bins = floats >> dp.t.then_find_bin(edges=[1.0, 2.0, 10.0])
print(bins([0.0, 1.0, 3.0, 15.0]))  # [0, 1, 2, 3]
label = bins >> dp.t.then_index(categories=["low", "mid", "high"], null="other")
print(label([0.0, 1.0, 3.0, 15.0]))  # ['low', 'mid', 'high', 'other']
counts = strings >> dp.t.then_count_by_categories(categories=["A", "B", "C"])
print(counts(["A", "B", "C", "A", "D"]))  # [2, 1, 1, 1]
print(counts.output_metric, counts.map(1))  # L1Distance(i32) 1
```

The aggregator that goes with them is `make_count_by_categories`. It counts how often each category occurs and appends a count of everything else, unless you turn that off with `null_category=False`. Its output is a vector of counts, so its output metric is a vector norm. The default is the L_1 distance, which chains with Laplace noise, and the `MO` argument switches it to the L_2 distance, which chains with Gaussian noise. Adding one row changes one count by one, so the stability is 1 in either norm.

9.3. The sum

The sum is the aggregator everything else is built on, and its documentation page is a study in where sensitivity comes from. Start with integers in public bounds and no known size.

```
import opendp.prelude as dp

dp.enable_features("contrib")
bounded = dp.vector_domain(dp.atom_domain(bounds=(0, 10))), dp.symmetric_distance()
total = bounded >> dp.t.then_sum()
print(total([1, 2, 4]), total.map(1))  # 7 10
sized = dp.vector_domain(dp.atom_domain(bounds=(-10, 10)), size=3), dp.symmetric_distance()
sized_total = sized >> dp.t.then_sum()
print([sized_total.map(d) for d in range(5)])  # [0, 0, 20, 20, 40]
```

Adding one row changes the sum by at most the larger of the two bounds in absolute value, here 10, and that is what `map(1)` reports. When the dataset's size is public, you write it into the domain and the sum uses a tighter bound. Two datasets of the same size cannot differ by one row. The nearest neighbours differ by one row replaced, which is one removal and one addition, a symmetric distance of 2. A replacement changes the sum by at most the width of the bounds, and that is `map(2)`. Odd distances round down, which gives the staircase in the last line. A distance of 1 maps to 0 because no two datasets of the same size are one row apart.

On the survey the same two calls read as follows. The vector is built once with Polars, and the chain clamps the hours into the codebook's bounds, 0 to 98, and sums them.

```
def hours_usually_worked(frame: pl.LazyFrame) -> list[int]:
    """The `HWUSUAL` column without nulls and without the code 99, as a list of ints."""
    kept = frame.filter(pl.col("HWUSUAL").is_not_null() & (pl.col("HWUSUAL") != 99))
    return kept.select(pl.col("HWUSUAL").cast(pl.Int64)).collect()["HWUSUAL"].to_list()
```

```
def int_sum(size: int | None = None) -> dp.Transformation:
    """Clamp to the public bounds, then sum; with a size the domain says how many rows there
    space = dp.vector_domain(dp.atom_domain(T=int), size=size), dp.symmetric_distance()
    return space >> dp.t.then_clamp(BOUNDS) >> dp.t.then_sum()
```

```
hours vector: 1,496,192 values
true sum: 56,315,594  true mean: 37.6393  true median: 37
int sum: value 56,315,594  map(1)=98  map(36)=3528
sized int sum: map(36)=1764  map(35)=1666
```

The privacy unit of this survey is one person, who appears at most 36 times, so every map on this page is read at an input distance of 36. Without a size the sensitivity is that many added rows of 98 hours each, which is 3,528 hours. With the size in the domain it is 18 replacements, which is half as much, and an odd distance of 35 rounds down to 17 replacements. The Polars filter that builds the vector runs outside OpenDP, as data preparation this chapter does not protect.

Preparing microdata runs the same filter inside the guarantee, and Essential statistics releases this sum through the Context API.

Floats behave differently, for two reasons that the documentation's sum page sets out. The first is rounding. Adding floats rounds each partial sum to the nearest representable number. So two datasets at distance 2 can have sums that differ by a little more than the width of the bounds, and the stability map adds a term for it.

```
import opendp.prelude as dp

dp.enable_features("contrib")
metric = dp.symmetric_distance()
sized = dp.vector_domain(dp.atom_domain(bounds=(-10.0, 10.0)), size=1000), metric
print((sized >> dp.t.then_sum()).map(2))  # 20.00000000004426
unsized = dp.vector_domain(dp.atom_domain(bounds=(-10.0, 10.0))), metric
print((unsized >> dp.t.then_sum()).map(1))  # 20.00000009313226
negative = dp.vector_domain(dp.atom_domain(bounds=(-10.0, 0.0))), metric
print((negative >> dp.t.then_sum()).map(1))  # 10.00000009313226
print(dp.t.make_bounded_float_checked_sum(size_limit=100, bounds=(-10.0, 0.0)).map(1))  # 10
```

With a known size of a thousand rows the term is tiny, in the eleventh decimal place. The second reason appears when the size is unknown. Then the rounding error has no bound, so the float sum first truncates its input to at most 2^{20} rows, a little over a million, by a random sample. Truncation changes the sensitivity analysis, because a neighbouring dataset with one more row may have to drop a random row to stay within the limit. The sensitivity therefore becomes the largest of the magnitudes of the bounds and their width. That doubles it for bounds symmetric about zero, as the second line shows, and changes nothing for bounds of the same sign, as the third line shows. The specialized constructor `make_bounded_float_checked_sum` lets you set the size limit yourself, and a limit of 100 shrinks the rounding term from the eighth decimal place to the twelfth.

The notebook's examples are too small to show why the limit matters. The hours vector has 1,496,192 values, more than 2^{20}. A plain `then_sum` on it as floats keeps a random million of them and sums those. The result is about 70% of the true sum, as the one-run block at the end of the chapter shows. Nothing warns you. The privacy guarantee holds, because the sensitivity is right for the truncated computation, but the number answers a different question from the one you asked. The fix is a size limit above any size the data could have, from public knowledge, here four million rows, or the exact size in the domain when it is public.

```
def float_sum(size_limit: int | None = None) -> dp.Transformation:
    """The float sum: `then_sum` truncates to 2^20 rows; a size limit of your own lifts that
    space = dp.vector_domain(dp.atom_domain(T=float, nan=False)), dp.symmetric_distance()
    clamped = space >> dp.t.then_clamp(FLOAT_BOUNDS)
    if size_limit is None:
        return clamped >> dp.t.then_sum()
    return clamped >> dp.t.make_bounded_float_checked_sum(size_limit=size_limit, bounds=FLOA
```

```
float sum, size limit 2^20=1,048,576: map(36)=3528.0000009127
float sum, size limit 4,000,000: value 56,315,594  map(36)=3528.0000038179
```

Both sensitivities are 3,528 hours plus a rounding term, and the larger limit buys a larger term, in the sixth decimal place rather than the seventh. The term also depends on the summation algorithm, which the specialized constructors take as the `S` argument.

```
import opendp.prelude as dp

dp.enable_features("contrib")
pairwise = dp.t.make_sized_bounded_float_checked_sum(1_000, (0.0, 10.0), S="Pairwise<f64>")
sequential = dp.t.make_sized_bounded_float_checked_sum(1_000, (0.0, 10.0), S="Sequential<f64
print(f"{pairwise.map(0):.2e} {sequential.map(0):.2e}")  # 4.43e-11 4.44e-09
pairwise = dp.t.make_sized_bounded_float_checked_sum(1_000_000, (0.0, 10.0), S="Pairwise<f64
sequential = dp.t.make_sized_bounded_float_checked_sum(1_000_000, (0.0, 10.0), S="Sequential
print(f"{pairwise.map(0):.2e} {sequential.map(0):.2e}")  # 8.85e-08 4.44e-03
```

The map at distance 0 isolates the rounding term, since two datasets with the same rows in a different order can already sum to different floats. The default is the pairwise sum, which adds the values as a balanced tree, so its term grows a little faster than the number of rows. The sequential sum adds the values one after another, and its term grows with the square of the number of rows. At a million rows the sequential term is 0.0044 against the pairwise 0.0000001. The reason to ask for the sequential analysis is a sum computed elsewhere, because database engines add floats in sequence, and a sensitivity claimed for their result must match how they add. Integer sums have no rounding term, which is one reason to cast hours to integers.

To release the integer sum, chain a Laplace measurement onto the chain. Its scale is the sensitivity over ε, which the full edition's chapter on the Laplace and Gaussian mechanisms derives. Each of the three measurements on this page is calibrated to $\varepsilon = 0.5$ on its own, and composing them is the job of Combinators. At that budget the scale is 7,056 hours. The error bar is about 21,000 hours on a total of 56 million, which is 0.04%.

```
sum release: scale=7056  map(36)=0.5000  95% accuracy=21,138.4
```

9.4. The mean and the variance

The mean is a sum divided by a size, and `make_mean` divides by the size in its input domain, not by the length of the data it is given. So the domain must carry a size, and without one the constructor refuses.

```
mean without a size: dataset size must be known. Either specify size in the input domain or
```

```
import opendp.prelude as dp

dp.enable_features("contrib")
space = dp.vector_domain(dp.atom_domain(bounds=(0.0, 10.0)), size=10), dp.symmetric_distance
mean = space >> dp.t.then_mean()
print(mean([5.0] * 10), mean.map(2))  # 5.0 1.0000000000000169
print(mean([5.0]))  # 0.5
print(mean.input_domain.member([5.0]))  # False
```

With ten values between 0 and 10, replacing one value moves the mean by at most the width over the size, which is 1. That is `map(2)`, up to a rounding term. The second print is the warning the documentation attaches to the mean. The mean of a one-element list came out as 0.5, because the division is by the size the domain declares, and nothing checks the data. A transformation assumes its input is a member of its input domain, and `member` checks that.

On the survey the size is not public, so the chain resizes first. The constant pads a vector that comes out short, and here it is the true median of 37 hours. A real release would resize to a released count, as A first release did, or to a public total. The script resizes to the true size, which is allowed here because the data are public, so nothing is padded.

```
def mean_chain(size: int, constant: float) -> dp.Transformation:
    """Clamp, resize to a known size (padding with the constant), then the mean over that si
    space = dp.vector_domain(dp.atom_domain(T=float, nan=False)), dp.symmetric_distance()
    return space >> dp.t.then_clamp(FLOAT_BOUNDS) >> dp.t.then_resize(size=size, constant=co
```

```
mean chain: value 37.6393  map(36)=0.0023580
mean release: scale=0.0047160  map(36)=0.5000  95% accuracy=0.0141
```

Read the map against the sized sum. The sized sum's map was 18 replacements of 98 hours, and dividing that by the size gives 0.0012. The mean's map is twice that, because the resize in front of it is 2-stable. The input distance of 36 becomes an output distance of 72, which the mean reads as twice as many replacements. That is the price of not knowing the size, and it is paid once per chain. At $\varepsilon = 0.5$ the Laplace scale is 0.0047 hours, and the error bar is 0.014 hours.

The documentation's mean page also shows the alternative, which it calls the plug-in approach: release a sum and a count separately and divide. The division is post-processing, so it costs nothing beyond the two releases, by the property that the full edition's chapter on properties proves. The resize page compares the two by simulation and finds the resized mean tighter at the same budget, because its numerator and denominator agree on how many rows there are. The variance takes the same preprocessing and the same kind of map. On the resized chain `then_variance` gives 137.63 hours squared, with a stability of 0.2311 at distance 36, and a Laplace measurement would release it exactly as the mean.

```
variance chain: value 137.6290  map(36)=0.2311
```

9.5. Quantile scores

A quantile is not a sum, and noise alone cannot release it. The documentation's quantile page opens with the reason. Take a dataset in which half the values sit at the lower bound and half at the upper. Its median jumps from the middle of the range to the upper bound when one value is added, so the median's sensitivity is half the range, and Laplace noise at that scale would drown it. Instead OpenDP scores every candidate quantile by how far it is from the truth and lets a selection mechanism pick one. That is the exponential mechanism of the full edition's chapter on selection, and `make_quantile_score_candidates` computes the scores.

```
import opendp.prelude as dp

dp.enable_features("contrib")
space = dp.vector_domain(dp.atom_domain(T=int)), dp.symmetric_distance()
median = space >> dp.t.then_quantile_score_candidates(candidates=[0, 25, 50, 75, 100], alpha
print(median([10, 20, 30, 40, 50, 60, 70, 80, 90]))  # [9, 5, 0, 5, 9]
print(median.output_metric, median.map(1))  # LInfDistance(T=u64) 1
quartile = space >> dp.t.then_quantile_score_candidates(candidates=[0, 25, 50, 75, 100], alp
print(quartile([10, 20, 30, 40, 50, 60, 70, 80, 90]), quartile.map(1))  # [9, 1, 8, 19, 27]
```

The score of a candidate for the median is the difference between the number of values below it and the number above it, in absolute value. In the snippet the candidate in the middle has four values on either side and scores zero, while the lowest candidate has all nine above it. The lowest score marks the best candidate, which is why the selection in the next chapter is told to pick the minimum. For another quantile the two counts get weights. The quarter quantile weights the values below by three and the values above by one, so that the score is zero where a quarter of the data lie below. Its stability rises to 3, because one added row can move a weighted count by that much. The output metric is the L_∞ distance, since one row moves every candidate's score at once, and that is the metric the noisy max measurement expects.

On the survey the candidates are every whole number of hours from 0 to 98.

```
def median_scores() -> dp.Transformation:
    """One score per candidate: how far it is from being the median; the lowest score is the
    space = dp.vector_domain(dp.atom_domain(T=int)), dp.symmetric_distance()
    return space >> dp.t.then_quantile_score_candidates(candidates=CANDIDATES, alpha=0.5)


def median_measurement(scale: float) -> dp.Measurement:
    """Noisy max over the scores releases an index; `negate` makes the lowest score the one
    return median_scores() >> dp.m.then_noisy_max(dp.max_divergence(), scale=scale, negate=T
```

```
median scores: lowest 15,190 at 37 hours, next 89,573 at 36  map(36)=36
median release: scale=144  map(36)=0.5000  score gap=74,383  gap/scale=517
```

The lowest score is at 37 hours, the true median. The next best candidate is one hour less and scores worse by 74,383. The scores' stability at distance 36 is 36. The measurement that

turns scores into a quantile is the noisy max of Measurements. It adds noise to every score and reports the index of the best one. Its privacy map under the max divergence is twice the stability over the scale, so a scale of 144 gives $\varepsilon = 0.5$. The released index names a candidate, which is post-processing. The gap between the two best scores is over five hundred times the scale, so the released median is 37 on essentially every run. The bundled `make_private_quantile` of the next chapter chains these same steps. Its scale is doubled before it reaches the noisy max for the median, so a private quantile at scale 72 is this chain at scale 144.

9.6. Reading a chain

Two methods read a chain's promise. `map` gives the stability, or the privacy loss once a measurement ends the chain, and `check` asks whether the promise at a given input distance stays within a bound. The sum chain's map at 36 is 3,528, so `check(36, 3528)` is true and `check(36, 3527)` is false. Both cost nothing and touch no data. The context of the first chapter runs `check` in a loop to find the largest scale that fits a budget, and Utilities and the Context API inside shows that search.

```
int sum chain: check(36, 3528)=True  check(36, 3527)=False
```

The other thing to read is a refusal. The rule every chain obeys is that the output space of one step is the input space of the next, and `>>` checks it when the chain is built. The sum below is built on the vectors bounded by 0 and 100, then chained after a clamp to 0 and 98.

```
try:
    _ = clamped >> dp.t.make_sum(dp.vector_domain(dp.atom_domain(bounds=(0, 100))), dp.symme
    mismatch = "built"
except dp.OpenDPException as err:
    mismatch = f"{err.variant}: {err.message}"
```

```
mismatched chain: DomainMismatch: Intermediate domains don't match. See https://github.com/o
    output_domain: VectorDomain(AtomDomain(bounds=[0, 98], T=i32))
    input_domain:  VectorDomain(AtomDomain(bounds=[0, 100], T=i32))
```

The message names both spaces. The fix is to build the sum from the clamp's output space, as `make_sum(*clamped.output_space)`, or to use the `then_` form, which does so for you. A metric mismatch reads the same way: an ordered sum needs the insert-delete distance, so it refuses to follow a clamp under the symmetric distance, and `make_ordered_random` between the two converts the metric. The third kind of refusal is a constructor whose input domain lacks what it needs, as the mean showed for a missing size and the sum shows for missing bounds.

```
import opendp.prelude as dp

dp.enable_features("contrib")
space = dp.vector_domain(dp.atom_domain(T=int)), dp.symmetric_distance()
clamp = space >> dp.t.then_clamp(bounds=(0, 10))
```

```
try:
    clamp >> dp.t.make_bounded_int_ordered_sum(bounds=(0, 10))
except dp.OpenDPException as err:
    print(err.variant)  # MetricMismatch
try:
    space >> dp.t.then_sum()
except dp.OpenDPException as err:
    print(err.message)  # `input_domain` must be bounded. Use `make_clamp` to bound data.
```

Every refusal happens at construction, before any data are touched. Finally, the releases of one run, read against their error bars.

```
released (one run):
  sum 56,307,689  (true 56,315,594, within 21,138 with probability 0.95)
  float sum truncated to 2^20 rows 39,467,741  (true 56,315,594; about 70% of it)
  mean 37.6367  (true 37.6393, within 0.0141 with probability 0.95)
  median 37  (true 37)
```

The sum and the mean landed within their bars, as they should about 19 times in 20. The truncated float sum is not a noisy release but a wrong computation, short by 30% with no noise added, and no error bar covers it. The median is exact, as the gap between its scores said it would be.

9.7. Summary

Table 9.1.: The transformations of this chapter, what each does, and the rule of thumb that goes with it.

Call or concept	What it does	Rule of thumb
`make_x(domain, metric, ...)`, `space >> then_x(...)`	Two forms of every constructor	Read `output_space` after each step
`transformation.map(d_in)`	The stability at input distance `d_in`	Read it at the privacy unit's distance, 36 here
`make_cast`, `make_cast_default`	Parse strings into a type; failures become options or the default	Impute or drop options before any aggregator
`make_impute_constant`, `make_impute_uniform_float`	Replace missing values by a constant or a uniform draw from bounds	Stability 1; the constant or bounds are public
`make_clamp(bounds)`	Pin values into public bounds	Bounds come from a codebook, never from the data
`make_resize(size, constant)`	Pad or sample to a known size, written into the domain	Stability 2; a wrong size biases silently

Call or concept	What it does	Rule of thumb
`make_find`, `make_find_bin`, `make_index`, `make_count_by_categories`	Values to indices, numbers to bins, indices to labels, counts per category	Counts carry an L_1 or L_2 metric
`make_sum`	Sum of bounded values; with a size, per replacement	Unsized float sums truncate to 2^{20} rows: set `size_limit`
`make_mean`, `make_variance`	Divide by the size in the domain	A resize before them doubles the input distance
`make_quantile_score_candidates`	A score per candidate, lowest best	Feed the scores to the next chapter's noisy max

9.8. References

- OpenDP documentation: "Transformations", "Preprocessing: Resize", "Aggregation: Sum", "Aggregation: Mean" and "Aggregation: Quantile" in the API user guide.
- Casacuberta, Shoemate, Vadhan and Wagaman, "Widespread Underestimation of Sensitivity in Differentially Private Libraries and How to Fix It" (2022): the rounding term and the truncation of the float sum.
- Smith, "Privacy-Preserving Statistical Estimation with Optimal Convergence Rates" (2011): the quantile scores and their sensitivity.
- McKenna and Sheldon, "Permute-and-Flip: A New Mechanism for Differentially Private Selection" (2020): the selection behind the noisy max.
- The full edition's chapters on sensitivity in the full edition, the Laplace and Gaussian mechanisms in the full edition, post-processing and composition in the full edition and selection in the full edition derive the results this chapter states.

10. Measurements

A transformation moves data to data and promises how far its output can move. A **measurement** is the step that ends the chain. It takes data and returns a random release, and its promise is a bound on how much the distribution of that release can change when one person's data changes. The previous chapter, Transformations, built the deterministic half of a query. This chapter builds the random half, which is where the privacy guarantee actually comes from.

OpenDP ships a small catalogue of measurements, and every one of them is read the same way. Its input domain says what it accepts, and its input metric says how two inputs are compared. Its output measure says in which currency the privacy is counted, and its privacy map turns a distance between inputs into a bound in that currency. Once you can read those four things, you can calibrate any mechanism in the library and compare two mechanisms honestly. The chapter goes through the catalogue in the documentation's order. It starts with the additive noise mechanisms and the canonical noise mechanism. Then come the thresholded mechanisms, which release counts for keys the data chose, and the approximate Laplace projection. It ends with the noisy max family, the private quantile built on it, and randomized response.

The script opendp_measurements.py builds every mechanism on this page on OpenDP 0.16.0 and prints its metric, its measure and its map. Two of them run on the Labour Force Survey (LFS): a thresholded count of the `ILOSTAT` categories and a private median of `HWUSUAL`. The rest take the documentation's toy inputs. Every map, check, threshold, score and error message is the same on every run. The released values are not, because OpenDP's noise cannot be seeded, so they appear in one block marked as one run at the end.

```
uv run python code/part03-framework/opendp_measurements.py
```

10.1. Additive noise

The simplest measurement adds a random draw to a number. OpenDP offers two draws, and the choice between them is a choice of currency. The constructor `make_laplace` adds Laplace noise and counts privacy in the budget ε of pure differential privacy (DP), under the output measure `MaxDivergence`. The constructor `make_gaussian` adds Gaussian noise and counts privacy in the budget ρ of zero-concentrated differential privacy (zCDP), under `ZeroConcentratedDivergence`. Both are built on the same space, so the input domain and the input metric do not decide which you get.

```
def scalar_mechanisms(scale: float) -> tuple[dp.Measurement, dp.Measurement]:
    """Laplace and Gaussian noise on one float; the input space is the same, the output meas
    space = dp.atom_domain(T=float, nan=False), dp.absolute_distance(T=float)
    laplace = space >> dp.m.then_laplace(scale=scale)  # output measure: MaxDivergence, read
```

```
    gaussian = space >> dp.m.then_gaussian(scale=scale)  # output measure: ZeroConcentratedD
    return laplace, gaussian
```

The map of the Laplace mechanism is the sensitivity divided by the scale, so a unit sensitivity at unit scale costs $\varepsilon = 1$. The map of the Gaussian mechanism is the squared sensitivity over twice the squared scale, which at the same inputs gives $\rho = 0.5$. The `check` method compares the map with a budget. A sensitivity of 2 does not fit within the budget that a sensitivity of 1 fits.

```
laplace float: map(1.0)=1.0 measure=MaxDivergence
gaussian float: map(1.0)=0.5 measure=ZeroConcentratedDivergence
laplace float: check(1.0, 1.0)=True check(2.0, 1.0)=False
```

What the two currencies promise is different. Pure DP with parameter ε says that one person changes the probability of any release by a factor of at most e^{ε}. Zero-concentrated DP with parameter ρ bounds the privacy loss on average and in its tails at once, which suits the Gaussian's thin tails. Combinators converts a ρ into an ε with a slack δ. Assessing utility and choosing the noise weighs the two for a release. The full edition's chapter on the Laplace and Gaussian mechanisms derives both maps.

10.1.1. Integers, floats and vectors

The same constructors accept integers. On an integer domain, `make_laplace` draws from the discrete Laplace distribution and `make_gaussian` from the discrete Gaussian, so the release is an integer too. The maps are unchanged, and the sensitivity is an integer as well.

```
laplace int: map(1)=1.0 output type=int
gaussian int: map(1)=0.5
```

A vector input changes the metric. Laplace noise on a vector wants a bound on the L1 distance between two inputs, which is the sum of the absolute differences. That is because its map adds up the cost of every coordinate. Gaussian noise on a vector wants a bound on the L2 distance, the square root of the summed squares, because its cost adds up in squares. The constructor refuses a space whose metric does not match.

```
def vector_mechanisms(scale: float) -> tuple[dp.Measurement, dp.Measurement]:
    """The same two mechanisms on a vector of floats: Laplace wants an L1 bound, Gaussian an
    domain = dp.vector_domain(dp.atom_domain(T=float, nan=False))
    laplace = (domain, dp.l1_distance(T=float)) >> dp.m.then_laplace(scale=scale)
    gaussian = (domain, dp.l2_distance(T=float)) >> dp.m.then_gaussian(scale=scale)
    return laplace, gaussian
```

The maps read as before. Doubling the L2 sensitivity quadruples ρ, which is the squared sensitivity showing through, while the Laplace map stays linear. Passing a plain float to a vector mechanism is a type error, and the message says what was expected.

```
laplace vector: metric=L1Distance(f64) map(1.0)=1.0
gaussian vector: metric=L2Distance(f64) map(1.0)=0.5
gaussian vector: map(2.0)=2.0 map(4.0)=8.0
laplace vector on a scalar: Expected type is Vec<f64> but input data is not a list.
gaussian on an L1 space: No match for concrete type L1Distance<f64>. See https://github.com/
```

10.1.2. Why OpenDP rounds, and the parameter `k`

Floating-point noise is a known source of broken implementations. A computer cannot sample a continuous distribution, and the gaps between floats leak which input was used. OpenDP avoids the problem by never sampling a continuous draw. A float input is turned into an exact rational, integer noise is added to its numerator by the discrete distribution, and the result is rounded back to a float. That is the same as sampling true continuous noise and then rounding, which is post-processing, so the analysis stays exact.

The grid of that rational is 2^k, where the integer `k` is a parameter of the constructor. The default `k` is the smallest exponent a 64-bit float can express, so the grid is exact and costs nothing. A coarser grid samples faster. But rounding the input to the grid can move two neighbouring inputs apart by up to one extra grid step, so the map adds that step to the sensitivity. The script shows the penalty. At the default there is none, while a grid of a half adds a half to every map, even at zero sensitivity.

```
laplace k=-1074: map(0.0)=0.0 map(1.0)=1.0
laplace k=-1: map(0.0)=0.5 map(1.0)=1.5
laplace k=0: map(0.0)=1.0 map(1.0)=2.0
laplace k=1: map(0.0)=2.0 map(1.0)=3.0
laplace k on integers: k is only valid for domains over floats
```

Integers need no grid, so the constructor rejects `k` on an integer domain. Leave `k` alone unless a vector release is too slow, and if you change it, read the map again. The full edition explains the discrete distributions and why they are safe in its chapter on discrete mechanisms in the full edition.

10.1.3. The geometric mechanism and `make_noise`

The constructor `make_geometric` is the Laplace mechanism restricted to integers, with one extra option. Passing `bounds` makes the sampler run in constant time, which closes a timing side channel, and it clamps the release into the bounds. On floats the constructor fails, because the geometric distribution has no float version.

```
geometric = int_space >> dp.m.then_geometric(scale=1.0)
bounded = int_space >> dp.m.then_geometric(scale=1.0, bounds=(0, 10))  # runs in constant ti
```

```
geometric: map(1)=1.0 measure=MaxDivergence
geometric bounds (0, 10): map(1)=1.0
geometric on floats: No match for concrete type f64. See https://github.com/opendp/opendp/di
```

Finally, `make_noise` is the general constructor behind the two. You name the output measure, and the library picks the distribution that goes with it: Laplace for the max divergence and Gaussian for zero-concentrated divergence. It is the constructor to reach for in code that lets the caller choose the currency.

```
noise_pure = space >> dp.m.then_noise(dp.max_divergence(), scale=1.0)  # the Laplace mechani
noise_zcdp = space >> dp.m.then_noise(dp.zero_concentrated_divergence(), scale=1.0)  # the G
```

```
make_noise MaxDivergence: map(1.0)=1.0 measure=MaxDivergence
make_noise ZeroConcentratedDivergence: map(1.0)=0.5 measure=ZeroConcentratedDivergence
```

10.2. Canonical noise

The mechanisms above take a scale and tell you what it costs. The canonical noise mechanism works the other way round. You state the sensitivity and the budget, and it draws from the distribution that spends exactly that budget. Approximate DP relaxes the pure guarantee with a slack δ, the probability with which the factor e^{ε} may fail. For that guarantee the canonical distribution is the Tulap distribution, a discrete Laplace draw plus a uniform one. It is the noise that leaves the most powerful hypothesis test possible at a given budget. So it is the mechanism to use when a release feeds a statistical test or a confidence interval rather than a table.

```
def canonical_mechanism(d_in: float, epsilon: float, delta: float) -> dp.Measurement:
    """Canonical noise on one float, calibrated so that inputs at most d_in apart cost exact
    space = dp.atom_domain(T=float, nan=False), dp.absolute_distance(T=float)
    return space >> dp.m.then_canonical_noise(d_in=d_in, d_out=(epsilon, delta))
```

The map is a step rather than a line, because the calibration is fixed at construction. For any sensitivity up to the one you declared it returns the pair you declared. Above that sensitivity it refuses, since the mechanism was never calibrated for it. The `check` method behaves accordingly.

```
canonical noise: measure=Approximate(MaxDivergence)
canonical noise: map(1.0)=(1.0, 1e-07) map(0.5)=(1.0, 1e-07)
canonical noise: check(1.0, (1.0, 1e-07))=True check(1.0, (0.9, 1e-07))=False
canonical noise map(2.0): d_in from the map (2) must be in [0, 1]
```

The installed version supports only a scalar float input and only an (ε, δ) budget. In the Context API the same mechanism is `query.canonical_noise()`. Passing `binomial_size` to it returns an object with confidence intervals and p-values for a binomially distributed count, which the documentation's notebook walks through on a count of married people.

10.3. Thresholded noise

A histogram over keys that come from the data has a problem that noise alone does not fix. If a category appears in the release at all, then someone in the data belongs to it. When that someone

is one person, the key itself is the leak. The thresholded noise mechanisms release a dictionary of noisy counts and keep only the keys whose noisy count clears a threshold. A key with many contributors survives on either of two neighbouring datasets, so its presence says nothing about any one person. A key with one contributor survives only if its noise is huge, and the probability of that is what the slack δ pays for. Grouping used this mechanism through Polars without naming it. This is what runs underneath.

The input domain is a map from keys to counts, and the input metric is a triple. The first entry bounds how many keys one person can touch. The second bounds the total change across all keys, in L1 for Laplace and in L2 for Gaussian. The third bounds the change to any one key.

```
def threshold_mechanisms(scale: float, threshold: float) -> tuple[dp.Measurement, dp.Measure
    """Laplace and Gaussian noise on a dictionary of float counts, releasing only the keys w
    domain = dp.map_domain(dp.atom_domain(T=str), dp.atom_domain(T=float, nan=False))
    laplace = dp.m.make_laplace_threshold(
        domain, dp.l01inf_distance(dp.absolute_distance(T=float)), scale=scale, threshold=th
    )
    gaussian = dp.m.make_gaussian_threshold(
        domain, dp.l02inf_distance(dp.absolute_distance(T=float)), scale=scale, threshold=th
    )
    return laplace, gaussian
```

The documentation's example has a person who touches one key by at most one. At scale 1 the Laplace version prices that at $\varepsilon = 1$ plus a δ. The δ is the chance that a count one person can create clears the threshold by noise alone. The Gaussian version prices it in ρ with a far smaller δ, because its tails fall faster.

```
laplace threshold: metric=L01InfDistance(AbsoluteDistance(f64)) measure=Approximate(MaxDiver
laplace threshold scale 1 threshold 20: map((1, 1.0, 1.0))=(1.0, 2.8014e-09)
gaussian threshold scale 1 threshold 20: map((1, 1.0, 1.0))=(0.5, 1.1102e-16)
gaussian threshold: measure=Approximate(ZeroConcentratedDivergence)
```

A smaller δ is not a smaller cost, because the two maps are in different currencies. The script converts the Gaussian guarantee to approximate DP at the Laplace mechanism's δ, with the combinators of Combinators. Then the honest comparison comes out the other way, and the Gaussian's ε is about six times larger. The picture flips when a person touches many keys by a little, which is the case the second line prices. There the L2 bound is much smaller than the L1 bound, and the Gaussian wins.

```
gaussian threshold as approximate DP at delta=2.8014e-09: epsilon=6.3036
spread (100, 10.0, 0.001): laplace (0.1000, 1.0316e-07) gaussian epsilon=0.0500
```

Notice that the Laplace map charged a tenth rather than ten over one. A person who touches a hundred keys by at most a thousandth can change the total by at most a tenth. The library knows this, so it tightens the stated L1 bound to that product. Integer counts take the discrete distributions, as in the previous section. The general form is `make_noise_threshold`, which takes the measure as an argument. The installed version wants that measure wrapped as approximate, which its docstring does not say.

```
discrete laplace threshold scale 1 threshold 10: map((1, 1, 1))=(1.0, 3.3190e-05)
make_noise_threshold Approximate(MaxDivergence): map((1, 1, 1))=(1.0, 3.3190e-05)
make_noise_threshold MaxDivergence: output_measure must be MaxDivergence or ZeroConcentrated
```

10.3.1. Counts of `ILOSTAT`

The LFS codes each response's labour status in `ILOSTAT`, and in this sample only four codes occur. Suppose you did not know which, so the keys must come from the data. The script counts the responses per code with Polars. Then it hands the dictionary to a thresholded Laplace mechanism over integer keys and integer counts.

```
def ilostat_counts(frame: pl.LazyFrame) -> dict[int, int]:
    """The number of responses in each ILOSTAT category, keyed by the category: the input of
    counts = frame.group_by("ILOSTAT").len().sort("ILOSTAT").collect()
    return {int(key): int(count) for key, count in zip(counts["ILOSTAT"], counts["len"])}


def ilostat_mechanism(scale: float, threshold: int) -> dp.Measurement:
    """Discrete Laplace noise on integer counts keyed by an integer category, with a thresho
    domain = dp.map_domain(dp.atom_domain(T=int), dp.atom_domain(T=int))
    metric = dp.l01inf_distance(dp.absolute_distance(T=int))
    return dp.m.make_laplace_threshold(domain, metric, scale=scale, threshold=threshold)
```

A person answers at most 36 times. Without further public knowledge the triple is therefore 36 in every entry, since a person could touch that many keys by one each, or one key that many times. The scale is the sensitivity over ε, which at a budget of 1 is 36. The error bar at that scale is about 108 responses. The threshold is then the only free choice, and it buys δ. The script prices three thresholds. Then it searches for the smallest one that keeps δ under the chapter's default of one in ten million.

```
ILOSTAT counts: {1: 1517011, 2: 155661, 3: 1450283, 9: 688977}
ILOSTAT d_in: (36, 36, 36) scale: 36 95% accuracy: 108.3
ILOSTAT threshold 500: map=1.0, delta=4.4835e-05
ILOSTAT threshold 1000: map=1.0, delta=4.1667e-11
ILOSTAT threshold 1500: map=1.0, delta=3.9968e-15
ILOSTAT smallest threshold with delta <= 1e-07: 720
ILOSTAT threshold 10: threshold (10) must not be smaller than l-infinity sensitivity 36
```

Every δ comes from the same tail. It is the chance that the noise lifts a count one person can make alone, which is 36 at most, over the threshold. That chance is then taken over every key the person could create. Raising the threshold shrinks it fast. A threshold below the per-key sensitivity is refused outright, because one person could then create a key with no noise at all. Here the smallest count is in the hundreds of thousands, so a threshold of 720 drops nothing, and all four counts come back within their error bars. The threshold only bites on rare categories, which is exactly where it should. Why it costs a δ and not an ε is derived in the full edition's chapter on sensitivity, under partition selection.

10.4. The approximate Laplace projection

Thresholding is one answer to unknown keys. The approximate Laplace projection is another. Instead of releasing a dictionary, it releases a compressed, hashed sketch of the whole key space. You then query the sketch for any key you like, including keys that never occurred. Nothing is dropped and nothing is listed, so no key leaks, and queries after the release are post-processing. The price is twofold. Answers are rounded to multiples of a parameter `alpha`, four by default, and two frequent keys can collide in the hash unless the sketch is large enough. Its size grows with two public bounds, `total_limit` on the sum of all counts and `value_limit` on any one count. The documentation's example sets them for a table of first names.

```
def projection_mechanism(scale: float) -> dp.Measurement:
    """A private projection of counts keyed by strings; releases a queryable that answers fo
    domain = dp.map_domain(dp.atom_domain(T=str), dp.atom_domain(T=int))
    metric = dp.l01inf_distance(dp.absolute_distance(T=int))
    return dp.m.make_alp_queryable(domain, metric, scale=scale, total_limit=50_000, value_li
```

One trap: here `scale` is not a noise scale but the privacy loss per unit of sensitivity, so a larger value spends more, not less. The map at unit sensitivity equals the scale, and it grows with the L1 entry of the triple.

```
alp scale 1: map((1, 1, 1))=1.0 measure=MaxDivergence
alp scale 2: map((1, 1, 1))=2.0
alp scale 1: map((2, 2, 1))=2.0
```

The documentation finds the scale for a budget with `binary_search_chain`, which Utilities and the Context API inside covers. On the toy table, the one-run answers at the end of the page land within a few units of the true counts. The name that is not in the table comes back near zero.

10.5. Noisy max and selection

Not every release is a number. Often the question is which one: which category is largest, which candidate value is the median, which model is best. The noisy max mechanisms take a vector of scores, add noise to each, and release the index of the largest. Their input metric is the L-infinity distance, a bound on how much any single score can change. That is the right bound because the release depends on how the scores compare, not on their sum. As with additive noise, the output measure chooses the distribution: exponential noise under the max divergence, and Gumbel noise under zero-concentrated divergence.

```
def noisy_max_mechanisms(scale: float) -> tuple[dp.Measurement, dp.Measurement]:
    """Select the index of the largest score: exponential noise under pure DP, Gumbel noise
    space = dp.vector_domain(dp.atom_domain(T=float, nan=False)), dp.linf_distance(T=float)
    exponential = space >> dp.m.then_noisy_max(dp.max_divergence(), scale=scale)
    gumbel = space >> dp.m.then_noisy_max(dp.zero_concentrated_divergence(), scale=scale)
    return exponential, gumbel
```

The map of the exponential version is twice the sensitivity over the scale, so at a scale of 2 a unit sensitivity costs $\varepsilon = 1$. The factor of two is there because one person can raise one score and lower another at the same time. If the scores can only move in one direction, you declare the metric monotonic, and the factor goes away. The Gumbel version is the exponential mechanism of the theory, which picks a candidate with probability growing exponentially in its score. Its map is the square of that same ratio, divided by eight. Negating the scores selects the smallest, at no extra cost. The constructor `make_noisy_top_k` releases the best k indices in order, for k times the cost.

```
noisy max: metric=LInfDistance(T=f64) measure=MaxDivergence
noisy max exponential scale 2: map(1.0)=1.0
noisy max gumbel scale 2: map(1.0)=0.125 measure=ZeroConcentratedDivergence
noisy max monotonic scale 2: exponential map(1.0)=0.5 gumbel map(1.0)=0.03125
noisy min scale 2: map(1.0)=1.0
noisy top 2 scale 2: exponential map(1.0)=2.0 gumbel map(1.0)=0.25
```

The older `make_report_noisy_max_gumbel` still exists and carries the pure-DP map. But the installed version marks it deprecated and points to `make_noisy_max`. The exponential mechanism, why selection costs twice what a count costs, and why exponential noise beats Gumbel under pure DP are in the full edition's chapter on selection.

```
report_noisy_max_gumbel scale 2: map(1.0)=1.0 measure=MaxDivergence
report_noisy_max_gumbel: Call to deprecated function (or staticmethod) make_report_noisy_max
```

10.5.1. A private median of `HWUSUAL`

Selection is how OpenDP releases a quantile. The candidates are a list of possible answers, here every whole number of hours from 1 to 98. The transformation `make_quantile_score_candidates` from the previous chapter scores each one by how far it is from splitting the data in half. The best candidate has the smallest score, so the chain ends with a noisy max over negated scores. The released index is looked up in the candidate list afterwards, which is post-processing. The constructor `make_private_quantile` does the three steps in one call.

```python
def hours_of_the_working(frame: pl.LazyFrame) -> list[float]:
    """HWUSUAL for the people at work (ILOSTAT 1), keeping 1 to 98 hours: 0 means varying, 9
    at_work = (pl.col("ILOSTAT") == 1) & pl.col("HWUSUAL").is_between(1, 98)
    return frame.filter(at_work).select("HWUSUAL").collect()["HWUSUAL"].to_list()


def median_by_hand(scale: float) -> tuple[dp.Transformation, dp.Measurement]:
    """The private median from its pieces: score each candidate, then pick an index by noisy
    space = dp.vector_domain(dp.atom_domain(T=float, nan=False)), dp.symmetric_distance()
    scores = space >> dp.t.then_quantile_score_candidates(candidates=HOURS, alpha=0.5)
    select = scores >> dp.m.then_noisy_max(dp.max_divergence(), scale=scale, negate=True)  #
    return scores, select  # the released index is looked up in HOURS afterwards, which is p
```

```
def median_mechanism(scale: float) -> dp.Measurement:
    """The same median in one constructor."""
    space = dp.vector_domain(dp.atom_domain(T=float, nan=False)), dp.symmetric_distance()
    return space >> dp.m.then_private_quantile(dp.max_divergence(), candidates=HOURS, alpha=
```

The data are the usual hours of the people at work, with the codes for varying and not applicable removed. A row filter keeps a person's contribution at most 36 rows, so the input distance is still 36 under the symmetric distance. The scores are integers, and each one moves by at most the input distance, so their L-infinity sensitivity is 36.

```
HWUSUAL at work, 1 to 98 hours: n=1,490,113 true median=37
quantile scores: metric=LInfDistance(T=u64) map(36)=36
best candidate: 37 hours, score 9,111
second candidate: 36 hours, score 95,652
gap in noise scales: 1,202
scores >> noisy max scale 72: map(36)=1.0
make_private_quantile scale 36: map(36)=1.0
make_private_quantile zCDP scale 36: map(36)=0.125
```

Two scales appear because the one-step constructor doubles the scale you pass. The scores of a median are integers scaled by two, the denominator of one half. So `make_private_quantile` multiplies the scale by that same two before handing it to noisy max, and the two maps agree at $\varepsilon = 1$. Read the scores before trusting the release. The best candidate beats the runner-up by over a thousand noise scales, so the mechanism returns 37 hours on practically every run, and the test checks for exactly that. On a smaller dataset the gaps shrink and the release starts to wander among nearby candidates. That wandering is the error you should expect from a private quantile.

10.6. Randomized response

Every mechanism so far runs on a trusted server that holds the data. Randomized response runs on the respondent's own device before the answer is sent, so that no server ever sees the truth. That is the local model of DP, in which the privacy unit is one answer. The metric is the discrete distance: two inputs are at distance 1 if they differ and 0 if they do not. OpenDP provides the randomizers only, and leaves the transport and the aggregation to you.

```
def randomized_response_mechanisms(prob: float) -> tuple[dp.Measurement, dp.Measurement]:
    """Randomized response on one yes/no answer and on one ILOSTAT category, each kept truth
    boolean = dp.m.make_randomized_response_bool(prob=prob)
    categorical = dp.m.make_randomized_response(categories=[1, 2, 3, 9], prob=prob)
    return boolean, categorical
```

The boolean version keeps the true answer with the probability you pass and flips it otherwise. Its map at distance 1 is the logarithm of the odds of telling the truth, so a probability of three

quarters costs about 1.1. A probability below one half is refused, because the answer would then be more often false than true. The categorical version keeps the truth with the same probability and otherwise picks one of the other categories uniformly. Its map grows with the logarithm of the number of alternatives, so four `ILOSTAT` codes cost twice what a yes/no answer costs.

```
randomized response bool: domain=AtomDomain(T=bool) metric=DiscreteDistance() measure=MaxDiv
randomized response bool prob 0.75: map(1)=1.0986 ln(0.75/0.25)=1.0986
randomized response bool prob 0.9: map(1)=2.1972
randomized response bool prob 0.4: probability must be within [0.5, 1]
randomized response 4 categories prob 0.75: map(1)=2.1972 ln(3*0.75/0.25)=2.1972
```

A single randomized answer is close to useless, and that is the point. The answers become useful in aggregate, after a correction for the known flipping rate, which the documentation derives for both versions. The third randomizer, for a bit vector, is the primitive behind RAPPOR, the first large deployment of local DP. The domain bounds how many bits can be set, and each bit is rerandomized with probability `f`. The map is then twice the weight times the logarithm of the odds of a bit surviving.

```
def bitvector_mechanism(f: float, max_weight: int) -> dp.Measurement:
    """Randomized response on a bit vector with at most max_weight ones, each bit rerandomiz
    return dp.m.make_randomized_response_bitvec(dp.bitvector_domain(max_weight=max_weight),
```

```
bit vector f=0.8 max_weight=3: domain=BitVectorDomain(weight=3) map(1)=2.4328 formula=2.4328
```

The script sends the documentation's 24-bit vector through the randomizer ten thousand times. Then it runs `debias_randomized_response_bitvec` over the answers. The three set bits come back near ten thousand and the others near zero, with a spread of a few hundred. That is what the correction can do at that many reports. Local DP buys its trust at a steep price in accuracy, and the full edition's chapter on trust models quantifies it.

The releases of the run behind this page close the chapter. The additive releases sit within a few units of 100, the thresholded count kept all four codes, the projection answered the unknown name with zero, and both medians are 37.

```
released (one run):
  laplace on 100.0: 104.05
  gaussian on 100.0: 100.26
  discrete laplace on 100: 100
  laplace vector on [0.0, 2.0, 2.0]: [-1.4, 1.17, 4.28]
  laplace k=-1 on 100.0, five draws: [99.5, 99.5, 102.5, 101.5, 100.0]
  bounded geometric on 10, five draws: [10, 10, 10, 9, 7]
  canonical noise on 100.0: 99.74
  laplace threshold on {a: 0, b: 20, c: 40}: {'c': 37.74439496773442}
  ILOSTAT counts: {1: 1517058, 2: 155697, 3: 1450247, 9: 689013}
  alp answers for Michael, James, Sharon, Lancelot: [748.0, 500.0, 48.0, 0.0]
  noisy max on [0, 2, 4, 6]: 3
  noisy min on [0, 2, 4, 6]: 0
```

```
noisy top 2 on [0, 2, 4, 6]: [3, 1]
median by hand: 37 hours
median by make_private_quantile: 37 hours
randomized response on True, ten draws: [True, True, False, True, True, False, False, Fals
randomized response on category 1, ten draws: [1, 1, 1, 1, 1, 1, 1, 2, 1, 3]
bit vector release: 10011011100000010001110
bit vector frequencies over 10,000 copies, bits 12, 20, 21: [10050, 10235, 10115]
bit vector frequencies, largest other bit: 565
```

10.7. Summary

Table 10.1.: The measurements of the chapter: their input metric, their output measure, the map the script printed at scale 1 for inputs at distance 1, and what each is for.

Constructor	Input metric	Output measure	Map at scale 1 for inputs at distance 1	What it is for
`make_laplace`	absolute or L1 distance	`MaxDivergence`	$\varepsilon = 1$	One number or a vector under pure DP; integers get the discrete Laplace
`make_gaussian`	absolute or L2 distance	`ZeroConcentratedDivergence`	$\rho = 0.5$	The same under zCDP; vectors cost by the L2 bound
`make_geometric`	absolute distance on integers	`MaxDivergence`	$\varepsilon = 1$	Integer Laplace with `bounds` for constant time
`make_noise`	as above	the one you pass	as above	Lets the caller choose the currency
`make_canonical_noise`	absolute distance	`Approximate(MaxDivergence)`	the one you declared, up to your `d_in`	Exact (ε, δ) calibration; inference on counts
`make_laplace_threshold`	L0, L1, L-infinity triple	`Approximate(MaxDivergence)`	$(1.2801, 10^{-9})$ at threshold 20	Counts over keys from the data; δ pays for the threshold
`make_gaussian_threshold`	L0, L2, L-infinity triple	`Approximate(ZeroConcentratedDivergence)`	$(0.5, 1.1102 \times 10^{-16})$ at threshold 20	The same when a person touches many keys by a little

Constructor	Input metric	Output measure	Map at scale 1 for inputs at distance 1	What it is for
`make_alp_queryable`	L0, L1, L-infinity triple	`MaxDivergence`	$\varepsilon = 1$ (the scale is a loss, not a noise)	Queries for any key after the release
`make_noisy_max`	L-infinity distance	`MaxDivergence` or `ZeroConcentratedDivergence`	$\varepsilon = 2$ or $\rho = 0.5$	Select an index by score; monotonic halves the cost
`make_noisy_top_k`	L-infinity distance	either	$\varepsilon = 4$ for $k = 2$	The best k indices, at k times the cost
`make_private_quantile`	symmetric distance	either	$\varepsilon = 1$	Scores plus noisy max plus lookup in one call
`make_randomized_response_bool`	discrete distance	`MaxDivergence`	1.099 at probability 0.75	One yes/no answer on the respondent's device
`make_randomized_response`	discrete distance	`MaxDivergence`	2.197 for four categories at 0.75	One category; the cost grows with the alternatives
`make_randomized_response_bitvec`	discrete distance	`MaxDivergence`	2.433 for weight 3 at `f` = 0.8	The RAPPOR primitive; debias the aggregate

10.8. References

- OpenDP documentation: "Measurements", "Additive Noise Mechanisms", "Canonical Noise Mechanism", "Thresholded Noise Mechanisms", "Approximate Laplace Projection", "Report Noisy Max Mechanisms" and "Randomized Response" in the API user guide.
- Dwork, McSherry, Nissim and Smith, "Calibrating Noise to Sensitivity in Private Data Analysis" (2006): the Laplace mechanism.
- Canonne, Kamath and Steinke, "The Discrete Gaussian for Differential Privacy" (2020): the discrete Gaussian and the discrete Laplace samplers OpenDP uses.
- Awan and Vadhan, "Canonical Noise Distributions and Private Hypothesis Tests" (2023): the Tulap distribution.
- Rogers, "A Unifying Privacy Analysis Framework for Unknown Domain Algorithms in Differential Privacy" (2023): the thresholded mechanisms' maps.
- Aumüller, Lebeda and Pagh, "Differentially Private Sparse Vectors with Low Error, Optimal Space, and Fast Access" (2021): the approximate Laplace projection.
- McKenna and Sheldon, "Permute-and-Flip: A New Mechanism for Differentially Private Selection" (2020): the exponential-noise noisy max.

- Smith, “Privacy-Preserving Statistical Estimation with Optimal Convergence Rates” (2011): the quantile by selection.
- Warner, “Randomized Response: A Survey Technique for Eliminating Evasive Answer Bias” (1965), and Erlingsson, Pihur and Korolova, “RAPPOR: Randomized Aggregatable Privacy-Preserving Ordinal Response” (2014).
- The full edition’s chapters on the Laplace and Gaussian mechanisms in the full edition, selection in the full edition, sensitivity and partition selection in the full edition, discrete mechanisms in the full edition and trust models in the full edition derive the results this chapter states.

11. Combinators

The previous two chapters built transformations and measurements one at a time. A real release is never one of them. It is a clamp, then a sum, then noise, then a second query that depends on the first. At the end it is reported as one guarantee of differential privacy (DP). The pieces that join measurements and transformations into bigger ones are the **combinators**. Each takes one or more objects of the framework and returns a new one, with a privacy map that says what the new object costs. You never add budgets by hand, because the combinator's map does it, and it refuses to join pieces whose promises do not fit.

The chapter follows the documentation's Combinators pages in order. Chaining joins a transformation to what comes after it. Composition releases several measurements on the same data, first all at once, then one at a time, then without deciding the budget of each in advance. Measure casting converts a guarantee between privacy measures, so that a Gaussian release can be reported with the privacy loss ε and the slack δ of approximate DP. Amplification by subsampling reads a smaller guarantee for a sample of a larger population. By the end you will be able to read the privacy map of any chain, say why a context refuses a query, and convert a budget between measures.

The script opendp_combinators.py runs every step on OpenDP 0.16.0. Its toy data are twenty integers from 0 to 10 drawn by a seeded generator. Its toy measurements are a Laplace count and a Laplace sum of the clamped values, as in the documentation. One example runs on the Labour Force Survey (LFS) sample, where the number of queries in a composition changes the noise scale of a real sum. The maps, profiles, scales and error messages on this page are the same on every run. The released values are not, because OpenDP's noise cannot be seeded, so they appear in one block marked as one run.

```
uv run python code/part03-framework/opendp_combinators.py
```

11.1. Chaining

The simplest way to combine two pieces is to run one after the other, and three constructors do that. `make_chain_tt` joins two transformations. `make_chain_mt` puts a measurement after a transformation, and `make_chain_pm` attaches a post-processing function to a measurement. There is no chainer for two measurements, because anything computed after a measurement is post-processing and needs no further accounting. The script builds the clamped sum and its Laplace release with the long forms first.

```
def chained_by_hand() -> tuple[dp.Transformation, dp.Measurement]:
    """The clamped sum and its Laplace release, built with the long-form chainers."""
    clamp = INT_SPACE >> dp.t.then_clamp(BOUNDS)
```

```
    total = dp.t.make_sum(clamp.output_domain, clamp.output_metric)
    clamped_sum = dp.c.make_chain_tt(total, clamp)  # a transformation: clamp, then sum
    laplace = dp.m.make_laplace(clamped_sum.output_domain, clamped_sum.output_metric, scale=
    noisy_sum = dp.c.make_chain_mt(laplace, clamped_sum)  # a measurement: the chain, then n
    return clamped_sum, noisy_sum
```

The outer piece comes first in each call and the inner piece second, as in function composition. Each constructor takes its input space from the output space of the piece before it. The chained transformation has a stability map, which Transformations introduced as the bound on how far the output moves when the input moves by a given distance. Adding or removing one record changes the clamped sum by at most the width of the bounds, so the map at distance 1 is 10. The chained measurement has a privacy map, which Measurements introduced, and it is the stability of the inner part divided by the noise scale. A sum that moves by 10 under Laplace noise of half that scale costs $\varepsilon = 2$.

```
clamp >> sum: map(1)=10
clamp >> sum >> laplace: map(1)=2.0 check(1, 2.0)=True
with >>: map(1)=2.0
```

The `>>` operator chooses the right chainer for you, and it takes a `then_` constructor on its right so that the input space never has to be repeated. Below is the same measurement in one line.

```
import opendp.prelude as dp

dp.enable_features("contrib")
space = dp.vector_domain(dp.atom_domain(T=int)), dp.symmetric_distance()
noisy_sum = space >> dp.t.then_clamp((0, 10)) >> dp.t.then_sum() >> dp.m.then_laplace(scale=
print(noisy_sum.map(1))  # 2.0
print(noisy_sum.input_domain)  # VectorDomain(AtomDomain(T=i32))
print(noisy_sum.output_measure)  # MaxDivergence
```

Post-processing is any computation on a release that does not touch the data again. It costs nothing, because whatever an adversary could learn from the processed value they could have learned from the release itself. The full edition's chapter on properties proves that. In OpenDP you attach a Python function to a measurement with `>>`, and the result is a measurement with the same privacy map.

```
def divide_by_size(total: int) -> float:
    return total / N  # the released sum over the public size: a mean, at no further cost

mean_of_sum = shortcut >> divide_by_size
```

```
post-processed: map(1)=2.0
```

In version 0.16.0 this needs a second feature flag, `honest-but-curious`, which the script switches on just before the chain and off just after. The reason is that the library cannot inspect a Python

function. A post-processor must not read global state, such as the clock, and must not raise an exception whose presence depends on the data. Either would open a channel from the data to the caller that the privacy map does not account for.

A chain only forms when the pieces fit. The output domain of the inner piece must equal the input domain of the outer one, and the same holds for the metrics. The script tries to feed a sum of floats into a Laplace mechanism built for integers, and the library refuses.

```
try:
    chain = float_sum >> int_laplace  # a float sum feeding a Laplace mechanism on integers
    domain_error = f"chained: {chain}"
except dp.OpenDPException as err:
    domain_error = str(err.message)
```

```
domain mismatch: Intermediate domains don't match. See https://github.com/opendp/opendp/disc
    output_domain: AtomDomain(T=f64)
    input_domain:  AtomDomain(T=i32)
metric mismatch: Intermediate metrics don't match. See https://github.com/opendp/opendp/disc
    output_metric: AbsoluteDistance(i32)
    input_metric:  AbsoluteDistance(f64)
```

The second message comes from a count, whose output is an integer at an integer absolute distance, chained to a Laplace mechanism whose metric measures distances in floats. With a mismatched domain, the outer piece's proof would cover inputs the inner piece never promised to produce. When you meet one of these errors, read the two lines it prints and insert the cast, clamp or impute step that makes them equal.

The sum and count that the rest of the chapter composes have the error bars below, which Assessing utility and choosing the noise explains how to read.

```
sum: scale=5 95% accuracy=15.4537
count: scale=1 95% accuracy=3.3756
```

11.2. Non-adaptive composition

Suppose you want both the sum and the count of the toy vector. Running the two measurements on the same data spends both of their budgets, and the question is what the total is. **Basic composition** says that it is the sum of the parts. A release that is ε_1-DP followed by one that is ε_2-DP on the same data is $(\varepsilon_1 + \varepsilon_2)$-DP, which the full edition's chapter on properties proves. The combinator `make_composition` builds that rule into one measurement, which runs every part on the data and returns their releases as a list.

```
both = dp.c.make_composition([shortcut, count])  # one measurement that releases the sum and
print(f"composition: map(1)={both.map(1):.1f}  parts: {shortcut.map(1):.1f} + {count.map(1):
```

```
composition: map(1)=3.0  parts: 2.0 + 1.0
composition output measure: MaxDivergence
```

The parts must agree on their input domain, their input metric and their output measure. That is because a composition is a statement about one dataset under one notion of neighbours and one way of measuring privacy. The sum and the count share all three. The composition is called non-adaptive because every part is fixed before any data are seen, so no part can depend on another's answer.

The same combinator composes measurements in other measures, and the rule it applies is the measure's own. **Zero-concentrated differential privacy (zCDP)** measures privacy by a single parameter ρ, which bounds the Rényi divergence between the output distributions on neighbours at every order. Under zCDP the parameters add just as ε does, which the full edition's chapter on concentrated DP derives. The Gaussian sum at scale 5 has $\rho = 2$, the Gaussian count at scale 1 has $\rho = 0.5$, and the composition reports their sum.

```
zCDP composition: map(1)=2.50  parts: 2.00 + 0.50
mixed measures: All output measures must be the same
```

The second line shows the check that stops you from mixing measures, because the maps of a Laplace count and of a Gaussian count return numbers that mean different things. Section 11.7 shows the combinators that bring them to a common measure first.

11.3. Adaptive composition

Often you do not know the second query until you have seen the first answer. You release a count, and it tells you whether a mean is worth asking for. Composition still holds when each query is chosen after the earlier answers, as long as the budget of each is fixed in advance. The full edition's chapter on composition proves this adaptive form. The combinator for it is `make_adaptive_composition`. Its constructor is more verbose than the non-adaptive one, because it cannot read the supporting elements off parts it has not seen yet.

```
def adaptive_compositor(d_mids: list[float]) -> dp.Measurement:
    """An adaptive compositor over the toy space: one share of pure-DP budget per query, fix
    return dp.c.make_adaptive_composition(
        input_domain=INT_SPACE[0],
        input_metric=INT_SPACE[1],
        output_measure=dp.max_divergence(),
        d_in=1,  # neighbours add or remove one record
        d_mids=d_mids,  # the epsilon each query may spend, in the order the queries arrive
    )
```

The argument `d_in` is the distance between neighbouring datasets, here 1 for adding or removing one record. The argument `d_mids` lists the privacy loss each query may spend, in order, and the privacy map of the compositor is their sum, here 3.

```
adaptive: map(1)=3.0
```

Invoking the compositor on the data does not release anything. It returns a **queryable**, which is a small state machine. It holds the data and the shares that remain, and it accepts a measurement as a query. It checks that the measurement's map at `d_in` fits within the next share, and it answers with the measurement's release. The script submits the sum, then tries the sum again, then submits the count.

```
queryable = compositor(data)  # the data go in once; what comes back answers measurements
adaptive_sum = queryable(shortcut)  # spends the first share, 2.0
try:
    queryable(shortcut)  # asks 2.0 of the second share, 1.0
    over_share = "answered"
except dp.OpenDPException as err:
    over_share = str(err.message)
adaptive_count = queryable(count)  # spends the second share, 1.0
```

```
second query over its share: insufficient budget for query: 2.0 > 1.0
third query: out of queries
```

The shares are consumed in order, so a query that asks for more than the next share is refused, and the share is not spent. After the count takes the second share, a third query of any size is refused as well. This is the object a context wraps. When A first release built a context with `split_evenly_over=3`, the library built an adaptive compositor with three equal shares and kept its queryable. The context's `accountant` attribute is that compositor, and its `d_mids` are the shares. A `release()` is a submission to the queryable, and the refusal you saw there is the "out of queries" above, translated.

On real data the number of shares decides the noise of every query. The script builds two contexts over the LFS sample, one with the whole budget of $\varepsilon = 1$ on a single query and one split evenly over three. Both ask for the sum of `HWUSUAL`, the hours usually worked per week, with the code 99 for "not applicable" filtered out. The values are bounded to 0 to 80 hours. A person appears in at most 36 quarters, so the privacy unit is 36 contributions. The sum therefore moves by up to 36 times the upper bound when one person is added or removed.

```
LFS context over 1: accountant map(36)=1.0 d_mids=[1.0]
LFS HWUSUAL sum over 1: scale=2,881.04 95% accuracy=8,630.8
LFS context over 3: accountant map(36)=1.0 d_mids=[0.3333, 0.3333, 0.3333]
LFS HWUSUAL sum over 3: scale=8,643.11 95% accuracy=25,892.4
true HWUSUAL sum: 56,315,594
```

Both accountants map the privacy unit's distance to 1, because the shares always add up to the budget you declared. What changes is the scale the context finds for the query, which is the sensitivity over the share. Three shares triple the scale and the error bar. On a sum of about 56 million hours an error bar of 26 thousand is still small. On a small group it would not be. The scale sits slightly above the sensitivity over the share, because the sum of floats carries a rounding term that Transformations explains.

11.4. Odometers

Adaptive composition fixes each query's budget before the first answer. Sometimes even that is too early, because the answer to the first query decides how much the second one deserves. **Fully adaptive composition** lets the privacy loss of each query be chosen as you go. The object that tracks it is an **odometer**, a running total of the loss spent so far that stays valid at whatever point you choose to stop. The full edition's chapter on adaptive composition shows why plugging adaptively chosen parameters into the ordinary theorems fails, and which running totals are valid. For pure DP the sum of the losses spent is one, and that is what OpenDP's odometer reports.

```
odometer = dp.c.make_fully_adaptive_composition(
    input_domain=INT_SPACE[0], input_metric=INT_SPACE[1], output_measure=dp.max_divergence()
)
odometer_queryable = odometer(data)
loss_before = odometer_queryable.privacy_loss(1)  # nothing spent yet
odometer_sum = odometer_queryable(shortcut)
loss_after_sum = odometer_queryable.privacy_loss(1)
odometer_count = odometer_queryable(count)
loss_after_count = odometer_queryable.privacy_loss(1)
```

The constructor takes the same three supporting elements as the adaptive compositor but no `d_in` and no `d_mids`, because nothing about the budget is decided yet. What it returns is an `Odometer`, not a `Measurement`, and an odometer has no privacy map, because it makes no promise about the total. Invoking it on the data gives an **odometer queryable**. It answers measurements like the compositor's queryable did, and it also reports `privacy_loss(d_in)`, the total spent so far at the given input distance.

```
odometer loss: before=0.0 after sum=2.0 after count=3.0 after another count=4.0
odometer loss at d_in=2 after three queries: 8.0
```

The total starts at zero, grows by the map of each query as it is answered, and never refuses anything. The last line shows why the argument matters. At distance 2, each Laplace query costs twice as much, so the total doubles. The odometer answers "how much have I spent?" and leaves "may I spend more?" to you. That suits an open-ended exploration, but nothing stops the total from growing past what you may claim. The next combinator adds the stop.

11.5. Privacy filters

A **privacy filter** is an odometer with a cap. It refuses any query that would take the running total past a bound fixed in advance. The total can then never exceed the bound, so the filter is a measurement with an ordinary privacy map. The full edition's chapter on adaptive composition proves that for pure DP the cap on the sum of the realized losses is a valid guarantee. The combinator `make_privacy_filter` takes the odometer, the input distance and the cap.

```
privacy_filter = dp.c.make_privacy_filter(odometer, d_in=1, d_out=2.0)
filter_queryable = privacy_filter(data)
filter_count_1 = filter_queryable(count)
filter_count_2 = filter_queryable(count)
try:
    filter_queryable(count)  # would take the loss to 3.0
    refusal = "answered"
except dp.OpenDPException as err:
    refusal = str(err.message)
```

```
filter: map(1)=2.0
filter third query: filter is now exhausted: pending privacy loss (3.0) would exceed privacy
filter loss after refusal: filter is exhausted: no more queries can be answered
```

The filter's map at distance 1 is the cap, whatever queries later arrive, which is what makes it a measurement. Two counts at a cost of 1 each fit under a cap of 2, and the third is refused with a message that names the pending total and the budget. The installed version then treats the filter as finished. After one refusal, even `privacy_loss` raises, so read the running total before the query that might exhaust it, not after. Unlike the adaptive compositor, a filter needs no list of shares.

The filter is also what a context wraps when you give it a budget but no split. The script builds a context on the toy data with `dp.loss_of(epsilon=2.0)` and neither `split_evenly_over` nor `split_by_weights`. Its queryable is then an odometer queryable behind a filter, and `current_privacy_loss` reads the odometer. The script gives its count query an explicit scale, since there is no share for the context to search within.

```
context without a split: queryable=OdometerQueryable spent=1.0 remaining=1.0
```

11.6. Chaining and nesting compositors

Compositors and filters are measurements, so the chainers and the compositors apply to them too. Two patterns come up in practice. The first is a transformation before a compositor, which saves repeating the same preprocessing in every query. The script chains the clamped sum in front of an adaptive compositor, so that the compositor's queries are measurements on the sum, a single integer, rather than on the vector.

```
sum_compositor = clamped_sum_only >> dp.c.then_adaptive_composition(
    output_measure=dp.max_divergence(),
    d_in=clamped_sum_only.map(1),  # the sum moves by at most 10 when one record changes
    d_mids=[2.0, 1.0],
)
sum_space = clamped_sum_only.output_domain, clamped_sum_only.output_metric
laplace_on_sum = sum_space >> dp.m.then_laplace(scale=SUM_SCALE)  # a query on the sum, not
```

```
transformation >> compositor: d_in=10 map(1)=3.0
laplace on the sum: map(10)=2.0
```

The one thing to get right is `d_in`. The compositor sits after the sum, so its input distance is the sum's output distance, which is the stability map of the sum at the privacy unit's distance, here 10. The `then_` form takes the domain and metric from the transformation, but it cannot guess the distance, so compute it with `map` rather than typing it. A query to this compositor is checked at that distance, and a Laplace mechanism of scale 5 on the sum then costs 2, which fits the first share.

The second pattern is a compositor inside a compositor. The inner compositor is a measurement whose map is the sum of its shares, so it can be submitted as a query to an outer one. The answer is the inner queryable.

```
outer = adaptive_compositor(d_mids=[3.0, 1.0])
inner = adaptive_compositor(d_mids=[2.0, 1.0])  # a measurement costing 3.0: it fits the out
outer_queryable = outer(data)
inner_queryable = outer_queryable(inner)  # the answer to a compositor query is a queryable
inner_sum = inner_queryable(shortcut)
outer_count = outer_queryable(count)  # the outer's second share, while the inner queryable
inner_count = inner_queryable(count)  # pure DP composes concurrently, so the inner one stil
```

```
nested: outer map(1)=4.0 inner map(1)=3.0
```

The outer compositor charges 3 for the inner one, whatever the inner one later answers, because 3 is the inner map. The script interleaves the queries. It asks the inner queryable for the sum, the outer one for the count, and the inner one again for the count. That works because pure DP supports concurrent composition, where several queryables spawned from one compositor stay open at the same time. For approximate zCDP and approximate Rényi differential privacy (RDP) the library instead freezes an earlier queryable when a new query arrives, because post-processing has not been proved to hold there while an analyst can still interact with it. The documentation uses nesting to run a zCDP compositor inside an approximate-DP one, after casting the inner compositor's measure with the combinators of the next section.

11.7. Measure casting

Every measurement carries an output measure, and a composition needs all of its parts to carry the same one. A Laplace release is measured in pure DP, a Gaussian release in zCDP, and a report is usually wanted as (ε, δ). The casting combinators move a measurement from one measure to another, each by a conversion the full edition derives, so that the pieces can be composed.

The first conversion goes from pure DP to zCDP. An ε-DP mechanism is ρ-zCDP with $\rho = \varepsilon^2/2$, which the full edition's chapter on concentrated DP proves, and `make_pureDP_to_zCDP` applies it to the map. `make_approximate` turns a pure-DP measurement into an approximate-DP one with a slack of zero. It can then be composed with measurements that have a real δ.

```
import opendp.prelude as dp

dp.enable_features("contrib")
space = dp.atom_domain(T=float, nan=False), dp.absolute_distance(T=float)
laplace = space >> dp.m.then_laplace(scale=1.0)
print(dp.c.make_pureDP_to_zCDP(laplace).map(1.0))  # 0.5
print(dp.c.make_approximate(laplace).map(1.0))  # (1.0, 0.0)
```

The conversion you will use most goes from zCDP to (ε, δ), because that is how a Gaussian release is reported. A ρ-zCDP mechanism satisfies approximate DP at every slack at once, with a privacy loss that grows as the slack shrinks. So the honest output of the conversion is not a pair but a curve, the **privacy profile**, which gives the smallest privacy loss for each slack and the reverse. `make_zCDP_to_approxDP` returns a measurement whose map is that curve, with the methods `epsilon(delta)` and `delta(epsilon)`. The script builds a Gaussian mechanism with $\rho = 0.1$, which the scale $1/\sqrt{2\rho}$ gives at unit sensitivity. It then reads the curve at a slack of one in ten million.

```
gaussian = FLOAT_SPACE >> dp.m.then_gaussian(scale=math.sqrt(1 / (2 * rho)))  # rho = 1 / (2
profile_measurement = dp.c.make_zCDP_to_approxDP(gaussian)
profile = profile_measurement.map(1.0)  # a curve, not a number
epsilon_at_delta = profile.epsilon(delta=delta)
delta_at_one = profile.delta(epsilon=1.0)
fixed = dp.c.make_fix_delta(profile_measurement, delta=delta)  # the curve at one delta: an
```

```
gaussian: rho=0.1000
profile at rho=0.1: epsilon(delta=1e-07)=2.3484 delta(epsilon=1)=0.008933
simple conversion: 2.6391  improved conversion: 2.3484
fix_delta: map(1.0)=(2.3484, 1e-07)  measure=Approximate(MaxDivergence)
```

A budget of $\rho = 0.1$ is $(2.35, 10^{-7})$-DP. The third line compares that with the two conversions the full edition states. The simple one is

$$\varepsilon = \rho + 2\sqrt{\rho \ln(1/\delta)}, \tag{11.1}$$

which gives 2.64 here. The improved conversion of Canonne, Kamath and Steinke minimizes a tighter expression over the Rényi order, and the script's numerical minimum agrees with OpenDP to four decimals. So the library implements the improved conversion, which matters when you check its numbers against a paper that quotes the simple one. The full edition's chapter on RDP conversions explains where the simple formula loses.

A curve cannot be composed with a pair, so `make_fix_delta` evaluates the curve at one δ of your choice and returns a measurement whose map is the pair. Its output measure is then the approximate max divergence, the same as `make_approximate` produces, and the two can meet in a composition. The last casting combinator, `make_fixed_approxDP_to_approxDP`, goes the other way and turns a fixed pair back into a curve. For a pure-DP release cast with `make_approximate`, that curve is flat, since the mechanism is ε-DP at zero slack and therefore at every larger slack too.

```
approximate: map(1.0)=(1.0, 0.0)  measure=Approximate(MaxDivergence)
fixed to profile: epsilon(delta=1e-07)=1.0 epsilon(delta=0)=1.0
laplace count + gaussian count: map(1)=(6.6709, 1e-07)
```

The last line is the payoff. The Laplace count, which Section 11.2 could not compose with the Gaussian count, is cast with `make_approximate`. The Gaussian count is cast to a curve and fixed at the same slack, and `make_composition` accepts the two. The total privacy loss is large because a Gaussian count with $\rho = 0.5$ is expensive at so small a slack. Casting makes a composition legal, not cheap. When the Gaussian parts dominate, account the whole release in zCDP and convert once at the end, as the practitioner edition's release workflow does.

11.8. Amplification by subsampling

The last combinator changes what a privacy map promises without changing the computation. Suppose your dataset is a simple random sample from a larger population, and nobody knows who was sampled. Then the release is more private with respect to the population than to the sample, because a person who may not be in the sample is harder to detect. The full edition's chapter on subsampling derives this amplification and shows why the sampling scheme must match the neighbouring relation. Write n for the size of the sample and N for the size of the population. Under pure DP the amplified loss is

$$\varepsilon' = \ln\Big(1 + \frac{n}{N}\,(e^{\varepsilon} - 1)\Big). \tag{11.2}$$

The script prints it beside the library's map. `make_population_amplification` needs a measurement whose input domain carries the sample size, because the ratio of the two sizes is part of the guarantee.

```
def sample_mean() -> dp.Measurement:
    """The Laplace mean of a sample of exactly SAMPLE values in [0, 10]: the domain carries
    space = dp.vector_domain(dp.atom_domain(bounds=(0.0, 10.0), nan=False), size=SAMPLE), dp
    return space >> dp.t.then_mean() >> dp.m.then_laplace(scale=0.5)


def amplified_mean(population_size: int) -> dp.Measurement:
    """The same mean, with its guarantee read for the population the sample was drawn from."
    dp.enable_features("honest-but-curious")  # the library cannot check that the data are s
    return dp.c.make_population_amplification(sample_mean(), population_size=population_size
```

```
sample mean: map(2)=2.0000
amplified, population 100: map(2)=0.4940  formula=0.4940
amplified, population 1000: map(2)=0.0619  formula=0.0619
amplifying an unsized domain: elements of the vector domain have unknown size
```

The maps are read at distance 2, because two datasets of the same fixed size differ by a swap of one record, which the symmetric distance counts as one removal and one addition. The mean of

ten values in a range of 10 moves by 1 under a swap, and Laplace noise of half that costs $\varepsilon = 2$. Read for a population of a hundred, the same release costs about 0.49, and for a population of a thousand about 0.06. The formula matches the library's map, and the gain grows with the population. A measurement on a vector domain without a size is refused, as the last line shows.

The combinator needs the `honest-but-curious` flag, for the same reason as the Python post-processor. The guarantee is only true if the data really are a uniform sample of a population of the stated size, and if the sample's membership is unknown to whoever reads the release. The library cannot verify either from the data, so the flag records that the claim rests on your word. Use the unamplified map when the sample was chosen in a way that depends on the people in it, or when the sampled identities are public.

11.9. Summary

Table 11.1.: The combinators of this chapter, what each does, and the rule of thumb that goes with it.

Call or concept	What it does	Rule of thumb
`make_chain_tt`, `make_chain_mt`, `make_chain_pm`	Join two transformations, a transformation and a measurement, or a measurement and a post-processor	Write `>>`; it picks the right one
`measurement >> python_function`	Post-processing, at no cost	Needs `honest-but-curious` in 0.16.0; no global state, no data-dependent exceptions
`DomainMismatch`, `MetricMismatch`	A chain whose intermediate domain or metric differs	Read the two printed lines and insert the step that makes them equal
`make_composition([m1, m2])`	One measurement that releases every part; its map is the sum of the parts' maps	Parts share domain, metric and measure; ρ adds like ε
`make_adaptive_composition(..., d_in, d_mids)`	A queryable that answers measurements one at a time, each within the next share	What a context with a split wraps; a query over its share is refused
`make_fully_adaptive_composition`	An odometer whose queryable reports `privacy_loss(d_in)` so far	Never refuses; for exploration, not for a release
`make_privacy_filter(odometer, d_in, d_out)`	An odometer with a cap, which makes it a measurement	What a context without a split wraps; after one refusal it answers nothing
`transformation >> then_adaptive_composition(...)`	Preprocessing shared by every query of a compositor	Set `d_in` to the transformation's map at the privacy unit's distance

Call or concept	What it does	Rule of thumb
A compositor submitted to a compositor	A sub-budget, answered by a queryable	Costs the inner compositor's map; pure DP lets both queryables stay open
`make_pureDP_to_zCDP`, `make_approximate`	ε to $\rho = \varepsilon^2/2$; ε to $(\varepsilon, 0)$	Cast to make a composition legal, then read the total once
`make_zCDP_to_approxDP`	ρ to a privacy profile with `epsilon(delta)` and `delta(epsilon)`	OpenDP uses the improved conversion; $\rho = 0.1$ gives 2.35 at $\delta = 10^{-7}$
`make_fix_delta`, `make_fixed_approxDP_to_approxDP`	A profile at one δ, and a pair (ε, δ) to a flat profile	Fix δ last, after composing in zCDP
`make_population_amplification`	A smaller map for a secret simple random sample of a larger population	Sized input domain, `honest-but-curious`, and a sampling you can vouch for

11.10. References

- OpenDP documentation: "Combinators", "Chaining", "(Non-Adaptive) Composition", "Adaptive Composition", "Fully Adaptive Composition and Odometers", "Privacy Filters", "Compositor Chaining and Nesting", "Measure Casting" and "Amplification" in the API user guide.
- Rogers, Roth, Ullman and Vadhan, "Privacy Odometers and Filters: Pay-as-you-Go Composition" (2016): the odometer and the filter behind `make_fully_adaptive_composition` and `make_privacy_filter`.
- Whitehouse, Ramdas, Rogers and Wu, "Fully Adaptive Composition in Differential Privacy" (2023): the fully adaptive composition theorem the odometer implements.
- Haney, Shoemate, Tian, Vadhan, Vyrros, Xu and Zhang, "Concurrent Composition for Interactive Differential Privacy with Adaptive Privacy-Loss Parameters" (2023): why several queryables may stay open at once.
- Bun and Steinke, "Concentrated Differential Privacy: Simplifications, Extensions, and Lower Bounds" (2016): zCDP, its composition and the cast from pure DP.
- Canonne, Kamath and Steinke, "The Discrete Gaussian for Differential Privacy" (2020): the improved conversion from zCDP to (ε, δ) that the privacy profile implements.
- The full edition's chapters on post-processing and basic composition in the full edition, composition in the full edition, concentrated DP in the full edition, RDP conversions in the full edition, subsampling in the full edition and adaptive composition in the full edition derive the results this chapter states.

12. Utilities and the Context API inside

Every query in this book so far ended with a call such as `.laplace()` that named no scale, and the context filled it in. This chapter shows what fills it in. The answer is a handful of utilities that the framework exposes on their own. Type descriptors say what kind of number a chain carries, a binary search turns a budget into a scale, and a serializer turns a measurement into text. The Context API is a thin layer over these utilities and over the combinators of the previous chapter. Once you can see the layer, you can predict what a query will do before you release it. You can also rebuild any part of it by hand when the layer refuses.

The chapter follows the documentation's pages on typing, parameter search and serialization, then its page on the Context API and its comparison of the two APIs. It ends by taking apart the count query of the first chapter, piece by piece, and finding the same scale of 3. By the end you will be able to read a type error, run the search a context runs, and store a measurement and reload it. You will also be able to say which framework object each line of a Context API program creates.

The script opendp_utilities.py runs every step on OpenDP 0.16.0 with Polars 1.36.1. Its plain-data queries use the first chapter's hundred integers from a seeded generator, and its one Polars query counts the rows of the France Labour Force Survey (LFS) sample. The type descriptors, the error messages, the scales the searches find and the privacy maps are the same on every run. The released counts are not, because OpenDP's noise cannot be seeded, so they appear in one block marked as one run.

```
uv run python code/part03-framework/opendp_utilities.py
```

12.1. Types

OpenDP's core is written in Rust, and Rust has no single integer type. It has signed and unsigned integers of several widths, from 8 bits to 128, and floats of two widths. The library keeps that precision because the constants of a privacy proof depend on it. A sum of 32-bit integers can overflow where a sum of 64-bit integers cannot, and a float's rounding error depends on how many bits it has. So every domain, metric and measure carries a **type descriptor**, and the library never casts one type to another on its own. The framework chapter introduced the descriptors as the `T=` arguments of the constructors. This section says how to write and read them.

A descriptor can be written in three ways, and the snippet shows that they agree. The canonical form is a string such as `"i32"` or `"Vec<f64>"`. A Python type such as `int`, `float`, `str` or `bool` is an alias, and `list[int]` is an alias for a vector. The objects in `opendp.typing`, such as `dp.i32`, `dp.f64` and `dp.Vec[dp.i32]`, spell the same strings with completion in an editor.

```
import opendp.prelude as dp

print(dp.atom_domain(T=float))  # AtomDomain(nan=true, T=f64)
print(dp.atom_domain((0, 1), T=dp.i32))  # AtomDomain(bounds=[0, 1], T=i32)
print(dp.RuntimeType.parse(list[int]))  # Vec<i32>
print(dp.Vec[dp.i32] == dp.RuntimeType.parse(list[int]))  # True
```

The aliases hide one decision. A Python `int` means a 32-bit integer and a `float` a 64-bit float, which the script confirms. The function `dp.RuntimeType.infer` reads the type off a value in the same way. NumPy's default integer has 64 bits, so an integer from an array arrives as `i64` while a domain built with `T=int` expects `i32`, and the two do not meet. The function `set_default_int_type` changes what the alias means for the rest of the program, which is the clean fix when all your integers have 64 bits.

```
import numpy as np
import opendp.prelude as dp

print(dp.RuntimeType.parse(int))  # i32
print(dp.RuntimeType.infer(np.int64(1)))  # i64
dp.set_default_int_type(dp.i64)
print(dp.RuntimeType.parse(int))  # i64
print(dp.vector_domain(dp.atom_domain(T=int)))  # VectorDomain(AtomDomain(T=i64))
```

When the types do not meet, the error reads `inferred type is XXX, expected YYY`, and the troubleshooting page of the documentation gives it an entry. The script provokes it in the simplest way. It builds a Laplace mechanism on an integer domain with an integer metric, then asks for its privacy map at a sensitivity of 1.5.

```
int_space = dp.atom_domain(T=int), dp.absolute_distance(T=int)
laplace_int = int_space >> dp.m.then_laplace(scale=1.0)  # Laplace noise on a 32-bit integer
try:
    laplace_int.map(1.5)  # a float sensitivity, on a metric whose distances are integers
    message = "no error"
except TypeError as err:
    message = first_line(err)
```

```
map(1.5) on an integer metric: TypeError: inferred type is f64, expected i32
map(2) on an integer metric: 2.0
float d_in on the count: TypeError: inferred type is f64, expected u32
integer d_in on the count: 0.3333
```

The message names the type the library inferred from your argument and the type the metric expects, and the fix is to pass what it expects, here the integer 2. The second pair of lines shows the case you will meet most often. The count query's input metric is the symmetric distance, whose distances are unsigned integers, so `map(1.0)` fails and `map(1)` succeeds. One more cause the documentation names is a scalar ε passed to a measurement whose measure expects a pair of

ε and the slack δ of approximate differential privacy. In 0.16.0 that case reads differently, so the script prints it too.

```
scalar epsilon on an approximate measure: TypeError: Cannot coerce a non-tuple type to a tup
pair on an approximate measure: True
```

The chapter on limitations and troubleshooting collects the other errors you can meet. The rule for this one is short. Read the expected type, then change your argument, your bounds or your data to match it. Never change the domain to match the argument, because the library chose that type for the proof.

12.2. Parameter search

A measurement has three kinds of parameter. The input distance `d_in` is the privacy unit's bound, the output distance `d_out` is the budget, and the constructor's own arguments include the noise scale. With every parameter fixed, `check(d_in, d_out)` answers whether the measurement stays within the budget, as the measurements chapter showed. The answer is monotone. If a scale passes at some budget, every larger scale passes too, and if a budget passes, every larger budget passes. A monotone yes-or-no function has one boundary, and a binary search finds it, so you can solve for any one parameter from the others.

The library wraps the search three ways. The first, `dp.binary_search`, takes a predicate on one number and returns the value nearest the boundary on the passing side. The second, `dp.binary_search_param`, takes a function that builds a chain from one number, with `d_in` and `d_out`, and uses `check` as the predicate. The third, `dp.binary_search_chain`, does the same and returns the chain built at the value it found.

```
import opendp.prelude as dp

print(dp.binary_search(lambda x: x >= 5.0))  # 5.0
print(dp.binary_search(lambda x: x > 5, T=int))  # 6
print(dp.binary_search(lambda x: x <= 5, bounds=(0, None)))  # 5
```

The three lines show the arguments that steer a search. Without bounds, the search first looks for them by stepping outwards from zero, which is why the first call needs nothing else. The type `T` says whether the parameter is a float or an integer when the predicate alone does not tell, and the integer search returns the first passing integer. A pair of bounds with `None` on one side gives a one-sided search. The script then runs the search a context runs, on the Laplace mechanism of the first chapter with a share of $\varepsilon/3$.

```
scale = dp.binary_search_param(make_laplace, d_in=1.0, d_out=share)  # the smallest scale wi
chain = dp.binary_search_chain(make_laplace, d_in=1.0, d_out=share)  # the measurement at th
largest_d_in = dp.binary_search(lambda d_in: make_laplace(1.0).check(d_in, EPSILON))  # the
```

```
search: scale=3.0000000000000004 chain.map(1.0)=0.3333 chain.check(1.0, share)=True
search: largest d_in at scale 1 within epsilon 1: 1.0
search: gaussian scale for rho=1 at d_in=1: 0.7071
```

The scale is the smallest one whose map at a sensitivity of 1 stays within a third of $\varepsilon = 1$. It comes out a hair above 3, because the search stops on the passing side of the boundary. That is the scale `param()` reported in the first chapter. The chain is the Laplace measurement at that scale, and its check confirms the budget. The third call turns the question round and asks for the largest sensitivity that a scale of 1 can hide within $\varepsilon = 1$, which is 1. The Gaussian mechanism is measured in zero-concentrated differential privacy (zCDP), which states the privacy loss as a parameter ρ. For Gaussian noise that loss is half the square of the sensitivity over the scale, so the same search for $\rho = 1$ gives a scale of 0.7071. The full edition's chapter on the two mechanisms derives both maps.

To see what the search does, the script writes it by hand. The predicate is `check`, the interval starts at 0 to 100, and each step keeps the half that contains the boundary.

```
def search_by_hand(passes: Callable[[float], bool], low: float, high: float, tolerance: floa
    """Bisection on a predicate that fails at `low` and passes at `high`: the smallest passi
    steps = 0
    while high - low > tolerance:
        middle = (low + high) / 2
        if passes(middle):
            high = middle  # the boundary is at or below the middle
        else:
            low = middle  # the boundary is above it
        steps += 1
    return high, steps
```

```
search by hand: scale=3.000000 after 27 steps
search with bounds (0, 100): 3.0000000000000004
```

Twenty-seven halvings of an interval of 100 bring it below a millionth, and the passing end of the last interval is the scale. The library's search on the same bounds lands on the float just above 3, so it bisects further than that tolerance. Two things go wrong with a search, and the script shows both messages.

```
search with bounds (0, 1): ValueError: the decision boundary of the predicate is outside the
search on a constant predicate: ValueError: unable to infer bounds
```

Bounds that do not contain the boundary give the first error, so widen them. A predicate that never changes gives the second, and here it is a predicate that always passes. With a real chain it means that no value of the parameter changes the answer, so look at the chain before the bounds.

12.3. Serialization

A measurement is a recipe. It names the constructors it was built from and their arguments, and `dp.serialize` writes that recipe as a JSON string, which `dp.deserialize` turns back into a measurement. The use is a split deployment. An analyst builds and inspects a query on a machine without the data, and sends the text to the machine that holds them. The script serializes the first chapter's count measurement.

```
measurement = count_laplace(3.0)
text = dp.serialize(measurement)  # JSON that names each constructor and its arguments
restored = dp.deserialize(text)
same_space = restored.input_space == measurement.input_space
```

```
serialized: 2152 characters, starting {"__function__": "make_chain_mt",
restored: map(1)=0.3333 same input space: True
```

The text begins with the outermost constructor, the chaining of a transformation and a measurement, and nests the rest inside it. The restored measurement has the same input space and the same privacy map, so a reviewer who receives the text can check the guarantee before the data are touched. The format is JSON, but the documentation does not promise that it stays the same between versions, and it asks you not to write it by hand.

Two things do not survive. The first is the `param` attribute that a context attaches to a measurement whose scale it searched for, which a later section explains. It is a Python attribute rather than part of the recipe, so it is gone after a round trip. The second is anything with Python state that the constructors did not set, which includes a context, a query and a plugin built from a Python function.

```
param before a round trip: 3.0000000000000004
param after a round trip: None
serialize a context: OpenDP JSON Encoder does not handle instances of <class 'opendp.context
```

A Polars query has its own route, because its plan is a Polars `LazyFrame` and Polars can serialize a plan to bytes. The method `Context.deserialize_polars_plan` takes those bytes and wraps them in a query on a context of its own. So a client can write the plan against the survey's schema and the server can run it against the rows.

```
client = lfs_context(pl.LazyFrame(schema=frame.collect_schema()))  # the schema, but no rows
plan = client.query().select(dp.len(signed=True)).polars_plan.serialize()  # bytes
server = lfs_context(frame)  # the real data
query = server.deserialize_polars_plan(plan)
summary = query.summarize(alpha=0.05)
```

```
plan: 1792 bytes
plan on the server: scale=36.0 95% accuracy=108.3429
```

The client's context holds an empty frame with the survey's schema, and the server's holds the file. Both state the privacy unit of 36 rows per person and the budget of the chapter on the data and the context. They must, because the plan carries the query but not the guarantee. The scale of 36 is the unit's distance over the budget, and the error bar is about 108 rows on a count of millions. The release itself is in the block at the end of the chapter.

12.4. The compositor behind a context

The rest of the chapter rebuilds the Context API from the combinators of the previous chapter. The constructor `dp.Context.compositor` builds one measurement, an **adaptive composition** over the data's space, and calls it on the data. Adaptive composition takes a list of per-query budgets, the `d_mids`, and lets you submit one measurement per share. Each may be chosen after seeing the earlier answers, and the promise is that together they stay within the sum. The full edition's chapter on composition proves that the budgets add. The script reads the context's accountant and builds the same one by hand.

```
by_hand_accountant = dp.c.make_adaptive_composition(
    input_domain=SPACE[0],
    input_metric=SPACE[1],
    output_measure=dp.max_divergence(),
    d_in=1,  # the privacy unit's distance: one row added or removed
    d_mids=[share] * QUERIES,  # the shares, which add up to the budget
)
queryable = by_hand_accountant(data)  # answers measurement queries on the data, and keeps c
```

```
context accountant: VectorDomain(AtomDomain(nan=true, T=f64)) SymmetricDistance() MaxDiverge
context d_in: 1  d_mids: [0.3333, 0.3333, 0.3333]
context accountant map(1): 1.0000
by hand accountant map(1): 1.0000  queryable: Queryable(Q=AnyMeasurement)
d_mids by weights [1, 2, 1]: [0.25, 0.5, 0.25]
```

The domain was inferred from the data, a list of floats, and the metric and the measure are the halves of `unit_of` and `loss_of` that the first chapter printed. The privacy map of the composition at the unit's distance is the whole budget, 1, from either construction. Calling the accountant on the data returns a **queryable**, an object that answers one measurement at a time and refuses when the shares run out. That queryable is what the context keeps. The data are inside it and nowhere else, which is why the first chapter told you to drop your own reference.

The shares come from `split_evenly_over` or `split_by_weights`, and the context does not divide the budget itself. It hands the weights to `binary_search_param` and scales them until the composition's map at `d_in` meets the budget, which for pure differential privacy is the proportional split you would expect. Weights of 1, 2 and 1 give a quarter, a half and a quarter of ε.

12.5. The query behind a release

`context.query()` returns a **Query**, an object that holds a chain and the share it must fit. On plain data it is a `Query`, and on a LazyFrame it is the `LazyFrameQuery` of the Polars chapters, built on the same machinery. The chain starts as the data's space. Each method you call, such as `count` or `laplace`, looks up the constructor of that name and chains it on. So `query.count()` is `space >> dp.t.then_count()`, with the space supplied by the context. The script watches the chain change type.

```
query = plain_query(context)  # the chain is the data's space: a domain and a metric
counted = query.count()  # the chain is a Transformation
noisy = counted.laplace()  # no scale: the chain is a PartialChain, waiting for one number
is_partial = isinstance(noisy._chain, dp.PartialChain)
```

```
query chain: tuple -> Transformation -> PartialChain
partial chain: True
resolve a transformation: ValueError: Query is not yet a measurement or odometer.
```

The interesting step is `laplace()` without a scale. The query counts the arguments against the constructor's signature and finds one missing. Instead of calling the constructor, it wraps it in a **partial chain**, a function from the missing number to the finished chain. Only one number may be missing at a time, which is why a second noise step without a scale fails below. A query that is still a transformation cannot be resolved into a measurement, which the third line shows.

`resolve` is where the number is found. It calls the partial chain's `fix`, which runs `binary_search` with the predicate "the chain at this value checks within the query's share". That is the search of Section 12.2, and `fix` stores the value it finds on the measurement as `param`. So `param()` reports the scale the search found, and `None` when nothing was searched for.

```
param of .laplace(): 3.0000000000000004
param of .laplace(scale=3.0): None
two missing parameters: ValueError: gaussian is missing 1 parameter(s).
```

The script now runs the same search by hand on the same chain, and releases through the by-hand queryable.

```
by_hand_param = dp.binary_search_param(count_laplace, d_in=1, d_out=share)  # what resolve r
by_hand_measurement = count_laplace(by_hand_param)
dp_count_by_hand = queryable(by_hand_measurement)  # what release does: the queryable runs t
```

```
resolve: context param=3.0000000000000004 by hand=3.0000000000000004 equal=True
d_mids after context(measurement): [0.3333, 0.3333]
```

The two scales are not just close but equal, because they are the same search on the same predicate. `release` does two things with the resolved measurement. It passes the measurement to the context, which hands it to the queryable and records the share as spent, so the first share

leaves `d_mids`. The by-hand version does the same with `queryable(measurement)`, except that nothing records the spend. That bookkeeping is what the context adds, and the reason to use it.

12.6. Nested compositors and other measures

Two features of the Context API are easier to understand once the chain is visible. The first is `Query.compositor`, which nests an adaptive composition inside one share. The query's share becomes the inner budget, split again, and releasing the query returns a new `Context` rather than a number.

```
inner = plain_query(context).compositor(split_evenly_over=2).release()  # a Context inside o
inner_scale = plain_query(inner).count().laplace().param()
```

```
inner context: d_in=1 d_out=0.3333 d_mids=[0.1667, 0.1667]
inner count scale: 6.0000
outer d_mids after the inner compositor: [0.3333]
```

The inner context has the outer share, a third of ε, as its whole budget, and splits it in two, so a count inside it gets a scale of 6. The outer context counts the nested compositor as one query spent, whatever happens inside it, because the composition's guarantee covers everything its queryable answers.

The second feature is a budget stated in one measure and a mechanism proved in another. The Laplace mechanism's measure is the max divergence of pure differential privacy, and the Gaussian mechanism's is zCDP. When the context's measure differs, `resolve` casts the measurement with the converters of the combinators chapter before it checks the budget.

```
approx = dp.Context.compositor(
    data=data,
    privacy_unit=dp.unit_of(contributions=1),
    privacy_loss=dp.loss_of(epsilon=EPSILON, delta=DELTA),  # approximate differential priva
    split_evenly_over=QUERIES,
)
laplace_cast = plain_query(approx).count().laplace().resolve()  # pure DP, cast to approxima
gaussian_cast = plain_query(approx).count().gaussian().resolve()  # zCDP, converted and its
```

```
approx share: (0.3333, 3.3e-08)
laplace under (epsilon, delta): Approximate(MaxDivergence) map(1)=(0.3333, 0.0e+00)
gaussian under (epsilon, delta): Approximate(MaxDivergence) map(1)=(0.3333, 3.3e-08)
param after a cast: None
```

Under an (ε, δ) budget the Laplace mechanism is wrapped by `make_approximate`, which reports its loss as the pair $(\varepsilon, 0)$. The slack goes unused, and the search fits only the ε part. The Gaussian mechanism goes through `make_zCDP_to_approxDP` and `make_fix_delta`, which spend the share's

slack to convert ρ into an ε. One side effect matters. The cast returns a new measurement that wraps the old one, so the `param` attribute sits on the inner measurement and `param()` reads `None`. To see the scale, run `binary_search_param` on the chain yourself, as the previous section did.

```
laplace under rho: ZeroConcentratedDivergence map(1)=0.0333
gaussian under rho: scale=3.8730
gaussian under epsilon: ValueError: Unable to cast measure from ZeroConcentratedDivergence t
```

Under a ρ budget the Laplace mechanism is cast with `make_pureDP_to_zCDP`, and the Gaussian mechanism needs no cast, so its `param()` works and reports a scale of 3.873. The one direction that cannot be cast is a Gaussian mechanism under a pure ε budget, because zCDP never implies pure differential privacy, and the context says so instead of guessing. The full edition's chapter on concentrated differential privacy derives the conversions behind each cast.

Finally the releases of one run, read against their error bars.

```
released (one run):
  survey rows 3811875  (true 3811932, within 108.3 with probability 0.95)
  count by hand 94  (true 100, within 9.4 with probability 0.95)
  count through the context 101  (true 100)
```

All three landed within their bars. The two counts of the hundred integers came from the same measurement, once through the by-hand queryable and once through the context, and differ only by their noise.

12.7. Summary

Table 12.1.: The utilities and the pieces of the Context API, what each does, and the rule of thumb that goes with it.

Call or concept	What it does	Rule of thumb
`"i32"`, `int`, `dp.i32`	Three spellings of one type descriptor	`int` is `i32` and `float` is `f64`; NumPy's integers are `i64`
`dp.set_default_int_type(dp.i64)`	Changes what `int` means for the rest of the program	The clean fix when all your integers have 64 bits
`inferred type is XXX, expected YYY`	Your argument's type does not match the metric's or the domain's	Change the argument to the expected type, not the domain to the argument
`dp.binary_search(predicate, bounds, T)`	The value nearest the boundary of a monotone predicate	Pass bounds when the chain is only valid in a range, and `T=int` for an integer
`dp.binary_search_param(make_chain, d_in, d_out)`	The smallest scale whose chain checks within the budget	What a context runs for `.laplace()` without a scale

Call or concept	What it does	Rule of thumb
`dp.binary_search_chain(...)`	The chain built at the value the search found	It carries no `param`; use `binary_search_param` for the number
`dp.serialize`, `dp.deserialize`	A measurement as JSON, and back	The recipe survives; `param`, contexts and Python plugins do not
`Context.deserialize_polars_plan(bytes)`	A Polars plan from a client, run on the server's context	The plan carries the query; each context states the guarantee
`dp.Context.compositor(...)`	An adaptive composition over the data's space, called on the data	Its queryable holds the data; `d_mids` are the shares
`context.query()`	A `Query` whose chain grows with each constructor	A missing number makes a partial chain; one at a time
`query.param()`	The scale that `resolve` found	`None` when the scale was given or the measure was cast
`Query.compositor(...)`	A context nested inside one share	The outer context counts it as one query
A Gaussian query under a pure budget	`ValueError: Unable to cast measure ...`	zCDP never implies pure differential privacy; state the budget with a δ or as ρ

12.8. References

- OpenDP documentation: "Typing", "Parameter Search" and "Serialization" under "Utilities" in the API user guide; "Context" and "API Comparison" in the same guide; and the entry `Inferred type is XXX, expected YYY` of "Troubleshooting" in Getting Started.
- Gaboardi, Hay and Vadhan, "A Programming Framework for OpenDP" (2020): the domains, metrics, measures and maps that the type descriptors and the searches work on.
- Dwork, Rothblum and Vadhan, "Boosting and Differential Privacy" (2010): composition of adaptively chosen mechanisms, which the compositor behind a context implements.
- Bun and Steinke, "Concentrated Differential Privacy: Simplifications, Extensions, and Lower Bounds" (2016): zCDP and its conversions to approximate differential privacy, which the measure casts use.
- The full edition's chapters on the Laplace and Gaussian mechanisms in the full edition, composition in the full edition and concentrated differential privacy in the full edition derive the results this chapter states.

Part IV.

Extending the library

The framework is open. Anything with a stability map or a privacy map can be a transformation or a measurement, whether OpenDP wrote it or you did. This part uses that openness. Its thread is that a plugin is only as private as its map. So the work of writing one is the proof, and the library's job is to make everything around the proof safe, from chaining to accounting.

Plugins writes a transformation and a measurement in Python and registers constructors so that they work from the Context API. It then follows the documentation's two larger examples, a Theil-Sen regression and a private choice of grouping columns. Statistical modelling fits the two models that ship with the library. One is a differentially private linear regression, and the other a differentially private principal component analysis. The chapter reads what each guarantees. Limitations and troubleshooting says what OpenDP does not promise, from floating-point arithmetic to side channels. It explains the feature flags that gate the less vetted parts, and it decodes the error messages you will meet most often.

Table 12.2.: The chapters of this part and the questions they answer.

Chapter	Question it answers
Plugins	How do I add my own transformation or measurement, and what must I prove about it?
Statistical modelling	How do I fit a private regression or a private principal component analysis with the library's estimators?
Limitations and troubleshooting	What does the library not guarantee, and what does each common error mean?

By the end of this part you will be able to extend OpenDP with a mechanism of your own and fit its built-in models. You will also be able to say which parts of a deployment the library covers and which parts remain yours.

13. Plugins

Suppose the mechanism you need is not in the library, such as a new estimator from a paper or a score that only makes sense for your data. The framework of the previous part was built for this case. A transformation is any function with a stability map, and a measurement is any function with a privacy map. So you can write both in Python and hand them to the library as **plugins**. From then on they chain, compose and run through a context like the built-in ones.

The one idea of this chapter is that a plugin is exactly as private as its map, and the library cannot check the map. Every built-in constructor comes with a written proof of its map, and for a plugin that proof is yours. So each example below shows its calls and then says in a sentence why its map is right, or where the documentation leaves that open. You will write a measurement, a transformation, and your own domains, metrics and measures, then register a constructor for the Context API. The chapter ends with the documentation's two larger examples, a Theil-Sen regression and a private choice of grouping columns.

The script opendp_plugins.py runs every step on OpenDP 0.16.0. Its regression runs on the documentation's toy data, a hundred points on a line with normal noise from a seeded generator. Its choice of grouping columns runs on the Labour Force Survey (LFS) sample. Every map, check, error message, score and candidate range on this page is the same on every run. The released values are not, because OpenDP's noise cannot be seeded, so they appear in one block marked as one run at the end.

```
uv run python code/part04-extending/opendp_plugins.py
```

13.1. Trust and the honest-but-curious flag

The documentation gives two reasons for plugins. Differential privacy is a wide and growing field, so the library cannot ship every mechanism. And an idea is easier to try in Python than in Rust, so a plugin is how a contribution usually starts. Both reasons put a Python function at the heart of a guarantee, and that changes who is responsible for it.

The library can inspect its own constructors, but it cannot read a Python function. So it cannot tell whether your privacy map is right, whether your function has side effects, or whether it raises only when the data make it. The constructors of this chapter therefore need a second feature flag next to `contrib`, named `honest-but-curious`, and the name is the trust model it describes. Whoever writes the plugin is honest, in that the map they write is one they have proved, and the library guards only against curiosity from everyone downstream. Without the flag a plugin constructor refuses to build.

```
without the flag: Attempted to use function that requires "contrib", "honest-but-curious", b
```

Each constructor's docstring spells out the promise. Take any two inputs in the domain that are at most `d_in` apart under the input metric. Then the two output distributions must be at most `privacy_map(d_in)` apart under the output measure, the function must have no side effects, and the map must be pure. A wrong map breaks the guarantee silently, because everything downstream believes it. The context charges what the map reports, the composition adds up the maps, and the release goes out. The snippet is the extreme case, a measurement that returns its input and claims a privacy loss of zero.

```
import opendp.prelude as dp

dp.enable_features("contrib", "honest-but-curious")
space = dp.vector_domain(dp.atom_domain(T=int)), dp.symmetric_distance()
leak = dp.m.make_user_measurement(*space, dp.max_divergence(), function=lambda x: x, privacy
print(leak.map(1))  # 0.0
print(leak([1, 2, 3]))  # [1, 2, 3]
```

The library accepts it, and a context would release the raw data at no charge. Nothing in the rest of this chapter protects you from this. The flag records that the claim rests on your word, and the sentence that justifies each map below is the habit that makes the word worth something.

13.2. A measurement plugin

The documentation's first example is deliberately useless, a measurement that ignores its input and returns a constant. It is useful here because its map is the easiest one to justify. The constructor `make_user_measurement` takes the five things that define a measurement in A framework to understand differential privacy. They are an input domain, an input metric, an output measure, the function and the privacy map.

```
def make_base_constant(constant: str) -> dp.Measurement:
    """The documentation's example: a measurement that ignores its input and returns a const

    def function(_arg: int) -> str:
        return constant  # the output does not depend on the data

    def privacy_map(_d_in: int) -> float:
        return 0.0  # so two neighbouring inputs give the same distribution: a privacy loss

    return dp.m.make_user_measurement(
        input_domain=dp.atom_domain(T=int),
        input_metric=dp.absolute_distance(T=int),
        output_measure=dp.max_divergence(),
        function=function,
        privacy_map=privacy_map,
```

```
        TO=str,
    )
```

The input domain says the function takes one integer, and the input metric compares two inputs by their absolute difference. The output measure is the max divergence, the measure of pure differential privacy, so the map's value is an ε. The map returns zero at every distance, and that is correct because the output does not depend on the input, so two neighbouring inputs give the same output distribution. The last argument, `TO`, names the type the function returns. The script chains the result after a sum on a vector of bounded integers.

```
constant: input=AtomDomain(T=i32) metric=AbsoluteDistance(i32) measure=MaxDivergence
sum >> constant: map(1)=0.0 release=denied
```

The chain works because the sum's output space, an integer under the absolute distance, is exactly the plugin's input space. The library checks that match, because domains and metrics are its own objects, but it does not check the map.

A plugin with a real map shows what the justification looks like. The next constructor adds Laplace noise drawn by NumPy rather than by OpenDP. It takes its input space as its first two arguments, which is the convention the Context API will need later.

```
def make_numpy_laplace(input_domain: dp.Domain, input_metric: dp.Metric, scale: float) -> dp
    """Laplace noise drawn by NumPy instead of OpenDP: the same map as the built-in, but on
    rng = np.random.default_rng()

    def function(arg: float) -> float:
        return float(arg + rng.laplace(scale=scale))

    def privacy_map(d_in: float) -> float:
        return d_in / scale  # the Laplace mechanism's loss: the sensitivity over the scale

    return dp.m.make_user_measurement(
        input_domain, input_metric, dp.max_divergence(), function=function, privacy_map=priv
    )
```

The map is the sensitivity over the scale. Laplace noise of a given scale on a quantity that one person moves by at most the sensitivity costs exactly that ratio in ε. The full edition's chapter on the Laplace and Gaussian mechanisms derives the bound. The built-in `then_laplace` at the same scale reports the same map, which is the check to run on any plugin with a built-in cousin.

```
python laplace scale 2: map(1.0)=0.5 built-in map(1.0)=0.5
halved map, same noise: map(1.0)=0.25 check(1.0, 0.25)=True
laplace scale 2: 95% accuracy=5.9915
```

The second line is the same noise with a map that claims half the loss. The library accepts it, and `check` confirms a fit within a quarter of a unit of ε, which is false. A context would hand such a

plugin a quarter and spend half. The third line is the error bar at scale 2, which the released value at the end of the page is read against. In production, prefer the Rust core, because NumPy's float sampling leaks a little through rounding and OpenDP's own samplers avoid that.

13.3. A transformation plugin

A transformation plugin takes one more argument than a measurement, an output domain. Its map is a stability map, which bounds how far apart two outputs can be when the inputs are a given distance apart, as Transformations introduced. The documentation's example repeats each record a fixed number of times.

```
def make_repeat(multiplicity: int) -> dp.Transformation:
    """The documentation's example: a transformation that repeats each record `multiplicity`

    def function(arg: list[int]) -> list[int]:
        return arg * multiplicity

    def stability_map(d_in: int) -> int:
        return d_in * multiplicity  # one added record of the input is `multiplicity` added

    return dp.t.make_user_transformation(
        input_domain=dp.vector_domain(dp.atom_domain(T=int)),
        input_metric=dp.symmetric_distance(),
        output_domain=dp.vector_domain(dp.atom_domain(T=int)),
        output_metric=dp.symmetric_distance(),
        function=function,
        stability_map=stability_map,
    )
```

Both metrics are the symmetric distance, the number of records added or removed between two datasets. The map multiplies that distance by the multiplicity. Every input record becomes `multiplicity` output records, so adding or removing one input record adds or removes that many output records. The output domain is a second claim you make. Every output must belong to it, or the function must fail in a way that does not depend on the data. Here the output is a vector of integers whenever the input is.

The documentation chains the plugin between built-in steps, a cast from strings and a clamp before a sum, and ends with Laplace noise.

```
repeat 2: [1, 2] -> [1, 2, 1, 2] map(1)=2 map(3)=6
cast >> repeat 2 >> clamp (1, 2) >> sum: map(1)=4 on ['0', '1', '2', '3'] = 12
... >> laplace scale 1: map(1)=4.0
domain mismatch: DomainMismatch("Intermediate domains don't match
```

The stability of a chain is the product of the stabilities of its links, as Combinators explains. The cast and the clamp have stability 1, and the repeat has 2. The clamp bounds every value by 2, so

one added input record, repeated twice, moves the sum by at most 4. Laplace noise of scale 1 then costs that much ε. The last line is what happens when the cast produces floats instead. The plugin's input domain is a vector of integers, and the library refuses the chain, because the spaces are its objects and it can compare them.

It cannot compare the function with the output domain, and that is the second way a transformation plugin goes wrong. The snippet declares that its output lies between 0 and 1 and then returns its input unchanged.

```
import opendp.prelude as dp

dp.enable_features("contrib", "honest-but-curious")
ints = dp.vector_domain(dp.atom_domain(T=int))
bounded = dp.vector_domain(dp.atom_domain(bounds=(0, 1)))
claim = dp.t.make_user_transformation(ints, dp.symmetric_distance(), bounded, dp.symmetric_d
print(claim([5, 6]))  # [5, 6]
print((claim >> dp.t.then_sum()).map(1))  # 1
```

The sum after it trusts the bounds and reports a sensitivity of 1, while the real sum moves by the whole value of a record. A wrong output domain is a wrong map one step later, so a transformation plugin needs two justifications, one for each.

13.4. Your own domains, metrics and measures

Sometimes a plugin works on data the library has no domain for, such as a data frame or a graph, or compares neighbours by a metric it does not define. Three constructors let you name your own, and each carries its own promise.

A **user domain** is built from an identifier and a membership function. Two user domains are equal when their identifiers are equal, which is how the library decides whether two steps chain. So the identifier must name one set and nothing else, or two different domains would chain and break the stability of the chain. The membership function must return true only for members. A **user distance** is a metric that is nothing but a name, and giving it is a promise that the distance you have in mind is a pseudo-metric. It is zero from a point to itself, never negative and symmetric, and it obeys the triangle inequality. The snippet declares a domain of even integers and an edit distance on strings, and uses the distance in a transformation that measures a string's length.

```
import opendp.prelude as dp

dp.enable_features("contrib", "honest-but-curious")
even = dp.user_domain("EvenInts", member=lambda x: x % 2 == 0)
print(even.member(4), even.member(3))  # True False
edits = dp.user_distance("EditDistance")
length = dp.t.make_user_transformation(dp.atom_domain(T=str), edits, dp.atom_domain(T=int),
print(length("hello"), length.map(1))  # 5 1
print((length >> dp.m.then_laplace(scale=1.0)).map(1))  # 1.0
```

The stability map of the length is the identity, because one edit changes the length by at most one. The output metric is the absolute distance on integers, which is what the Laplace measurement wants. So the chain closes, and a scale of 1 costs an ε of 1 per edit.

A **user divergence** is a privacy measure of your own. The one property a measure must have is that post-processing cannot increase it, since the promise that a release stays private under further computation rests on it. The full edition's chapter on properties proves that for the max divergence. Two more constructors complete the set. The function `new_privacy_profile` turns a curve from ε to the slack δ into a privacy profile the library can invert. And `new_function` wraps a Python post-processor, which Combinators did behind the scenes for a bare function after `>>`. The post-processor must not read global state or raise on the data, because either would open a channel from the data to the caller that no map accounts for.

```
import numpy as np
import opendp.prelude as dp

dp.enable_features("contrib", "honest-but-curious")
space = dp.atom_domain(T=float, nan=False), dp.absolute_distance(T=float)
renyi = dp.user_divergence("RenyiDivergence(alpha=2)")
rng = np.random.default_rng()
gaussian = dp.m.make_user_measurement(*space, renyi, function=lambda x: float(x + rng.normal
print(gaussian.map(1.0), gaussian.output_measure)  # 1.0 RenyiDivergence(alpha=2)
try:
    dp.c.make_composition([gaussian, gaussian])
except dp.OpenDPException as err:
    print(str(err).split(". See")[0].strip())  # Type("No match for concrete type ExtrinsicD
profile = dp.new_privacy_profile(lambda eps: 1.0 if eps < 0.5 else 1e-8)
print(profile.delta(epsilon=0.499), profile.epsilon(delta=1e-8))  # 1.0 0.5
double = dp.new_function(lambda x: 2 * x, TO=float)
laplace = space >> dp.m.then_laplace(scale=1.0)
print((laplace >> double).map(1.0))  # 1.0
```

The measurement adds standard normal noise and reports its loss under the Rényi divergence of order 2. Its map is the squared sensitivity, because the Rényi divergence of order 2 between two unit Gaussians is the square of the distance between their centres. The full edition's chapter on Rényi differential privacy derives that. The map works, but the composition does not, because the combinators add up only the measures the library defines. So a plugin measure gets `map` and `check` and nothing more. The profile answers both ways, the slack at a given ε and the smallest ε at a given slack, and its curve must be decreasing. The post-processor leaves the Laplace measurement's map unchanged, which is the point of post-processing.

13.5. Registering a constructor for the Context API

A plugin built with the framework API is a loose measurement, and the lesson of A first release was to put every query through a context. The function `dp.register` adds a constructor to the Context API, as a method of `context.query()`. Two rules govern it. The constructor's name

must start with `make_`, and the method is the name without that prefix, unless you pass a `name`. And if its first two parameters are named `input_domain` and `input_metric`, the context fills them in. They come from the previous step's output space, or from its own input space for a first step. The documentation calls that partial application, and it is the convention that gives every built-in `make_` constructor its `then_` form.

```
def make_anything_constant(input_domain: dp.Domain, input_metric: dp.Metric, constant: str)
    """The documentation's constructor for the Context API: it accepts whatever space the pr

    def function(_arg: Any) -> str:
        return constant

    def privacy_map(_d_in: Any) -> float:
        return 0.0

    return dp.m.make_user_measurement(
        input_domain, input_metric, dp.max_divergence(), function=function, privacy_map=priv
    )


dp.register(make_anything_constant)  # from now on, context.query().anything_constant(...)
```

The script builds the documentation's context over three integers, with a privacy unit of 36 contributions and a budget of $\varepsilon = 1$ split over two queries. Then it calls the plugin through it.

```
anything_constant: map(36)=0.0 release=denied
budget spent by the plugin query: [0.5]
clamp >> sum >> int_constant: release=denied
register twice: 'anything_constant' is already registered in the Context API. Please choose
register without make_: constructor.__name__ must start with 'make_', found select_cols
```

On the second line the plugin's map is zero, but the context charged the query its full share of the budget. A context splits the budget before it sees a query and only checks that each query fits within its share, as Utilities and the Context API inside shows. A plugin that costs less than its share does not give the difference back. The third line uses the documentation's other style, `make_int_constant`, without the two leading parameters. It registers too, but its input space is fixed to integers under the absolute distance, so it works after a clamp and a sum but not elsewhere. The documentation recommends the first style, with the caution that a constructor accepting any space should check that the space suits its function. The last two lines are the errors `register` raises, for a name already taken and for a name without the prefix.

13.6. A Theil-Sen regression

The documentation's first larger example fits a line through noisy points. Least squares is a poor private estimator, because one far-away point can move the fitted line by any amount. The

Theil-Sen estimator takes the median of the slopes through pairs of points instead, and a median is a quantile, which OpenDP already releases privately. The documentation's variant pairs the points at random and fits a line through each pair. It records where that line crosses two fixed vertical cuts, at a quarter and three quarters of the range of x. The medians of the crossings at each cut are two points on the fitted line, which give the slope and the intercept. The library ships a vetted version of this estimator, which Statistical modelling uses, so this one is for learning how plugins combine.

The first plugin is the pairing, a transformation from a two-column array to a two-column array of predictions.

```
def pairwise_predict(data: NDArray[np.float64], x_cuts: NDArray[np.float64]) -> NDArray[np.f
    """Pair the rows at random, fit a line through each pair, and predict y at the two cut p
    data = np.array(data, copy=True)[: len(data) // 2 * 2]
    np.random.shuffle(data)
    p1, p2 = np.array_split(data, 2)
    dx, dy = (p2 - p1).T
    x_bar, y_bar = (p1 + p2).T / 2
    points = dy / dx * (x_cuts[None].T - x_bar) + y_bar
    return points.T[dx != 0]
```

The transformation wraps that function.

```
def make_pairwise_predict(x_cuts: NDArray[np.float64], runs: int = 1) -> dp.Transformation:
    """A transformation from the (x, y) rows to the predictions: one row per pair and run.""

    def function(arg: NDArray[np.float64]) -> NDArray[np.float64]:
        return np.vstack([pairwise_predict(arg, x_cuts) for _ in range(runs)])

    def stability_map(d_in: int) -> int:
        return d_in * runs  # each record sits in one pair, so it changes one prediction per

    return dp.t.make_user_transformation(
        input_domain=dp.numpy.array2_domain(num_columns=2, T=float),
        input_metric=dp.symmetric_distance(),
        output_domain=dp.numpy.array2_domain(num_columns=2, T=float),
        output_metric=dp.symmetric_distance(),
        function=function,
        stability_map=stability_map,
    )
```

The domains come from the library's NumPy extension, a two-dimensional array with two columns. The documentation states the map, the input distance times the number of runs, without an argument, so here is the one it relies on. Each record sits in exactly one pair, so adding or removing a record changes at most one prediction per run. Two things make that claim weaker than a built-in map. The pairing is random, so the claim is really about a coupling of two runs rather than about a function. And the function drops the last record when the count is odd, so

removing one record can also change another's pair. This is the kind of gap the flag asks you to notice.

The second plugin selects one column of the predictions, and the library's quantile does the rest.

```
def make_select_column(j: int) -> dp.Transformation:
    """Column j of the predictions as a vector; one row in is one row out, so the stability

    def function(arg: NDArray[np.float64]) -> NDArray[np.float64]:
        return arg[:, j]

    def stability_map(d_in: int) -> int:
        return d_in

    return dp.t.make_user_transformation(
        input_domain=dp.numpy.array2_domain(num_columns=2, T=float),
        input_metric=dp.symmetric_distance(),
        output_domain=dp.vector_domain(dp.atom_domain(T=float)),
        output_metric=dp.symmetric_distance(),
        function=function,
        stability_map=stability_map,
    )
```

The library's quantile then runs once for each cut.

```
def make_private_medians(output_measure: dp.Measure, y_bounds: tuple[float, float], scale: f
    """The private median of the predictions at each cut, by the library's quantile, compose
    median = dp.m.then_private_quantile(output_measure, candidates=np.linspace(*y_bounds, 10
    return dp.c.make_composition(
        [
            make_select_column(0) >> dp.t.then_drop_null() >> median,
            make_select_column(1) >> dp.t.then_drop_null() >> median,
        ]
    )
```

Selecting a column keeps one row per row, so its stability is the input distance. The private quantile scores a hundred candidate values between the public bounds of y and picks one by noisy max, as Measurements showed. Its map is the input distance over the scale. The composition adds the two maps, by the basic composition rule of the full edition's chapter on composition. The third piece solves for the line, which is post-processing of two released numbers and costs nothing.

```
def make_private_theil_sen(
    output_measure: dp.Measure, x_bounds: tuple[float, float], y_bounds: tuple[float, float]
) -> dp.Measurement:
    """Pair and predict, release the two medians, then solve for the slope and the intercept
    x_cuts = x_bounds[0] + (x_bounds[1] - x_bounds[0]) * np.array([0.25, 0.75])
```

```
    p_inv = np.linalg.inv(np.vstack([x_cuts, np.ones_like(x_cuts)]).T)  # the line through t

    def to_coefficients(medians: list[float]) -> tuple[float, float]:
        slope, intercept = p_inv @ np.array(medians)
        return float(slope), float(intercept)

    return make_pairwise_predict(x_cuts) >> make_private_medians(output_measure, y_bounds, s
```

The script runs it with the documentation's bounds and a scale of 1, on the toy points. Their true line has slope 2 and intercept 1.

```
theil-sen: x cuts -1.5 and 1.5, 100 candidates from -10.0 to 10.0 step 0.2020
pairwise predict: map(1)=1 medians: map(1)=2.0 theil-sen: map(1)=2.0 map(2)=4.0
toy data: n=100 least-squares slope=2.0573 intercept=1.0376
pairs: 50 medians at the cuts -2.2424 and 4.2857
selection bound: 26.2 score units at noise scale 2.0
cut -1.5: best candidate -2.3232 score 2; within the bound: -2.9293 to -1.5152 (8 candidates
cut 1.5: best candidate 4.3434 score 8; within the bound: 3.3333 to 6.3636 (16 candidates)
```

The pairing has stability 1 and the two medians cost 2 together, so the whole estimator costs an ε of 2 per record. The least-squares fit of the toy is close to the truth, and the exact medians of the fifty pairwise predictions are the numbers the private medians aim at.

The last three lines are the bound the released medians are read against. The private quantile adds exponential noise of scale 2 to each candidate's score and picks the best. The difference of two such draws is a Laplace draw of the same scale. So the chance that a worse candidate wins falls exponentially in its gap to the best, measured in scales. A union bound over the other 99 candidates gives the printed bound, which a losing candidate beats at most once in ten thousand runs. The candidates within the bound span about one and a half units of y at the lower cut and about three at the upper cut. The predictions there are more spread out, so neighbouring candidates score alike.

13.7. Choosing grouping columns privately

The documentation's second larger example solves a problem you meet before any grouped release. You want to group the rows by some columns before releasing counts, but you may not look at the data to choose them. Some columns are too uniform, with most rows sharing a value, and some are too diverse, with most rows having their own. Grouping by too many columns leaves most groups too small to survive the thresholds of Grouping. The example scores each candidate set by how many of its groups reach a public minimum of rows. It then picks a set with the **exponential mechanism**, which chooses a candidate with probability growing exponentially in its score. The full edition's chapter on selection derives it.

The score is a transformation plugin, and it needs a user domain because the library has no domain for a whole data frame. The documentation scores a pandas frame, and the same score runs here on a Polars frame.

```
def score(frame: pl.DataFrame, columns: tuple[str, ...], min_rows: int) -> float:
    """How many groups of these columns have at least `min_rows` rows."""
    return float(frame.group_by(list(columns)).len().filter(pl.col("len") >= min_rows).heigh
```

The transformation applies it to every candidate set.

```
def make_grouping_cols_score(
    input_domain: dp.Domain, input_metric: dp.Metric, candidates: list[tuple[str, ...]], min
) -> dp.Transformation:
    """A transformation from the frame to one score per candidate set of grouping columns.""
    dp.assert_features("contrib")

    def function(frame: pl.DataFrame) -> list[float]:
        return [score(frame, columns, min_rows) for columns in candidates]

    def stability_map(d_in: int) -> float:
        return float(d_in)  # one row changes one group's size, so it moves each score by at

    return dp.t.make_user_transformation(
        input_domain=input_domain,
        input_metric=input_metric,
        output_domain=dp.vector_domain(dp.atom_domain(T=float, nan=False)),
        output_metric=dp.linf_distance(T=float),  # not monotonic: a person's rows may be ad
        function=function,
        stability_map=stability_map,
    )
```

The output metric is the L-infinity distance, the largest change in any one score, because noisy max compares scores rather than summing them. The map is the identity on the input distance, and the reason is that one row belongs to one group of each candidate. Adding or removing it changes that group's size by one, so it moves each score by at most one, and `d_in` rows move each score by at most `d_in`.

The documentation declares the metric monotonic, which halves the cost of noisy max. That is right when neighbours differ by one row added or one removed, since every score then moves the same way. Here the privacy unit is a person with up to 36 rows, and a neighbouring dataset can add some of them and remove others. The scores can then move both ways, so the script keeps the metric plain and pays the factor of two. The full edition's chapter on sensitivity is where these bounds come from.

The measurement chains the scores into noisy max and names the chosen candidate, and it is registered so that a context can run it.

```
def make_select_grouping_cols(
    input_domain: dp.Domain, input_metric: dp.Metric, candidates: list[tuple[str, ...]], min
) -> dp.Measurement:
    """Score the candidates, pick one by the exponential mechanism, and name its columns."""
```

```
    def name(index: int) -> tuple[str, ...]:
        return candidates[index]  # post-processing: the index is already private

    scores = make_grouping_cols_score(input_domain, input_metric, candidates, min_rows)
    return scores >> dp.m.then_noisy_max(dp.max_divergence(), scale) >> name


dp.register(make_select_grouping_cols)


def frame_domain() -> dp.Domain:
    """A domain for a Polars DataFrame: the library has none, so membership is a Python chec

    def member(value: object) -> bool:
        return isinstance(value, pl.DataFrame)

    return dp.user_domain("PolarsDataFrameDomain", member=member)
```

On the survey the candidates are every non-empty subset of six columns. Five are `SEX`, `ILOSTAT`, `YEAR`, `QUARTER` and `AGE`, and the sixth is the person identifier `PIDENT`. This sample codes age in seven bands. A group counts when it has at least 100 rows, a public choice standing in for the smallest group worth a noisy count. The context is built over the collected frame with the user domain passed as `domain`, a privacy unit of 36 contributions and a budget of $\varepsilon = 1$. The registered method is called with the scale left out, so that the context searches for it against the plugin's map.

```
LFS: 3,811,932 rows; candidate columns SEX, ILOSTAT, YEAR, QUARTER, AGE, PIDENT; 63 candidat
distinct values: SEX 2 ILOSTAT 4 YEAR 9 QUARTER 4 AGE 7 PIDENT 1,682,306
score: groups of at least 100 rows
rank 1: SEX, ILOSTAT, YEAR, QUARTER, AGE score 996 of 1,105 groups
rank 2: ILOSTAT, YEAR, QUARTER, AGE score 504 of 570 groups
rank 3: SEX, YEAR, QUARTER, AGE score 432 of 470 groups
sets with PIDENT: 32, largest score 0
select scale 72: map(36)=1.0 monotonic would be 0.5
context query: param scale=72.0
within 910 of the best: 10 candidates
```

The scores are computed at development time, which is allowed because the survey is public, and they show the shape the documentation describes. The five columns without the identifier score best, with most of their groups above the minimum, and the next set scores about half of that. Every set that includes `PIDENT` scores zero, because no person has a hundred rows. The map of the selection is twice the sensitivity over the scale, so a scale of 72 costs an ε of 1, and the context's search lands on that scale.

The bound follows the same reasoning as for the medians, with exponential noise at scale 72 and a union bound over 62 other candidates. A set that scores more than 910 below the best therefore wins at most once in ten thousand runs, and ten sets lie within that bound. The best beats the

runner-up by 492, under seven noise scales, so it wins most runs, but any of the ten can appear. That spread is the price of the person-level unit on data with few, coarse columns. A person can move each score by 36, while the scores themselves are only in the hundreds. Raising the minimum group size lowers every score and brings the candidates closer together, until the choice is nearly random, and spending more budget is the only other lever.

```
released (one run):
  python laplace of 10.0: 9.6979  (within 6.0 with probability 0.95)
  repeated sum of ['0', '1', '2', '3'] with laplace scale 1: 13  (true 12)
  theil-sen: slope 2.1549 intercept 1.1111  (medians at the cuts -2.1212 and 4.3434)
  grouping columns: SEX, ILOSTAT, YEAR, QUARTER, AGE
```

The Python Laplace landed within its error bar, and the repeated sum within one of the truth. Both private medians fell inside their candidate ranges, so the slope and the intercept land near 2 and 1. The chosen grouping columns are the five that score best, and your run can differ in each of these.

13.8. Summary

Table 13.1.: The plugin constructors, what each does, and the rule of thumb that goes with it.

Call or concept	What it does	Rule of thumb
`dp.enable_features("honest-but-curious")`	Unlocks the plugin constructors	The library cannot check a Python map; the proof is yours
`make_user_measurement(...)`	A measurement from your function and privacy map	Compare the map with a built-in cousin where one exists
`make_user_transformation(...)`	A transformation from your function and stability map	Justify the map and the output domain
`user_domain`, `user_distance`, `user_divergence`	Your own domain, metric and measure	Identifiers name one thing; a user measure cannot be composed
`new_privacy_profile`, `new_function`	A profile from a curve; a post-processor	No global state, no data-dependent exceptions
`dp.register(make_thing)`	`context.query().thing(...)`	Take `input_domain` and `input_metric` first
A plugin in a context	Charged its share of the budget, not its map	A cheap plugin does not give its share back
Theil-Sen from plugins	Pair, predict at two cuts, two private medians, solve	Use the vetted estimator for real work
Grouping columns by noisy max	Score each set by its groups above a minimum, pick one	Monotonic only for one-row units

13.9. References

- OpenDP documentation: “Plugins”, “Measurement example”, “Transformation example”, “Context API Plugins”, “Thiel-Sen Regression” and “Privately Selecting Grouping Columns” in the API user guide.
- Theil (1950) and Sen, “Estimates of the regression coefficient based on Kendall’s tau” (1968): the estimator behind the regression example.
- McSherry and Talwar, “Mechanism Design via Differential Privacy” (2007): the exponential mechanism behind noisy max.
- Gaboardi, Hay and Vadhan, “A Programming Framework for OpenDP” (2020): the maps a plugin must satisfy.
- The full edition’s chapters on the Laplace and Gaussian mechanisms in the full edition, composition in the full edition, selection in the full edition and sensitivity in the full edition derive the results this chapter states.

14. Statistical modelling

Every release so far has been a table of statistics, such as a count, a sum, a mean or a quantile per group. Often the question is about a relation instead. How many hours do people actually work for each hour they usually work? Which directions of a numeric table carry its variance? OpenDP ships two models for such questions, a differentially private linear regression and a differentially private principal component analysis (PCA), both with interfaces modelled on scikit-learn. The one idea of this chapter is that a fitted model is a release like any other. Behind each `fit` is a measurement with a privacy map, and the map, not the argument named `epsilon`, says what the fit costs.

You will fit the regression on the documentation's generated points and on the Labour Force Survey (LFS), and read its privacy loss from its measurement. You will see what the estimator targets and why its sensitivity is bounded. You will fit the PCA on the documentation's generated table and compare its release with NumPy's exact decomposition. Then you will meet the NumPy domain and the clamp underneath both estimators, and finish with the questions a reviewer should ask about a private model.

The script opendp_statistical_modelling.py runs every step on OpenDP 0.16.0 with scikit-learn 1.8.0. Its toys come from a seeded NumPy generator, and the seed also fixes how the regression pairs its rows. So the exact fits, the bounds, the cut points and every privacy map are the same on every run. The released fits are not, because the private medians and the PCA's noise come from OpenDP's cryptographic generator, which cannot be seeded. They appear in one block marked as one run.

```
uv run python code/part04-extending/opendp_statistical_modelling.py
```

14.1. A model or a table?

The documentation's page on statistical modelling makes one point before it shows any code. When you know which relation you need, a model tailored to it spends the whole budget on the few numbers that answer the question. A slope costs two private medians, as the next section shows. A table fine enough to read the slope off would cost a noisy count in every cell. The same holds for the PCA, which releases a handful of directions rather than a full covariance matrix.

The flexible alternative is synthetic data. A synthetic copy of the table answers any later question, including ones nobody has asked yet, and it can be handed to people who need microdata. The price is accuracy. A statistic computed from a synthetic copy is less accurate than the same statistic released directly under the same budget, because the copy had to serve every question at once. AIM and MST generates such copies. So release a table when the questions are counts, a model when the question is a relation, and synthetic data when the questions are not yet known.

The two models are narrow on purpose. The regression takes one explanatory column and one response, and the PCA takes a numeric table whose rows can be bounded in length. A model with many explanatory variables is trained by the methods of the full edition's chapter on private empirical risk minimization, which OpenDP does not ship.

14.2. Linear regression

A least-squares line is a poor thing to release directly. Its slope is a ratio of sums whose sensitivity is unbounded unless both columns are clipped, and clipping changes the line. The estimator OpenDP ships is a **Theil-Sen regression**, which rests on medians instead. It pairs the rows at random, fits the line through each pair, and reads each line at two fixed cut points on the x axis. At each cut point it then releases the median of those readings with a private quantile. Finally it solves the two released medians for a slope and an intercept, which is post-processing and costs nothing more.

The sensitivity is bounded because of how the private median chooses. It scores every candidate on a fixed grid by how many readings lie on either side of it. The exponential mechanism then picks a candidate with a probability that favours high scores. One person's row sits in one pair, so it moves one reading, and that changes any candidate's score by at most one. How far the reading moved never enters, which is why the estimator needs no clipping and survives a person who reports a thousand hours. The full edition's chapter on selection derives the exponential mechanism and the private quantile. Its chapter on sensitivity in the full edition explains why a count of readings is the right quantity to bound.

```
def make_regression(
    x_bounds: tuple[float, float], y_bounds: tuple[float, float], scale: float, runs: int =
) -> dp.sklearn.linear_model.LinearRegression:
    """The private Theil-Sen regression under pure differential privacy, with public bounds
    return dp.sklearn.linear_model.LinearRegression(
        dp.max_divergence(),  # the output measure: the privacy loss is read in epsilon
        x_bounds=[x_bounds],  # fixes the two cut points, a quarter and three quarters of th
        y_bounds=y_bounds,  # the range over which the candidate medians are spread
        scale=scale,  # the noise scale of each of the two private medians
        runs=runs,  # how many random pairings to pool; each multiplies the privacy loss
    )
```

The first argument is the output measure. Here `dp.max_divergence()` asks for pure differential privacy (DP), whose parameter is the privacy loss ε. The bounds are public information, and they never clip the data. The x bounds only fix where the two cut points go, at the fractions `fraction_bounds` of the way across, a quarter and three quarters by default. The y bounds only fix the grid of `candidates_count` evenly spaced candidates from which each median is chosen, a hundred by default. The scale is the noise scale of each private median, and `runs` repeats the random pairing and pools the readings. The documentation also builds this estimator from plugins, which Plugins follows.

```
regression space: NPArray2Domain(num_columns=2, nan=True, T=f64) SymmetricDistance() MaxDive
regression map: scale=1 runs=1 -> epsilon=2.0
regression map: scale=2 runs=1 -> epsilon=1.0
regression map: scale=2 runs=3 -> epsilon=3.0
```

The estimator's measurement is reachable as its `measurement` attribute, and it reads like any other. Its input is a two-column NumPy array under the symmetric distance, so the privacy unit is a row. Its map at distance 1 is therefore the privacy loss for adding or removing one row. Two medians at scale 1 cost $\varepsilon = 2$, and doubling the scale halves it. Three runs at scale 2 cost $\varepsilon = 3$, because a row then sits in three pairs and moves three readings. More runs make the median better defined, but at a fixed budget they force a larger scale, so the gain is rarely worth it on a large dataset.

The documentation's toy is a hundred points with a true slope of 2 and an intercept of 1, drawn here by a seeded generator. The script prints the deterministic things to read a fit against.

```
toy: n=100 cut points=[-1.5, 1.5] candidate step=0.2020
toy least squares: slope=2.0275 intercept=0.9725
toy median line, no noise: slope=2.0379 intercept=1.0786 readings=50
```

The candidate step is the y range divided into 99 intervals, about 0.2 here. The cut points are 3 apart, so moving one median by one grid step changes the slope by about 0.07, before any noise is added. The median line is what the estimator would return with exact medians and the seeded pairing, from the 50 readings that 100 points give. It is close to least squares here because the noise in the toy is symmetric. The released fit is in the last section.

14.2.1. Actual hours against usual hours on the survey

The survey question is how many hours people actually worked in the reference week for each hour they usually work. The script takes the people working in the fourth quarter of 2013 whose usual and actual hours are both recorded. The survey codes hours from 0 to 98, so the bounds of both columns are public. The code 99 means "not applicable" and is dropped. A person can appear more than once in a quarter, so the script keeps each person's first row, and the privacy unit of the fit is one person. That step and the row filter run in Polars before the array reaches the measurement, so the library does not account for them. A row filter keeps neighbouring datasets neighbouring, and keeping one row per person is the truncation of Identifier truncation. Here both are your claim rather than the library's.

```
survey: working people in 2013 Q4 with hours recorded: 36,914 (one row each)
survey: of them with 0 actual hours that week: 4,109
survey: bounds=(0.0, 98.0) cut points=[24.5, 73.5] candidate step=0.9899
survey least squares: slope=0.8582 intercept=-0.1935
survey median line, no noise: slope=1.0000 intercept=0.0000 medians=[24.50, 73.50]
survey readings: 16,463 pairs; within one candidate step of the medians: 5,606 and 5,252
```

The two exact fits disagree, and the disagreement is the lesson. Among the 36,914 people, 4,109 worked no hours at all that week, through leave or illness. So least squares is pulled toward them, and it gives 0.86 actual hours per usual hour. The median line ignores them, because most pairs of people lie on the line where actual equals usual. So its medians at the cut points are the cut points themselves, and its slope is exactly 1. A private Theil-Sen fit therefore estimates the typical relation, not the average one, and the median line is the exact value to compare it with. The last line counts the readings, which are fewer than half the people because a pair with equal usual hours has no slope. Thousands of them lie within one grid step of each median.

```
model = make_regression(HOURS_BOUNDS, HOURS_BOUNDS, scale=REGRESSION_SCALE)
epsilon = model.measurement.map(1)  # the privacy loss for adding or removing one person's r
survey_slope, survey_intercept = fit_regression(model, hours, seed)
```

```
survey private fit: scale=2 epsilon=1.0
```

The fit at scale 2 costs $\varepsilon = 1$ for one person. The `fit` method takes the explanatory column as a two-dimensional array and the response as a vector, like scikit-learn. It returns a fitted scikit-learn `LinearRegression`, so `coef_`, `intercept_` and `predict` work as usual.

14.3. Principal component analysis

A PCA finds the directions along which a table varies most. OpenDP's version releases three things under one budget. The first is the mean of the rows, the second the singular values, which say how much variance each direction carries, and the third the components, which are the directions. The algorithm follows Amin and others (2019). It scales every row into a ball of radius `row_norm`, centres the rows on a privately estimated mean, and forms the matrix of sums of squares and cross-products. It releases that matrix's eigenvalues with Laplace noise, since replacing one row moves their sum by at most twice the squared row norm. It then draws the eigenvectors one at a time from a distribution that favours directions of high variance, which is the exponential mechanism on the sphere. After each draw it projects the matrix away from the drawn direction.

That is why two facts must be public. The row norm is the sensitivity of the whole algorithm, and every map below is in units of it. The number of rows is part of the domain. It makes neighbouring tables tables of the same size with one row replaced, which is distance 2 under the symmetric distance, and it gives the mean a known divisor. Both are declared, not checked, as the reviewer's section shows. The budget is split in three equal parts, for the mean, the eigenvalues and the eigenvectors, and the eigenvectors' part is split again over the number of eigenvectors drawn.

The table is the documentation's. Ten thousand centred rows of four columns are drawn from a normal distribution with a random covariance, here from a seeded generator so that the exact decomposition is reproducible.

```
pca table: 10,000 rows x 4 columns, row length mean=8.53 max=36.56
pca table: rows longer than 1: 99.9%  longer than 40: 0.0%
pca exact, raw rows: singular values=[901.9, 278.6, 222.2, 156.9]
pca exact, rows clamped to 1: singular values=[80.48, 40.67, 34.29, 26.29]
pca exact, rows clamped to 1: first component=[-0.614, -0.534, -0.526, -0.247]
```

The documentation fits with `row_norm=1.0`, and the first line says what that means here. Almost every row is longer than 1, so the clamp scales almost every row onto the unit sphere. The singular values the algorithm can estimate are then those of the clamped table, about 80 rather than about 900 for the first. The directions survive the clamp, because scaling a row does not turn it, so the first component of the clamped table is the one to compare the release with. A row norm of 40 clamps nothing, and the script fits with it too, to show the other side of the trade.

```
def make_pca(epsilon: float, row_norm: float, n_components: int | None = None) -> dp.sklearn
    """The private PCA of the generated table, with the two public facts the algorithm needs
    return dp.sklearn.decomposition.PCA(
        epsilon=epsilon,
        row_norm=row_norm,  # public: every row is scaled into a ball of this radius before
        n_samples=PCA_ROWS,  # public: the exact number of rows, which fixes the domain
        n_features=PCA_COLUMNS,
        n_components=n_components,  # None releases every component
    )
```

```
pca = make_pca(PCA_EPSILON, ROW_NORM)
measurement = pca.measurement()  # the measurement behind fit(), for its map and for composi
pca.fit(table)  # the private release: the mean, the singular values and the components
```

```
pca model: PCA(epsilon=1.0, n_components=4, n_features=4, n_samples=10000, row_norm=1.0)
pca space: NPArray2Domain(size=10000, num_columns=4, nan=True, T=f64) SymmetricDistance() Ma
pca map: map(2)=2.0 map(1)=2.0 check(2, 1.0)=False check(2, 2.0)=True
make_private_pca unit_epsilon=1: map(2)=1.0 map(1)=1.0
pca n_components=2: map(2)=2.0
pca composition of epsilon 1 and 0.5: map(2)=3.0
```

Read the third line twice. The model was built with `epsilon=1.0`, but its measurement's map at distance 2, which is one replaced row, is 2.0. The `check` call confirms that the fit does not stay within $\varepsilon = 1$. The reason is in the constructor. The `PCA` class calls the framework's `make_private_pca` with twice its `epsilon`, divided by `n_changes`, the number of rows one person may change. And `make_private_pca` with a unit budget of 1 has a map of 1.0 at distance 2, as the fourth line shows. So in version 0.16.0 a `PCA(epsilon=1.0)` fit costs $\varepsilon = 2$ per changed row by its own map, and the map is what you should quote. The map is also flat below distance 2, because the algorithm is an adaptive composition declared at that distance, which Combinators explains.

Asking for two components leaves the total unchanged, as the fifth line shows. What changes is inside. The library draws at most one eigenvector fewer than there are columns, since the last

direction is determined by the others. So the eigenvectors' third is split over three draws for four components and over two draws for two, and each draw gets a larger share. The `measurement()` method exists for the compositors of the same chapter. The last line composes a fit at `epsilon=1.0` with one at `epsilon=0.5`, and the maps add to 3.0 at distance 2. Calling the measurement on the table fits the model and returns it.

14.4. The NumPy domain underneath

Both estimators live on `dp.numpy.array2_domain`, the domain of two-dimensional NumPy arrays. Its members are described by a number of columns, an optional number of rows and the element type. Optionally they also have a norm, its order `p` and an origin, which say that every row lies within that norm of the origin. A domain with a norm is a promise about the data, and `make_np_clamp` is the transformation that makes the promise true.

```
import numpy as np
import opendp.prelude as dp

dp.enable_features("contrib", "honest-but-curious")
rows = dp.numpy.array2_domain(num_columns=2, T=float)
clamp = dp.numpy.make_np_clamp(rows, dp.symmetric_distance(), norm=1.0, p=2, origin=np.array
print(clamp(np.array([[1.0, 2.0], [4.0, 6.0], [1.5, 2.0]])).tolist())  # [[1.0, 2.0], [1.6,
print(clamp.map(1))  # 1
print(clamp.output_domain)  # NPArray2Domain(origin=[1. 2.], norm=1.0, p=2, num_columns=2, n
```

The clamp leaves the first and third rows alone, because they lie within distance 1 of the origin. It pulls the second row from distance 5 back to the sphere, along the line to the origin. Its stability map is the identity, since clamping each row on its own neither adds nor removes rows. Its output domain records the norm, the order and the origin, and that no value is missing. Transformations reads stability maps in general.

That output domain is what `make_private_pca` would rather receive. A domain built with the norm and the origin states that the rows are already bounded around a known centre. The algorithm then skips the private mean and its third of the budget.

```
bounded domain: NPArray2Domain(origin=[0. 0. 0. 0.], norm=1.0, p=2, size=10000, num_columns=
pca on the bounded domain: map(1)=0.50 map(2)=1.00 map(4)=2.00
```

The script passes an explicit split as a `PCAEpsilons`, with half for the eigenvalues, a sixth for each of three eigenvectors and nothing for the mean. The map is then linear in the distance rather than flat, because no compositor is declared in between. Use this route when the centre is public, for instance when the columns are scores on a known scale, and the class when it is not.

14.5. What a reviewer should ask

The first question is the privacy unit. Both estimators count rows, so the claim "one person" holds only if each person has one row, which on the survey was your own Polars step. The second is the bounds. The regression's bounds and the PCA's row norm and row count must come from the code book or the domain, never from the data. A bound read off the data leaks it. The third is which features are on, since a feature is a statement about what you accept on trust.

```
regression without honest-but-curious: Attempted to use function that requires "contrib", "h
pca without honest-but-curious: Attempted to use function that requires "contrib", "honest-b
pca without idealized-numerics: Attempted to use function that requires "contrib", "idealize
regression without idealized-numerics: constructed
pca with n_samples one short of the table: fitted without complaint
```

Both estimators need `honest-but-curious`, because both are built from user-defined transformations whose functions and stability maps are taken as written. The regression's pairing step, for example, claims that one row moves `runs` readings, and the library records that claim rather than proving it. The PCA also needs `idealized-numerics`, which concedes that its proofs hold for real arithmetic and not for floating point. Every guarantee of the PCA rests on that concession. The eigenvalues, their sensitivity and the eigenvector sampler are all computed in floating point, and the library's own source notes that it knows no floating-point-safe sampler for the eigenvector draw. The regression constructs without the flag, because its private medians run in the Rust core and only the pairing is Python.

The last line is the reviewer's trap. The number of rows is part of the PCA's domain, but the library does not check that the array has that many rows. A wrong `n_samples` is therefore a wrong guarantee that nothing reports. The row norm, by contrast, is enforced by the clamp, and the regression's bounds need no enforcing because its sensitivity does not depend on them. Limitations and troubleshooting collects the other things the library does not check.

14.6. The releases of one run

```
released (one run):
  toy private fit: slope=1.9529 intercept=1.0101
  survey private fit: slope=1.0101 intercept=-0.9899
  survey private fit at 35 usual hours: 34.36 actual
  pca row_norm=1: singular values=[80.48, 40.66, 34.28, 26.37]
  pca row_norm=1: |cos| with exact components=[1.000, 0.998, 0.996, 0.996]
  pca row_norm=1: mean=[0.001, 0.002, 0.001, 0.001]  explained variance ratio=[0.648, 0.165,
  pca row_norm=40: singular values=[906.8, 268.6, 212.6, 153.2] (exact [901.9, 278.6, 222.2,
  pca row_norm=40: |cos| with exact components=[0.996, 0.659, 0.426, 0.086]
  pca n_components=2: returned the model: True  singular values=[80.41, 40.64]
  pca n_components=2: |cos| with exact components=[0.999, 0.999]
```

The toy fit is off the least-squares line by about one grid step at one cut point, which is what a private median on 50 readings at $\varepsilon = 2$ gives. The test allows the slope within 0.3 of the least-squares slope and the intercept within 0.5. That is about three grid steps at either cut point. A candidate three steps from the best one scores lower by every reading in between, and the exponential mechanism makes such a candidate rare at this budget.

The survey fit is the same on every run. The medians sit exactly at the cut points, and the nearest candidates on the grid give a slope of 1.0101 and an intercept of about -1. Thousands of readings lie within one grid step of each median, so the next candidate on the grid scores thousands of readings worse than the best one. Noise at scale 2 never overturns a gap that large. The error of the fit is therefore the grid, not the noise, and a finer `candidates_count` would remove it at no cost in privacy. The test allows the slope within 0.1 of the median line and the intercept within 4 hours of it, which is two grid steps at the lower cut point. The fit predicts about 34.4 actual hours for someone who usually works 35.

The PCA at row norm 1 recovers the singular values of the clamped table to within a tenth and all four directions to a cosine above 0.99. Its private mean is within a few thousandths of the true zero. The test allows each singular value within 1 of the exact one and each cosine above 0.9. Both bounds leave a wide margin, because the eigenvalues' Laplace noise is a few units against eigenvalues in the thousands and the four directions are well separated.

At row norm 40 the singular values are the raw table's, within a few percent, but only the first direction is right. The eigenvector sampler's concentration falls with the square of the row norm, so a norm of 40 leaves little of it, and the last three directions are close to random. The test checks only the first singular value, within 30, and the first cosine, above 0.9. The two-component fit returns the model itself, and its two directions are as good as the four-component fit's first two.

14.7. Summary

Table 14.1.: The calls and concepts of private modelling, what each does, and the rule of thumb that goes with it.

Call or concept	What it does	Rule of thumb
`dp.sklearn.linear_model.LinearRegression(measure, x_bounds, y_bounds, scale, runs)`	A Theil-Sen regression from two private medians	Bounds fix the cut points and the grid; they never clip the data
`model.measurement.map(1)`	The privacy loss of the fit per added or removed row	Two medians at scale s cost $2\,\text{runs}/s$; quote this, not the arguments
`candidates_count`	The grid the medians are chosen from	On a large dataset the grid, not the noise, is the error; refine it for free
The median line against least squares	What the estimator targets	Compare a Theil-Sen fit with the median line; outliers pull least squares only

Call or concept	What it does	Rule of thumb
`dp.sklearn.decomposition.PCA(epsilon, row_norm, n_samples, n_features, n_components)`	Releases the mean, the singular values and the components	`row_norm` and `n_samples` are public facts the library does not check
`pca.measurement()`	The measurement behind `fit`, for maps and compositors	In 0.16.0 its map at distance 2 is twice `epsilon`; read the map
`n_components`	How many directions to draw	Fewer draws give each more budget at the same total
`row_norm`	The sensitivity scale of the whole algorithm	Tight: magnitudes are the clamped table's, directions good. Loose: magnitudes right, directions poor
`dp.numpy.array2_domain(norm, p, origin)` and `make_np_clamp`	The bounded domain and the transformation that enforces it	A known origin skips the private mean and its third of the budget
`honest-but-curious`, `idealized-numerics`	Trust the plugins' maps; accept real-arithmetic proofs	Both estimators need the first; the whole PCA rests on the second

14.8. References

- OpenDP documentation: "Statistical Modeling", "Linear Regression" with the API reference of `opendp.extras.sklearn.linear_model`, "Differentially Private PCA", and "Theil-Sen Regression" in the plugins user guide.
- Theil, "A Rank-Invariant Method of Linear and Polynomial Regression Analysis" (1950), and Sen, "Estimates of the Regression Coefficient Based on Kendall's Tau" (1968): the median-of-slopes estimator.
- Dwork and Lei, "Differential Privacy and Robust Statistics" (2009): why robust estimators such as medians are the natural private ones.
- Amin, Dick, Kulesza, Munoz and Vassilvitskii, "Differentially Private Covariance Estimation" (2019): the eigenvalue and eigenvector mechanisms behind the PCA.
- The full edition's chapters on selection in the full edition, sensitivity in the full edition and private empirical risk minimization in the full edition derive the results this chapter states.

15. Limitations and troubleshooting

Every chapter so far has shown what OpenDP guarantees. This one shows where the guarantee stops, and what to do when the library stops you. The one idea is that the guarantee covers exactly what passes through a measurement whose proof has been checked. The arithmetic that proof assumes, the code that runs before the data reach the context, and the channels an attacker can watch besides the output are yours to account for. Most of the library's refusals exist to keep you on the right side of that line, so an error message is usually the guarantee doing its job.

The chapter follows the documentation's page on limitations, its feature listing and its troubleshooting page. You will read what the library does not promise and what each limit means for a deployment. Then you will switch each feature flag off and on to see what it gates. You will mark out what lies outside the guarantee, and meet the error messages of the earlier chapters again, each with its cause and its fix. By the end you will be able to read an `OpenDPException`, and to say what a reviewer verifies before trusting a release made with OpenDP.

The script opendp_limitations.py reproduces every message on this page with OpenDP 0.16.0 and Polars 1.36.1. It uses tiny lists for the framework calls and the France Labour Force Survey (LFS) sample behind a context for the Polars ones. The messages, the sensitivities, the scales and the error bar are the same on every run. The one released count is not, because OpenDP draws its noise from a cryptographic generator that cannot be seeded. It appears at the end in a block marked as one run.

```
uv run python code/part04-extending/opendp_limitations.py
```

15.1. What OpenDP does not promise

The documentation's limitations page names five things, and each changes something about how you deploy the library. The first is arithmetic. A privacy proof in the literature is a statement about real numbers, but a computer adds floating-point numbers, which round. Casacuberta, Shoemate, Vadhan and Wagaman studied what that does to a sum in "Widespread Underestimation of Sensitivity in Differentially Private Libraries and How to Fix It" (2022). Several libraries computed the sensitivity of a sum as if the arithmetic were exact. The rounding made the true sensitivity larger, so the noise was too small and the stated ε was wrong. The full edition's chapter on discrete mechanisms in the full edition shows the older attack on the Laplace sampler itself, where the low-order bits of a float reveal which dataset produced it.

OpenDP's answer is to put the rounding into the proof. The script builds a float sum and an integer sum of values between 0 and 80 with the framework API of Transformations. It then asks each for its sensitivity when 36 rows are added or removed.

```
def sum_sensitivities(d_in: int) -> tuple[float, int]:
    """The sensitivity of a float sum and of an integer sum of values in [0, 80], at `d_in`
    float_sum = (dp.vector_domain(dp.atom_domain(bounds=HOURS_BOUNDS)), dp.symmetric_distanc
    int_sum = (dp.vector_domain(dp.atom_domain(bounds=(0, 80))), dp.symmetric_distance()) >>
    return float_sum.map(d_in), int_sum.map(d_in)
```

```
float sum of 36 rows in [0, 80]: sensitivity 2880.0000007451
int sum of 36 rows in [0, 80]: sensitivity 2880
```

In exact arithmetic both would be 36 times 80. The float sum's map adds a small term for the error of rounding floats as they are added, and the integer sum's does not, because integers add exactly. The term is tiny here, but it grows with the number of rows. That is why a float sum on the LFS needs a public bound on that number (Essential statistics). Where a component still assumes exact arithmetic, the library puts it behind the `idealized-numerics` flag, which the next section tries. The rule for a deployment is to prefer integer statistics where you can. Where you cannot, read a float statistic's scale as the library computes it rather than as the formula predicts.

The second limitation is side channels. OpenDP has not been hardened against them, so the time a query takes, the memory it uses and the cache lines it touches may depend on the data. An attacker who can time the system that runs OpenDP may learn something the output does not say, and no proof in the library covers that. The deployment question is therefore who can interact with the running system. An analyst who submits queries through a queue and reads results later is in a different position from one with a shell on the machine. Keep the second kind out, or accept that the guarantee is weaker than the ε you record.

The third is vetting. Every component of OpenDP is meant to come with a written proof and a review of that proof, and until the review is finished the component needs the `contrib` flag. The documentation says plainly that the process is expected to find bugs in code and in proofs. So a release made with `contrib` on, which is every release in this book, rests on proofs that are written but not yet checked by a second reader. Record which components a release used, so that you can revisit it if one is corrected.

The fourth is stability. OpenDP follows semantic versioning, so until 1.0.0 its interfaces may change between minor releases. When they do, the release notes say how to migrate. Pin the version, as this book pins 0.16.0, and keep a test that reruns your release code. Then an upgrade that changes a default is caught before it changes a number. The fifth is software quality. Like any project OpenDP has bugs, and its issue tracker is where to report one. But no deployment can rule out an unknown bug, so the practical answer is the one the practitioner edition gives in Release and governance in the practitioner edition. Keep a record of the run from which the guarantee can be recomputed, and a plan for what reopens a release.

15.2. The feature flags

The feature flags are how the library asks for your consent to each of those limitations. There are four, plus a deprecated alias, and all are off until you call `dp.enable_features`. A first release

switched `contrib` on without showing what happens otherwise. The script switches it off, tries the Laplace mechanism of that chapter, and switches it back on.

```
dp.disable_features("contrib")
without_contrib = refusal(lambda: space >> dp.m.then_laplace(scale=1.0))
dp.enable_features("contrib")
```

```
laplace without contrib: OpenDPException: Attempted to use function that requires "contrib",
```

The helper `refusal` runs a call and returns the exception's type and message, and the message here names the flag and the call that enables it. The same wording appears for every flag. Running a finished script with a flag off, through `disable_features`, is the quickest way to find out which of its calls depend on that flag.

The second flag is `honest-but-curious`. It gates constructors whose guarantee depends on the arguments you pass being correct. The plugins of Plugins are the main case, because `make_user_measurement` takes a privacy map as a Python function, and the library cannot check that the function tells the truth. The name describes the adversary the flag assumes: one who is honest when building the measurement and only curious when reading its outputs. The script builds the smallest possible plugin, which releases its input unchanged and claims a privacy loss equal to the distance between inputs.

```
def plugin_measurement() -> dp.Measurement:
    """A plugin measurement whose privacy map is only a claim: the library cannot check `ide
    return dp.m.make_user_measurement(
        dp.atom_domain(T=float, nan=False),
        dp.absolute_distance(T=float),
        dp.max_divergence(),
        function=identity,
        privacy_map=identity_map,
    )
```

```
plugin without honest-but-curious: OpenDPException: Attempted to use function that requires
plugin with honest-but-curious: map(1.0)=1.0
```

With the flag on, the plugin reports whatever its map says, here a loss of 1 at distance 1, and nothing about that number was verified. So the flag belongs in the environment where you write and prove plugins, not in one where an untrusted analyst submits queries. Such an analyst could register a map that claims any loss they like. The same flag gates `OnceFrame.lazy()`, which hands back a released Polars plan that could be run twice.

The third flag is `idealized-numerics`, which marks components whose proof assumes exact arithmetic. In 0.16.0 none of the constructors this book uses asks for it. The float Laplace mechanism builds with the flag off, and so do the float sums above, because their proofs account for rounding. The flag's older name `floating-point` still works and warns.

```
float laplace without idealized-numerics: no error
enable_features("floating-point"): DeprecationWarning: "floating-point" is deprecated. Use "
assert_features("idealized-numerics") with the alias on: no error
assert_features("idealized-numerics") after disabling it: OpenDPException: Attempted to use
assert_features("contrib", "honest-but-curious"): OpenDPException: Attempted to use function
```

The last three lines use `dp.assert_features`, which raises the error a gated constructor would raise, without building anything. It is what every gated constructor calls first. It is also useful on its own at the top of a script, where one line then states every flag the script relies on. The fourth flag, `rust-stack-trace`, changes no guarantee. It adds the Rust frames to an error message, and the section on reading an exception uses it.

15.3. What lies outside the guarantee

The guarantee covers what happens to the data inside a measurement. Four things happen outside it in every analysis. The first is loading the data. The documentation says that loading is not covered and should be done by a trusted curator, and gives the reason: reading a CSV infers each column's type from its values. The script reads three two-row files that differ in one value, then a longer one whose odd value comes after the rows the inference looked at.

```
for raw in (b"x\n1\n2\n", b"x\n1\n2.5\n", b"x\n1\nabc\n"):
    print(f"csv {raw!r}: column x inferred as {pl.read_csv(io.BytesIO(raw)).schema['x']}")
late_text = b"x\n" + b"1\n" * 200 + b"abc\n"  # a text value after 200 numbers
failed = refusal(lambda: pl.read_csv(io.BytesIO(late_text), infer_schema_length=100))
print(f"csv with a text value on row 201, inferred from 100 rows: {first_line(failed)}")
as_strings = pl.read_csv(io.BytesIO(late_text), infer_schema=False).schema["x"]
print(f"the same file with infer_schema=False: column x read as {as_strings}")
```

```
csv b'x\n1\n2\n': column x inferred as Int64
csv b'x\n1\n2.5\n': column x inferred as Float64
csv b'x\n1\nabc\n': column x inferred as String
csv with a text value on row 201, inferred from 100 rows: ComputeError: could not parse `abc
the same file with infer_schema=False: column x read as String
```

One person's value decides whether the column is integer, float or text. When the value arrives after the rows the inference read, it decides whether the load succeeds at all. Inside a trusted environment that failure is useful, since it reveals a malformed file before anything is released. But if the success or failure of the load is visible outside, it is a release that nothing accounted for. The fixes are the documentation's: read a format that carries its schema, as this book does with Parquet, or read every column as text with `infer_schema=False` and cast it yourself. The LFS loader in `practitioner_datasets` does the first, once, when it converts the documentation's CSV to Parquet. That conversion is the curator's step, and nothing on these pages claims a guarantee for it.

The second thing outside the guarantee is public information. Every bound, imputed value, key list and margin in this book was chosen from the domain or from a published source. The data

and the context took its bound of 150,000 responses a quarter from Eurostat. A bound read off the private data is a release of the data, and the library cannot tell the two apart.

The third is code that runs outside the context. The same chapter told you to delete your reference to the frame once the context holds it, for the same reason. Collecting the frame itself is not a query, so nothing counts it. The fourth is the error channel. Whether a query succeeds or fails is itself an output, and a failure that depends on the data is a release. OpenDP guards against this inside a query, which is why it replaces some Polars arguments so that an expression cannot fail on one value. The loading step and your own code around the context have no such guard.

15.4. The error messages you will meet

The documentation's troubleshooting page has one entry, the type mismatch, and the earlier chapters met several more. This section reproduces each and says what caused it and what fixes it. The first is the type mismatch. By the time you call a function, OpenDP has fixed the type of every argument from the data's domain. It refuses an argument of another type rather than converting it. The script puts three integers behind a context and clamps them with float bounds.

```
float bounds on integer data: TypeError: inferred type is f64, expected i32. See https://doc
integer bounds on integer data: scale=10.0
```

The message names the type it inferred from your argument and the type the data have. The fix is to pass bounds of the data's type, `(0, 10)` for integers, or to cast the data first. The documentation names one common cause: a scalar ε passed where a measurement expects an (ε, δ) pair. The script asks an approximate measurement, built with `make_approximate` as in Combinators, whether it stays within a scalar budget.

```
check(1.0, 1.0) on an approximate measurement: TypeError: Cannot coerce a non-tuple type to
check(1.0, (1.0, 0.0)) on the same measurement: True
```

An approximate measurement's privacy loss is a pair, so its budget must be one too, and a pure budget is written as the pair $(\varepsilon, 0)$. The same message comes from `binary_search_param` with a scalar `d_out` on an approximate chain (Utilities and the Context API).

Next, a domain that allows missing values. A list of Python floats is inferred as a domain that may contain NaN, and a clamp refuses such a domain. The reason is that NaN compared with a bound is neither inside nor outside it.

```
clamp on floats that may be nan: OpenDPException: FailedFunction("Domain contains nan")
impute, then clamp: scale=80.0
```

The fix is the first chapter's: `impute_constant` before `clamp`, which maps the domain to one without NaN. The same need shows up in Polars as `fill_null` before a `dp.sum`.

Next, mismatched domains in a chain. The `>>` operator checks that the output domain of the left side equals the input domain of the right side, and refuses otherwise. The script clamps to one range and then chains a sum that was built for a wider one.

```
vectors = dp.vector_domain(dp.atom_domain(T=float, nan=False)), dp.symmetric_distance()
clamp = vectors >> dp.t.then_clamp((0.0, 10.0))
wide_sum = dp.t.make_sum(dp.vector_domain(dp.atom_domain(bounds=(0.0, 100.0))), dp.symmetric
mismatch = refusal(lambda: clamp >> wide_sum)  # the clamp's output has bounds [0, 10], the
chained = clamp >> dp.t.then_sum()  # a partial constructor takes the clamp's output space,
```

```
clamp to [0, 10] then a sum built for [0, 100]: OpenDPException: DomainMismatch("Intermediat
    output_domain: VectorDomain(AtomDomain(bounds=[0.0, 10.0], T=f64))
    input_domain:  VectorDomain(AtomDomain(bounds=[0.0, 100.0], T=f64))
")
clamp to [0, 10] then then_sum(): sensitivity 10.0
```

The message prints both domains, so the mismatch is there to read: the bounds differ. The fix is the partial constructor `then_sum()`, which takes its input space from the clamp's output, as Transformations explained.

The remaining messages come from the Polars side, with the LFS behind a context of one query, a unit of 36 rows and a budget of $\varepsilon = 1$.

```
def make_context(
    frame: pl.LazyFrame, delta: float | None = None, margins: Sequence[dp.polars.Margin] | N
) -> dp.Context:
    """The LFS behind a unit of 36 rows and a budget of epsilon = 1, with or without a delta
    return dp.Context.compositor(
        data=frame,
        privacy_unit=dp.unit_of(contributions=CONTRIBUTIONS),
        privacy_loss=dp.loss_of(epsilon=EPSILON, delta=delta) if delta else dp.loss_of(epsil
        split_evenly_over=1,
        margins=margins,
    )
```

A float sum needs a margin with `max_length`, for the reason of the first section. Its sensitivity includes a rounding term that grows with the number of rows, and the proof needs a public bound on that number.

```
float sum without a margin: OpenDPException: MakeTransformation("must specify 'max_length' i
float sum with max_length=5,400,000: scale=2,881.04
```

The fix is the documentation's margin, `dp.polars.Margin(max_length=150_000 * 36)`. The scale then comes out a little above the 2880 of the first section, which is the sensitivity of the sum over $\varepsilon = 1$ in exact arithmetic. The extra is the rounding term at 5.4 million rows. A grouped sum needs the same bound in the margin of its grouping, which is what `by=[]` in the message refers to.

The next three come from one query. The script builds a count and tries to collect it before releasing it. Then it releases the count, collects the result twice, and submits another query to a context whose one share is spent.

```
count_query = context.query().select(dp.len(signed=True))
before_release = refusal(lambda: count_query.collect())  # the plan has no data; only a rele
once = count_query.release()
dp_count = int(once.collect()["len"][0])
twice = refusal(lambda: once.collect())  # a second collection would be a second release
exhausted = refusal(lambda: context.query().select(dp.len(signed=True)).release())
```

```
count: scale=36.0 95% accuracy=108.34
collect before release: ValueError: You must call `.release()` before executing a query.
collect a OnceFrame twice: OpenDPException: FailedFunction("OnceFrame has been exhausted")
query after the budget is spent: ValueError: Privacy allowance has been exhausted
```

A `LazyFrameQuery` is a plan without data, so `collect` on it would compute nothing private. The library refuses anyway, so that the habit of collecting a plan never meets a frame that holds data. The call to `release` returns a `OnceFrame`, which can be collected once. The noise is drawn when the plan runs, so running it twice would be two releases for one share of the budget. The third message is the one the first chapter ended on. None of the three is a bug to work around. If you need a value twice, keep the DataFrame that the first `collect` returned.

The last message is from Contingency tables. A contingency table whose keys or cuts cover every column has no private key set to protect, so its release spends no δ. The library refuses a budget that carries one rather than let it go unspent. The script states the slack as $\delta = 10^{-7}$, the value the grouping chapters use.

```
contingency table with keys for every column under (epsilon, delta): ValueError: delta (1e-0
the same table under pure epsilon: ContingencyTableQuery
```

The fix is to state the budget as pure ε when the keys are all public, or as the zero-concentrated budget ρ of Utility and noise. Keep δ for the tables whose keys the library must discover. Two more messages were decoded where they arose, but they belong in this list. One is a plain `mean`, which is refused as not recognized because OpenDP has no proof for it. The other is a grouping without keys or a δ, which is refused because its key set is private (Grouping). For a message that is on none of these pages, the documentation's troubleshooting page says to ask on OpenDP's Slack or by email. Its next-steps page points to the user guide section behind each tutorial.

15.5. Reading an OpenDPException

Most of the messages above came as an `OpenDPException`, which is how an error in the Rust core reaches Python. It has three attributes. The `variant` is the kind of error, one of a fixed list such as `DomainMismatch`, `MakeTransformation` or `FailedFunction`, which code can match on. The `message` is the text. The `raw_traceback` is the Rust stack at the point of the error, which is recorded whether or not you display it. The script raises the domain mismatch again and reads the three, then switches on `rust-stack-trace` and prints the exception a second time.

```
try:
    _ = clamp >> wide_sum
except dp.OpenDPException as err:
    print(f"variant: {err.variant}")
    print(f"message: {first_line(err.message or '')}")
    print(f"rust frames recorded: {err.raw_traceback is not None}")
    dp.enable_features("rust-stack-trace")
    print(str(err).strip())
    dp.disable_features("rust-stack-trace")
```

```
variant: DomainMismatch
message: Intermediate domains don't match. See https://github.com/opendp/opendp/discussions/
rust frames recorded: True
Continued Rust stack trace:
    opendp_combinators__make_chain_tt
    opendp::combinators::chain::make_chain_tt::<opendp::ffi::any::AnyDomain, opendp::ffi::an
    opendp::combinators::chain::mismatch_error::<opendp::ffi::any::AnyDomain>
  DomainMismatch("Intermediate domains don't match. See https://github.com/opendp/opendp/dis
    output_domain: VectorDomain(AtomDomain(bounds=[0.0, 10.0], T=f64))
    input_domain:  VectorDomain(AtomDomain(bounds=[0.0, 100.0], T=f64))
")
```

The frames read from the outside in: the entry point that Python called, the chaining function, and the helper that formats the mismatch. They matter when the message alone does not say where in a long chain the library stopped. They also matter when you report a bug, since they locate it in the Rust source. The list of variants may change between versions, as the class's own documentation warns. So match on a variant in tests you control rather than in code that must outlive an upgrade. The errors that are not an `OpenDPException` come from the Python layer: a `TypeError` from argument conversion, and a `ValueError` from the context and the query classes.

15.6. What a reviewer verifies

A release made with OpenDP comes with a claim: this table satisfies (ε, δ)-differential privacy for this privacy unit. The chapters so far gave the reviewer everything needed to check it, and Table 15.1 gathers the checks. The first is the unit, because an ε means nothing until you know what it protects, and `dp.unit_of(contributions=36)` protects a person only if nobody has more than 36 rows. Identifier truncation is the fix when someone does. The second is the budget, read back from the context rather than from the code's intent.

The third is the flags, because `honest-but-curious` means a privacy map was taken on trust, and the reviewer must then read that map. The fourth is the public information, each bound and key traced to a source outside the data. The fifth is what ran outside the context, from the load to any code that held the frame. The full edition's chapter on the definition in the full edition says what the finished claim must state, and the practitioner edition's release chapter turns it into a record.

Table 15.1.: What a reviewer verifies before trusting a release made with OpenDP.

Check	Question	Where to look
Unit	What does one privacy unit contain, and does anyone exceed it?	`privacy_unit` in the context; the identifier and any truncation
Budget	What was spent, under which measure, and was any release collected twice?	`context.current_privacy_loss()`; one `collect` per `OnceFrame`
Features	Which flags were on, and if `honest-but-curious`, which maps were claimed?	the `enable_features` line; every `make_user_*` call and `lazy()`
Public information	Where did each bound, imputed value, key list and margin come from?	the constants of the script, traced to their sources
Outside the context	What touched the data before and after the context?	the load and its schema; any code holding the frame

15.7. Summary

Table 15.2.: The flags, the limits and the error messages of this chapter, with the rule of thumb for each.

Call or concept	What it does	Rule of thumb
`dp.enable_features("contrib")`	Opts in to components whose proofs are not yet vetted	Every script in this book needs it; record it with the release
`honest-but-curious`	Opts in to constructors whose guarantee rests on your arguments, such as plugins and `OnceFrame.lazy()`	Only where you wrote and proved the map, never where an untrusted analyst queries
`idealized-numerics`	Opts in to components whose proof assumes exact arithmetic	No constructor in this book needs it in 0.16.0; its alias `floating-point` warns
`dp.assert_features(...)`	Raises the flag error without building anything	One line at the top of a script states what it relies on
`rust-stack-trace`	Adds the Rust frames to an exception's text	Switch on to locate a bug; it changes no guarantee
Loading the data	Outside the guarantee; CSV inference reads private values	Read Parquet, or `infer_schema=False` and cast
`inferred type is f64, expected i32`	An argument's type differs from the data's	Match the data's type or cast first; a scalar ε for a pair is the common case

Call or concept	What it does	Rule of thumb
`Domain contains nan`	A float domain may hold NaN, and clamp refuses it	`impute_constant` or `fill_null` before `clamp`
`DomainMismatch`	Two chained components disagree on the domain between them	Use `then_*` constructors; read the two domains in the message
`must specify 'max_length' in a margin`	A float sum needs a public bound on the rows	`dp.polars.Margin(max_length=...)` from a public source
`You must call .release() before executing a query`	A plan was collected before it was released	`release()` first, then `collect()` once
`OnceFrame has been exhausted`	A released frame was collected twice	Keep the DataFrame from the first `collect`
`Privacy allowance has been exhausted`	Every share of the budget is spent	The refusal is the guarantee; a new context is a new budget
`delta (...) must be zero because keys and cuts span all columns`	A table with public keys spends no δ	Budget the query with pure ε or with ρ
`OpenDPException`	Carries `variant`, `message` and `raw_traceback`	Match on the variant only in tests; expect the list to change before 1.0

15.8. References

- OpenDP documentation: "Limitations" and the feature listing in the API user guide; "Troubleshooting" and "Next Steps" in Getting Started; the note on data loading in "Working with Tabular Data" and the margin and float-sum notes in "Essential Statistics".
- Casacuberta, Shoemate, Vadhan and Wagaman, "Widespread Underestimation of Sensitivity in Differentially Private Libraries and How to Fix It" (2022): the finite-precision sensitivity errors in earlier libraries, and the corrected proofs that OpenDP's sums follow.
- Mironov, "On Significance of the Least Significant Bits for Differential Privacy" (2012): the floating-point attack on the Laplace sampler.
- Jin, McMurtry, Rubinstein and Ohrimenko, "Are We There Yet? Timing and Floating-Point Attacks on Differential Privacy Systems" (2022): timing and floating-point side channels in differential privacy libraries.
- The full edition's chapters on the definition in the full edition and discrete and floating-point-safe mechanisms in the full edition derive the results this chapter states; the practitioner edition's Release and governance in the practitioner edition gives the record a reviewer asks for.

Part V.

Synthetic data

The last part releases not a statistic but a dataset. A synthetic dataset is a table of made-up rows whose statistics match the real table's as closely as a privacy budget allows. An analyst who never sees the real data can still run queries on it, fit models and draw charts. OpenDP builds one in two steps. First it releases a contingency table, which holds the noisy counts of every combination of a few columns at a time. Then it fits a model to those noisy marginals and samples rows from it. The second step is post-processing, so it costs no further budget. The thread of the part is that the quality of the synthetic data is decided by which marginals are measured and how the budget is split among them.

Contingency tables releases the marginals of a fixed workload with the `Fixed` algorithm. It explains keys, cuts and the thresholds that handle categories the data reveals, and shows the `ContingencyTable` object with its projections and error estimates. Synthetic data with AIM and MST lets the library choose the marginals adaptively with the AIM algorithm, or along a spanning tree with MST, and generates the synthetic rows. Evaluating and tuning synthetic data measures the result against the real data. It checks the marginals, a query the workload did not contain and a downstream model, and shows how the budget and the workload change the answer.

Table 15.3.: The chapters of this part and the questions they answer.

Chapter	Question it answers
Contingency tables	How do I release the noisy counts of a set of marginals, and what does the table object hold?
Synthetic data with AIM and MST	How do I let the library choose the marginals, and how do I generate synthetic rows?
Evaluating and tuning synthetic data	How good is the synthetic data, and what changes it?

By the end of this part you will be able to generate a differentially private synthetic copy of a table and state its guarantee. You will also be able to measure how far the copy is from the original.

16. Contingency tables

Suppose you want to publish not one count but a whole table. It would show how many people of each age group usually work each number of hours, by employment status and sex. A **contingency table** is that table. It counts the records in every combination of the values of several columns. Grouping released one for two columns, and this chapter asks what changes when there are four, or forty. The answer is that you no longer release the table itself. You release a few **marginals**, which are the same counts for two or three columns at a time, and fit one model that agrees with all of them. The model then answers any count over those columns, consistently, from the budget already spent.

The chapter follows the documentation's page on contingency tables. You will set up a context under zero-concentrated differential privacy (zCDP) and say which columns have known keys or cut points. Then you will release a fixed workload of marginals and read the `ContingencyTable` that comes back. At the end you will read a projection of the table against its error bar and add a marginal that the first release left out.

The script opendp_contingency_tables.py runs every step on OpenDP 0.16.0 with `mbi` 1.1.0, the Private-PGM version OpenDP pins, against the Labour Force Survey (LFS) sample of the earlier chapters. The budgets, scales, thresholds, keys and error bars it prints are the same on every run. The released counts are not, because OpenDP's noise cannot be seeded, so the page shows them in one block marked as one run.

```
uv run python code/part05-synthetic-data/opendp_contingency_tables.py
```

16.1. From grouped counts to marginals

Why not release the full table with the grouped counts of Grouping? Two things go wrong as columns are added. The first is size. Each column multiplies the number of cells by its number of values, and the four raw columns of this sample already make 4,648 cells. The second is noise. The noise on a cell does not depend on how many records the cell holds. So spreading the same records over more cells makes each count smaller while its noise stays the same. Soon most cells are noise.

A marginal is the contingency table of a subset of the columns. The two-way marginal of `AGE` and `HWUSUAL` has sixteen cells once the two columns are binned as below, each with a count large enough to survive its noise. Releasing several low-order marginals keeps the relations between the columns that share a marginal and gives up the higher-order ones. The documentation's workload has two marginals, `AGE` with `HWUSUAL` and `HWUSUAL` with `ILOSTAT`.

Noisy marginals released separately do not agree with each other. Both marginals above contain the one-way counts of `HWUSUAL`, and two noisy copies of the same count differ. OpenDP therefore

does not hand you the noisy marginals. It fits one probability model over all the columns whose own marginals come as close as possible to the noisy ones, and every count you read comes from that model.

The method is **Private-PGM**, which McKenna, Sheldon and Miklau introduced in 2019 in their paper "Graphical-model based estimation and inference for differential privacy". The model is a graphical model with one factor per measured marginal, fitted by mirror descent, a gradient method that keeps the model a valid distribution at every step. The fitting sees only the noisy marginals, so it is post-processing and costs no budget. Private-PGM is the `mbi` package, and the full edition's chapter on query release in the full edition covers the problem it solves, which is answering many counting queries from a few noisy measurements.

Install the contingency-table extra with `pip install 'opendp[mbi]'`. It brings `mbi` 1.1.0, the version OpenDP 0.16.0 pins, and the documentation's code then runs as written. Every algorithm fits its model with `mbi.estimation.mirror_descent` at that package's default of 1,000 steps. The check at the end of the chapter looks at whether that is enough on this data.

16.2. The context under zCDP, with a delta

The context is the one of Assessing utility and choosing the noise, with a larger budget. As there, the privacy unit is 36 contributions, because a person answers at most once per quarter over nine years. The budget is a privacy parameter ρ of 0.2 under zero-concentrated differential privacy, together with a slack δ of 10^{-7} for the stable keys. Of that, the two queries of the chapter take $\rho = 0.1$ and $\rho = 0.05$, so part of the budget stays unspent.

```
def make_context(frame: pl.LazyFrame) -> dp.Context:
    """The context: 36 contributions per person, rho = 0.2 of zCDP with delta = 1e-7, spent
    return dp.Context.compositor(
        data=frame,
        privacy_unit=dp.unit_of(contributions=CONTRIBUTIONS),  # a person answers in at most
        privacy_loss=dp.loss_of(rho=RHO, delta=DELTA),  # the delta pays for the stable keys
    )
```

```
privacy unit: (SymmetricDistance(), 36)
privacy loss: (Approximate(ZeroConcentratedDivergence), (0.2, 1e-07))
```

Why zCDP rather than pure differential privacy? A contingency table is many counts released together. Under zCDP the Gaussian noise on all of them composes by adding up their values of ρ. That sum grows more slowly than the privacy parameters ε of pure differential privacy would add up. Combinators showed that rule, and the full edition's chapter on concentrated differential privacy in the full edition derives it. The mechanism also accepts a budget in ε, with Laplace noise. The error bars below are then only approximate, because a sum of Laplace draws is not Laplace.

Why a δ? One column, `ILOSTAT`, will have neither keys nor cuts. Its keys will be read from the data, which is the stable-keys release of Grouping. There a key is shown only when its noisy count clears a threshold, set so that one person alone reaches it with probability at most δ. The

slack pays for that threshold and for nothing else, and the library enforces the connection in both directions. Give every column keys or cuts and also a δ, and the query is refused. Leave a column without them and set $\delta = 0$, and it is refused too. The script provokes both refusals on this data.

```
delta with keys for every column: delta (1e-07) must be zero because keys and cuts span all
no delta with ILOSTAT unkeyed: delta (0.0) must be nonzero because keys and cuts don't span
```

The snippet below shows the first refusal on a toy frame, together with one more fact. When every column is keyed, nothing is spent on discovering keys, and the query reports no one-way scale or threshold at all.

```
import opendp.prelude as dp
import polars as pl

dp.enable_features("contrib")
frame = pl.LazyFrame({"SEX": [1, 2, 1, 2]})
fixed = dp.mbi.Fixed(queries=[dp.mbi.Count(("SEX",))])
unit = dp.unit_of(contributions=1)
pure = dp.Context.compositor(data=frame, privacy_unit=unit, privacy_loss=dp.loss_of(rho=0.1)
query = pure.query().contingency_table(keys={"SEX": [1, 2]}, algorithm=fixed)
print(query.oneway_scale, query.oneway_threshold)  # None None
slack = dp.loss_of(rho=0.1, delta=1e-7)
approximate = dp.Context.compositor(data=frame, privacy_unit=unit, privacy_loss=slack, split
try:
    approximate.query().contingency_table(keys={"SEX": [1, 2]}, algorithm=fixed)
except ValueError as err:
    print(err)  # delta (1e-07) must be zero because keys and cuts span all columns
```

16.3. Keys and cuts

The model needs a fixed, finite set of values for every column, because its cells are indexed by them. OpenDP takes that set from one of three places. For a categorical column you pass **keys**, the public list of its values, as `keys={"SEX": [1, 2]}` from the survey's codebook. For a numeric column you pass **cuts**, the public bin edges, as `cuts={"AGE": [20, 40, 60]}`. The column is then binned into left-open intervals, four of them for three cuts, labelled `(-inf, 20.0]`, `(20.0, 40.0]`, `(40.0, 60.0]` and `(60.0, inf]`. For a column with neither, here `ILOSTAT`, the library spends part of the budget on **stable keys**, the values whose noisy counts clear the threshold.

Each of the three sources can miss values, and the library handles every miss the same way. A value outside the key list, a code 3 in `SEX` for instance, becomes `null`. A stable key with too small a count is not released and becomes `null` too. So `null` stands for "something else", and a true null in the data joins it. If you need to tell true nulls apart, recode them to a value of their own with `with_columns` before the query, as the documentation advises. A cut column has no null bin, so a null `AGE` or `HWUSUAL` lands in the last bin. The check at the end of the chapter confirms that on the 255 rows without an age and the 20,819 without usual hours.

The codebook of `ILOSTAT` lists six codes, from 1 for working to 9 for a child under 15. Two of them, 4 for conscripts and 5 for other inactive people, do not occur in this sample at all, so the stable keys should find exactly four. The documentation describes codes 4 and 5 as absorbed into `null`. In this sample `null` is their cell, with no rows behind it, and the released key set below shows the same thing.

16.4. The Fixed algorithm and its workload

`dp.mbi.Fixed` releases a workload you name in advance, as a list of `Count` queries, one per marginal. The algorithms of the next chapter choose the marginals themselves, and a fixed workload is the right tool when you already know which relations you need. The query names the columns, the keys and cuts, and the algorithm.

```
def workload_query(context: dp.Context) -> ContingencyTableQuery:
    """The documentation's query: keys for SEX, cuts for AGE and HWUSUAL, stable keys for IL
    return (
        context.query(rho=TABLE_RHO, delta=DELTA)
        .select("SEX", "AGE", "HWUSUAL", "ILOSTAT")
        .contingency_table(
            keys={"SEX": [1, 2]},  # public: the codebook's two codes
            cuts={"AGE": [20, 40, 60], "HWUSUAL": [1, 20, 40]},  # left-open bins: (-inf, 20
            algorithm=dp.mbi.Fixed(
                queries=[dp.mbi.Count(("AGE", "HWUSUAL")), dp.mbi.Count(("HWUSUAL", "ILOSTAT
                oneway_split=ONEWAY_SPLIT,  # the share of rho for the one-way marginal of I
            ),
        )
    )
```

The budget of the query is split in two parts. A share `oneway_split` goes to the one-way marginals of the columns without keys, which is where the stable keys of `ILOSTAT` come from. The rest goes to the workload, equally over its counts unless you give them weights. When you do not set the share, `Fixed` uses half the budget times the fraction of columns without keys, which here is a half of a quarter. The documentation sets it to one eighth explicitly, which is the same number, and the script prints the split.

```
oneway split: 0.125 of rho=0.1 -> rho=0.0125 one-way, rho=0.04375 per count
oneway scale: 227.68  oneway threshold: 1364
```

Read the two attributes before releasing. The scale is the standard deviation of the Gaussian noise on the one-way counts of `ILOSTAT`, and the threshold is the count a noisy key must exceed to be kept. Both follow from the privacy unit. One person's 36 rows can all carry the same status, so adding or removing a person changes one count by up to 36. That number is the sensitivity of the one-way marginal. The Gaussian scale that spends ρ on a count of sensitivity Δ is $\Delta/\sqrt{2\rho}$, as Assessing utility and choosing the noise stated. The script checks the arithmetic for both parts of the budget.

```
scale from the formula 36 / sqrt(2 x 0.0125): 227.68
scale of each count of the workload, 36 / sqrt(2 x 0.04375): 121.70
```

So each cell of the two measured marginals carries Gaussian noise with standard deviation 121.70. The threshold 1364 is set as in Grouping. At scale 227.68, a key held by one person alone, in all 36 of their rows, clears it with probability at most δ. A smaller threshold needs more keys, a larger budget, or a tighter bound on how many groups one person can touch. The sensitivity 36 is the price of the privacy unit. Truncating to one row per person and year, as Identifier truncation does, would lower it. The sensitivity of each marginal is derived like that of any grouped count, in the full edition's chapter on sensitivity.

16.5. The released table

```
table = query.release()
ilostat_std = table.std(("ILOSTAT",))
ilostat_accuracy = dp.gaussian_scale_to_accuracy(ilostat_std, alpha=ALPHA)
```

The release returns a `ContingencyTable`. The table holds the key sets the model was fitted on, the cuts, the thresholds that produced any stable keys, and the data type of each column's keys.

```
keys: SEX=[1, 2, null]  AGE=[(-inf, 20.0], (20.0, 40.0], (40.0, 60.0], (60.0, inf]]  HWUSUAL
cuts: AGE=[20.0, 40.0, 60.0]  HWUSUAL=[1.0, 20.0, 40.0]
thresholds: {'ILOSTAT': 1364}
schema: {'SEX': Int64, 'AGE': String, 'HWUSUAL': String, 'ILOSTAT': Int64}
```

Every key set that is not a cut set ends in `null`, the cell for values outside the keys. `SEX` has one although the data hold only 1 and 2. The stable keys of `ILOSTAT` are the four codes with rows, plus `null`. The schema shows that a cut column now holds strings, the bin labels, so a query on the table names bins and not numbers.

The method `project` sums the model over every column but the ones you name and returns the counts as an array, in the order of the keys. The method `project_melted` returns the same counts as a data frame with one row per combination of keys. Both are post-processing. You can project onto any set of columns, including ones no marginal measured, which the next section shows is a mixed blessing.

`std` estimates the standard deviation of a projection, and `dp.gaussian_scale_to_accuracy` turns it into an error bar, as for any Gaussian release. Projecting a two-way marginal onto one of its columns sums four noisy cells, so the standard deviation doubles, from 121.70 to 243.40. With probability 0.95, each count of `ILOSTAT` is therefore within 477.06 of the truth. That estimate leaves out the one-way marginal of `ILOSTAT` itself. It was measured only over the keys that were released, a prefix of the marginal rather than the whole of it. OpenDP counts no cell-wise precision for such a measurement. The column `HWUSUAL` sits in both measured marginals, and the two estimates combine by their inverse variances to 181.42.

```
std ILOSTAT: 243.40  95% accuracy: 477.06
std AGE x HWUSUAL: 121.70  95% accuracy: 238.53
std HWUSUAL: 181.42  95% accuracy: 355.58
```

16.6. Adaptive estimation

`SEX` has keys, but no marginal of the workload contains it. The model has an estimate of the total number of rows and nothing about how they split by sex. So it spreads the total evenly over the three keys, and `std` refuses to put a bar on that.

```
std SEX: attrs (('SEX',)) are not covered by the query set
```

A second query repairs this without starting over. Passing the released table as `table=` tells the library to keep its keys, cuts, marginals and model, measure the new marginals, and refit. Every column now has keys, the ones found by the first release included, so this query must carry $\delta = 0$ and spends nothing on key discovery.

```
def sex_query(context: dp.Context, table: dp.mbi.ContingencyTable) -> ContingencyTableQuery:
    """A second query that adds the marginal of SEX to the released table's model; every key
    return (
        context.query(rho=SEX_RHO, delta=0.0)
        .select("SEX")
        .contingency_table(
            table=table,  # keep the keys, cuts, marginals and model of the first release
            algorithm=dp.mbi.Fixed(queries=[dp.mbi.Count(("SEX",))]),
        )
    )
```

The new marginal is a one-way count at $\rho = 0.05$ with the same sensitivity as before, so its scale is 113.84. The three cells of `SEX` now have an error bar of 223.13. The model has four marginals, and the context has spent $\rho = 0.15$ of its 0.2. The second query's budget was chosen after seeing the first query's table, which is the adaptive composition that Combinators described. The refit starts from the first model's potentials, a warm start, and runs its own 1,000 steps of mirror descent from there.

```
std SEX after the second query: 113.84  95% accuracy: 223.13
marginals in the model: [('ILOSTAT',), ('AGE', 'HWUSUAL'), ('HWUSUAL', 'ILOSTAT'), ('SEX',)]
privacy loss spent: (0.15, 1e-07)
```

16.7. Checking the release against the truth

The data are public, so the script computes what the table estimates and compares. The true one-way counts of `ILOSTAT` and the true `AGE` by `HWUSUAL` table for the documentation's cuts come from Polars. Every null age or null hours is placed in the last bin, as the library places them.

```
true ILOSTAT counts: 1=1,517,011  2=155,661  3=1,450,283  9=688,977
true AGE x HWUSUAL (a null AGE or HWUSUAL counts in the last bin):
  AGE (-inf, 20.0]: (-inf, 1.0]=2,733  (1.0, 20.0]=13,651  (20.0, 40.0]=95,686  (40.0, inf]=
  AGE (20.0, 40.0]: (-inf, 1.0]=1,441  (1.0, 20.0]=31,415  (20.0, 40.0]=370,616  (40.0, inf]
  AGE (40.0, 60.0]: (-inf, 1.0]=1,285  (1.0, 20.0]=46,743  (20.0, 40.0]=437,820  (40.0, inf]
  AGE (60.0, inf]: (-inf, 1.0]=1,502  (1.0, 20.0]=28,084  (20.0, 40.0]=122,405  (40.0, inf]=
```

The releases of one run follow, read against the error bars above. The noise is only part of what they show.

```
released (one run):
  ILOSTAT 1: 1,515,589  (true 1,517,011, off by -1,422)
  ILOSTAT 2: 155,622  (true 155,661, off by -39)
  ILOSTAT 3: 1,450,332  (true 1,450,283, off by +49)
  ILOSTAT 9: 688,890  (true 688,977, off by -87)
  ILOSTAT null: 1,142  (true 0, off by +1,142)
  AGE x HWUSUAL largest |released - true|: 521  (accuracy 238.53)
  at AGE (60.0, inf] HWUSUAL (40.0, inf]: 1,038,588  (true 1,039,109)
  last bin of both, the nulls' cell: 1,038,588  (true 1,039,109, off by -521)
  SEX before the second query: 1,270,525 1,270,525 1,270,525  (uniform)
  SEX after: 1: 1,828,537  (true 1,828,992)  2: 1,982,506  (true 1,982,940)  null: 610  (tru
  ILOSTAT after the second query:
    ILOSTAT 1: 1,516,288  (true 1,517,011, off by -723)
    ILOSTAT 2: 155,608  (true 155,661, off by -53)
    ILOSTAT 3: 1,450,301  (true 1,450,283, off by +18)
    ILOSTAT 9: 688,867  (true 688,977, off by -110)
    ILOSTAT null: 590  (true 0, off by +590)
```

Start with the cells the noise alone explains. The largest of the sixteen two-way errors is 521 rows, past twice the per-cell bar of 238.53 but inside three times it. That largest error sits in the last bin of both columns, which holds the 20,819 rows without usual hours and the 255 without an age. A null placed in any other bin would have left this cell some 20,000 rows short, so the run still confirms where the library puts a null. The `null` keys of `ILOSTAT` and `SEX` have no rows behind them, and their estimates are positive, because a fitted distribution cannot go negative. Read a cell whose truth may be zero with that in mind.

Now the counts that the noise does not explain. The working code 1 of `ILOSTAT` came out 1,422 rows short, past twice its bar of 477.06. The empty `null` key holds 1,142 rows. Both are the fit, not the noise. The fit starts from the uniform model, the one that spread `SEX` evenly above, so `null` begins with a fifth of all rows. A key with no rows is the slowest cell to fit, because the model can empty it only in the limit. One thousand steps bring it down to about a thousand rows and no further, and the rows it keeps are missing from the other keys.

The second query shows the same thing in `SEX`. Its fit starts from the first model, in which the `null` key of `SEX` holds a third of all rows. After its own thousand steps that key still holds 610 rows, more than twice its bar. The two real keys are short by 455 and 434 rows. Those steps also carried on fitting `ILOSTAT`, so its working code is now 723 rows short and its `null` holds 590.

A refit settles it. The script refits the four released marginals of the second table from scratch, with 10,000 steps of mirror descent instead of the default 1,000, by calling `mbi.estimation.mirror_descent` on them. That touches no data, so it is post-processing and a dev-time check, never part of a release.

```
the same marginals refitted at 10,000 steps (dev time, post-processing):
  ILOSTAT 1: 1,516,829  (true 1,517,011, off by -182)
  ILOSTAT 2: 155,587  (true 155,661, off by -74)
  ILOSTAT 3: 1,450,277  (true 1,450,283, off by -6)
  ILOSTAT 9: 688,859  (true 688,977, off by -118)
  ILOSTAT null: 103  (true 0, off by +103)
  SEX: 1: 1,828,829  (true 1,828,992)  2: 1,982,798  (true 1,982,940)  null: 27  (true 0)
  AGE x HWUSUAL largest |refit - true|: 295  (accuracy 238.53)
```

The refit leaves 103 rows in the `null` key of `ILOSTAT` and 27 in that of `SEX`, inside their bars. Code 1 is within 182 rows of the truth, and each real key of `SEX` is within 163. The largest two-way error is 295 rows, inside three times its bar. A fit that stops early biases the empty cells upward and the largest ones downward, and nothing in the release warns you. A dev-time check against public or historical data is how you notice it.

The test behind the script holds each two-way cell and each refitted count of `ILOSTAT` and `SEX` within three times its bar. A correct run leaves that band less than once in a thousand. The released counts of the default fit it checks as this section describes. Each `null` holds fewer than a tenth of a percent of the rows and more than the refit leaves there. Each real key is within three times its bar plus the rows left in `null`, because those rows were taken from the real keys.

16.8. Summary

Table 16.1.: The calls and concepts of a contingency-table release, what each does, and the rule of thumb that goes with it.

Call or concept	What it does	Rule of thumb
`dp.loss_of(rho=0.2, delta=1e-7)`	A zCDP budget with a slack for stable keys	Gaussian noise composes tightly over many marginals; the δ is needed exactly when a column has neither keys nor cuts
`keys={"SEX": [1, 2]}`	The public values of a categorical column	Values outside the list become `null`
`cuts={"AGE": [20, 40, 60]}`	Left-open bins of a numeric column	Three cuts make four bins; a null lands in the last one
A column with neither	Stable keys, bought with `oneway_split` of the budget	Rare values fall below the threshold and join `null`
`dp.mbi.Fixed(queries=[dp.mbi.Count(...), ...], oneway_split=1/8)`	A workload named in advance	The rest of the budget is shared equally over the counts, or by their weights

Call or concept	What it does	Rule of thumb
`query.oneway_scale`, `query.oneway_threshold`	The noise and the cut-off of the key discovery	Read them before releasing; both are `None` when every column is keyed
`table.keys`, `table.cuts`, `table.thresholds`, `table.schema`	What the model was fitted on	Cut columns come back as bin labels, not numbers
`table.project`, `table.project_melted`	Any count over the table's columns, from the model	Post-processing; a column no marginal covers comes back uniform
`table.std` with `dp.gaussian_scale_to_accuracy`	The error bar of a covered projection	Raises for an uncovered projection; a cell near zero is biased upward
`contingency_table(table=table, ...)`	Adds marginals to an existing model	Every key is then known, so $\delta = 0$
`mbi.estimation.mirror_descent(domain, marginals, iters=...)`	Refits the released marginals with more steps	Post-processing; the default 1,000 steps leave rows in the empty `null` keys on this data, so check the fit on public data

16.9. References

- OpenDP documentation: "Contingency Tables" in Getting Started, and the opening of "Synthetic Data", which lists the algorithms and the rules for null values.
- McKenna, Sheldon and Miklau, "Graphical-model based estimation and inference for differential privacy" (2019): Private-PGM, the model that one consistent table is fitted from, and the mirror-descent fit.
- Bun and Steinke, "Concentrated Differential Privacy: Simplifications, Extensions, and Lower Bounds" (2016): zCDP and the Gaussian mechanism under it.
- The full edition's chapters on query release in the full edition, sensitivity in the full edition and concentrated differential privacy in the full edition derive the results this chapter states.

17. Synthetic data with AIM and MST

Contingency tables released the marginals you asked for, which works when you know the questions in advance. The analyst who will use a synthetic dataset usually does not, and a workload that guesses wrong leaves the relationships they care about to chance. This chapter hands the choice to the library. The **AIM** algorithm picks the marginals it measures one round at a time, going each time for the one its current model gets most wrong. The **MST** algorithm picks a spanning tree of column pairs in one go. Both fit a model to the noisy marginals, and `synthesize()` turns the model into rows. Only the measurements and the choices touch the data. Fitting the model and drawing rows post-process released values, so they cost nothing.

The chapter follows the documentation's synthetic-data page and the docstrings of `dp.mbi.AIM`, `dp.mbi.MST`, `dp.mbi.Sequential` and `ContingencyTable.synthesize`. You will release three tables over the same four survey columns and read which marginals each algorithm chose, at what noise scale. You will generate a synthetic frame of 3.8 million rows, state the guarantee of the whole release, and compare the synthetic marginals with the real ones. That comparison turns up a problem with nulls that the documentation warns about, and you will fix it. Keys, cuts, `Fixed` and the `ContingencyTable` object are the previous chapter's, used here without re-teaching them.

The script opendp_aim_and_mst.py runs everything on the Labour Force Survey (LFS) sample with OpenDP 0.16.0 and `mbi` 1.1.0, the Private-PGM version OpenDP pins. It runs JAX, which `mbi` uses to fit the models, on the CPU, and filters the warnings `mbi` may print when it imports JAX. The noise scales, the threshold, the budget arithmetic, the conversion to ε and the facts about the data are the same on every run. The marginals the algorithms choose, the noisy counts, the synthetic rows, the run times and the distances to the real data are not, because OpenDP's noise cannot be seeded. The page shows them in blocks marked as one run. The whole script takes about 75 seconds on this machine.

```
uv run python code/part05-synthetic-data/opendp_aim_and_mst.py
```

17.1. When you do not know the questions

A fixed workload spends the budget on the marginals you name and on nothing else. If the analyst later asks about a pair of columns you did not name, the model answers from the one-way marginals alone, as if the two columns were independent. The documentation's contingency-table page shows this on `SEX`, which comes out uniform over its keys because no measured marginal contains it. With four columns you could name every pair and every triple. But the budget would then be spread over ten marginals, most of which the model would have fitted well anyway. The question is how to spend it where the model is wrong.

AIM answers it adaptively. It first measures the one-way marginal of every column with half the budget, which also releases the keys of a column that has none, as the previous chapter explained. Then it fits a model to those marginals and runs rounds. In each round it scores every candidate marginal, by default every marginal of up to three columns. The score is the distance between the model's version of the marginal and the true one, as a sum of absolute cell differences, minus what measuring it would cost in noise. Candidates are weighted by how much they overlap the workload, so the three-way marginals, which are the workload itself, weigh most.

A noisy selection then picks one candidate with a tenth of the round's budget. The pick is differentially private because Gaussian noise is added to the scores, the report-noisy-max form of the exponential mechanism that the full edition's chapter on selection derives. The chosen marginal is measured with Gaussian noise with the other nine tenths, and the model is refitted.

The adaptive part is in the budget per round. AIM starts with a small share per round, the budget divided by the number of rounds. A measurement may change the model's version of that marginal by less than the noise it carried. Then the round taught little, and the next round gets four times the share. The rounds stop when the budget is spent, which a privacy filter enforces. The `rounds` argument sets the number of planned rounds and so the starting share. Its default is sixteen per column, which is 64 here, and the script uses 10 so that a release takes seconds. The `max_size` argument caps the memory of the model in megabytes, and a candidate that would push the model past the cap is skipped. With four small columns the cap never binds.

```
import opendp.prelude as dp

dp.enable_features("contrib")
aim = dp.mbi.AIM()
print(aim.queries, aim.measure_split, aim.max_size, aim.rounds)  # 3 0.9 80.0 None
print(dp.mbi.MST().num_selections, dp.mbi.MST().measure_split)  # None 0.9
print(dp.mbi.Count(("SEX", "ILOSTAT")))  # Count(by=('SEX', 'ILOSTAT'), weight=1.0)
```

The `queries` argument is either the largest number of columns a candidate may have, 3 by default, or an explicit workload. A workload is a list of `dp.mbi.Count` objects, each a set of columns with a weight, and AIM then favours the candidates that overlap the heavy ones. The `measure_split` of 0.9 is the share of each round that goes to measuring rather than selecting. MST has the same split and one argument of its own, `num_selections`, which Section 17.3 explains.

17.2. The AIM release

The context is the documentation's, with three times the budget because this chapter releases three tables. The privacy unit is 36 contributions, the most quarters one person can answer over nine years, as the data and the context explained. The budget is $\rho = 0.3$ under zero-concentrated differential privacy (zCDP), where ρ bounds the privacy loss on average rather than in the worst case and the noise is Gaussian. Assessing utility and choosing the noise introduced it, and the full edition's chapter on concentrated differential privacy (DP) derives its composition and conversion rules. The slack δ of 3×10^{-7} is the small failure probability that pays for releasing the keys of `ILOSTAT`, which the query does not name.

```
def make_context(frame: pl.LazyFrame) -> dp.Context:
    """The context: a privacy unit of 36 rows, a budget of rho = 0.3 with delta = 3e-7, one
    return dp.Context.compositor(
        data=frame,
        privacy_unit=dp.unit_of(contributions=CONTRIBUTIONS),  # neighbours add or remove on
        privacy_loss=dp.loss_of(rho=RHO, delta=DELTA),  # zCDP with Gaussian noise; delta pa
    )
```

Every release goes through one function, because the three queries differ only in the algorithm plugged in. The query takes a third of the budget, $\rho = 0.1$ with $\delta = 10^{-7}$. It selects the documentation's four columns, gives the keys of `SEX` and the cuts of `AGE` and `HWUSUAL`, and leaves `ILOSTAT` to the threshold. The `fill` argument is for later.

```
def table_query(context: dp.Context, algorithm: Algorithm, fill: bool = False) -> Contingenc
    """The documentation's contingency-table query at rho = 0.1 and delta = 1e-7, with the a
    query = context.query(rho=RHO_QUERY, delta=DELTA_QUERY)
    if fill:
        query = query.with_columns(fill_nulls())  # a row-by-row transformation before the t
    return query.select(*COLUMNS).contingency_table(
        keys=KEYS,  # public categories of SEX; ILOSTAT's are released with the threshold
        cuts=CUTS,  # bins for the two numeric columns
        algorithm=algorithm,
    )
```

The first release plugs in AIM with ten rounds. Before releasing, the query reports the scale and the threshold of its one-way step, which cost nothing to read.

```
aim_query = table_query(context, dp.mbi.AIM(rounds=ROUNDS))
oneway_scale, oneway_threshold = aim_query.oneway_scale, aim_query.oneway_threshold
table_aim, seconds_aim = timed_release(aim_query)
```

```
privacy unit: (SymmetricDistance(), 36)
privacy loss: (Approximate(ZeroConcentratedDivergence), (0.3, 3e-07))
query budget: rho=0.1 delta=1e-07
aim: oneway scale=227.6840 threshold=1364
aim: rho per one-way marginal=0.0125  scale 36/sqrt(2 rho)=227.6840
aim: rounds=10  rho per round=0.0050  measure=0.0045  select=0.0005  scale of a round's marg
aim keys ILOSTAT: [1, 2, 3, 9, None]  thresholds: {'ILOSTAT': 1364}
```

The scales follow from the budget split. Half of the query's ρ goes to the one-way step, and AIM measures all four one-way marginals there, so each gets 0.0125. Adding or removing one person changes the cells of a marginal by at most 36 rows in all, and in the worst case all in one cell, so the sensitivity in the Euclidean norm is 36. Write Δ_2 for that sensitivity and σ for the standard deviation of the Gaussian noise. The mechanism is ρ-zCDP when

$$\sigma = \frac{\Delta_2}{\sqrt{2\rho}}, \tag{17.1}$$

and that formula gives the 227.68 the query reports.

The other half of the share is divided over the ten rounds. Nine tenths of a round's 0.005 goes to measuring, and the formula gives 379.47 for a round's marginal. The threshold of 1364 is the noisy count a category of `ILOSTAT` must clear to be released. The keys came out as the four codes in the data plus the null that stands for anything else.

```
aim release (one run): 13.4 s
aim measured: ('AGE', 'HWUSUAL', 'ILOSTAT') std=379.47
aim measured: ('SEX', 'HWUSUAL', 'ILOSTAT') std=379.47
aim measured: ('SEX', 'AGE', 'HWUSUAL') std=379.47
aim measured: ('SEX', 'AGE', 'HWUSUAL') std=189.74
aim measured: ('SEX', 'AGE', 'ILOSTAT') std=379.47
aim measured: ('SEX', 'HWUSUAL') std=379.47
aim measured: ('AGE', 'HWUSUAL') std=379.47
```

The release took 13 seconds in this run. The table's `marginals` attribute holds every measurement with its clique, the columns it spans, and its noise. The script lists them grouped by clique, in the order the cliques were first measured. In this run AIM measured all four three-way marginals before any pair, as the weights predict. One of them appears twice, the second time at half the scale, which is the adaptive rule at work. A round had moved the model by less than its noise, so the next round took four times the share, which was all that was left, and measured that marginal again. So there were seven measurements rather than ten. Another run chooses differently, which is why you read this list after every release.

17.3. A spanning tree of pairs with MST

MST is the mechanism that won the 2018 NIST synthetic-data contest, and it is simpler than AIM. After the same one-way step it considers only pairs of columns. Each pair is scored by how far the model's version is from the true one, again as a sum of absolute cell differences. MST then selects pairs one at a time with a noisy maximum, each time among the pairs that join two columns not yet connected by an earlier pick. With four columns that gives three pairs, and they form a tree that spans every column, which is where the name comes from. The `num_selections` argument sets how many pairs to pick and defaults to one fewer than the number of columns. Then all selected pairs are measured at once with nine tenths of the remaining budget, and the model is fitted.

```
mst_query = table_query(context, dp.mbi.MST())
table_mst, seconds_mst = timed_release(mst_query)
```

```
mst: oneway scale=227.6840 threshold=1364
mst: pairs=3  rho per pair=0.0150  scale of a pair=207.8461
mst release (one run): 3.3 s
mst measured: ('HWUSUAL', 'ILOSTAT') std=207.85
mst measured: ('AGE', 'ILOSTAT') std=207.85
mst measured: ('SEX', 'ILOSTAT') std=207.85
mst pairs form a spanning tree: True
```

The one-way step is identical, so its scale and threshold are the same. The three pairs share nine tenths of the remaining budget, 0.015 each, and the formula gives 207.85. Each pair is therefore measured more precisely than one of AIM's rounds, because MST commits to three measurements where AIM keeps budget back for rounds it may need. In this run the tree is a star on `ILOSTAT`, the column most related to the other three. The script checks that the pairs connect all four columns without a cycle, which the selection guarantees. The release took three seconds.

17.4. A first look at the one-way marginals

The data are public, so the releases can be compared with the truth, which no real release allows. The script draws the synthetic rows of each table, puts their numeric columns back into their bins, and compares each one-way marginal with the real one. The distance it reports is the total variation distance between the two marginals read as distributions, which is half the sum of the absolute differences of their shares. It also reports the largest error of any cell in rows, and the table's own standard deviation for that marginal from `std`.

The last column is a bound. Write k for the number of cells of the marginal and n for the number of rows. If every cell were off by three standard deviations at once, the distance would be

$$\frac{3k\sigma}{2n}. \tag{17.2}$$

A well-fitted marginal stays far under that, because the cell errors have different signs and few reach three standard deviations.

```
one-way SEX (aim): tv=0.00007 largest cell error=2,813 rows  std=204.6 bound=0.00024
one-way AGE (aim): tv=0.00015 largest cell error=1,547 rows  std=201.1 bound=0.00032
one-way HWUSUAL (aim): tv=0.00200 largest cell error=11,310 rows  std=192.3 bound=0.00030
one-way ILOSTAT (aim): tv=0.00039 largest cell error=1,670 rows  std=792.7 bound=0.00156
one-way SEX (mst): tv=0.00004 largest cell error=2,779 rows  std=204.5 bound=0.00024
one-way AGE (mst): tv=0.00017 largest cell error=1,981 rows  std=204.5 bound=0.00032
one-way HWUSUAL (mst): tv=0.00257 largest cell error=13,620 rows  std=204.5 bound=0.00032
one-way ILOSTAT (mst): tv=0.00040 largest cell error=1,963 rows  std=227.7 bound=0.00045
```

`SEX` and `AGE` are right as shares, with distances below their bounds. Their largest cell errors of two or three thousand rows are more than the noise explains, and the next section traces them to the model's total. `HWUSUAL` is wrong even as shares. Its distance is many times its bound in both releases, and one cell is off by more than eleven thousand rows, over fifty standard deviations. Both algorithms measured that marginal at the same scale as the others, so noise cannot explain it. Something systematic is wrong with the column.

17.5. Nulls and the key set

The model behind every table needs a fixed set of keys for every column, which the documentation's section on nulls explains. The keys come from you, through `keys` and `cuts`, or from the stable-key release with the threshold. Either way the set need not cover the data, and every value outside it is

replaced by null before counting. For a cut column there is no null key. The four bins of `HWUSUAL` are its whole key set, and the column has 20,819 real nulls, hours that were not recorded.

```
rows: 3,811,932  AGE nulls: 255  HWUSUAL nulls: 20,819
HWUSUAL = 99 (not applicable): 2,294,921
```

In version 0.16.0 those nulls take two different paths. The one-way step counts a cut column by joining its counts onto the bin labels, so a null row is not counted at all. The multi-way step indexes every row into a bin before it counts, and a row whose bin is null gets the last index, so the same row is counted in the bin above 40 hours. The model is then fitted to measurements that disagree about how many rows there are, and it compromises. The script shows the compromise on the fitted one-way marginal of `HWUSUAL`, against the real counts with the nulls in the last bin.

```
aim model total (one run): 3,806,638.8  real rows 3,811,932
HWUSUAL bins (aim, one run): fitted count against the real count with nulls in the last bin
  (-inf, 1.0]: fitted 8,953  real 6,961  difference +1,992
  (1.0, 20.0]: fitted 121,660  real 119,893  difference +1,767
  (20.0, 40.0]: fitted 1,028,785  real 1,026,527  difference +2,258
  (40.0, inf]: fitted 2,647,240  real 2,658,551  difference -11,311
```

The last bin is eleven thousand rows short and every other bin is two thousand rows over. The model's total, which it estimates from the one-way marginals, is five thousand rows below the real count. Those are the missing nulls, spread by the fit. The smallest bin suffers most in relative terms, since two thousand extra rows on seven thousand is more than a quarter.

The documentation's remedy is to turn true nulls into a key before the table, with `select` or `with_columns`, and that is what the `fill` argument of the query does. It is a row-by-row transformation inside the query, so it changes nothing about the privacy unit. The script sends the nulls of `HWUSUAL` to 99, the survey's own code for not applicable, which already accounts for most of the last bin. It sends the 255 nulls of `AGE` to zero, the youngest bin. Which code a null gets is a modelling decision. What matters for the fit is that every row lands in a key, the same way in both steps.

17.6. Guaranteeing marginals with Sequential

Adaptive selection has a cost. If a particular marginal must be in the release, AIM may or may not measure it. `Sequential` runs a list of algorithms one after another on the same table, each starting from the model the previous one left, and splits the budget evenly between them unless `weights` says otherwise. A `Fixed` workload first and AIM second gives you both. The named marginals are measured whatever happens, and AIM spends the rest where the model is still wrong. The third release does this with the pairs of `ILOSTAT` with `SEX` and with `AGE`, and it fills the nulls.

```
sequential = dp.mbi.Sequential(
    algorithms=[
        dp.mbi.Fixed(queries=[dp.mbi.Count(by) for by in GUARANTEED]),  # these two marginal
        dp.mbi.AIM(rounds=ROUNDS),  # then AIM chooses the rest
    ],
)
sequential_query = table_query(context, sequential, fill=True)
table_seq, seconds_seq = timed_release(sequential_query)
```

```
sequential: rho per guaranteed marginal=0.0125  scale=227.6840
sequential: rho per AIM round=0.0025  scale of a round's marginal=536.6563
sequential release (one run): 12.1 s
sequential model total (one run): 3,811,712.7  real rows 3,811,932
sequential measured: ('SEX', 'ILOSTAT') std=227.68
sequential measured: ('AGE', 'ILOSTAT') std=227.68
sequential measured: ('SEX', 'HWUSUAL', 'ILOSTAT') std=536.66
sequential measured: ('SEX', 'AGE', 'HWUSUAL') std=536.66
sequential measured: ('SEX', 'AGE', 'HWUSUAL') std=268.33
sequential measured: ('SEX', 'AGE', 'ILOSTAT') std=536.66
sequential measured: ('HWUSUAL', 'ILOSTAT') std=536.66
sequential measured: ('HWUSUAL', 'ILOSTAT') std=536.66
sequential measured: ('SEX', 'HWUSUAL') std=536.66
sequential guaranteed marginals present: True
```

The one-way step again takes half, and the rest is split evenly between the two algorithms. `Fixed` divides its quarter between its two marginals, which come out at the familiar 227.68. AIM runs its ten rounds on the other quarter, so each round's marginal is noisier than in the first release, 536.66 against 379.47. That is the price of the guarantee, and `weights` can shift it either way. The `oneway` and `oneway_split` arguments belong to the outer algorithm here, `Sequential`, which measures all four one-way marginals as AIM does. The model's total is now 219 rows from the truth, within one standard deviation of a one-way measurement. The nulls were the problem, and the one-way marginals confirm it.

```
one-way SEX (sequential): tv=0.00004 largest cell error=240 rows  std=198.2 bound=0.00023
one-way AGE (sequential): tv=0.00006 largest cell error=233 rows  std=200.7 bound=0.00032
one-way HWUSUAL (sequential): tv=0.00009 largest cell error=273 rows  std=206.2 bound=0.0003
one-way ILOSTAT (sequential): tv=0.00008 largest cell error=360 rows  std=271.5 bound=0.0005
```

Every distance is under its bound by a factor of three or more, and no cell is more than 360 rows off, which is under two standard deviations. `HWUSUAL` went from eleven thousand rows of error to 273.

17.7. Generating the rows

A `ContingencyTable` is a model, and `synthesize()` turns it into a frame. The model is a graphical model over the four columns whose marginals match the noisy measurements as closely

as they can be matched at once, fitted by Private-PGM with mirror descent. The full edition's chapter on query release explains why answering many queries through one consistent model beats answering them separately. Drawing rows from the model looks only at released values, so it is post-processing and spends nothing, however often you call it.

```
synthesize (aim, one run): shape (3806638, 4)  model total 3,806,638.8  11.9 s
```

SEX	AGE	HWUSUAL	ILOSTAT
i64	f64	f64	i64
1	19.813324	0.482324	1
1	19.500078	0.94265	1
1	19.338927	0.741979	9
1	19.031755	0.118982	9
1	19.232563	0.676039	1

```
synthetic AGE from 19.00 to 61.00  HWUSUAL from 0.00 to 41.00
synthetic dtypes: {'SEX': Int64, 'AGE': Float64, 'HWUSUAL': Float64, 'ILOSTAT': Int64}
synthetic SEX values: [1, 2]  ILOSTAT values: [1, 2, 3, 9]
```

Without a `rows` argument the frame has as many rows as the model's total, which is the noisy estimate of the row count and not the true one. The frame has the four columns with their original types, and `SEX` and `ILOSTAT` take values from their keys, the null key included. The default `method="round"` scales the model's expected counts to the number of rows and rounds them at random, so the synthetic marginals match the model's to within a row per cell. The alternative, `method="sample"`, draws each row independently from the model. That adds sampling noise of the order of the square root of each count, and it is the right choice when the rows should behave like a sample.

```
sample = table_aim.synthesize(rows=SAMPLE_ROWS, method="sample")  # draws rows independently
```

```
synthesize rows=100,000 method=sample: shape (100000, 4)
```

The numeric columns deserve a second look, because the first rows all show an `AGE` just under 20. The model knows only which bin a row is in. For a cut column, `synthesize()` draws a value uniformly inside the bin, and for the two open-ended bins it draws within one unit of the edge. So every synthetic age lies between 19 and 61, whatever the real range was, and the rows coded 99 in the real data all become hours just above 40. An analyst who computes a mean age from this frame gets a number that reflects the bins and the sampling rule, not the data. The ages are right as bins and wrong as numbers. If the numbers matter, give finer cuts, at the cost of more cells per marginal and more noise per cell, which Evaluating and tuning synthetic data weighs.

The script seeds NumPy's global generator, which the rounding and the within-bin draws use, so the frame is a function of the released table.

17.8. The privacy statement

The context has spent its three shares, and the statement of the whole release is written from the variables of the script, not typed in.

```
budget spent: (0.3, 3e-07)
rho=0.3 at delta=1e-07: epsilon=4.2709
whole context converted: epsilon=4.2709 delta=4e-07
```

A budget in ρ is stated to readers in ε, and the conversion is the one Assessing utility and choosing the noise used. A Gaussian measurement whose scale makes a sensitivity of 1 cost exactly $\rho = 0.3$ is converted with `make_zCDP_to_approxDP` and `make_fix_delta` at a further δ of 10^{-7}. That gives $\varepsilon = 4.27$. The same two combinators applied to the context's `accountant` give the same ε with the total δ, three parts for the keys and one for the conversion. So three adaptive releases cost $\varepsilon = 4.27$, which is a loose guarantee.

```
print(
    f"This release of three contingency tables and the synthetic rows drawn from them satisf
    f"(rho = {RHO:g}, delta = {DELTA:g})-zero-concentrated differential privacy, which is "
    f"(epsilon = {epsilon:.2f}, delta = {total_delta:g})-differential privacy, "
    f"for adding or removing all of one person's survey responses, at most {CONTRIBUTIONS} o
    f"under the central model, using OpenDP {dp.__version__} with Gaussian noise. "
    f"The keys of SEX and the cuts of AGE and HWUSUAL were fixed in advance; the keys of ILO
    f"with a threshold of {oneway_threshold}. Generating the rows is post-processing and spe
)
```

The statement covers the synthetic rows without a separate term, because post-processing adds nothing to the loss. It names what was public, since a cut chosen by looking at the data would leak it. The unit of 36 contributions is the expensive part of this guarantee. Every scale above carries the factor 36, and Identifier truncation and bounds showed that hardly anyone in the file answers more than a few times. The same query runs under the identifier unit with a truncation to four rows per person. The script builds it to read the scale and does not release it.

```
context_id = dp.Context.compositor(
    data=frame,
    privacy_unit=dp.unit_of(contributions=1, identifier="PIDENT"),  # neighbours add or remo
    privacy_loss=dp.loss_of(rho=RHO_QUERY, delta=DELTA_QUERY),
)
query_id = (
    context_id.query(rho=RHO_QUERY, delta=DELTA_QUERY)
    .truncate_per_group(TRUNCATE)  # at most 4 rows per person, so a person moves a count by
    .select(*COLUMNS)
    .contingency_table(keys=KEYS, cuts=CUTS, algorithm=dp.mbi.AIM(rounds=ROUNDS))
)
```

```
identifier unit, 4 rows per person: oneway scale=25.2982 threshold=142
identifier unit: scale ratio 9.00 = 36/4
```

The one-way scale falls nine-fold, to 25.30, because a person now moves a count by at most 4 rows rather than 36. The threshold falls from 1364 to 142, and every other scale of the release carries the same factor. The cost is the truncation, which drops every row beyond the fourth of anyone who answered more often, before anything is counted.

17.9. The measured marginals against the real ones

The last check compares every measured multi-way marginal of the three releases with the real one, in the same way as the one-way marginals. A measured marginal is what each release spent its budget on, so it is where the synthetic data should be closest to the truth. The lines are from the same run as the lists above.

```
measured (AGE, HWUSUAL) (aim): tv=0.00200 largest cell error=2,992 rows  std=224.1 bound=0.0
measured (SEX, HWUSUAL) (aim): tv=0.00200 largest cell error=5,730 rows  std=242.4 bound=0.0
measured (AGE, HWUSUAL, ILOSTAT) (aim): tv=0.00219 largest cell error=1,382 rows  std=379.5
measured (SEX, AGE, HWUSUAL) (aim): tv=0.00200 largest cell error=1,556 rows  std=169.7 boun
measured (SEX, AGE, ILOSTAT) (aim): tv=0.00093 largest cell error=699 rows  std=379.5 bound=
measured (SEX, HWUSUAL, ILOSTAT) (aim): tv=0.00200 largest cell error=1,940 rows  std=379.5
measured (AGE, ILOSTAT) (mst): tv=0.00116 largest cell error=2,366 rows  std=207.8 bound=0.0
measured (HWUSUAL, ILOSTAT) (mst): tv=0.00258 largest cell error=3,700 rows  std=207.8 bound
measured (SEX, ILOSTAT) (mst): tv=0.00041 largest cell error=1,085 rows  std=207.8 bound=0.0
measured (AGE, ILOSTAT) (sequential): tv=0.00027 largest cell error=387 rows  std=221.1 boun
measured (HWUSUAL, ILOSTAT) (sequential): tv=0.00068 largest cell error=1,035 rows  std=351.
measured (SEX, HWUSUAL) (sequential): tv=0.00017 largest cell error=338 rows  std=342.9 boun
measured (SEX, ILOSTAT) (sequential): tv=0.00019 largest cell error=335 rows  std=218.1 boun
measured (SEX, AGE, HWUSUAL) (sequential): tv=0.00094 largest cell error=712 rows  std=240.0
measured (SEX, AGE, ILOSTAT) (sequential): tv=0.00099 largest cell error=751 rows  std=536.7
measured (SEX, HWUSUAL, ILOSTAT) (sequential): tv=0.00073 largest cell error=567 rows  std=5
```

Two things stand out. First, in the two releases that left the nulls alone, every marginal containing `HWUSUAL` breaks its bound, and the pairs without it are further from the truth than in the third release, because the wrong total leaks into every marginal. Second, in the release that filled the nulls, every measured marginal is under its bound by a factor of four or more. The bound counts three standard deviations in every cell at once, which never happens, so a three-way marginal with a distance of one thousandth is a good fit.

What this check does not say is how the synthetic data answers a question nobody measured, or how a model trained on it compares with one trained on the real rows. Those are the questions of Evaluating and tuning synthetic data, which also shows what changes when the budget, the cuts or the number of rounds change.

17.10. Summary

Table 17.1.: The calls and concepts of synthetic data with AIM and MST, what each does, and the rule of thumb that goes with it.

Call or concept	What it does	Rule of thumb
`dp.mbi.AIM(rounds=10)`	One-way marginals, then rounds that measure the marginal the model fits worst	Fewer rounds for a quick release; read `table.marginals` to see what it chose
`queries`, `measure_split`, `max_size`	The candidates or a weighted workload; the share of a round spent measuring; the memory cap	The defaults serve a handful of columns
`dp.mbi.MST()`	One-way marginals, then pairs that form a spanning tree, measured at once	Faster than AIM when pairs are what matter; it never measures a triple
`dp.mbi.Sequential(algorithms=[Fixed, AIM])`	Run the algorithms in turn on one table, splitting the budget evenly or by `weights`	Put the marginals you must have in `Fixed`; AIM spends the rest
`query.oneway_scale`, `query.oneway_threshold`	The one-way step's noise scale and key threshold, before anything is spent	Under the identifier unit, 4 rows per person cost a ninth of 36
`table.synthesize(rows, method)`	Rows drawn from the fitted model; `round` matches its marginals, `sample` draws independently	Post-processing, so free; the row count defaults to the model's noisy total
Cut columns in the synthetic frame	Values drawn uniformly inside the bin, within one unit of the edge for the open bins	Right as bins, wrong as numbers; give finer cuts if the numbers matter
Nulls in a cut column	Dropped by the one-way step and counted in the last bin by the multi-way step in 0.16.0	Fill them with a public code in `with_columns` before the table
Total variation distance against `std`	How far a synthetic marginal is from the real one, against three standard deviations per cell	A measured marginal over the bound points to the data, not the noise

17.11. References

- OpenDP documentation: "Synthetic Data" in the statistical modelling guide, "Contingency Tables" in the tabular data guide, and the API reference for `opendp.extras.mbi` (`AIM`, `MST`, `Sequential`, `Fixed`, `ContingencyTable`).

- McKenna, Mullins, Sheldon and Miklau, "AIM: An Adaptive and Iterative Mechanism for Differentially Private Synthetic Data" (2022): the selection score, the budget schedule and the experiments behind the defaults.
- McKenna, Miklau and Sheldon, "Winning the NIST Contest: A scalable and general approach to differentially private synthetic data" (2021): the MST mechanism and the Private-PGM engine that fits the model.
- McKenna, Sheldon and Miklau, "Graphical-model based estimation and inference for differential privacy" (2019): Private-PGM itself, the inference step that both algorithms share.
- The full edition's chapters on selection in the full edition, query release in the full edition and concentrated DP in the full edition derive the results this chapter states.

18. Evaluating and tuning synthetic data

Synthetic data with AIM and MST ended with a synthetic table and a privacy statement. The statement says how little the table reveals about any one person, but it says nothing about whether the table is any good. This chapter measures that. The one idea is that "good" is three separate questions, and only the first has an answer that the release itself can give. You will compare an AIM release with the real data on the marginals it measured, on a table it never measured, and through a classifier trained on its rows. Then you will repeat the comparison at three budgets and for MST, and write down what to publish alongside the rows.

The documentation stops once the rows are generated, so this chapter goes beyond it. Its methods come from the paper that introduced AIM and from a benchmark of private synthetic-data generators, both cited at the end. The Labour Force Survey (LFS) sample is public, and that is what makes the comparison possible. On private data you would run the same comparison on a public or historical stand-in before the release, never on the data you protect.

The script opendp_evaluating_synthetic_data.py runs every step on OpenDP 0.16.0 with `mbi` 1.1.0, and it imports the previous chapter's script for the functions the two share. The budgets, noise scales, thresholds, conversions to the privacy parameter ε and the classifier trained on real rows are the same on every run. The releases, the fitted models and every distance to the truth are not, because OpenDP's noise cannot be seeded. So the page shows them in blocks and tables marked as one run. The whole script, with four releases, took 55 seconds in one run.

```
uv run python code/part05-synthetic-data/opendp_evaluating_synthetic_data.py
```

18.1. What good means

A synthetic table can be judged by three questions. The first is whether the marginals the algorithm measured match the real ones. The second is whether queries it never measured come out right, such as a count over all four columns or a share within a small group. The third is whether a model trained on the synthetic rows works on real rows. These are the checks of the benchmark by Tao and coauthors, and each one catches failures that the other two miss.

Only the first question has an error bar that comes from the release. Every measured marginal carried Gaussian noise of a known scale. So the table's `std` method can say how far its fitted counts should be from the truth, as Contingency tables showed. A table that no measurement covers has no such bar, and `std` refuses it. A downstream model has no bar either. The second and third questions can therefore be answered only against real data. On a real release that means public or historical data of the same kind, which the full edition's chapter on public data discusses.

The release under evaluation is the previous chapter's AIM release with its nulls filled. It uses the same privacy unit of 36 contributions, the same four columns with their keys and cuts, and ten rounds. The budget is $\rho = 0.1$ under zero-concentrated differential privacy (zCDP), where ρ bounds the privacy loss on average and the noise is Gaussian. A slack δ of 10^{-7} pays for releasing the keys of `ILOSTAT`. Two functions build the context and the query from a budget, so that later sections can change it.

```
def make_context(frame: pl.LazyFrame, rho: float) -> dp.Context:
    """A context for one release at budget rho, with the previous chapter's unit and slack."
    return dp.Context.compositor(
        data=frame,
        privacy_unit=dp.unit_of(contributions=CONTRIBUTIONS),  # neighbours add or remove on
        privacy_loss=dp.loss_of(rho=rho, delta=DELTA),  # zCDP with Gaussian noise; delta pa
    )


def table_query(context: dp.Context, rho: float, algorithm: Algorithm) -> ContingencyTableQu
    """The previous chapter's query with its nulls filled, at the budget rho, with the algor
    return (
        context.query(rho=rho, delta=DELTA)
        .with_columns(synth.fill_nulls())  # every null lands in a public key before anythin
        .select(*COLUMNS)
        .contingency_table(keys=synth.KEYS, cuts=synth.CUTS, algorithm=algorithm)
    )
```

The release itself is one line, and the next one prepares the comparison. It puts the real rows through the same steps as the query, filling the nulls, cutting the numbers into bins and mapping codes outside the keys to null. So every real marginal has exactly the cells of the synthetic one.

```
aim_query = table_query(make_context(frame, RHO), RHO, dp.mbi.AIM(rounds=ROUNDS))
table_aim, seconds_aim = synth.timed_release(aim_query)
real = synth.binned(frame, table_aim.keys, fill=True).collect()  # the real rows in the tabl
```

```
privacy unit: (SymmetricDistance(), 36)
privacy loss: (Approximate(ZeroConcentratedDivergence), (0.1, 1e-07))
aim rho=0.1: oneway scale=227.6840 threshold=1364  scale of a round's marginal=379.4733  eps
rows: 3,811,932  keys ILOSTAT: [1, 2, 3, 9, None]
aim rho=0.1 release (one run): 13.7 s
aim rho=0.1 measured: ('AGE', 'HWUSUAL', 'ILOSTAT') std=379.47
aim rho=0.1 measured: ('SEX', 'HWUSUAL', 'ILOSTAT') std=379.47
aim rho=0.1 measured: ('SEX', 'AGE', 'HWUSUAL') std=379.47
aim rho=0.1 measured: ('SEX', 'AGE', 'ILOSTAT') std=379.47
aim rho=0.1 measured: ('SEX', 'ILOSTAT') std=189.74
```

The scales are the previous chapter's, since the query is the same. The one-way marginals carry noise of scale 227.68 and each round's marginal 379.47. Converted at a further δ of 10^{-7}, the budget is $\varepsilon = 2.35$. In this run AIM measured all four three-way marginals and then a pair at half the scale.

18.2. The workload error

The first question asks how far each measured marginal of the model is from the real one. The AIM paper calls the average of that distance over a workload the **workload error**, and it is the number AIM itself tries to make small. The distance here is the total variation distance of the previous chapter, half the sum over cells of the absolute differences between the two tables' shares. It is the largest difference between the shares that the two tables give to any set of cells. So the worst distance below, 0.00102, means that no question about those columns gets a share that is off by more than that.

The function below computes it for a list of cliques, the sets of columns each marginal spans. It reads the model's own counts with `project_melted` rather than drawing rows, because drawing rows adds noise of its own, which Section 18.6 measures. Next to each distance it puts the bound of the previous chapter. That bound is the distance the marginal would have if every one of its cells were off by three of its standard deviations at once.

```
def marginal_errors(
    table: ContingencyTable, real: pl.DataFrame, cliques: Sequence[tuple[str, ...]]
) -> list[MarginalError]:
    """Each clique's distance between the fitted model's marginal and the real one, with the
    deviation for it and the bound that three standard deviations in every cell would give."
    errors: list[MarginalError] = []
    for clique in cliques:
        fitted = table.project_melted(clique)  # the model's counts: no rows are drawn
        tv, worst = distance(synth.marginal(real, clique), fitted, clique)
        std = table.std(clique)
        bound = synth.tv_bound(synth.cells(table, clique), std, real.height)
        errors.append(MarginalError(clique, tv, worst, std, bound))
    return errors
```

Table 18.1.: The workload error of the AIM release at $\rho = 0.1$ in one run: the total variation distance between each one-way and each measured marginal of the model and the real one, the largest cell error, the table's own standard deviation, and the bound from it.

Marginal	Distance	Largest cell error (rows)	`std`	Bound
`SEX`	0.00004	158	196.2	0.00023
`AGE`	0.00005	146	219.9	0.00035
`HWUSUAL`	0.00006	246	219.9	0.00035
`ILOSTAT`	0.00013	307	201.0	0.00040
(`SEX`, `ILOSTAT`)	0.00022	425	178.9	0.00106
(`AGE`, `HWUSUAL`, `ILOSTAT`)	0.00102	509	379.5	0.01195
(`SEX`, `AGE`, `HWUSUAL`)	0.00081	503	379.5	0.00717
(`SEX`, `AGE`, `ILOSTAT`)	0.00072	568	379.5	0.00896
(`SEX`, `HWUSUAL`, `ILOSTAT`)	0.00065	470	379.5	0.00896

Every distance is under its bound by a factor of three or more. The worst measured marginal, the triple of `AGE`, `HWUSUAL` and `ILOSTAT`, is under it by a factor of 11.7. No cell is off by more than a few hundred rows out of 3.8 million. The one-way standard deviations are not the one-way scale of 227.68, because the model combines each one-way measurement with every measured marginal that contains the column. That is why `ILOSTAT`, which sits in four marginals, gets 201.0. The triples have larger distances than the one-way marginals although their largest cell errors are similar, since they spread the same noise over many more cells.

Read the table as a check of the pipeline, not only of the noise. A distance that breaks its bound means something systematic, and the previous chapter traced one such break to nulls that the two steps of the release counted differently. On private data you cannot fill this table in, because it needs the real marginals. You can fill in the `std` column, which comes from the release, and that is the error bar you publish.

18.3. A query nobody measured

The second question asks about a table that no measurement covers. AIM's candidates have at most three columns, so it never measures the four-way table of `SEX`, `AGE`, `HWUSUAL` and `ILOSTAT`, whose 240 cells hold every combination at once. The model still answers it. It fills those cells with the distribution that agrees with the measured marginals and assumes nothing else. The full edition's chapter on query release explains why many queries are best answered through one such model. But the table can no longer put a bar on the answer.

```
try:
    four_std = f"{table_aim.std(COLUMNS):.1f}"
except ValueError as err:
    four_std = str(err)  # no measured marginal covers all four columns, so the table has no
```

```
four-way cells: 240  measured by AIM: False
std of the four-way table: attrs (('SEX', 'AGE', 'HWUSUAL', 'ILOSTAT')) are not covered by t
four-way (aim rho=0.1, one run): tv=0.00141 largest cell error=450 rows  best measured (SEX,
cells of the share with at least 1,000 real rows: 26
share working by SEX, AGE, HWUSUAL (aim rho=0.1, one run): largest error=2.05 points at {'SE
```

In this run the four-way table is further from the truth than every measured marginal. Its errors add up from two sources. One is the noise of the overlapping triples it is assembled from. The other is any dependence among all four columns that no triple shows, which the model cannot know about. Here the four measured triples overlap so much that little is left to guess, and the distance stays near the worst measured one. A release that measured fewer marginals would leave more to the model's assumptions, as Section 18.5 shows for MST.

A conditional share is a harder test, because it divides one small count by another. The script reads the share of people working in each combination of `SEX`, `AGE` band and `HWUSUAL` band off the four-way table. It compares the 26 cells with at least a thousand real rows. The largest error in this run was two points, in the cell of men under 20 who usually work 1 to 20 hours. Everyone in that cell works, since a usual number of hours implies a job, but the model gave the cell a share of 97.95%. Noise is added per count, not per person, so a small group gets the largest relative

error. When an analyst will ask about small groups, measure them, for instance with `Fixed` inside `Sequential` as the previous chapter did.

18.4. Train on synthetic, test on real

The third question is whether a model trained on the synthetic rows works on real rows. The benchmark calls this "train on synthetic, test on real". The task here is to predict whether a person works, `ILOSTAT` equal to 1, from `SEX`, the `AGE` band and the `HWUSUAL` band. The model is a logistic regression from scikit-learn on eleven indicator columns, one per key of each feature.

The script draws 200,000 synthetic rows with `method="sample"`, which draws each row independently from the model, as a sample from the population would be. It also takes 400,000 real rows in one seeded Polars shuffle. The first 200,000 train the yardstick, a model on real rows, and the rest are held out to score both models.

```
rows = synth.synthesize(table, seed, rows=ROWS, method="sample")  # independent draws from t
rows = synth.binned(rows.lazy(), table.keys, fill=False).collect()  # the numbers back in th
model = classifier(rows, table.keys, seed)  # train on synthetic
```

```
def classifier(train: pl.DataFrame, keys: dict[str, Any], seed: int) -> LogisticRegression:
    """A logistic regression that predicts ILOSTAT == 1 from SEX, the AGE band and the HWUSU
    `train`, which is either synthetic rows or real rows."""
    return LogisticRegression(max_iter=1000, random_state=seed).fit(design(train, keys), lab


def accuracy(model: LogisticRegression, test: pl.DataFrame, keys: dict[str, Any]) -> float:
    """The share of the held-out real rows the model labels correctly."""
    return float((model.predict(design(test, keys)) == labels(test)).mean())
```

```
held-out real rows: 200,000  real training rows: 200,000  synthetic training rows: 200,000
features: 11 indicators
share working in the held-out rows: 0.3976
train on real, test on real: accuracy=0.9313
standard error of an accuracy on 200,000 rows: 0.0006
train on synthetic (aim rho=0.1, one run), test on real: accuracy=0.9313  gap=+0.0000  mean
```

The model trained on real rows labels 93.13% of the held-out rows correctly, against 39.76% working. The model trained on synthetic rows scored the same in this run, so the gap is zero, well inside the standard error of 0.0006. The test allows a gap of one percentage point. That is generous against the standard error, and it is the size of gap a model would show if it got a few small cells wrong.

Accuracy is a coarse check, though. Two models that put every cell's probability on the same side of one half make the same predictions, whatever the probabilities are. So the script also reports the mean difference between the two models' predicted probabilities, 0.0015 in this run, which sees changes that accuracy cannot. A richer model, one with interactions between the features,

would also depend on the parts of the table that are hardest to get right. Statistical modelling fits private models directly, and for a single model that is often better than training on synthetic rows.

18.5. What the budget and the algorithm buy

Tuning means running the evaluation under different settings and choosing between them. The script repeats it with AIM at a fifth of the budget, $\rho = 0.02$, and at five times it, $\rho = 0.5$. Then it runs MST at the original budget. Each release gets a context of its own, because the comparison is between alternatives and you would publish only one of them. Released from the same private data, the four would compose to $\rho = 0.72$, and so tuning on the private data spends budget too. Tune on a public stand-in, and release once.

```
mst_query = table_query(make_context(frame, RHO), RHO, dp.mbi.MST())
table_mst, seconds_mst = synth.timed_release(mst_query)
mst_eval = evaluate("mst rho=0.1", table_mst, seconds_mst, real, test_real, baseline, seed)
```

```
aim rho=0.02: oneway scale=509.1169 threshold=3004  scale of a round's marginal=848.5281  ep
aim rho=0.5: oneway scale=101.8234 threshold=630  scale of a round's marginal=169.7056  epsi
mst rho=0.1: oneway scale=227.6840 threshold=1364  scale of a pair=207.8461  epsilon at delt
mst rho=0.1 measured: ('HWUSUAL', 'ILOSTAT') std=207.85
mst rho=0.1 measured: ('AGE', 'ILOSTAT') std=207.85
mst rho=0.1 measured: ('SEX', 'ILOSTAT') std=207.85
share working by SEX, AGE, HWUSUAL (mst rho=0.1, one run): largest error=24.03 points at {'S
budget the four contexts add up to if all were released from the same data: rho=0.72
```

The scales follow the formula of the previous chapter, the sensitivity of 36 over the square root of twice the budget. Twenty-five times the budget therefore divides every scale by five, from 509.12 to 101.82 for the one-way marginals. Converted to ε, the smallest budget is 1.00 and the largest 5.67. The full edition's chapter on choosing epsilon discusses which of them a release can defend.

Table 18.2.: The same evaluation at three budgets with AIM and at one with MST, in one run: ε at $\delta = 10^{-7}$, the seconds the release took, the number of marginals measured, the total variation distances of the one-way marginals (mean), of the worst measured marginal and of the four-way table, the largest error of the share working, and the mean difference in predicted probability from the model trained on real rows.

Release	ε	Seconds	Measured	One-way mean	Worst measured	Four-way	Share error (points)	Probability difference
AIM, $\rho = 0.02$	1.00	6.4	4	0.00016	0.00157	0.00252	15.64	0.0016
AIM, $\rho = 0.1$	2.35	13.7	5	0.00007	0.00102	0.00141	2.05	0.0015

Release	ε	Seconds	Measured	One-way mean	Worst measured	Four-way	Share error (points)	Probability difference
AIM, $\rho = 0.5$	5.67	7.1	6	0.00005	0.00052	0.00077	1.65	0.0014
MST, $\rho = 0.1$	2.35	3.3	3	0.00021	0.00114	0.05610	24.03	0.0280

The budget buys accuracy everywhere, but not evenly. Across the three budgets, the worst measured marginal and the four-way table improve steadily, though by less than the scales shrink. The one-way marginals, already close at $\rho = 0.1$, improve least. The share error falls fastest at the low end, from 15.64 points to 2.05, because small groups are the first to drown in noise. A larger budget also lets AIM measure more marginals in its rounds, and in this run it measured four, five and six. All four classifiers scored the same accuracy of 0.9313, so for this task the budget makes no visible difference.

The algorithm changes more than the budget does. MST measures three pairs, all of them with `ILOSTAT` in this run, and so treats `SEX`, `AGE` and `HWUSUAL` as independent once employment status is known. That assumption is false, since hours depend on sex and age among those who work. The four-way table comes out tens of times further from the truth than AIM's at the same budget. The share error reaches 24 points, in the cell of women under 20 who report at most one usual hour. The predicted probabilities move by 0.028 on average, while the accuracy still does not change. MST is the better choice when the questions are about pairs and the release must be fast, since in this run it took 3.3 seconds against 13.7 for AIM.

The cuts and the number of rounds are the other two settings. Finer cuts multiply the cells of every marginal, and since every cell carries noise of the same scale, the bound on each marginal grows with them. More rounds give each round a smaller share, so AIM measures more marginals, each less precisely. Evaluate either change on a public stand-in in the same way.

18.6. How many rows, and how to draw them

The evaluation of the marginals read the model, not the rows, and this section shows why. Drawing a fixed number of rows independently adds a sampling error on top of the model's error. The script computes its expected size for each one-way marginal from the table's own shares, at 200,000 rows and at the size of the real table.

```
sampling noise on one-way SEX: expected tv at 200,000 rows=0.00090  at 3,811,932 rows=0.0002
sampling noise on one-way AGE: expected tv at 200,000 rows=0.00152  at 3,811,932 rows=0.0003
sampling noise on one-way HWUSUAL: expected tv at 200,000 rows=0.00100  at 3,811,932 rows=0.
sampling noise on one-way ILOSTAT: expected tv at 200,000 rows=0.00140  at 3,811,932 rows=0.
aim rho=0.1 one-way tv summed (one run): model=0.00029  round=0.00028  sample=0.00745
aim rho=0.1 four-way tv (one run): model=0.00141  round=0.00139  sample=0.00483
mst rho=0.1 four-way tv (one run): model=0.05610  round=0.18410  sample=0.05791
mst rho=0.1 pair ('AGE', 'HWUSUAL') tv (one run): model=0.01504  round=0.12370  sample=0.012
```

At 200,000 sampled rows the sampling error of `SEX` is many times the model's error in Table 18.1, so the rows are much worse than the release. Even at the real table's size the sampling error stays above the model's, because the privacy noise at this budget is small against 3.8 million rows. More rows than the model's total add no information, since they only sample the same model more finely.

The default `method="round"` avoids the sampling error, and for AIM its marginals matched the model's in this run. But in `mbi` 1.1.0 it draws each column by the rank of a row within the group of its parent columns. Two columns drawn from the same parents are then sorted together, which ties them. MST's star makes `AGE` and `HWUSUAL` such a pair, and rounding moved their marginal from 0.01504 to 0.12370 while sampling left it near the model. So sample when the joint distribution of the rows matters, and draw as many rows as the model's total if the marginals must be close. Check the pairs before you trust rounded rows.

18.7. What to publish

A synthetic dataset needs a statement of its guarantee and enough description for a user to judge what it can answer. The practitioner edition's chapter on release and governance builds such a statement for a trained model, and the same rules apply. The numbers come from the variables of the script, never typed in, and the statement names what was public.

```
print(
    f"This synthetic dataset of {ROWS:,} rows was generated by AIM (OpenDP {dp.__version__},
    f"{importlib.metadata.version('mbi')}, rounds={ROUNDS}, queries up to 3 columns) from on
    f"over {', '.join(COLUMNS)} released under (rho = {RHO:g}, delta = {DELTA:g})-zero-conce
    f"privacy, which is (epsilon = {epsilon:.2f}, delta = {2 * DELTA:g})-differential privac
    f"removing all of one person's survey responses, at most {CONTRIBUTIONS} of them, under
    f"The keys of SEX and the cuts of AGE and HWUSUAL were fixed in advance; nulls were reco
    f"{synth.FILL}; the keys of ILOSTAT were released with a threshold of {aim_query.oneway_
    f"Drawing the rows by independent sampling is post-processing and spent nothing."
)
```

```
This synthetic dataset of 200,000 rows was generated by AIM (OpenDP 0.16.0, mbi 1.1.0, round
```

The statement covers the rows without a further term, because drawing rows from a released model is post-processing, which the full edition's chapter on properties proves cannot weaken the guarantee. It covers this one table only. The other releases of this chapter, made from the same people, would add to it.

Table 18.3.: What to publish with a synthetic dataset, and where this chapter found each item.

Item	What to publish	In this chapter
The guarantee	ρ and δ, their conversion to ε, the privacy unit and the trust model, printed from the variables	$\rho = 0.1$, $\varepsilon = 2.35$, 36 contributions, central model
The columns	Each column with its keys or cuts, the null recoding, and which keys were released	`SEX` keys, `AGE` and `HWUSUAL` cuts, `ILOSTAT` keys at threshold 1364
The algorithm	Its name and parameters, the library versions, and how the rows were drawn	AIM with 10 rounds, OpenDP 0.16.0, `mbi` 1.1.0, 200,000 sampled rows
The error bars	The measured marginals and their `std`, which come from the release and cost nothing	Table 18.1, the `std` column
The utility	The marginal errors and the downstream check, measured on public or historical data	Never on the private data itself

The last row is the one most often done wrong. A distance computed on the private data depends on that data, so publishing it is a new release that the statement does not cover. Limitations and troubleshooting put it as a rule: the guarantee covers what passes through a measurement and nothing computed beside it. The error bars from `std` are safe to publish. The distances in this chapter's tables were safe only because the data are public.

18.8. Summary

Table 18.4.: Evaluating and tuning synthetic data, what each step does, and the rule of thumb that goes with it.

Call or concept	What it does	Rule of thumb
Three questions	Measured marginals, unmeasured queries, a downstream model	Only the first has an error bar from the release
`table.project_melted(clique)`	The fitted model's counts for a marginal, without drawing rows	Evaluate the model first, then the rows
Total variation distance against `std`	Workload error of a marginal against three standard deviations in every cell	A distance over the bound points to the pipeline, not the noise
`table.std` on an uncovered table	Raises `ValueError`: not covered by the query set	No measurement, no error bar; check unmeasured queries on public data

Call or concept	What it does	Rule of thumb
Train on synthetic, test on real	A model fitted to synthetic rows, scored on held-out real rows	Accuracy is coarse; compare predicted probabilities too
One context per setting	Alternatives that you compare but release only one of	On the same private data their budgets add up; tune on a stand-in
The budget	Divides every scale by the square root of its ratio	Small groups and unmeasured tables gain most; one-way marginals least
AIM against MST	Triples chosen adaptively, or a tree of pairs	MST is fast but loses what no pair shows
`synthesize(rows, method="sample")`	Independent rows, with sampling error of their own	Draw as many rows as the model's total when the marginals must be close
`method="round"` in `mbi` 1.1.0	Matches the model's marginals but ties columns drawn from the same parents	Check the pairs before trusting rounded rows
What to publish	Guarantee, columns, algorithm, error bars, utility on public data	Never publish a distance computed on the private data

18.9. References

- OpenDP documentation: "Synthetic Data" in the statistical modelling guide, "Contingency Tables" in the tabular data guide, and the API reference for `opendp.extras.mbi.ContingencyTable` (`std`, `project_melted`, `synthesize`). The documentation gives no evaluation; this chapter's is its own.
- McKenna, Mullins, Sheldon and Miklau, "AIM: An Adaptive and Iterative Mechanism for Differentially Private Synthetic Data" (2022): the workload error and the selection that minimizes it.
- Tao, McKenna, Hay, Machanavajjhala and Miklau, "Benchmarking Differentially Private Synthetic Data Generation Algorithms" (2021): marginal errors and downstream tasks as the measures of quality.
- The practitioner edition's chapter on release and governance in the practitioner edition: the statement of a guarantee and the evidence that goes with it.
- The full edition's chapters on post-processing and composition in the full edition, query release in the full edition, public data in the full edition and choosing epsilon in the full edition derive the results this chapter states.

This is the last chapter of the book. You can now state a privacy unit and a budget, release statistics, tables and models through a context, and read every scale and threshold before spending anything. You can build the framework's pieces by hand when the context is not enough, and turn a contingency table into synthetic rows. You can also measure how good those rows are, on

data you are allowed to look at, and publish them with a guarantee that says exactly what it covers.

A. Glossary

Acronyms used in the book and the pages that use them.

API Application programming interface. Used in Getting started, A first release, The data and the context, Grouping, Assessing utility and choosing the noise, The framework, A framework to understand differential privacy, Transformations, Measurements, Combinators, Utilities and the Context API inside, Extending the library, Plugins, Statistical modelling, Limitations and troubleshooting, Synthetic data with AIM and MST, Evaluating and tuning synthetic data.

CPU Central processing unit. Used in Synthetic data with AIM and MST.

CSV Comma-separated values. Used in The data and the context, Limitations and troubleshooting.

DP Differential privacy. Used in Preface, Essential statistics, Assessing utility and choosing the noise, A framework to understand differential privacy, Measurements, Combinators, Statistical modelling, Synthetic data with AIM and MST.

JSON JavaScript Object Notation. Used in Utilities and the Context API inside.

LFS Labour Force Survey. Used in Preface, Getting started, The data and the context, Essential statistics, Grouping, Identifier truncation and bounds, Preparing microdata, Assessing utility and choosing the noise, A framework to understand differential privacy, Transformations, Measurements, Combinators, Utilities and the Context API inside, Plugins, Statistical modelling, Limitations and troubleshooting, Contingency tables, Synthetic data with AIM and MST, Evaluating and tuning synthetic data.

PCA Principal component analysis. Used in Statistical modelling.

RDP Rényi differential privacy. Used in Combinators.

SQL Structured Query Language. Used in Identifier truncation and bounds.

B. Index

adaptive composition, 125, 135
approximate differential privacy, 87, 135

basic composition, 125

clipping, 32, 73, 74
composition, 11, 20, 32, 41, 87, 88, 100, 124, 125, 135, 151, 203

differential privacy, 4, 10, 20, 22, 32, 41, 52, 74, 76, 87, 88, 100, 112, 125, 135, 137, 151, 160, 170, 172, 181, 193, 203
discrete Gaussian distribution, 74, 112, 125
discrete Laplace distribution, 111, 112

empirical risk minimization, 160
exponential mechanism, 32, 151

fully adaptive composition, 125

Gaussian mechanism, 11, 32, 74, 87, 88, 100, 113, 135, 151, 181
group privacy, 20

honest-but-curious, 124, 125, 150, 160, 169

Laplace mechanism, 11, 20, 32, 52, 88, 112

one-way marginal, 192, 199, 203

permute-and-flip, 100, 112
post-processing, 11, 20, 32, 41, 88, 100, 124, 125, 172, 181, 192, 203
privacy filter, 125
privacy loss, 125, 159
privacy odometer, 125
privacy profile, 87, 125
pure differential privacy, 10, 20, 73, 87, 111, 125, 135

randomized response, 76, 112, 113
report-noisy-max, 112
Rényi differential privacy, 125

sensitivity, 10, 11, 20, 32, 41, 51, 52, 64, 72, 73, 87, 88, 100, 112, 113, 151, 160, 170, 181
smooth sensitivity, 64
sparse vector technique, 112

total variation distance, 192, 196, 199, 202

variance, 52, 74, 100

zero-concentrated differential privacy, 22, 72–74, 87, 111, 125, 135, 180, 181